

Numerical Optimization and Model Predictive Control

Prof. Dr.-Ing. Knut Graichen

Summer Semester 2026

Chair of Automatic Control

Faculty of Engineering

Friedrich-Alexander-Universität Erlangen-Nürnberg

“Cum enim mundi universi fabrica sit perfectissima atque a Creatore sapientissimo absoluta, nihil omnino in mundo contingit, in quo non maximi minimive ratio quaequam eluceat; quamobrem dubium prorsus est nullum, quin omnes mundi effectus ex causis finalibus ope methodi maximorum et minimorum aequè feliciter determinari queant, atque ex ipsis causis efficientibus.”

“As the construction of the entire world was most perfectly accomplished by the wisest Creator, nothing happens in this world in which the relation of a maximum or minimum does not shine forth; therefore there is absolutely no doubt that all effects in this world that are due to final causes can be equally determined from the method of maxima and minima as from the effected causes themselves.” (freely translated)

Leonhard Euler, *Calculus of Variations*, 1744.

Contents

1	Introduction	1
1.1	Parameter optimization problems	1
1.1.1	Mathematical formulation	1
1.1.2	Examples	2
1.2	Dynamic optimization problems	5
1.2.1	Mathematical formulation	5
1.2.2	Examples	6
1.3	References	10
2	Basics of optimization	11
2.1	Local/global minima and admissible set	11
2.2	Gradient and Hessian matrix	13
2.3	Convexity	14
2.3.1	Convex sets	14
2.3.2	Convex functions	15
2.3.3	Convex optimization problems	18
2.4	References	19
3	Unconstrained optimization	21
3.1	Necessary optimality conditions	21
3.2	Sufficient optimality conditions	22
3.3	Line search methods	23
3.3.1	Step size computation	25
3.3.2	Gradient method	28
3.3.3	Conjugate gradient method	30
3.3.4	Newton's method	30
3.3.5	Quasi-Newton method	32
3.4	Further numerical methods	33
3.4.1	Trust region method	33
3.4.2	Direct search methods	35
3.5	Example: Rosenbrock's "banana function"	37
3.6	References	41

4	Constrained optimization	43
4.1	Illustrative examples	43
4.1.1	One equality constraint	43
4.1.2	One inequality constraint	45
4.1.3	Two inequality constraints	47
4.2	Optimality conditions	48
4.2.1	Constraint qualification	48
4.2.2	First-order optimality conditions	49
4.2.3	Second-order optimality conditions	51
4.2.4	Interpretation of Lagrange multipliers	52
4.3	Linear optimization	53
4.3.1	Standard form	54
4.3.2	Optimality conditions	54
4.3.3	Simplex method	55
4.3.4	Interior point method	60
4.3.5	Example: Transport network	63
4.4	Quadratic optimization	64
4.4.1	Optimality conditions	65
4.4.2	Equality-constrained quadratic problems	65
4.4.3	Active set method	66
4.4.4	Further numerical methods	69
4.5	Nonlinear optimization	69
4.5.1	Sequential quadratic programming (SQP)	69
4.5.2	Further numerical methods	74
4.6	Lagrangian duality	76
4.6.1	Dual formulation	76
4.6.2	Augmented Lagrangian method	82
4.6.3	Consideration of inequality constraints	84
4.6.4	Remarks	85
4.7	Software overview	86
4.8	References	88
5	Dynamic optimization	89
5.1	Problem statement	89
5.2	Calculus of variations	91
5.2.1	Fundamentals	91
5.2.2	Euler-Lagrange equations	94
5.2.3	Example: Galileo's hanging chain	96
5.3	Unconstrained optimal control problems	97
5.3.1	Application of the calculus of variations	98
5.3.2	Optimality conditions	100

5.3.3	Example with free end time	103
5.3.4	General procedure and singular case	105
5.3.5	Interpretation of adjoint variables	108
5.3.6	General terminal conditions	109
5.4	Optimization of linear systems with quadratic cost functional	110
5.4.1	Riccati differential equation	110
5.4.2	Example: Swing-up of the pendulum on a cart	111
5.4.3	Algebraic Riccati equation	114
5.5	Optimal control problems with input constraints	117
5.5.1	Pontryagin's maximum principle	117
5.5.2	General procedure	119
5.5.3	Minimization of the Hamiltonian	120
5.5.4	Example: Double integrator	123
5.5.5	State-dependent input constraints	125
5.6	Numerical solution of optimality conditions (indirect methods)	126
5.6.1	Discretization method	126
5.6.2	Shooting method	128
5.6.3	Handling of general terminal constraints and free end time	129
5.6.4	Gradient method	130
5.7	Numerical solution by discretization (direct methods)	132
5.7.1	Partial discretization	133
5.7.2	Full discretization	134
5.8	Software overview	135
5.9	References	135
6	Model predictive control (MPC)	137
6.1	Basic concept behind MPC	137
6.1.1	Dynamic optimization on moving horizon	137
6.1.2	Continuous-time vs. discrete-time MPC	139
6.2	MPC formulations	140
6.2.1	Infinite horizon	140
6.2.2	Finite horizon and terminal condition	142
6.2.3	Finite horizon and terminal set	143
6.2.4	Finite horizon and terminal cost	145
6.2.5	Sufficiently long horizon	147
6.3	Real-time implementation	151
6.3.1	Suboptimal solution strategy	152
6.3.2	Stability and incremental improvement	153
6.3.3	Application example	155
6.4	Software overview	158
6.5	References	159

1 Introduction

The word “optimization” refers to the search for an optimal solution to a given problem with the meaning of “optimal” being more or less well defined. Optimization problems are widely present in industry and economy, some examples are:

- Economy and financial sector: portfolio optimization or company development (e.g. maximization of profit through optimal distribution of investments, expenses etc.)
- Process and product optimization: process scheduling, minimization of production costs, energy efficient operation, etc.
- Control engineering: optimal feedback control, optimal trajectory planning, parameter estimation, etc.

The mathematical formulation of these problems leads to optimization problems that can be classified as follows:

- *Parameter optimization*: minimization of a function with the optimization variables being elements of the Euclidean space
- *Dynamic optimization*: minimization of a functional with the optimization variables being elements of a Hilbert space (e.g. time functions).

In this chapter, we present some introductory examples to highlight the difference between parameter and dynamic optimization.

1.1 Parameter optimization problems

Mathematically speaking, a *parameter optimization problem* minimizes a function $f(\mathbf{x})$ subject to additional constraints, where the optimization variables $\mathbf{x}$ are elements of the Euclidean space $\mathbb{R}^n$.

1.1.1 Mathematical formulation

A general nonlinear parameter optimization problem is typically expressed as

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{cost function} \quad (1.1)$$

$$\text{s.t. } g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \quad \text{equality constraints} \quad (1.2)$$

$$h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q \quad \text{inequality constraints} \quad (1.3)$$

We speak of *constrained* optimization, if the cost function $f(\mathbf{x})$ to be minimized is subject to (s.t.) constraints of the form (1.2) or (1.3), otherwise (1.1) is referred to as *unconstrained* optimization. The min-formulation in (1.1) became standard over the decades. A maximization problem can equally be formulated as minimization problem by negating the cost function

$$\max_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) = \min_{\mathbf{x} \in \mathbb{R}^n} -f(\mathbf{x}).$$

Further well known terms for parameter optimization are *finite-dimensional optimization* or *mathematical programming*. The term “programming” is historically rooted, as the first applications of optimization were military action plans:

“It is a word used by the military for a plan, a schedule, or in general for a program of actions. This is exactly what it means for computers: a program of actions for the computer to execute.”

George Danzig
Founder of linear programming

Parameter optimization is typically differentiated in the following classes:

- *Linear programming*: The cost function and constraints are linear.
- *Quadratic programming*: The cost function is quadratic, the constraints are linear.
- *Nonlinear programming*: The cost function or at least one of the constraints is nonlinear.
- *Integer programming*: All optimization variables are discrete.
- *Mixed-integer programming*: The optimization variables are partly continuous and discrete.

1.1.2 Examples

Linear programming is often used in economic optimization tasks such as production planning and investment problems. The following example is a strongly simplified portfolio optimization problem.

Example 1.1 (Portfolio optimization) An investor wants to invest 10000 euros in three equity funds with different profit expectations and risk ratings:

Fund	Expected profit/year	Risk classification
A	10%	4
B	7%	2
C	4%	1

The investor wants to make a profit of at least 600 euros after one year. On the other hand, he would like to invest his money rather conservatively, i.e. at least 4000 euros shall be invested in fund C and the overall risk is to be minimized. What is the optimal allocation of the investor's 10000 euros in order to fulfill these criteria?

The first step for solving the portfolio optimization problem is its mathematical formulation. To this end, we define the optimization variables $x_1, x_2, x_3 \geq 0$ as the ratios of the 10000 euros invested in fund A, B, and C, respectively. Hence, the third variable x_3 can be substituted by the relation

$$x_3 = 1 - x_1 - x_2.$$

The requested minimum profit of 600 euros corresponds to the inequality constraint

$$10000[0.1x_1 + 0.07x_2 + 0.04(1 - x_1 - x_2)] \geq 600 \Rightarrow 6x_1 + 3x_2 \geq 2. \quad (1.4)$$

Likewise, the minimum investment of 4000 euros in fund C implies

$$10000(1 - x_1 - x_2) \geq 4000 \Rightarrow x_1 + x_2 \leq 0.6. \quad (1.5)$$

The optimization objective is to minimize the overall risk. This can be expressed by the function

$$f(\mathbf{x}) = 4x_1 + 2x_2 + (1 - x_1 - x_2) = 1 + 3x_1 + x_2. \quad (1.6)$$

In summary, the linear optimization problem becomes

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = 1 + 3x_1 + x_2 \quad (1.7a)$$

$$\text{s.t. } 6x_1 + 3x_2 \geq 2 \quad (1.7b)$$

$$x_1 + x_2 \leq 0.6 \quad (1.7c)$$

$$x_1 + x_2 \leq 1 \quad (1.7d)$$

$$x_1, x_2 \geq 0. \quad (1.7e)$$

Figure 1.1 shows the single constraints and the admissible set. The shape of the contour lines $f(\mathbf{x}) = \text{const.}$ of the cost function (1.7a) illustrates that the point of the admissible set with the lowest value of $f(\mathbf{x})$ is located in the corner at $\mathbf{x}^*$. Hence, the optimal distribution of the investment of 10000 euros over the three funds is

$$x_1^* = \frac{1}{15}, \quad x_2^* = \frac{8}{15}, \quad x_3^* = \frac{6}{15}. \quad (1.8)$$

The following (academic) optimization problem is intended to illustrate the influence of constraints on an optimal solution.

Example 1.2 We start with the unconstrained quadratic problem

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2. \quad (1.9)$$

The contour lines $f(\mathbf{x}) = \text{const.}$ of the function $f(\mathbf{x})$ are shown in Figure 1.2 as a function of the two optimization variables $\mathbf{x} = [x_1, x_2]^T$. It is easy to see that the minimum $f(\mathbf{x}^*) = 0$ is located at the position $\mathbf{x}^* = [2, 1]^T$.

In order to investigate the influence of constraints on the optimal solution, we add an equality constraint of the form (1.2)

$$g(\mathbf{x}) = x_2 - 2x_1 = 0. \quad (1.10)$$

The geometric interpretation is that a possible solution must lie on the straight line given by (1.10), see Figure 1.3. In particular, the optimal solution lies on the touchpoint of the straight line $g(\mathbf{x}) = 0$ with the contour line $f(\mathbf{x}) = 1.8$.

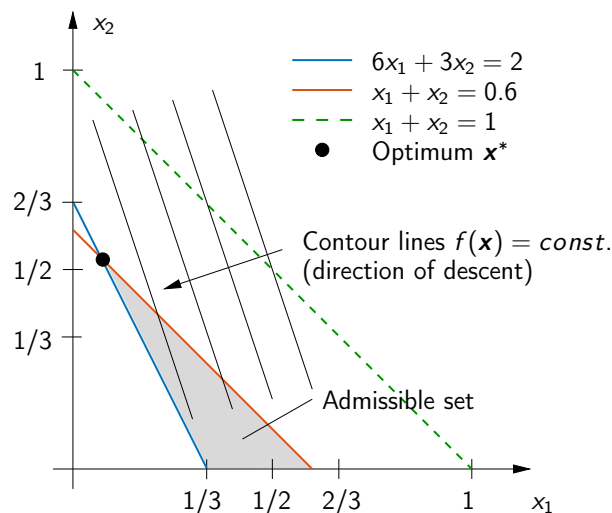


Figure 1.1: Illustration of the portfolio optimization problem in Example 1.1.

We now substitute the equality constraint (1.10) with the inequality constraint

$$h_1(\mathbf{x}) = x_1 + x_2 - 2 \leq 0, \quad (1.11)$$

which reduces the set of admissible points to the region below the straight line $h_1(\mathbf{x}) = 0$ (Figure 1.4). The optimum $f(\mathbf{x}^*) = 0.5$ at the point $\mathbf{x}^* = [1.5, 0.5]^T$ lies at the boundary of the admissible set on a contour line which touches $h_1(\mathbf{x}) = 0$.

The admissible set is further reduced if we consider a second inequality constraint

$$h_2(\mathbf{x}) = x_1^2 - x_2 \leq 0. \quad (1.12)$$

The optimal point $\mathbf{x}^* = [1, 1]^T$ with the minimum $f(\mathbf{x}^*) = 1$ now lies in the intersection of the curves $h_1(\mathbf{x}) = 0$ and $h_2(\mathbf{x}) = 0$, i.e. both constraints (1.11) and (1.12) are active, see Figure 1.5.

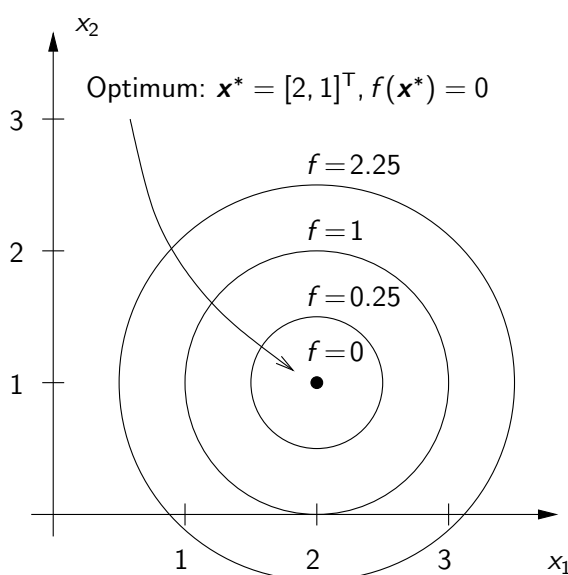


Figure 1.2: Geometric illustration of the unconstrained problem (1.9).

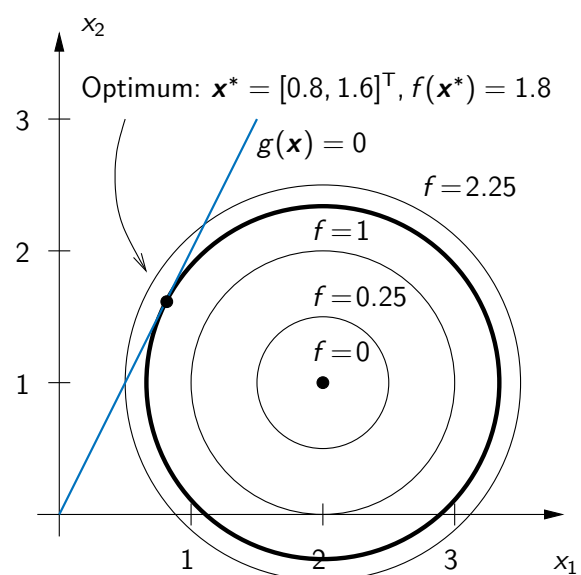


Figure 1.3: Geometric illustration of the constrained problem (1.9), (1.10).

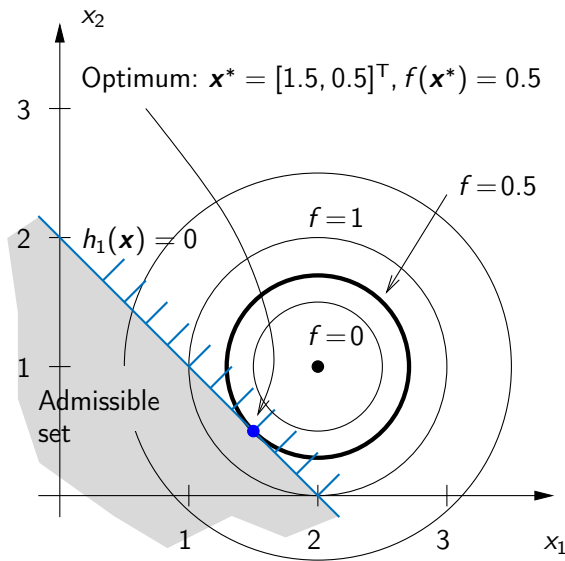


Figure 1.4: Geometric illustration of the constrained problem (1.9), (1.11).

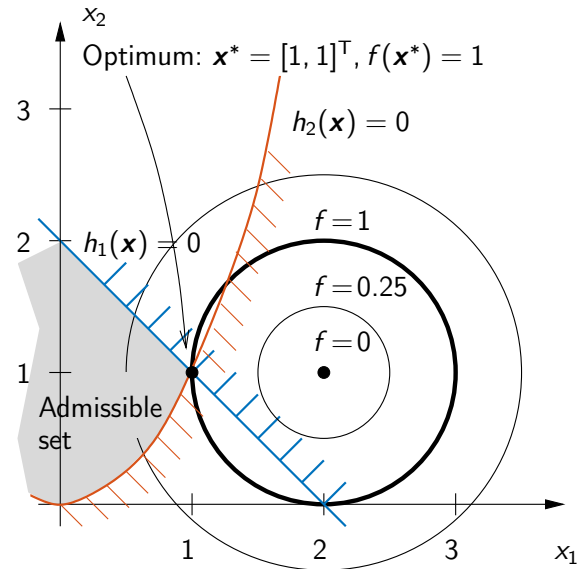


Figure 1.5: Geometric illustration of the constrained problem (1.9), (1.11), (1.12).

Example 1.2 illustrates the influence of equality and inequality constraints on the solution (and solvability) of the optimization problem (1.1)-(1.3). The systematic analysis of parameter optimization problems as well as numerical methods for their solution will be covered in the chapters to come.

1.2 Dynamic optimization problems

As shown in the last section, the optimization variables in parameter optimization problems are elements of the Euclidean space $\mathbb{R}^n$. In dynamic optimization, we are concerned with minimizing a functional subject to constraints, where the optimization variables are elements of a Hilbert space. In particular, we speak of *dynamic optimization* if the optimization variables are functions of time t .

1.2.1 Mathematical formulation

The general structure of a dynamic optimization problem is

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{u}) = V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt \quad \text{cost functional} \quad (1.13a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad \text{system dynamics \& IC} \quad (1.13b)$$

$$\mathbf{g}(\mathbf{x}(t_f), t_f) = \mathbf{0} \quad \text{terminal constraints} \quad (1.13c)$$

$$h_i(\mathbf{x}, \mathbf{u}) \leq 0, \quad i = 1, \dots, q \quad \text{inequality constr.} \quad (1.13d)$$

where $\mathbf{u} \in \mathbb{R}^m$ is the control input for the nonlinear system (1.13b) with the state $\mathbf{x} \in \mathbb{R}^n$ and the respective initial conditions (IC). The terminal conditions of the general form (1.13c) may

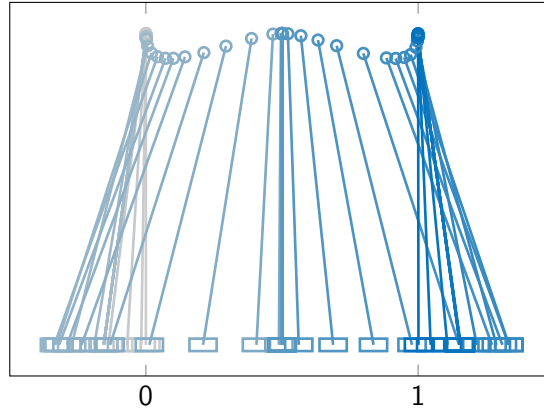


Figure 1.6: Snapshots of the side-stepping of the pendulum (Example 1.3).

e.g. be relevant to reach a desired state $\mathbf{x}_f$ at the end time t_f (i.e. $\mathbf{g}(\mathbf{x}(t_f), t_f) = \mathbf{x}(t_f) - \mathbf{x}_f$). The inequality constraints (1.13d) are also relevant in many practical applications, for instance, to account for constraints on the control $\mathbf{u}$ or on the state $\mathbf{x}$.

The problem of dynamic optimization is to find an optimal control trajectory $\mathbf{u}^*(t)$, $t \in [t_0, t_f]$ and associated state trajectory $\mathbf{x}^*(t)$ such that the cost functional (1.13a) is minimized while the dynamical system (1.13b), the terminal conditions (1.13c), and the constraints (1.13d) are satisfied. The end time t_f may be fixed (specified beforehand) or unknown. In the latter case, (1.13) is a *free end-time problem* and t_f is an additional optimization variable for solving (1.13).

Other well known terms for dynamic optimization are *infinite-dimensional optimization* or *dynamic programming*. The problem (1.13) is also called *optimal control problem*.

1.2.2 Examples

The following lines present some illustrative examples to familiarize ourselves with the concept of dynamic optimization.

Example 1.3 (Inverted pendulum) A classical benchmark problem in control is the inverted pendulum on a cart. We consider a side-stepping scenario formulated as the following optimal control problem

$$\min_{u(\cdot)} J(u) = \int_0^{t_f} 1 + cu^2 dt \quad (1.14a)$$

$$\text{s.t.} \quad \begin{bmatrix} 1 & \varepsilon \cos \theta \\ \cos \theta & 1 \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} \varepsilon \dot{\theta}^2 \sin \theta + u \\ -\sin \theta \end{bmatrix}, \quad \varepsilon = m/(M + m) \quad (1.14b)$$

$$\mathbf{x}(0) = [0, 0, 0, 0]^T, \quad \mathbf{x}(t_f) = [1, 0, 0, 0]^T \quad (1.14c)$$

$$-1 \leq u \leq 1. \quad (1.14d)$$

The simplified and normalized equations of motion for the states $\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$ are given by (1.14b) with the parameter ε that involves the mass of the pendulum and the cart, respectively. The input force u acting on the cart is box-constrained by (1.14d). Figure 1.6 shows an example of the lateral displacement of the pendulum to illustrate the movement.

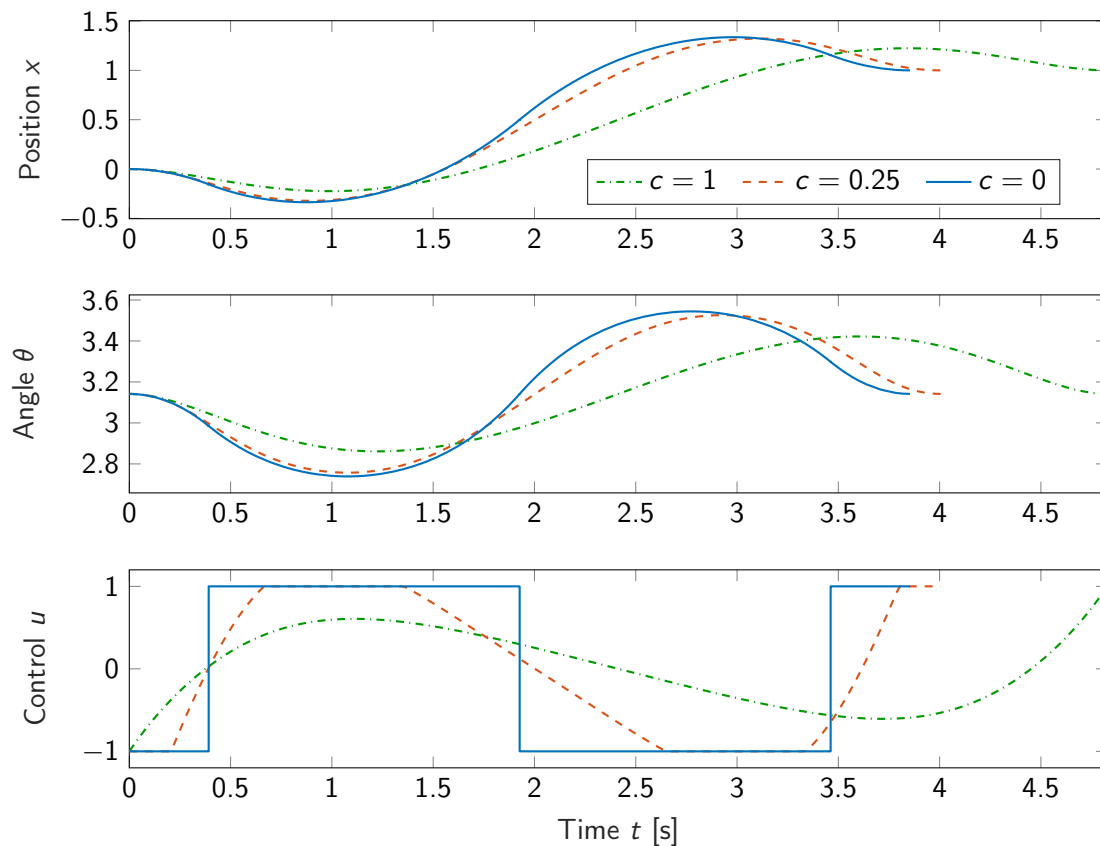


Figure 1.7: Optimal trajectories for the side-stepping of the pendulum (Example 1.3).

The cost functional (1.14a) and therefore the characteristics of the dynamic optimization problem depend on the parameter c . The choice $c = 0$ implies the minimization of the end time

$$J(u) = \int_0^{t_f} 1 \, dt = t_f, \quad (1.15)$$

which is also known as time optimality. For $c > 0$, the squared control u is weighted in the cost functional as a trade-off between time and energy optimality.

Figure 1.7 depicts the optimal trajectories for $c = 0$, $c = 0.25$, and $c = 1$. For $c = 0$, the input u shows a bang-bang behavior, whereas for $c > 0$ the control amplitudes decrease and the transition time t_f increases.

The inverted pendulum is a good example to illustrate that not every optimal control problem has a solution, especially when the end time t_f is not fixed. As shown in Figure 1.7, the end time t_f increases as the weighting of u^2 increases in comparison to the time-optimal part in the cost functional (1.14a). In particular, $c \rightarrow \infty$ corresponds to energy optimality, i.e.

$$J(u) = \int_0^{t_f} u^2 \, dt. \quad (1.16)$$

In this case, the optimal control problem has no finite solution, because the side-stepping maneuver of the pendulum would take infinitely long with $t_f \rightarrow \infty$.

Example 1.4 (Goddard problem [2, 1]) A classical aerospace optimal control problem is the maximization of the altitude of a rocket under the influence of drag and gravity. This problem was posed and solved by the American rocket pioneer Robert H. Goddard in 1919 and can be formulated as the normalized optimal control problem

$$\min_{u(\cdot)} -h(t_f) \quad (1.17)$$

$$\text{s.t.} \quad \dot{h} = v, \quad \dot{v} = \frac{u - D(h, v)}{m} - \frac{1}{h^2}, \quad \dot{m} = -\frac{u}{c}, \quad (1.18)$$

$$h(0) = 1, \quad v(0) = 0, \quad m(0) = 1, \quad m(t_f) = 0.6, \quad (1.19)$$

$$0 \leq u \leq 3.5. \quad (1.20)$$

The state variables are the altitude h , velocity v , and the mass m of the rocket. The drag is modeled with the function

$$D(h, v) = D_0 v^2 \exp[\beta(1 - h)], \quad D_0 > 0 \quad (1.21)$$

that depends on the states h und v . The boundary conditions in (1.19) consist of the (normalized) initial conditions and the terminal condition for $m(t_f)$ that corresponds to the net mass of the rocket after the fuel is completely burnt. The control input of the system is the thrust u subject to the constraint (1.20).

Figure 1.8 shows the optimal trajectories for the Goddard rocket compared to a flight with maximum thrust.¹ The optimal control trajectory $u^*(t)$ also starts with full thrust, but then breaks down and increases again with a parabolic shape until the fuel is consumed. This behavior is caused by the drag function $D(h, v)$. At the start of the flight, the velocity v is low and therefore the influence of drag is small which justifies to fly with full thrust. As the velocity increases, however, the drag becomes dominant and it is more economic to save fuel. As the altitude increases, the drag effect becomes smaller and the thrust increases again. This optimal strategy increases the final altitude by 2.7% compared to the full thrust flight, see Figure 1.8.

Example 1.5 (Economic model [4, 3]) The following example describes the behavior of a consumer who wants to maximize consumption, leisure time, and education over his lifetime. The level of education E and the capital C of an average consumer are described by the dynamics

$$\dot{E} = \overbrace{E^\epsilon u_2 u_3}^{\text{Education}} - \overbrace{\delta E}^{\text{Forgetting}}, \quad E(0) = E_0 \quad (1.22)$$

$$\dot{C} = \underbrace{iC}_{\text{Interest}} + \underbrace{E u_2 g(u_3)}_{\text{Income}} - \underbrace{u_1}_{\text{Consumption}}, \quad C(0) = C_0. \quad (1.23)$$

The control variables are the consumption u_1 , the ratio u_2 of working time w.r.t. the total time, and the ratio u_3 of education time w.r.t. the working time. The controls are subject to the constraints

$$u_1 > 0, \quad 0 \leq u_2 \leq 1, \quad 0 \leq u_3 < 1. \quad (1.24)$$

¹ The parameter values are $c = 0.5$, $D_0 = 310$, $\beta = 500$.

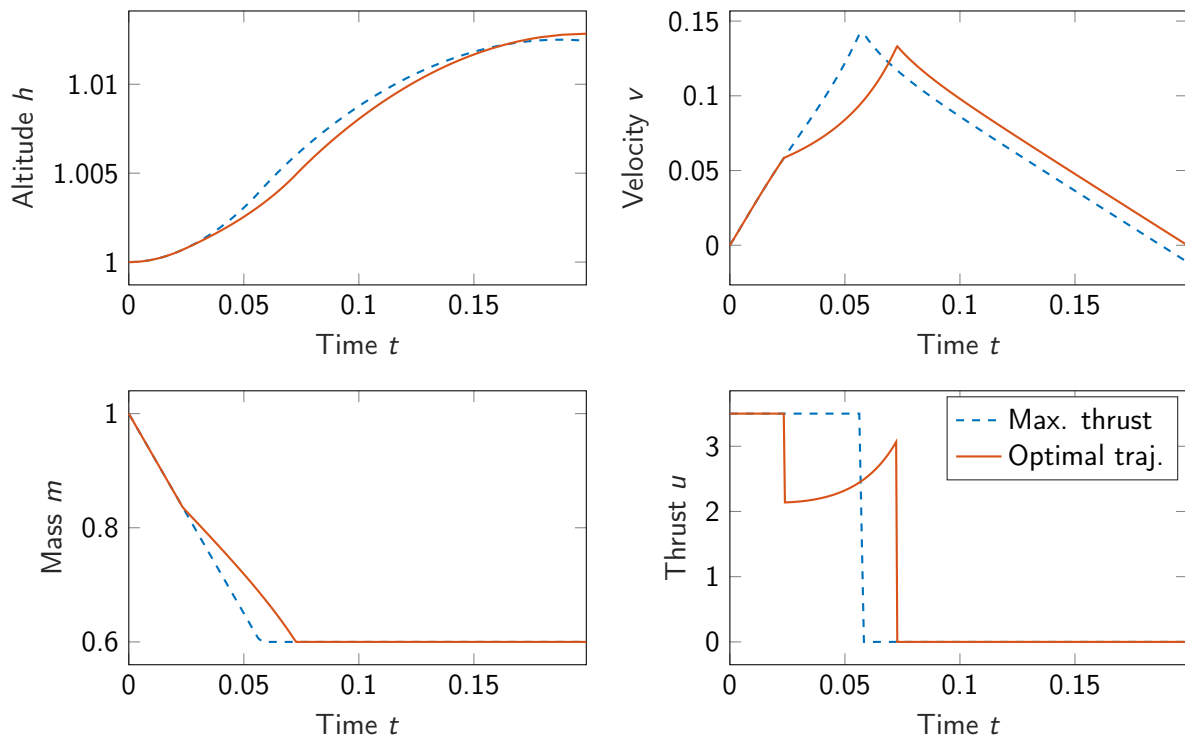


Figure 1.8: Optimal trajectories of the Goddard problem (Example 1.4).

As mentioned before, the consumer's optimization goal is to maximize consumption, leisure time, and education over the lifetime of $t_f = 75$ years, which is expressed in the following cost functional to be minimized

$$J(\mathbf{u}) = -C^\kappa(t_f) - \int_{t_0}^{t_f} U(t, u_1, u_2, E) e^{-\rho t} dt. \quad (1.25)$$

The integral cost function

$$U(t, u_1, u_2, E) = \alpha_0 u_1^\alpha + \beta_0 (1 - u_2)^\beta + \gamma_0 t E^\gamma \quad (1.26)$$

weights the consumption u_1 , the leisure time $(1 - u_2)$ and the level of education E . The term $-C^\kappa(t_f)$ in (1.25) additionally accounts for the capital of inheritance.

The optimal trajectories for the level of educational $E(t)$ and the capital $C(t)$ are shown in Figure 1.9.² The first 17 years are the education phase of the life (i.e. $u_3 = 1$), followed by a long working phase of 34 years with ongoing education. During the last working phase of 10 years (age 52 to 61), the working time is continuously reduced (control u_2). From the 62nd year onward, retirement sets in. The level of education E is particularly high at age 30 to 60. The capital C is negative during the first half of life, which corresponds to taking out a loan. In the course of life, however, this is compensated by the rising income.

² The following parameter values are used: $\alpha = -1$, $\alpha_0 = -1$, $\beta = -0.5$, $\beta_0 = -1$, $\gamma = 0.2$, $\gamma_0 = 5$, $\kappa = 0.2$, $\rho = 0.01$, $\varepsilon = 0.35$, $\delta = 0.01$, $i = 0.04$, $H_0 = 1$, $C_0 = 30$. The function $g(u_3)$ in (1.23) is defined by the parabola $g(u_3) = 1 - (1 - a)u_3 - au_3^2$.

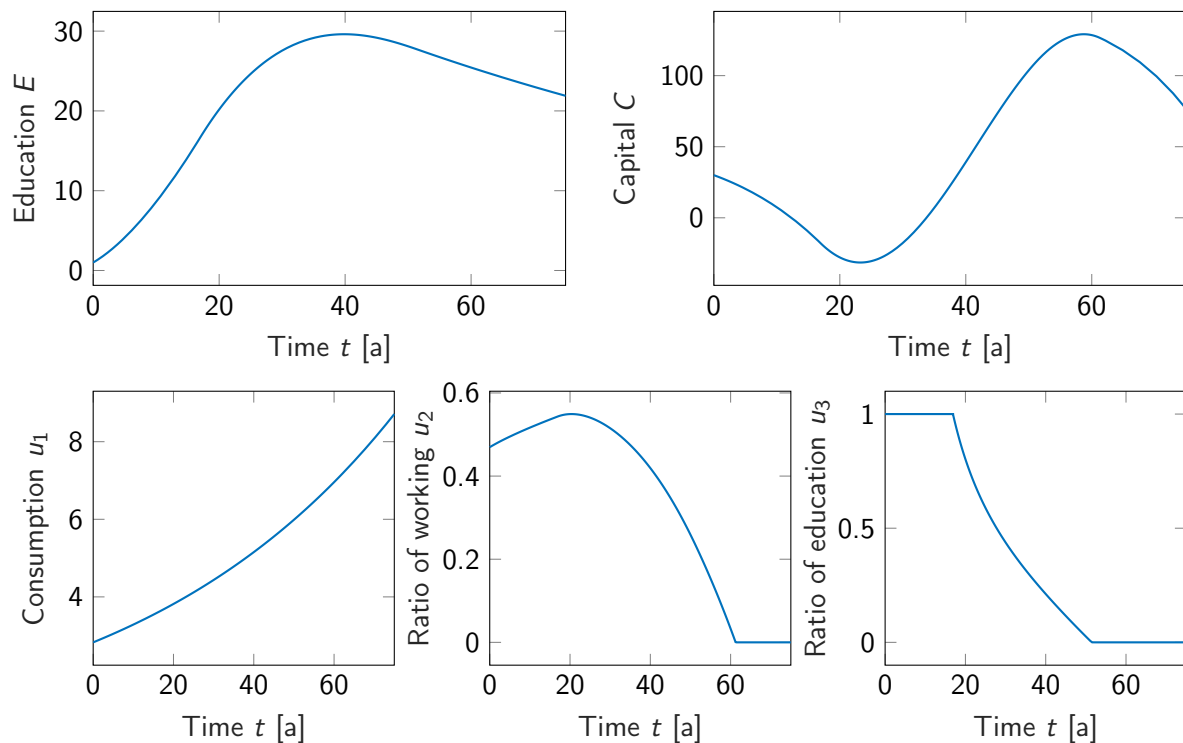


Figure 1.9: Optimal consumer behavior (Example 1.5).

1.3 References

- [1] A.E. Bryson. *Dynamic Optimization*. Menlo Park, CA: Addison-Wesley, 1999.
- [2] R.H. Goddard. "A method for reaching extreme altitudes". In: *Smithsonian Miscellaneous Collections* 71 (1919).
- [3] H.J. Oberle and R. Rosendahl. "Numerical computation of a singular-state subarc in an economic optimal control model". In: *Optimal Control Applications and Methods* 27 (2006), pp. 211–235.
- [4] K. Pohmer. *Mikroökonomische Theorie der personellen Einkommens- und Vermögensverteilung*. Studies in Contemporary Economics. Vol. 16. Springer, 1985.

2 Basics of optimization

This chapter introduces basic concepts and properties of parameter optimization problems

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (2.1)$$

$$\text{s.t. } g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \quad (2.2)$$

$$h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q \quad (2.3)$$

that are relevant for the following chapters.

2.1 Local/global minima and admissible set

Definition 2.1 (Local and global minima) *The function $f(\mathbf{x})$ with $f : \mathcal{X} \rightarrow \mathbb{R}$ and $\mathcal{X} \subseteq \mathbb{R}^n$ has*

- a) a local minimum at $\mathbf{x}^* \in \mathcal{X}$, if $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x}$ in a neighborhood of $\mathbf{x}^*$,
- b) a strict local minimum, if $f(\mathbf{x}^*) < f(\mathbf{x})$ for all $\mathbf{x}$ in a neighborhood of $\mathbf{x}^*$,
- c) a global minimum, if $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{X}$,
- d) a unique global minimum, if $f(\mathbf{x}^*) < f(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{X}$.

Definition (2.1) implies that a global minimum is always a local minimum. The different types of minima are illustrated in Figure 2.1.

Definition 2.2 (Admissible set) *The admissible set $\mathcal{X}_{ad}$ of the optimization problem (2.1)-(2.3) is defined as*

$$\mathcal{X}_{ad} = \left\{ \mathbf{x} \in \mathbb{R}^n : \begin{array}{ll} g_i(\mathbf{x}) = 0, & i = 1, \dots, p \\ h_i(\mathbf{x}) \leq 0, & i = 1, \dots, q \end{array} \right\}. \quad (2.4)$$

All *admissible points* $\mathbf{x} \in \mathcal{X}_{ad}$ satisfy the constraints $g_i(\mathbf{x}) = 0$ and $h_i(\mathbf{x}) \leq 0$. This shows that the problem (2.1)-(2.3) can also be formulated as

$$\min_{\mathbf{x} \in \mathcal{X}_{ad}} f(\mathbf{x}). \quad (2.5)$$

For unconstrained problems, we have $\mathcal{X}_{ad} = \mathbb{R}^n$.

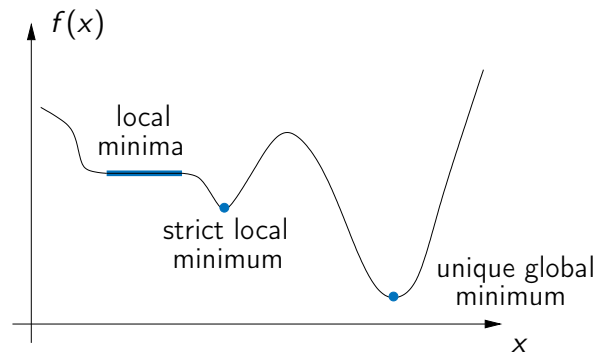


Figure 2.1: Different minima of a function $f(x)$ with $x \in \mathbb{R}$.

Example 2.1 The admissible set for the problem in Example 1.2 with the two inequality constraints (1.11) and (1.12) is

$$\mathcal{X}_{ad} = \{ \mathbf{x} \in \mathbb{R}^n \mid h_1(\mathbf{x}) = x_1 + x_2 - 2 \leq 0, \quad h_2(\mathbf{x}) = x_1^2 - x_2 \leq 0 \}.$$

This corresponds to the region enclosed by the line $h_1(\mathbf{x}) = 0$ and the parabola $h_2(\mathbf{x}) = 0$, see Figure 1.5.

It is obvious that the admissible set $\mathcal{X}_{ad}$ must be non-empty, because otherwise the optimization problem (2.5) has no solution. Another necessary condition for $\mathcal{X}_{ad}$ can be derived from the equality constraint (2.2), since the algebraic restrictions $g_i(\mathbf{x}) = 0$ reduce the number of free optimization variables $\mathbf{x} \in \mathbb{R}^n$. This implies that the number p of (independent) equality constraints (2.2) must not be larger than the number of optimization variables $\mathbf{x} \in \mathbb{R}^n$ as a necessary condition for the admissible set $\mathcal{X}_{ad}$ to be non-empty.

Theorem 2.1 (Weierstrass) Let $\mathcal{X}_{ad}$ be a compact non-empty set and $f : \mathcal{X}_{ad} \rightarrow \mathbb{R}$ a continuous function on $\mathcal{X}_{ad}$. Then, the problem

$$\min_{\mathbf{x} \in \mathcal{X}_{ad}} f(\mathbf{x}) \tag{2.6}$$

attains its minimum at least at one point $\mathbf{x}^* \in \mathcal{X}_{ad}$.

Example 2.2 Consider the constrained problem of Example 1.2 (also see Figure 1.5)

$$\min_{\mathbf{x} \in \mathbb{R}^2} (x_1 - 2)^2 + (x_2 - 1)^2 \tag{2.7}$$

$$\text{s.t. } x_1 + x_2 - 2 \leq 0 \tag{2.8}$$

$$x_1^2 - x_2 \leq 0. \tag{2.9}$$

The cost function is continuous and the admissible set $\mathcal{X}_{ad}$ is non-empty and compact. Then, Theorem 2.1 implies that there exists at least one minimizing solution $\mathbf{x}^*$.

Note that Weierstrass' theorem does not hold for problems that are defined on an *open set* (e.g. $\mathcal{X}_{ad} = \mathbb{R}^n$). An example for this is

$$\min_{x \in (0, \infty)} \frac{1}{x}. \tag{2.10}$$

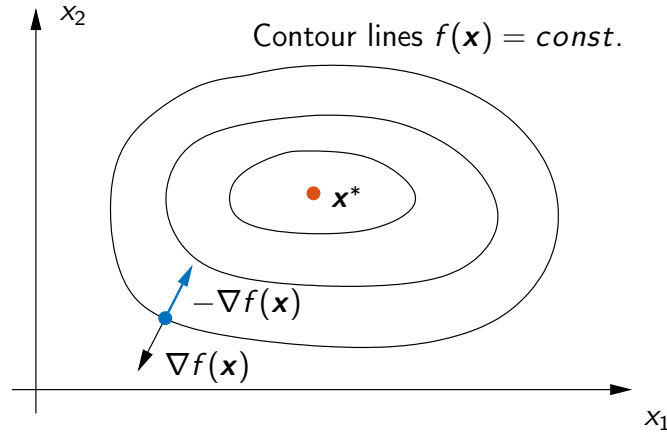


Figure 2.2: Direction of steepest descent $-\nabla f(\mathbf{x})$ for a function $f(\mathbf{x})$ with $\mathbf{x} \in \mathbb{R}^2$.

Although $f(x) = \frac{1}{x}$ is continuous on the open interval $\mathcal{X}_{ad} = (0, \infty)$, there exists no minimum. However, the existence of a solution to an unconstrained optimization problem can be guaranteed if the function $f(\mathbf{x})$ is *radially unbounded*, i.e. $\lim_{\|\mathbf{x}\| \rightarrow \infty} f(\mathbf{x}) = \infty$ for all $\mathbf{x} \in \mathbb{R}^n$.

2.2 Gradient and Hessian matrix

The calculation of first and second order derivatives of the cost function and the constraints is of fundamental importance in optimization. The case of discontinuous functions or discontinuous derivatives is somewhat delicate as the “classical” optimality conditions that are derived in the following chapters do not hold for these problems and further theoretical and numerical problems arise. It is therefore often assumed that all functions of an optimization problem are continuous and sufficiently often differentiable.

Definition 2.3 (Gradient) Suppose that $f : \mathcal{X} \rightarrow \mathbb{R}$ is continuously differentiable. Then, the gradient (i.e. the first partial derivative) of $f(\mathbf{x})$ at the point $\mathbf{x} = [x_1, \dots, x_n]^T$ is defined by

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial \mathbf{x}} \right)^T = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}. \quad (2.11)$$

The negative gradient has a special meaning for the solution of an optimization problem as it represents the direction of steepest descent of a cost function $f(\mathbf{x})$ at the point $\mathbf{x}$, see Figure 2.2. This property will be examined in more detail in the next chapter.

Definition 2.4 (Hessian matrix) Let $f : \mathcal{X} \rightarrow \mathbb{R}$ be twice continuously differentiable. Then,

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 x_n} \\ \vdots & & \vdots \\ \frac{\partial^2 f}{\partial x_n x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \quad (2.12)$$

is the Hessian matrix or simply Hessian (i.e. the second partial derivative) of $f(\mathbf{x})$ at the point $\mathbf{x} = [x_1, \dots, x_n]^T$.

In case of scalar dependency, the ∇ -notation is usually replaced by $f'(x)$ and $f''(x)$. According to the theorem of Schwarz, the continuity of the second partial derivative implies commutativity, i.e.

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i}.$$

Hence, the Hessian matrix is symmetric, i.e. $\nabla^2 f(\mathbf{x}) = \nabla^2 f(\mathbf{x})^\top$, and always has real eigenvalues. An important property of the Hessian in the context of optimization is its definiteness that can be investigated as follows.

Theorem 2.2 (Definiteness) *The definiteness of a symmetric $(n \times n)$ -matrix $\mathbf{A}$ is characterized by the following conditions:*

Matrix $\mathbf{A}$ is	(1): for all $\mathbf{p} \in \mathbb{R}^n$ with $\mathbf{p} \neq \mathbf{0}$	(2): all eigen- values λ_i are	(3): all principle mi- nors D_i satisfy
positive semidefinite ($\mathbf{A} \geq 0$)	$\mathbf{p}^\top \mathbf{A} \mathbf{p} \geq 0$	real and ≥ 0	
positive definite ($\mathbf{A} > 0$)	$\mathbf{p}^\top \mathbf{A} \mathbf{p} > 0$	real and > 0	$D_i > 0$
negative semidefinite ($\mathbf{A} \leq 0$)	$\mathbf{p}^\top \mathbf{A} \mathbf{p} \leq 0$	real and ≤ 0	
negative definite ($\mathbf{A} < 0$)	$\mathbf{p}^\top \mathbf{A} \mathbf{p} < 0$	real and < 0	$(-1)^{i+1} D_i < 0$

The eigenvalues λ_i , $i = 1, \dots, n$ of the matrix $\mathbf{A}$ are the roots of the characteristic equation

$$\det(\lambda \mathbf{I} - \mathbf{A}) = 0$$

with the $(n \times n)$ -unit matrix $\mathbf{I}$. Criterion (3) is also known as *Sylvester's criterion* for definite matrices. The principle minors D_i are the determinants of the upper left submatrices of $\mathbf{A} = [a_{ij}]$:

$$D_1 = a_{11}, \quad D_2 = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}, \quad \dots \quad D_n = \begin{vmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \dots & a_{nn} \end{vmatrix}.$$

Each of the three conditions (1)-(3) is necessary and sufficient. It is therefore only required to check one of them to determine the definiteness of the symmetric matrix $\mathbf{A}$, also see Exercise 2.1.

2.3 Convexity

The concept of convexity is of fundamental importance in optimization and in general leads to a simpler numerical solution of the optimization problem. Moreover, convexity is also related to the uniqueness of a solution. Convexity can be defined for sets and functions as well as for optimization problems.

2.3.1 Convex sets

Definition 2.5 (Convex set) *A set $\mathcal{X} \subseteq \mathbb{R}^n$ is convex, if the following condition is satisfied for all points $\mathbf{x}, \mathbf{y} \in \mathcal{X}$*

$$\mathbf{z} = k \mathbf{x} + (1 - k) \mathbf{y} \in \mathcal{X} \quad \text{for all } k \in [0, 1]. \quad (2.13)$$

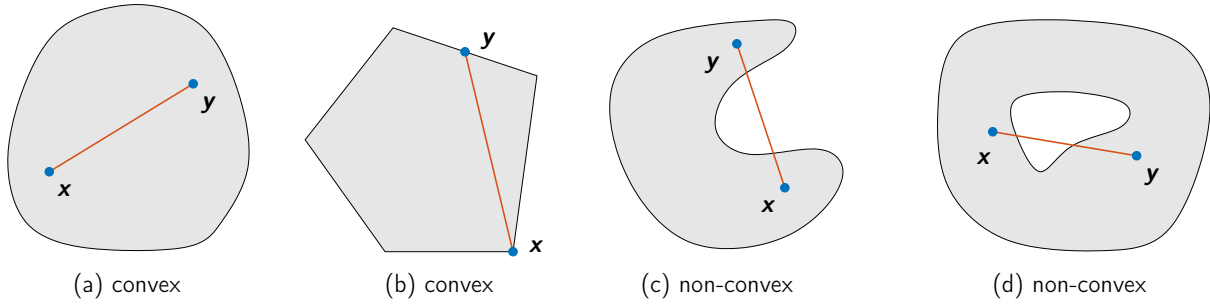


Figure 2.3: Examples of convex and non-convex sets.

A geometric interpretation of this definition is that a set $\mathcal{X} \subseteq \mathbb{R}^n$ is convex if and only if the connecting line between any two points $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ is completely contained in $\mathcal{X}$. Figure 2.3 shows some examples of convex and non-convex sets.

A well known property of convex sets is that their intersection is also a convex set. This property is convenient for characterizing the convexity of the admissible set $\mathcal{X}_{ad}$ of optimization problems (cf. Theorem 2.4).

2.3.2 Convex functions

Definition 2.6 (Convex and concave function) Let $\mathcal{X} \subseteq \mathbb{R}^n$ be a convex set. A function $f : \mathcal{X} \rightarrow \mathbb{R}$ is convex on $\mathcal{X}$, if the following condition holds for all points $\mathbf{x}, \mathbf{y} \in \mathcal{X}$, $\mathbf{x} \neq \mathbf{y}$ and $\mathbf{z} = k\mathbf{x} + (1-k)\mathbf{y}$

$$f(\mathbf{z}) \leq k f(\mathbf{x}) + (1-k) f(\mathbf{y}) \quad \forall k \in [0, 1]. \quad (2.14)$$

The function f is strictly convex, if

$$f(\mathbf{z}) < k f(\mathbf{x}) + (1-k) f(\mathbf{y}) \quad \forall k \in (0, 1). \quad (2.15)$$

The function f is (strictly) concave, if $-f$ is (strictly) convex.

If the function f is convex (concave), then all function values $f(\mathbf{z})$ with $\mathbf{z} = k\mathbf{x} + (1-k)\mathbf{y}$ lie below (above) the line connecting $f(\mathbf{x})$ and $f(\mathbf{y})$ for any $\mathbf{x}, \mathbf{y} \in \mathcal{X}$. Figure 2.4 shows some examples of convex and concave functions. Note that linear functions are both concave and convex.

Convex functions have some interesting properties:

- a) The weighted sum of convex functions

$$f(\mathbf{x}) = \sum_{i=1}^k a_i f_i(\mathbf{x}) \quad (2.16)$$

with the coefficients $a_i \geq 0, i = 1, \dots, k$ is convex.

- b) If the function $f(\mathbf{x})$ is convex, then the level set

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^n : f(\mathbf{x}) \leq k\} \quad (2.17)$$

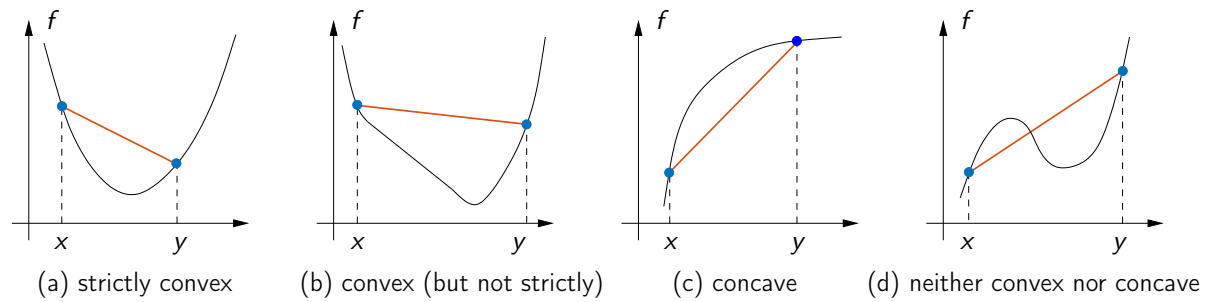


Figure 2.4: Examples of convex and concave functions.

with $k > 0$ is convex, see Figure 2.5.

c) A continuously differentiable function $f(\mathbf{x})$ is convex on $\mathcal{X}$ if the inequality

$$f(\mathbf{y}) \geq f(\mathbf{x}) + (\mathbf{y} - \mathbf{x})^\top \nabla f(\mathbf{x}) \quad (2.18)$$

holds for all $\mathbf{x}, \mathbf{y} \in \mathcal{X}$. This can be seen as an alternative to Definition 2.6 and is illustrated in Figure 2.6. The geometric interpretation of this inequality is that at every point $\mathbf{x}$ of a convex function $f(\mathbf{x})$ there exists a *supporting hyperplane* (*supporting tangent* in the scalar case) as a lower bound on $f(\mathbf{x})$.

Example 2.3 Figure 2.7 shows the non-convex function

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = -\frac{100}{(x_1 - 1)^2 + (x_2 - 1)^2 + 1} - \frac{200}{(x_1 + 1)^2 + (x_2 + 2)^2 + 1}. \quad (2.19)$$

The non-convexity of the function can also be seen from the contour lines in Figure 2.7 that define the non-convex level sets (2.17).

The following theorem characterizes the convexity of a function in terms of the definiteness of its Hessian matrix (also see Theorem 2.2).

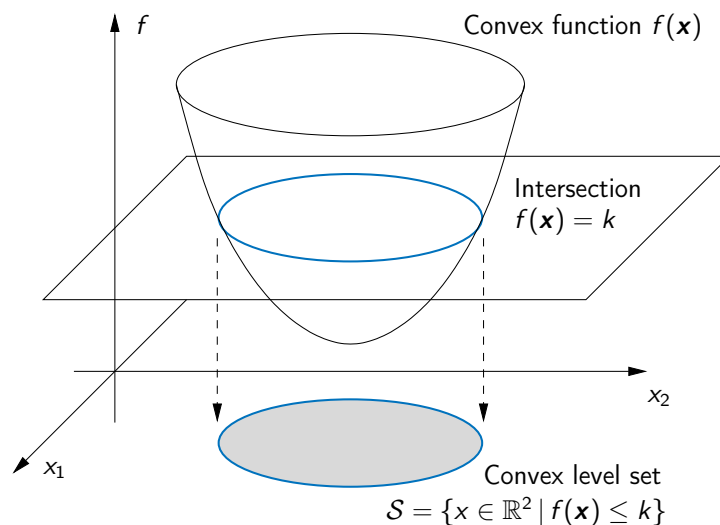


Figure 2.5: Convex level set $\mathcal{S}$ as the intersection of a convex function $f(\mathbf{x})$ with the plane $f(\mathbf{x}) = \text{const}$.

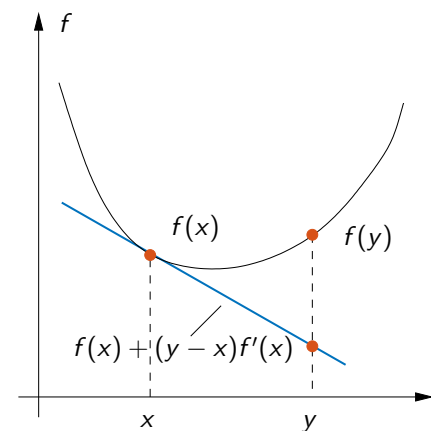


Figure 2.6: Supporting tangent of a convex function $f(\mathbf{x})$.

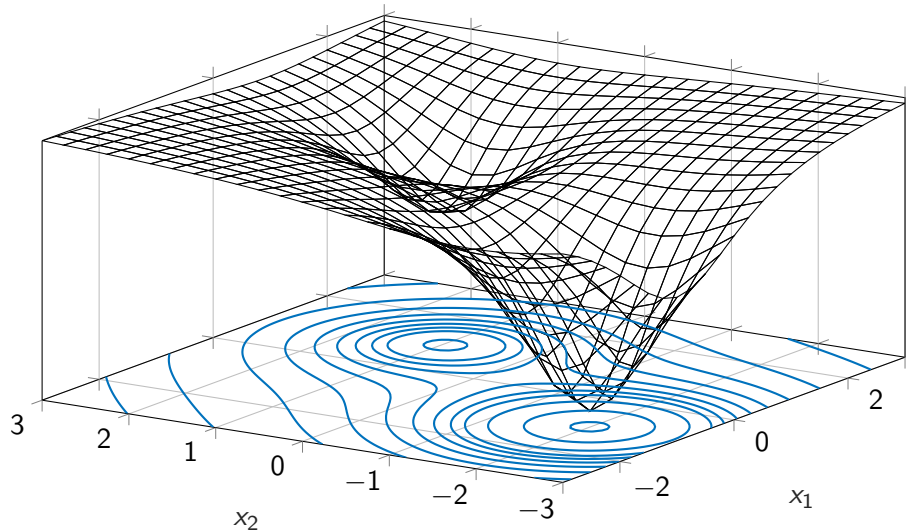


Figure 2.7: Shape and contour lines of the function (2.19).

Theorem 2.3 Suppose that $\mathcal{X} \subseteq \mathbb{R}^n$ is a convex set and $f : \mathcal{X} \rightarrow \mathbb{R}$ is twice continuously differentiable. Then, the following statements are true:

- a) f is convex, if $\nabla^2 f(\mathbf{x})$ is positive semidefinite for all $\mathbf{x} \in \mathcal{X}$,
- b) f is strictly convex, if $\nabla^2 f(\mathbf{x})$ is positive definite for all $\mathbf{x} \in \mathcal{X}$,
- c) f is concave, if $\nabla^2 f(\mathbf{x})$ is negative semidefinite for all $\mathbf{x} \in \mathcal{X}$,
- d) f is strictly concave, if $\nabla^2 f(\mathbf{x})$ is negative definite for all $\mathbf{x} \in \mathcal{X}$.

Note, however, that strict convexity of a function $f(\mathbf{x})$ does not necessarily imply the positive definiteness of the Hessian matrix. An example is the strictly convex function $f(x) = x^4$ with the second derivative $f''(0) = 0$ at point $x = 0$.

Exercise 2.1 Determine the convexity of the following functions by checking the definiteness of the Hessian $\nabla^2 f(\mathbf{x})$ using the criteria (1)-(3) in Theorem 2.2:

$$f(\mathbf{x}) = x_1^4 + x_1^2 - 2x_1x_2 + x_2^2, \quad \mathbf{x} \in \mathbb{R}^2 \quad (2.20)$$

$$f(\mathbf{x}) = x_2 + \frac{1}{x_1x_2}, \quad x_{1,2} > 0. \quad (2.21)$$

Exercise 2.2 Investigate if the following functions are convex or concave on the given domain and plot the functions (e.g. with MATLAB):

$$f(x) = e^x - 1, \quad x \in \mathbb{R} \quad (2.22)$$

$$f(\mathbf{x}) = x_1^\alpha x_2^{1-\alpha}, \quad x_1, x_2 > 0, \quad \alpha \in [0, 1]. \quad (2.23)$$

2.3.3 Convex optimization problems

The concept of convexity of sets and functions can be extended to optimization problems.

Definition 2.7 *The parameter optimization problem*

$$\min_{\mathbf{x} \in \mathcal{X}_{ad}} f(\mathbf{x})$$

is (strictly) convex if the admissible set $\mathcal{X}_{ad}$ is convex and the cost function f is (strictly) convex on $\mathcal{X}_{ad}$.

A sufficient condition for the convexity of the admissible set $\mathcal{X}_{ad}$ is given in the next theorem.

Theorem 2.4 *Suppose that the functions $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \dots, p$ are linear and that the functions $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \dots, q$ are convex. Then, the admissible set $\mathcal{X}_{ad}$ in (2.4) is convex.*

Linear and thus convex equality constraints $g_i(\mathbf{x}) = 0$ and convex inequality constraints $h_i(\mathbf{x}) \leq 0$ always have convex subsets, also see point b) on Page 15 and Figure 2.5. The theorem then directly follows from the already mentioned property that the intersection of convex sets is again a convex set. We can now state a sufficient condition for the convexity of optimization problems.

Theorem 2.5 *The optimization problem*

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \\ & h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q \end{aligned}$$

is (strictly) convex, if

- a) the functions $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \dots, p$ are linear,
- b) the functions $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \dots, q$ are convex,
- c) the cost function f is (strictly) convex on $\mathcal{X}_{ad} \subseteq \mathbb{R}^n$.

Some important characteristics of convex optimization problems are:

- a) Every local minimum is a global minimum.
- b) If a strict local minimum exists, then it is unique and global.
- c) A strictly convex optimization problem cannot possess more than one solution. In other words, if a local minimum exists, then it is a unique global minimum.

It is worth noting that the strict convexity of an optimization problem does not automatically imply the existence of a solution. To illustrate this point, we again consider the problem

$$\min_{x \in (0, \infty)} \frac{1}{x}. \quad (2.24)$$

The first and second derivatives of the function $f(x) = \frac{1}{x}$ are $f'(x) = -\frac{1}{x^2}$ and $f''(x) = \frac{2}{x^3}$. Obviously, we have $f''(x) > 0$ for all $x \in \mathcal{X} = (0, \infty)$, which shows that the optimization problem (2.24) is strictly convex. However, as discussed before, the problem has no solution.¹

The above listed properties a)-c) of convex optimization problems show that convexity simplifies the analysis of optimization problems considerably. In fact, the majority of theoretical results for optimization problems and convergence properties of numerical algorithms are based on this assumption. Since most numerical methods converge to a local minimum, this automatically implies a (unique) global solution in case of (strict) convexity.

Example 2.4 We again consider Example 2.2. The cost function (2.7) and the constraints (2.8), (2.9) are convex. Thus, the optimization problem (2.7)-(2.9) is convex and therefore the local solution $\mathbf{x}^* = [1, 1]^T$ in Figure 1.5 is also the global solution.

2.4 References

- [1] S. Boyd and L. Vandenberghe. *Convex Optimization*. http://www.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf. Cambridge: Cambridge University Press, 2004.
- [2] J. Nocedal and S.J. Wright. *Numerical Optimization*. New York: Springer, 2006.
- [3] M. Papageorgiou, M. Leibold, and M. Buss. *Optimierung*. Berlin: Springer, 2012.

¹ Remember that Weierstrass' Theorem 2.1 only guarantees the existence of a solution if the admissible set $\mathcal{X}$ (or $\mathcal{X}_{ad}$, respectively) is compact.

3 Unconstrained optimization

This chapter addresses unconstrained parameter optimization problems

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (3.1)$$

and derives necessary and sufficient conditions for an optimal solution. Afterwards, the most common optimization algorithms are presented for solving (3.1) numerically.

3.1 Necessary optimality conditions

Optimality conditions are useful to find a (local) extremum point $\mathbf{x}^*$ and to verify if it is indeed a minimum for the unconstrained problem (3.1), i.e.

$$f(\mathbf{x}) \geq f(\mathbf{x}^*) \quad (3.2)$$

for all $\mathbf{x}$ in a neighborhood of $\mathbf{x}^*$ (see Definition 2.1). Under the assumption that $f(\mathbf{x})$ is at least twice continuously differentiable, the function $f(\mathbf{x})$ can be expanded in a Taylor series at the point $\mathbf{x}^*$

$$f(\mathbf{x}^* + \delta\mathbf{x}) = f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top \delta\mathbf{x} + \frac{1}{2} \delta\mathbf{x}^\top \nabla^2 f(\mathbf{x}^*) \delta\mathbf{x} + \mathcal{O}(\|\delta\mathbf{x}\|^3). \quad (3.3)$$

If the point $\mathbf{x}^*$ is a local minimum, then (3.2) with $\mathbf{x} = \mathbf{x}^* + \delta\mathbf{x}$ must be satisfied for all sufficiently small $\delta\mathbf{x}$. Inserting (3.3) into (3.2) leads to the inequality

$$\nabla f(\mathbf{x}^*)^\top \delta\mathbf{x} + \frac{1}{2} \delta\mathbf{x}^\top \nabla^2 f(\mathbf{x}^*) \delta\mathbf{x} + \mathcal{O}(\|\delta\mathbf{x}\|^3) \geq 0. \quad (3.4)$$

The first term $\nabla f(\mathbf{x}^*)^\top \delta\mathbf{x}$ is dominant for sufficiently small $\delta\mathbf{x}$ and therefore determines the sign of the left side in (3.4). In other words, $\nabla f(\mathbf{x}^*)^\top \delta\mathbf{x} \geq 0$ must hold if $\mathbf{x}^*$ is a minimum point. Due to the fact that $\mathbf{x}^*$ is only a local minimum if the inequality (3.4) is satisfied for all (sufficiently small) $\delta\mathbf{x}$, we have

$$\nabla f(\mathbf{x}^*) = \mathbf{0}. \quad (3.5)$$

This is a first-order optimality condition that holds for all extrema, because $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is also satisfied by maxima and saddle points, see Figure 3.1. A point $\mathbf{x}^*$ satisfying (3.5) is therefore often called *stationary point* and (3.5) is the *stationarity condition*.

A necessary second-order condition for a local minimum $\mathbf{x}^*$ follows from (3.4)

$$\delta\mathbf{x}^\top \nabla^2 f(\mathbf{x}^*) \delta\mathbf{x} \geq 0. \quad (3.6)$$

This condition must be satisfied for all sufficiently small directions $\delta\mathbf{x}$ in a neighborhood of $\mathbf{x}^*$, which implies that the Hessian $\nabla^2 f(\mathbf{x}^*)$ must be positive semidefinite as a consequence of Theorem 2.2. We summarize the necessary optimality conditions for a local minimum in the following lines.

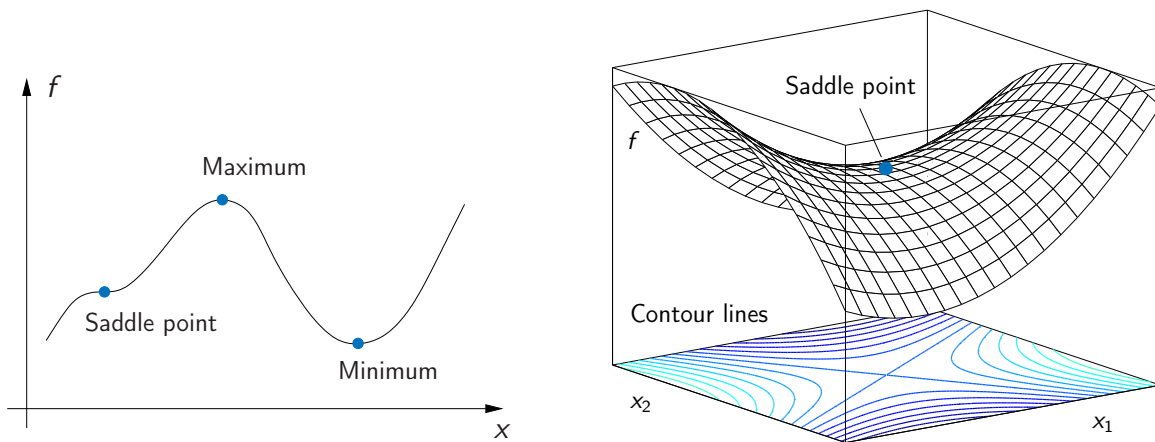


Figure 3.1: Examples of stationary points in $\mathbb{R}^1$ and $\mathbb{R}^2$.

Theorem 3.1 (Necessary optimality conditions) Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice continuously differentiable and that $\mathbf{x}^*$ is a local minimum point of f . Then, the following first- and second-order optimality conditions hold

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \quad (3.7)$$

$$\nabla^2 f(\mathbf{x}^*) \geq \mathbf{0} \quad (\text{positive semidefinite}). \quad (3.8)$$

Note that the optimality conditions are only necessary but not sufficient. An example is the function $f(x) = x^3$ that has an extremum at $x^* = 0$, which is easily verified by the first derivative $f'(x) = 3x^2$. Although the second derivative $f''(x^*) = 6x^* = 0$ is positive semidefinite, $x^* = 0$ is not a minimum point but a saddle point, see Figure 3.1.

3.2 Sufficient optimality conditions

A sufficient condition that guarantees the existence of a local minimum $\mathbf{x}^*$ can be extracted from the inequality (3.4). If $\nabla f(\mathbf{x}^*) = \mathbf{0}$ holds and the quadratic term in (3.4) is strictly positive, i.e.

$$\delta \mathbf{x}^T \nabla^2 f(\mathbf{x}^*) \delta \mathbf{x} > 0, \quad (3.9)$$

then the inequality (3.4) is strictly satisfied for all sufficiently small $\delta \mathbf{x}$ and therefore $\mathbf{x}^*$ is a strict local minimum.

Theorem 3.2 (Sufficient optimality conditions) Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a twice continuously differentiable function. If the following conditions are satisfied at some point $\mathbf{x}^*$

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \quad (3.10)$$

$$\nabla^2 f(\mathbf{x}^*) > \mathbf{0} \quad (\text{positive definite}), \quad (3.11)$$

then f has a strict local minimum at $\mathbf{x}^*$.

Note that a strict minimum point $\mathbf{x}^*$ does not automatically imply the positive definiteness of the Hessian $\nabla^2 f(\mathbf{x}^*)$. An example is the function $f(x) = x^4$, which obviously has a strict minimum at the point $x^* = 0$. However, the second derivative $f''(x) = 12x^2$ vanishes at the position $x^* = 0$ and is thus not positive definite.

The stationarity condition $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is a nonlinear system of equations of order n that can be solved for a stationary point $\mathbf{x}^* \in \mathbb{R}^n$. Afterwards, the positive definiteness of the Hessian matrix $\nabla^2 f(\mathbf{x}^*)$ can be analyzed using one of the criteria in Theorem 2.2 to investigate whether $f(\mathbf{x}^*)$ is a strict minimum.

Example 3.1 Consider the optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^2} x_1^2 + ax_2^2 - x_1x_2 \quad (3.12)$$

with the parameter $a \in \mathbb{R}$. To characterize the stationary points $\mathbf{x}^*$, we compute the gradient and the Hessian

$$\nabla f(\mathbf{x}) = \begin{bmatrix} 2x_1 - x_2 \\ 2ax_2 - x_1 \end{bmatrix}, \quad \nabla^2 f(\mathbf{x}) = \begin{bmatrix} 2 & -1 \\ -1 & 2a \end{bmatrix}. \quad (3.13)$$

From $\nabla f(\mathbf{x}^*) = \mathbf{0}$ we get $\mathbf{x}^* = [0, 0]^T$ as the unique stationary point for $a \neq \frac{1}{4}$. The definiteness of the Hessian $\nabla^2 f(\mathbf{x}^*)$ is checked via the principle minors, see Criterion (3) in Theorem 2.2,

$$D_1 = 2, \quad D_2 = 4a - 1. \quad (3.14)$$

This shows that $\nabla^2 f(\mathbf{x}^*)$ is positive definite for $a > \frac{1}{4}$, in which case $\mathbf{x}^* = [0, 0]^T$ is a strict minimum point. For $a < \frac{1}{4}$, we get $D_1 > 0$ and $D_2 < 0$ for which Sylvester's criterion does not apply. In fact, $\nabla^2 f(\mathbf{x})$ is indefinite for $a = -1$ and $\mathbf{x}^* = [0, 0]^T$ is a saddle point as shown in Figure 3.1b).

If the cost function $f(\mathbf{x})$ is convex or even strictly convex, we can obtain stronger results than in Theorem 3.1 and 3.2:

Theorem 3.3 (Sufficient optimality conditions for convex functions) Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a continuously differentiable and (strictly) convex function. If $\nabla f(\mathbf{x}^*) = \mathbf{0}$, then the function f has a (unique) global minimum at $\mathbf{x}^*$.

3.3 Line search methods

The stationarity condition $\nabla f(\mathbf{x}^*) = \mathbf{0}$ cannot be solved analytically in most cases and numerical methods must be used instead to determine a stationary point $\mathbf{x}^*$ in an iterative manner. A widely used class of numerical algorithms are *line search methods*. Starting from a suboptimal initial solution $\mathbf{x}^0$ with $f(\mathbf{x}^0) > f(\mathbf{x}^*)$, line search methods approach the minimum point $\mathbf{x}^*$ iteratively by enforcing

$$f(\mathbf{x}^{k+1}) < f(\mathbf{x}^k) \quad (3.15)$$

in each optimization step k , such that the minimum point $\mathbf{x}^*$ is reached in the limit

$$\lim_{k \rightarrow \infty} \mathbf{x}^k = \mathbf{x}^*. \quad (3.16)$$

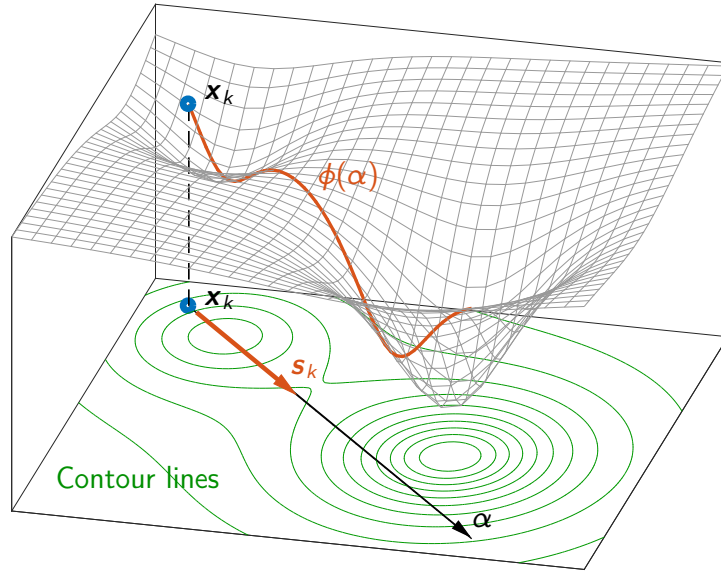


Figure 3.2: Illustration of line search problem (3.20).

All line search methods determine a *search direction* (or *descent direction*) $\mathbf{s}^k$, in which the inequality (3.15) is satisfied. To make this more precise, we consider the Taylor expansion of the cost function $f(\mathbf{x})$ at the point $\mathbf{x}^k$

$$f(\mathbf{x}^k + \mathbf{s}) = f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^T \mathbf{s} + \mathcal{O}(\|\mathbf{s}\|^2). \quad (3.17)$$

The higher-order terms $\mathcal{O}(\|\mathbf{s}\|^2)$ can be neglected for sufficiently small $\mathbf{s}$ and we see that a suitable search direction $\mathbf{s}^k$ at the current iterate $\mathbf{x}^k$ must satisfy the *descent condition*

$$\nabla f(\mathbf{x}^k)^T \mathbf{s}^k < 0 \quad (3.18)$$

in order for $f(\mathbf{x}^k + \mathbf{s}^k) < f(\mathbf{x}^k)$ to hold. There exist different methods for computing the search direction $\mathbf{s}^k$. The most common ones are introduced in Section 3.3.2 to 3.3.5.

After the search direction $\mathbf{s}^k$ is computed, a suitable step size α^k must be chosen to determine how far to go in direction of $\mathbf{s}^k$, i.e.

$$\boxed{\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{s}^k.} \quad (3.19)$$

The optimal step size α^k is the solution of the (scalar) optimization problem

$$\boxed{\alpha^k = \arg \min_{\alpha > 0} \phi(\alpha) \quad \text{with} \quad \phi(\alpha) = f(\mathbf{x}^k + \alpha \mathbf{s}^k),} \quad (3.20)$$

which corresponds to the maximum possible descent in the search direction $\mathbf{s}^k$. Figure 3.2 illustrates the idea behind the line search for a (non-convex) cost function $f(\mathbf{x})$ with $\mathbf{x} \in \mathbb{R}^2$ and a given descent direction $\mathbf{s}^k$.

The line search problem (3.20) is usually not solved exactly but in an approximate manner, see Section 3.3.1. The computed value of α^k then defines the step size in the k -th iteration and determines the next point (3.19). The basic line search algorithm is summarized in Table 3.1.

Initialization: $\mathbf{x}^0$ (Initial guess)
 $k \leftarrow 0$ (Iteration index)
 $\varepsilon_x, \varepsilon_f$ (Convergence tolerance)

repeat

Compute search direction $\mathbf{s}^k$
 Compute step size α^k
 $\mathbf{x}^{k+1} \leftarrow \mathbf{x}^k + \alpha^k \mathbf{s}^k$
 $k \leftarrow k + 1$

until $\|\mathbf{x}^k - \mathbf{x}^{k-1}\| \leq \varepsilon_x$ **or** $\|f(\mathbf{x}^k) - f(\mathbf{x}^{k-1})\| \leq \varepsilon_f$

Table 3.1: General structure of line search methods.

3.3.1 Step size computation

The calculation of a suitable step size α^k represents a compromise between computation time and accuracy. The optimal solution would be to solve the scalar optimization problem (3.20) exactly or at least sufficiently accurate. In practice, however, this would require too many evaluations of the cost function f and possibly of the gradient ∇f . Most numerical methods therefore use a suboptimal line search to determine a step size α^k that leads to an adequate reduction of the cost function f in the next step $\mathbf{x}^{k+1}$.

Decrease and curvature conditions (Wolfe conditions)

A first criterion for a sufficient decrease of the cost function f is the inequality

$$\underbrace{f(\mathbf{x}^k + \alpha^k \mathbf{s}^k)}_{= \phi(\alpha^k)} \leq \underbrace{f(\mathbf{x}^k)}_{= \phi(0)} + c_1 \alpha^k \underbrace{\nabla f(\mathbf{x}^k)^T \mathbf{s}^k}_{= \phi'(0)} \quad (3.21)$$

with the constant $c_1 \in (0, 1)$. The sufficient decrease condition is sometimes called *Armijo condition*. The right side of the inequality (3.21) represents a straight line with negative slope $c_1 \phi'(0)$ that is guaranteed to lie above the function value $\phi(\alpha^k)$ for sufficiently small α^k and $c_1 \in (0, 1)$. Thus, the sufficient decrease condition (3.21) declares a step size α^k valid only if the function value $\phi(\alpha^k)$ lies below this straight line. Figure 3.3 gives a graphical illustration of the sufficient decrease condition. In practice, c_1 is typically chosen to be very small (e.g. 10^{-4}) to make the decrease condition not too restrictive.

The Armijo condition (3.21) alone is often not sufficient to guarantee a noteworthy decrease of the cost function f as the condition is always satisfied for sufficiently small step sizes α^k , see Figure 3.3. Unnecessarily small values of α^k can be avoided by adding the so-called *curvature condition*¹

$$\underbrace{|\nabla f(\mathbf{x}^k + \alpha^k \mathbf{s}^k)^T \mathbf{s}^k|}_{= \phi'(\alpha^k)} \leq c_2 \underbrace{|\nabla f(\mathbf{x}^k)^T \mathbf{s}^k|}_{= \phi'(0)}. \quad (3.22)$$

The constant $c_2 \in (c_1, 1)$ is chosen w.r.t. the first constant c_1 from (3.21). Since the left side of the curvature condition represents the gradient $\phi'(\alpha^k)$, the criterion implies that the

¹ Strictly speaking, (3.22) is a *strengthened* curvature condition. A weaker form of (3.22) can be stated without taking the absolute values on both sides.

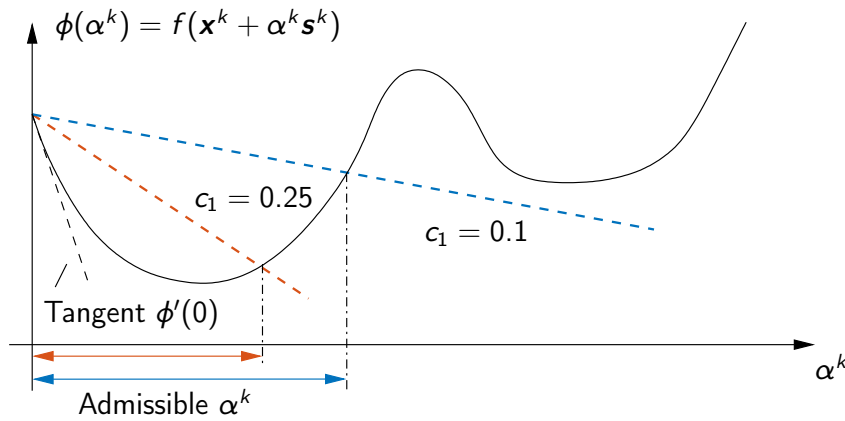


Figure 3.3: Illustration of the sufficient decrease condition.

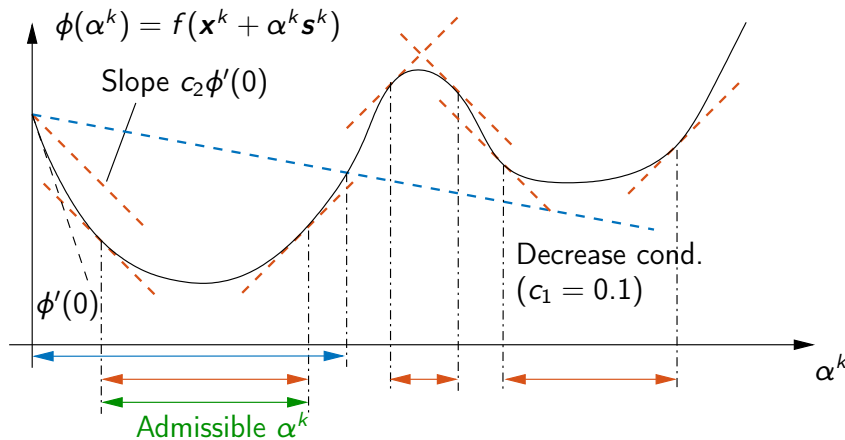


Figure 3.4: Illustration of the Wolfe conditions.

absolute value of the gradient $\phi'(\alpha^k)$ at point α^k is smaller or equal to c_2 times the absolute initial slope $\phi'(0)$, see Figure 3.4.

The formulation of the curvature condition indeed makes sense, since in the case of a strongly negative gradient $\phi'(\alpha^k)$ it can be assumed that ϕ and thus f can be further reduced by moving further along the α^k -axis. On the other hand, the curvature condition does not allow the slope $\phi'(\alpha^k)$ to become too large which indicates that one moves away from the local minimum of ϕ .

Both conditions together, (3.21) and (3.22), are also called *Wolfe conditions* and can be used together with most of the line search methods. Typically, c_2 is set to $c_2 = 0.9$ if Newton- or quasi-Newton method are used to compute the search direction $\mathbf{s}^k$. In case of the conjugate gradient method, $c_2 = 0.1$ is a standard choice. These methods are outlined in the following sections. It can be shown for continuously differentiable and lower bounded functions f that there always exists an interval where the Wolfe conditions are satisfied.

Backtracking

As already mentioned, the descent condition (3.21) alone is not sufficient to guarantee an adequate descent in the cost function f . However, if the step size candidates α^k are chosen appropriately, the curvature condition (3.22) can be omitted and the line search can be stopped based on the descent condition (3.21) alone. A suitable method in this regard is the *backtracking* algorithm, which in its simplest form is given in Table 3.2.

Initialization: $\alpha \leftarrow \bar{\alpha} > 0$ (Initial step size)
 $\rho \in (0, 1)$ (Backtracking parameter)
 c (Armijo parameter)

repeat
 $\alpha \leftarrow \rho \alpha$

until $f(\mathbf{x}^k + \alpha \mathbf{s}^k) \leq f(\mathbf{x}^k) + c \alpha \nabla f(\mathbf{x}^k)^\top \mathbf{s}^k$

$\alpha^k \leftarrow \alpha$ (Step size)

Table 3.2: Basic structure of the backtracking algorithm.

The procedure can be illustrated using Figure 3.3. If the initial value $\bar{\alpha}$ is far to the right, α is reduced by the factor ρ until $\phi(\alpha) = f(\mathbf{x}^k + \alpha \mathbf{s}^k)$ lies below the straight line (with $c = 0.1$ or 0.25). The successive reduction by factor ρ ensures that α^k becomes small enough to satisfy the descent condition (3.21) but *does not become too small*, which could lead to a stagnation of the process.

The backtracking procedure is often used together with the Newton method and the “natural” initial step size $\bar{\alpha} = 1$, see Section 3.3.4. However, backtracking is less appropriate for quasi-Newton or conjugate gradient methods, see Sections 3.3.3 and 3.3.5.

Golden section search

An alternative method for finding the step size α^k is the *golden section search* that generates a convergent series of interval reductions. The first step consists in finding an interval $[\alpha_l, \alpha_r]$ that contains a minimum of the function $\phi(\alpha)$, see Figure 3.5. This can be done, for instance, by starting with a sufficiently small α_l and successively increasing α_r from α_l onward until the function value $\phi(\alpha_r)$ starts to increase.

In the next step, two inner points $\alpha_l < \hat{\alpha}_l < \hat{\alpha}_r < \alpha_r$ are chosen as

$$\hat{\alpha}_l = \alpha_l + (1 - a)(\alpha_r - \alpha_l), \quad \hat{\alpha}_r = \alpha_l + a(\alpha_r - \alpha_l) \quad (3.23)$$

with the parameter $a \in (\frac{1}{2}, 1)$. If $\phi(\hat{\alpha}_l) < \phi(\hat{\alpha}_r)$, the minimum lies in the interval $[\alpha_l, \hat{\alpha}_r]$ and the outer right point α_r is discarded, see “Step 1” in Figure 3.5. If $\phi(\hat{\alpha}_l) > \phi(\hat{\alpha}_r)$ instead, the minimum must lie in the interval $[\hat{\alpha}_l, \alpha_r]$ and the outer left point α_l is omitted. The next

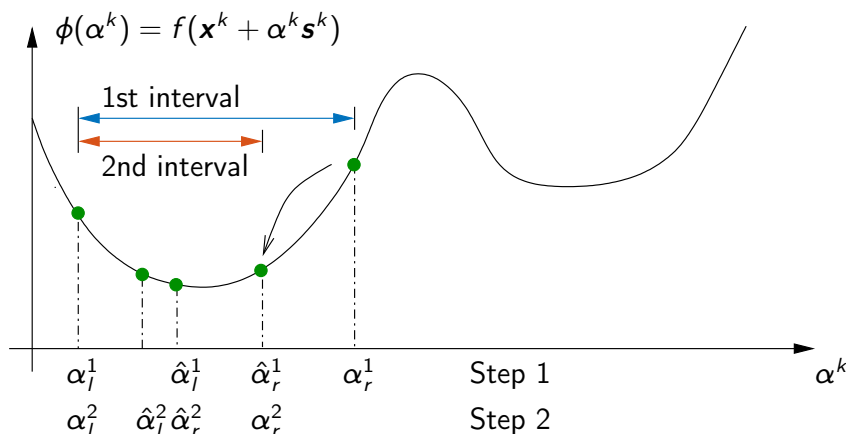


Figure 3.5: Illustration of the golden section search.

```

Initialization:   $\alpha_l, \alpha_r$  (Initial interval containing minimum)
                    $a = 0.618$  (Golden ratio parameter)
                    $\varepsilon_\alpha > 0$  (Convergence tolerance)
                    $\hat{\alpha}_l \leftarrow \alpha_l + (1 - a)(\alpha_r - \alpha_l)$  (Inner points)
                    $\hat{\alpha}_r \leftarrow \alpha_l + a(\alpha_r - \alpha_l)$ 

repeat
  if  $\phi(\hat{\alpha}_l) < \phi(\hat{\alpha}_r)$  do
     $\alpha_r \leftarrow \hat{\alpha}_r$ 
     $\hat{\alpha}_r \leftarrow \hat{\alpha}_l$ 
     $\hat{\alpha}_l \leftarrow \alpha_l + (1 - a)(\alpha_r - \alpha_l)$ 
  else (i.e..  $\phi(\hat{\alpha}_l) \geq \phi(\hat{\alpha}_r)$ )
     $\alpha_l \leftarrow \hat{\alpha}_l$ 
     $\hat{\alpha}_l \leftarrow \hat{\alpha}_r$ 
     $\hat{\alpha}_r \leftarrow \alpha_l + a(\alpha_r - \alpha_l)$ 
  end

until  $|\alpha_r - \alpha_l| \leq \varepsilon_\alpha$ 

```

Table 3.3: Golden section search.

iteration then considers the shortened interval $[\alpha_l, \hat{\alpha}_r]$ or $[\hat{\alpha}_l, \alpha_r]$ and the procedure starts again.

If the parameter a is chosen as

$$a = \frac{\sqrt{5} - 1}{2} \approx 0.618, \quad (3.24)$$

then only one function evaluation is necessary to determine a new interior point, see “Step 2” with $\hat{\alpha}_r^2 = \hat{\alpha}_l^1$ in Figure 3.5. This choice of a is known as the *golden ratio* and also gives the *golden section search* its name. In each step of the algorithm, the interval is reduced by approx. 38%. Table 3.3 summarizes the algorithm.

When the golden section search is stopped, the step size α^k can either be computed from the mean $\alpha^k = (\alpha_l + \alpha_r)/2$ or from a quadratic interpolation between the smallest three function values ϕ at the positions $(\alpha_l, \hat{\alpha}_l, \hat{\alpha}_r, \alpha_r)$. The golden section search is a simple and robust method to compute a suitable step size α^k , but may require more iterations to converge than more sophisticated algorithms.

3.3.2 Gradient method

The computation of a suitable search direction $\mathbf{s}^k$ is of fundamental importance in line search methods and comes prior to the line search problem (3.20) that eventually leads to the next iterate

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{s}^k \quad (3.25)$$

with a meaningful descent in the cost f . Various methods exist to compute the search direction $\mathbf{s}^k$ that differ in terms of accuracy, convergence speed, and computational effort.

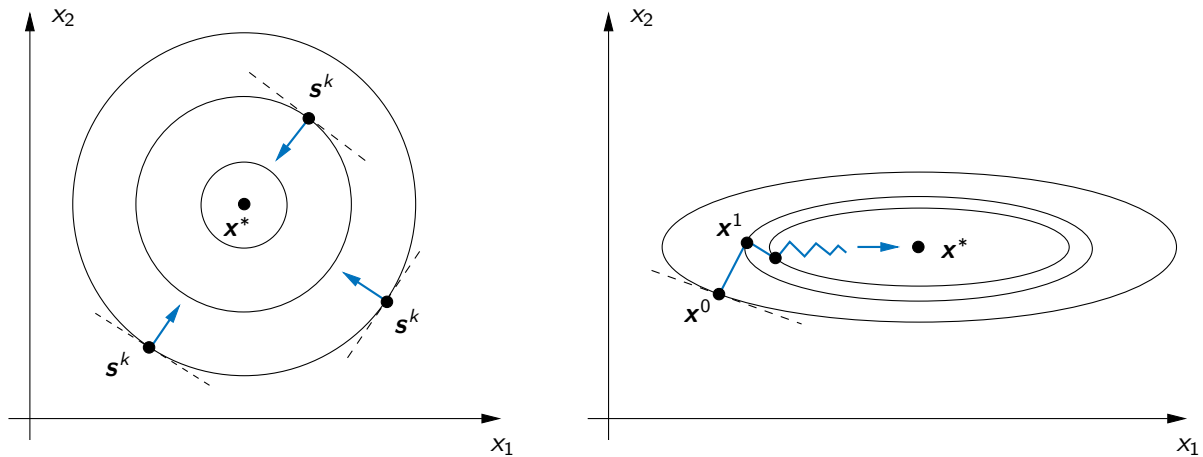


Figure 3.6: Well- and ill-conditioned problems (left/right) for the gradient method.

The simplest choice of $\mathbf{s}^k$ is to use the negative gradient at the point $\mathbf{x}^k$

$$\mathbf{s}^k = -\nabla f(\mathbf{x}^k). \quad (3.26)$$

It is easy to see that the descent condition (3.18) is satisfied for non-stationary points (i.e. $\nabla f(\mathbf{x}^k) \neq \mathbf{0}$)

$$-\nabla f(\mathbf{x}^k)^T \nabla f(\mathbf{x}^k) = -\|\nabla f(\mathbf{x}^k)\|^2 < 0. \quad (3.27)$$

In fact, the negative gradient minimizes the inequality (3.18) and is therefore also called direction of *steepest descent*. Geometrically interpreted, the gradient direction is orthogonal to the contour line $f(\mathbf{x}^k) = \text{const.}$ that passes through the point $\mathbf{x}^k$, see Figure 3.6.

A problem is well-conditioned if the negative gradient points towards the minimum $f(\mathbf{x}^*)$, see Figure 3.6 (left). In this case, the gradient method converges rapidly to the optimal solution. However, if the problem is ill-conditioned in the sense that the negative gradient does not, at least roughly, point in the direction of $\mathbf{x}^*$, then the gradient method typically converges very slowly as illustrated in Figure 3.6 (right). The next theorem makes this more precise for quadratic cost functions.

Theorem 3.4 (Convergence rate of gradient method) For quadratic functions $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c$ with positive definite matrix $\mathbf{Q}$, the gradient method with exact line search (3.20) converges according to

$$f(\mathbf{x}^{k+1}) - f(\mathbf{x}^*) \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 (f(\mathbf{x}^k) - f(\mathbf{x}^*)), \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (3.28)$$

with $\lambda_{\min}$ and $\lambda_{\max}$ being the smallest and largest eigenvalues of $\mathbf{Q}$.

The number κ is the spectral condition number of the matrix $\mathbf{Q}$ and directly affects the speed of convergence as demonstrated in the following exercise.

Exercise 3.1 Determine the convergence rate (3.28) of the gradient method for the quadratic problem

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = \frac{1}{2} \left(\frac{x_1 - x_1^*}{a} \right)^2 + \frac{1}{2} (x_2 - x_2^*)^2 \quad (3.29)$$

with the values $a = 1$ and $a = 5$, also see Figure 3.6.

The advantages and disadvantages of the gradient methods can be summarized as follows:

- + larger domain of convergence than in case of the Newton method (positive definiteness of the Hessian $\nabla^2 f(\mathbf{x}^k) > \mathbf{0}$ is not required)
- + simple and robust
- slow convergence for ill-conditioned or badly scaled problems
- limited accuracy (first-order method).

3.3.3 Conjugate gradient method

The *conjugate gradient method* is an important alternative to the conventional gradient method due to the fact that the computational effort is not much higher but the efficiency and convergence speed are significantly better. The conjugate gradient method uses information from the previous iteration to determine the search direction

$$\mathbf{s}^0 = -\nabla f(\mathbf{x}^0) \quad (3.30a)$$

$$\mathbf{s}^k = -\nabla f(\mathbf{x}^k) + \beta^k \mathbf{s}^{k-1}. \quad (3.30b)$$

The conjugate gradient method is initialized in steepest descent direction $-\nabla f(\mathbf{x}^0)$, whereas all further steps contain a correction term based on the previous search direction $\mathbf{s}^{k-1}$. The factor β^k is chosen in dependence of the gradient norms of the current and last iteration

$$\beta^k = \frac{\|\nabla f(\mathbf{x}^k)\|^2}{\|\nabla f(\mathbf{x}^{k-1})\|^2}. \quad (3.31)$$

This formula is due to *Fletcher* and *Reeves*, others can be found in the textbooks at the end of the chapter.

Conjugate gradient methods do not require any complex matrix operations and can therefore be applied to large problems, where memory efficiency must be taken into account. Interestingly, it can be shown for quadratic problems

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c, \quad \mathbf{x} \in \mathbb{R}^n$$

with positive definite Hessian $\mathbf{Q}$ that the conjugate gradient method with exact line search (3.20) converges in at most n iterations irrespective of the conditioning of $\mathbf{Q}$.

Exercise 3.2 Verify that the conjugate gradient method with exact line search produces a search direction $\mathbf{s}^k$ that satisfies the descent condition (3.18).

3.3.4 Newton's method

A special case of the line search method occurs if the search direction $\mathbf{s}^k$ is chosen such that the iteration rule $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{s}^k$ corresponds to the Newton method. In this case we speak of the *Newton direction* $\mathbf{s}^k$. To get to this point, we approximate the cost function $f(\mathbf{x}) = f(\mathbf{x}^k + \mathbf{s})$ at the point $\mathbf{x}^k$ by the quadratic model

$$m^k(\mathbf{s}) = f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^T \mathbf{s} + \frac{1}{2} \mathbf{s}^T \nabla^2 f(\mathbf{x}^k) \mathbf{s} \quad (3.32)$$

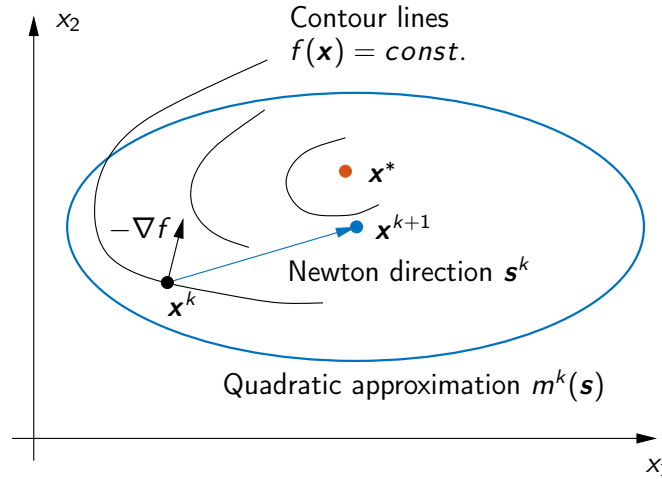


Figure 3.7: Illustration of Newton's method in comparison to the gradient method.

and compute the minimum point $\mathbf{s}$ from the stationarity condition

$$\nabla f(\mathbf{x}^k) + \nabla^2 f(\mathbf{x}^k) \mathbf{s} = \mathbf{0}. \quad (3.33)$$

If we assume that the Hessian matrix $\nabla^2 f(\mathbf{x}^k)$ is positive definite, the Newton direction $\mathbf{s}^k = \mathbf{s}$ at the point $\mathbf{x}^k$ becomes

$$\mathbf{s}^k = -[\nabla^2 f(\mathbf{x}^k)]^{-1} \nabla f(\mathbf{x}^k). \quad (3.34)$$

It is easy to verify that the descent condition (3.18)

$$\nabla f(\mathbf{x}^k)^T \mathbf{s}^k = -\nabla f(\mathbf{x}^k)^T [\nabla^2 f(\mathbf{x}^k)]^{-1} \nabla f(\mathbf{x}^k) < 0 \quad (3.35)$$

is always satisfied since the inverse of a positive definite matrix $\nabla^2 f(\mathbf{x}^k)$ is also positive definite.

The Newton direction (3.34) defines the next point $\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{s}^k$, which corresponds to the “natural” step size $\alpha^k = 1$ for the iteration rule (3.25) of the line search method. Therefore, Newton methods typically use a default step size of $\alpha^k = 1$ that is only reduced by backtracking if the sufficient decrease condition for the cost function $f(\mathbf{x}^{k+1})$ is violated.

Newton's method is a second-order method, since the error of the approximation (3.32) is of order $\mathcal{O}(\|\mathbf{s}\|^3)$. It is reliable and also well suited for high-dimensional systems, if the quadratic approximation of the function f at the point $\mathbf{x}^k$ is sufficiently accurate. Figure 3.7 illustrates the advantage of the quadratic approximation of the Newton method compared to the steepest descent direction $-\nabla f$ of the gradient method.

Exercise 3.3 Show that for quadratic problems

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{b}^T \mathbf{x} + c \quad (\mathbf{Q} \text{ positive definite}) \quad (3.36)$$

the Newton method converges within one iteration independent of the initial point $\mathbf{x}^0$.

The advantages and disadvantages of Newton's method are

- + quadratic convergence in the neighborhood of $\mathbf{x}^*$ (within few iterations)
- + high accuracy (second-order method)
- only local convergence (positive definiteness of $\nabla^2 f(\mathbf{x})$ usually not guaranteed for all $\mathbf{x}$)
- expensive computation of Hessian $\nabla^2 f$ in particular for large-scale problems.

The local convergence property of Newton methods can be globalized by keeping the Hessian matrix $\nabla^2 f(\mathbf{x})$ positive definite, e.g. by distorting its diagonal. Moreover, the complex computation of the Hessian matrix $\nabla^2 f$ can be mitigated by sparse numerics or finite-difference schemes or even completely avoided by the quasi-Newton method that is discussed in the following lines.

3.3.5 Quasi-Newton method

The *quasi-Newton method* is an attractive alternative to Newton's method, since it avoids the time-consuming calculation of the Hessian matrix $\nabla^2 f$ and its inverse. Instead, the Hessian is approximated and updated during the individual iterations by exploiting information about the gradients at the single iteration points.

To illustrate this idea, we approximate the gradient ∇f at the point $\mathbf{x}^k$ by the linear model

$$\nabla f(\mathbf{x}^k + \mathbf{s}) \approx \nabla f(\mathbf{x}^k) + \nabla^2 f(\mathbf{x}^k) \mathbf{s}. \quad (3.37)$$

For sufficiently small $\mathbf{s} = \mathbf{x}^{k-1} - \mathbf{x}^k$, we obtain an approximation for the gradient at the previous point $\mathbf{x}^{k-1}$

$$\nabla f(\mathbf{x}^{k-1}) \approx \nabla f(\mathbf{x}^k) + \nabla^2 f(\mathbf{x}^k)(\mathbf{x}^{k-1} - \mathbf{x}^k). \quad (3.38)$$

This relation shows that the Hessian at point $\mathbf{x}^k$ can be approximated by a matrix $\mathbf{H}^k \approx \nabla^2 f(\mathbf{x}^k)$ that must satisfy the so-called *secant equation*

$$\mathbf{H}^k(\mathbf{x}^k - \mathbf{x}^{k-1}) = \nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}). \quad (3.39)$$

Since the problem is underdetermined ($\mathbf{H}^k$ has n^2 elements, but (3.39) only n equations), there are typically further conditions that are imposed on $\mathbf{H}^k$ such as symmetry and minimal difference between $\mathbf{H}^k$ and $\mathbf{H}^{k-1}$. These additional conditions are combined with (3.39) in an optimization problem for $\mathbf{H}^k$, which can be solved analytically.

A popular formula for updating $\mathbf{H}^k$ is the *BFGS formula* due to *Broyden, Fletcher, Goldfarb, and Shanno*

$$\mathbf{H}^k = \mathbf{H}^{k-1} - \frac{\mathbf{H}^{k-1} \mathbf{d}^k (\mathbf{d}^k)^\top \mathbf{H}^{k-1}}{(\mathbf{d}^k)^\top \mathbf{H}^{k-1} \mathbf{d}^k} + \frac{\mathbf{y}^k (\mathbf{y}^k)^\top}{(\mathbf{d}^k)^\top \mathbf{y}^k} \quad (3.40)$$

with the vectors

$$\mathbf{d}^k = \mathbf{x}^k - \mathbf{x}^{k-1}, \quad \mathbf{y}^k = \nabla f(\mathbf{x}^k) - \nabla f(\mathbf{x}^{k-1}). \quad (3.41)$$

The initialization of the quasi-Newton method at point $\mathbf{x}^0$ is either done with a positive definite estimate $\mathbf{H}^0 \approx \nabla^2 f(\mathbf{x}^0)$ or in the most simple case by the unit matrix $\mathbf{H}^0 = \mathbf{I}$.

A drawback of the BFGS formula (3.40) is that Newton's method actually requires the inverse Hessian matrix to calculate the search direction $\mathbf{s}^k$, see (3.34). It is therefore more reasonable

to directly approximate the inverse Hessian matrix $\mathbf{B}^k = (\mathbf{H}^k)^{-1}$. The respective BFGS formula (3.40) for updating the inverse Hessian is

$$\mathbf{B}^k = [\mathbf{I} - \rho^k \mathbf{d}^k (\mathbf{y}^k)^\top] \mathbf{B}^{k-1} [\mathbf{I} - \rho^k \mathbf{y}^k (\mathbf{d}^k)^\top] + \rho^k \mathbf{d}^k (\mathbf{d}^k)^\top, \quad \rho^k = \frac{1}{(\mathbf{y}^k)^\top \mathbf{d}^k}. \quad (3.42)$$

Using $\mathbf{B}^k$ in the Newton direction (3.34) eventually gives the search direction

$$\mathbf{s}^k = -\mathbf{B}^k \nabla f(\mathbf{x}^k). \quad (3.43)$$

The quasi-Newton method usually has good convergence properties, although it does not match the convergence speed of the pure Newton method. The big advantage over Newton's method, however, is that the matrix-vector multiplications in (3.42) are typically significantly faster to perform than the computation of the Hessian $\nabla^2 f$ and its inverse. A further advantage of the quasi-Newton method is that there are no singularity issues (at least in theory) because $\mathbf{H}^k$ as well as $\mathbf{B}^k$ are positive definite as long as the line search (3.20) is exact.

Similar to the conjugate gradient method, it can be shown that the quasi-Newton method solves a quadratic problem

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c, \quad \mathbf{x} \in \mathbb{R}^n$$

with positive definite $\mathbf{Q}$ and exact line search (3.20) in at most n iterations.

3.4 Further numerical methods

This section presents two standard optimization algorithms that do not belong to the class of line search methods. The first one is the *trust region method* and the second one belongs to the class of *direct search methods* that does not rely on gradient information.

3.4.1 Trust region method

The *trust region method* constructs a local simplified model $m^k(\mathbf{s}) \approx f(\mathbf{x}^k)$ of the cost function $f(\mathbf{x}^k)$ at the point $\mathbf{x}^k$ that is solved instead of the original optimization problem (3.1):

$$\min_{\|\mathbf{s}\| \leq \Delta^k} m^k(\mathbf{s}) \quad \text{with} \quad m^k(\mathbf{s}) \approx f(\mathbf{x}^k) \quad \text{for} \quad \|\mathbf{s}\| \leq \Delta^k. \quad (3.44)$$

The search radius for the solution is restricted to the *trust region* $\|\mathbf{s}\| \leq \Delta^k$, which is motivated by the fact that the model $m^k(\mathbf{s})$ is in general only locally accurate, i.e. for sufficiently small $\mathbf{s}$. In most cases, the model m^k is constructed as the quadratic form

$$m^k(\mathbf{s}) = \frac{1}{2} \mathbf{s}^\top \mathbf{H}^k \mathbf{s} + \nabla f(\mathbf{x}^k)^\top \mathbf{s} + f(\mathbf{x}^k), \quad (3.45)$$

where the matrix $\mathbf{H}^k$ is either given by the Hessian $\nabla^2 f(\mathbf{x}^k)$ or an approximation of it. The solution of the new optimization problem (3.44) is a “candidate” $\mathbf{s}^k$ for the next iterate $\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{s}^k$. If the associated cost $f(\mathbf{x}^{k+1})$ does not lead to a sufficient cost reduction w.r.t. the previous one $f(\mathbf{x}^k)$, the trust region radius Δ^k is reduced and (3.45) is solved again.

During each of these iterations with shrinking trust region radius Δ^k , the distance between $\mathbf{x}^k$ and the new candidate $\mathbf{x}^{k+1}$ becomes smaller and $\mathbf{s}^k$ will in general change its direction. Thus, the trust region method differs clearly from line search methods in terms of how the search direction and step size are chosen:

- Line search methods fix the search direction $\mathbf{s}^k$ and determine the step size α^k afterwards.
- The trust region method chooses the trust region radius Δ^k and then simultaneously computes the direction and distance to the next point $\mathbf{x}^{k+1}$ in an iterative manner.

The trust region radius Δ^k is adapted in each iteration by evaluating the accuracy of the model m^k compared to the cost function f via the measure

$$\rho^k(\mathbf{s}^k) = \frac{f(\mathbf{x}^k) - f(\mathbf{x}^k + \mathbf{s}^k)}{m^k(\mathbf{0}) - m^k(\mathbf{s}^k)}. \quad (3.46)$$

The numerator describes the actual descent in the cost, the denominator the predicted one. For $\rho^k \approx 1$, the model m^k and the actual cost f show a good agreement in the current iteration, which motivates to enlarge the trust region radius Δ^k for the next step. If in contrast $\rho^k \ll 1$, then Δ^k should be reduced, since we do not have enough confidence in the model over the current trust region. The basic algorithm of the trust region method is sketched in Table 3.4.

Initialization:	$\bar{\Delta}, \Delta^0 \in (0, \bar{\Delta})$	(Trust region: initial & max. radius)
	$\eta \in [0, \frac{1}{4})$	(Parameter)
	$k \leftarrow 0$	(Iteration index)
	$\varepsilon_{\nabla f} > 0$	(Convergence tolerance)
repeat		
	$m^k(\mathbf{s}) \leftarrow (3.45)$	(Model)
	$\mathbf{s}^k \leftarrow \arg \min \{m^k(\mathbf{s}) : \ \mathbf{s}\ \leq \Delta^k\}$	(Often solved approximately)
	$\rho^k \leftarrow (3.46)$	(Model quality)
	if $\rho^k < \frac{1}{4}$ do	
	$\Delta^{k+1} \leftarrow \frac{1}{4}\Delta^k$	(Shrink trust region)
	else if $\rho^k > \frac{3}{4}$ and $\ \mathbf{s}^k\ = \Delta^k$ do	
	$\Delta^{k+1} \leftarrow \min\{2\Delta^k, \bar{\Delta}\}$	(Enlarge trust region)
	else	
	$\Delta^{k+1} \leftarrow \Delta^k$	
	end if	
	if $\rho^k > \eta$ do	
	$\mathbf{x}^{k+1} \leftarrow \mathbf{x}^k + \mathbf{s}^k$	(Next step)
	$\mathbf{H}^{k+1} \leftarrow \mathbf{H}^k + \dots$	(Update of Hessian matrix)
	else	
	$\mathbf{x}^{k+1} \leftarrow \mathbf{x}^k$	(Repeat step with $\Delta^{k+1} < \Delta^k$)
	end if	
	$k \leftarrow k + 1$	
until	$\ \nabla f(\mathbf{x}^k)\ \leq \varepsilon_{\nabla f}$	(Or alternative tolerance criterion)

Table 3.4: Trust region method.

3.4.2 Direct search methods

All of the algorithms introduced so far depend on information about the gradient ∇f (and the Hessian $\nabla^2 f$ if available) to compute the next iteration point $\mathbf{x}^{k+1}$ with a sufficient decrease in the cost. There are practical cases, however, where the gradient is not available or cannot be computed or approximated with reasonable effort because the problem at hand is either too complex or not continuously differentiable. An alternative for these kind of problems are *direct search methods* that use samples of function values to determine a new iteration point.

The probably best-known direct search algorithm in unconstrained nonlinear optimization is the *simplex method*² developed by *Nelder* and *Mead*, which is therefore also called *Nelder-Mead method*. The algorithm operates on a *simplex* with $n + 1$ points $\mathbf{x}^i$, $i = 1, \dots, n + 1$ in the n -dimensional parameter space $\mathbb{R}^n$.³ For instance, the simplex is a straight line in $\mathbb{R}^1$ and a triangle in $\mathbb{R}^2$.

The function values $f(\mathbf{x}^i)$ are computed for the $n + 1$ points $\mathbf{x}^i$ of the simplex and sorted in ascending order, i.e.

$$f(\mathbf{x}^1) \leq f(\mathbf{x}^2) \leq \dots \leq f(\mathbf{x}^{n+1}). \quad (3.47)$$

The algorithm replaces the “worst” point $\mathbf{x}^{n+1}$ by a new one that lies on the parametrized straight line

$$g(a) = \bar{\mathbf{x}} + a(\bar{\mathbf{x}} - \mathbf{x}^{n+1}), \quad \bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}^i \quad (3.48)$$

running through the point $\mathbf{x}^{n+1}$ with the highest cost value $f(\mathbf{x}^{n+1})$ and the center $\bar{\mathbf{x}}$ of the remaining simplex points. The simplex method uses different projections by varying a in order to generate a new simplex:

- a) **Reflexion** (Figure 3.8a):
 Compute the reflexion point $\mathbf{x}_r = g(1)$ and $f_r = f(\mathbf{x}_r)$.
 If $f(\mathbf{x}^1) \leq f_r < f(\mathbf{x}^n)$, set $\mathbf{x}^{n+1} = \mathbf{x}_r$.
- b) **Expansion** (Figure 3.8b):
 If $f_r < f(\mathbf{x}^1)$, compute $\mathbf{x}_e = g(2)$ and $f_e = f(\mathbf{x}_e)$.
 If $f_e < f_r$, set $\mathbf{x}^{n+1} = \mathbf{x}_e$. Otherwise, set $\mathbf{x}^{n+1} = \mathbf{x}_r$.
- c) **Outer contraction** (Figure 3.8c):
 If $f(\mathbf{x}^n) \leq f_r < f(\mathbf{x}^{n+1})$, compute $\mathbf{x}_c = g(\frac{1}{2})$ and $f_c = f(\mathbf{x}_c)$.
 If $f_c \leq f_r$, set $\mathbf{x}^{n+1} = \mathbf{x}_c$. Otherwise, go to Step e).
- d) **Inner contraction** (Figure 3.8d):
 If $f_r \geq f(\mathbf{x}^{n+1})$, compute $\mathbf{x}_c = g(-\frac{1}{2})$ and $f_c = f(\mathbf{x}_c)$.
 If $f_c < f(\mathbf{x}^{n+1})$, set $\mathbf{x}^{n+1} = \mathbf{x}_c$. Otherwise, go to Step e).
- e) **Shrinking** (Figure 3.8e):
 Shrink the simplex around the “best” point $\mathbf{x}^1$:
 $\mathbf{x}^i = \frac{1}{2}(\mathbf{x}^1 + \mathbf{x}^i)$, $i = 2, \dots, n + 1$.

² The simplex method of Nelder and Mead is not to be confused with the simplex algorithm of *G. Danzig* in linear programming.

³ We omit the iteration index k for the sake of readability.

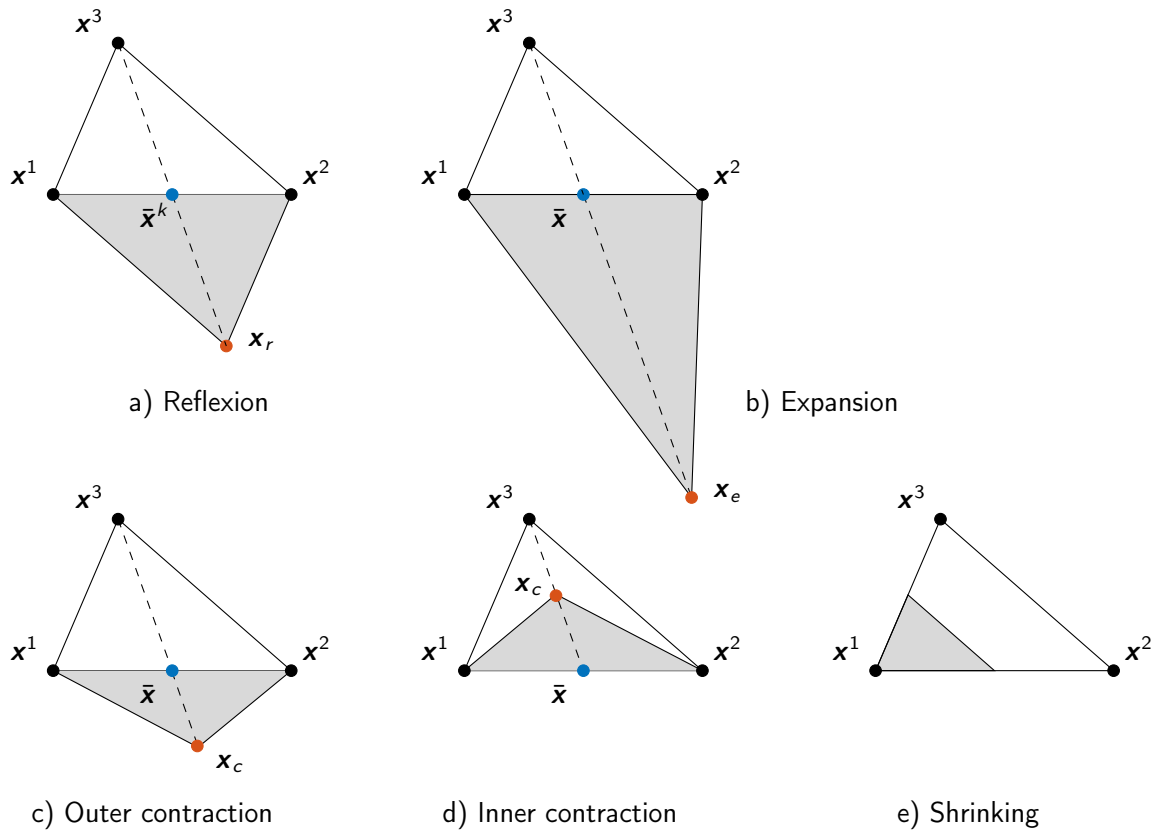


Figure 3.8: Operations of the simplex method of Nelder and Mead.

The algorithm and the individual simplex operations are summarized in Table 3.5. The simplex method is iteratively continued until a convergence criterion is satisfied. Over the iterations, the simplex moves towards the optimum and contracts.

Due to the fact that the simplex method is a heuristic algorithm, it is hard to guarantee convergence. In fact, the simplex may converge to a non-optimal point. In this case, a standard approach is to restart the algorithm with a new simplex at the last point. In practice, however, the simplex method usually performs well and robust and is one of the most popular optimization algorithms, albeit its moderate speed of convergence.

Initialization: $\mathbf{x}^i, \dots, \mathbf{x}^{n+1}$ (Initial simplex)
 $\varepsilon_x, \varepsilon_f > 0$ (Convergence tolerance)

repeat

Sorting (3.47)

$\bar{\mathbf{x}} \leftarrow$ (3.48) (Center of remaining simplex)

Exchange of $\mathbf{x}^{n+1}$ (Reflexion/Expansion/Contraction)
or of all points $\mathbf{x}^i$ (Shrinking)

until $\max_{2 \leq i \leq n+1} \|\mathbf{x}^i - \mathbf{x}^1\| \leq \varepsilon_x$ **or** $f(\mathbf{x}^{n+1}) - f(\mathbf{x}^1) \leq \varepsilon_f$

Table 3.5: Simplex method of Nelder and Mead.

3.5 Example: Rosenbrock's "banana function"

A well-known benchmark problem in optimization is *Rosenbrock's* problem

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) \quad \text{with} \quad f(\mathbf{x}) = 100(x_2 - x_1^2)^2 + (x_1 - 1)^2. \quad (3.49)$$

Figure 3.9 shows the profile and the contour lines of the function, which for apparent reasons is also known as *banana function*. We will use Rosenbrock's problem to compare the algorithms of this chapter numerically under MATLAB.

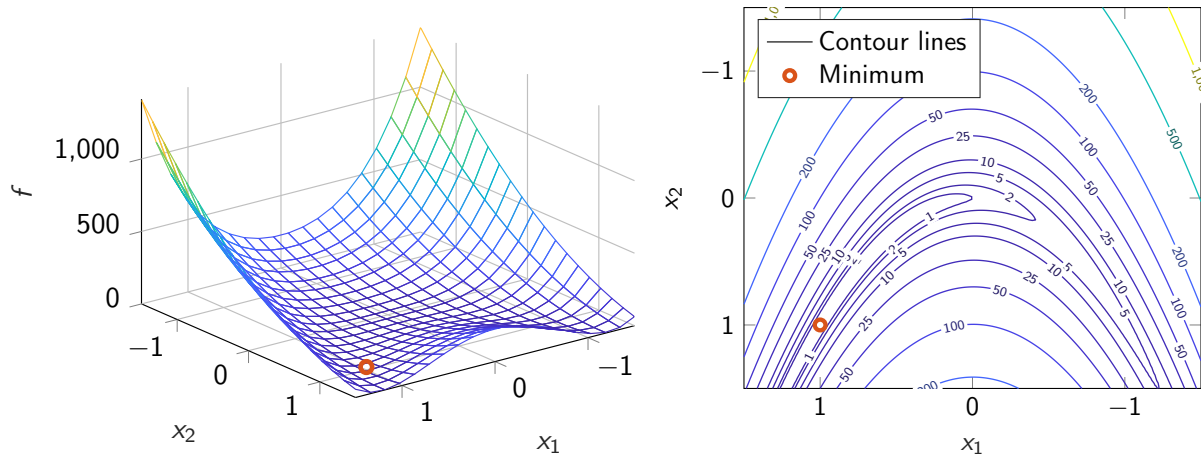


Figure 3.9: Profile and contour lines of Rosenbrock's banana function.

Exercise 3.4 Verify that the point $\mathbf{x}^* = [1, 1]^T$ is a unique minimum point. Are the function $f(\mathbf{x})$ and the optimization problem (3.49) convex?

MATLAB offers the *Optimization Toolbox* for the solution of parameter optimization problems. The following functions are particularly relevant for unconstrained optimization:

- `fminunc`: line search method (gradient, quasi-Newton) & trust region method (Newton)
- `fminsearch`: simplex method of Nelder and Mead.

An alternative is the free MATLAB function `minFunc` [5] that offers a wide range of line search methods. Table 3.6 shows selected key figures of the different algorithms for the numerical solution of Rosenbrock's problem with `fminunc`, `fminsearch`, and `minFunc`. The initial point was $\mathbf{x}^0 = [-1, -1]^T$ in all cases.

Figure 3.10 additionally shows the iteration profiles of the methods implemented in `fminunc` and `fminsearch`. For the sake of completeness, the MATLAB code for Rosenbrock's problem (3.49) is shown in Figure 3.12 at the end of the chapter to illustrate the option settings and function call for `fminunc` and `fminsearch`.

The gradient method shows a stagnating convergence, because even after reaching the maximum number of function evaluations (200), the minimum has not yet been reached. Figure 3.11 additionally shows the iterations of the gradient method plotted together with the contour lines of the Rosenbrock function (3.49). The single gradient steps in steepest descent

Method	Iter.	$f(\mathbf{x}^*)$	$\nabla f(\mathbf{x}^*)$	Function calls		
				$f(\mathbf{x})$	$\nabla f(\mathbf{x})$	$\nabla^2 f(\mathbf{x})$
LS: Gradient method	57	0.1232	1.1978	200	200	-
LS: Conj. grad. method.	31	$4.1 \cdot 10^{-17}$	$6.9 \cdot 10^{-9}$	91	90	-
LS: Newton's method	21	$3.5 \cdot 10^{-17}$	$3.5 \cdot 10^{-8}$	29	28	24
LS: Quasi-Newton (BFGS)	23	$5.4 \cdot 10^{-12}$	$9.2 \cdot 10^{-6}$	29	29	-
TR: Newton's method	25	$2.2 \cdot 10^{-18}$	$2.1 \cdot 10^{-8}$	26	26	26
Simplex method (Nelder-Mead)	67	$5.3 \cdot 10^{-10}$	-	125	-	-

Table 3.6: Comparison of optimization methods for Rosenbrock's problem (LS: line search method, TR: trust region method).

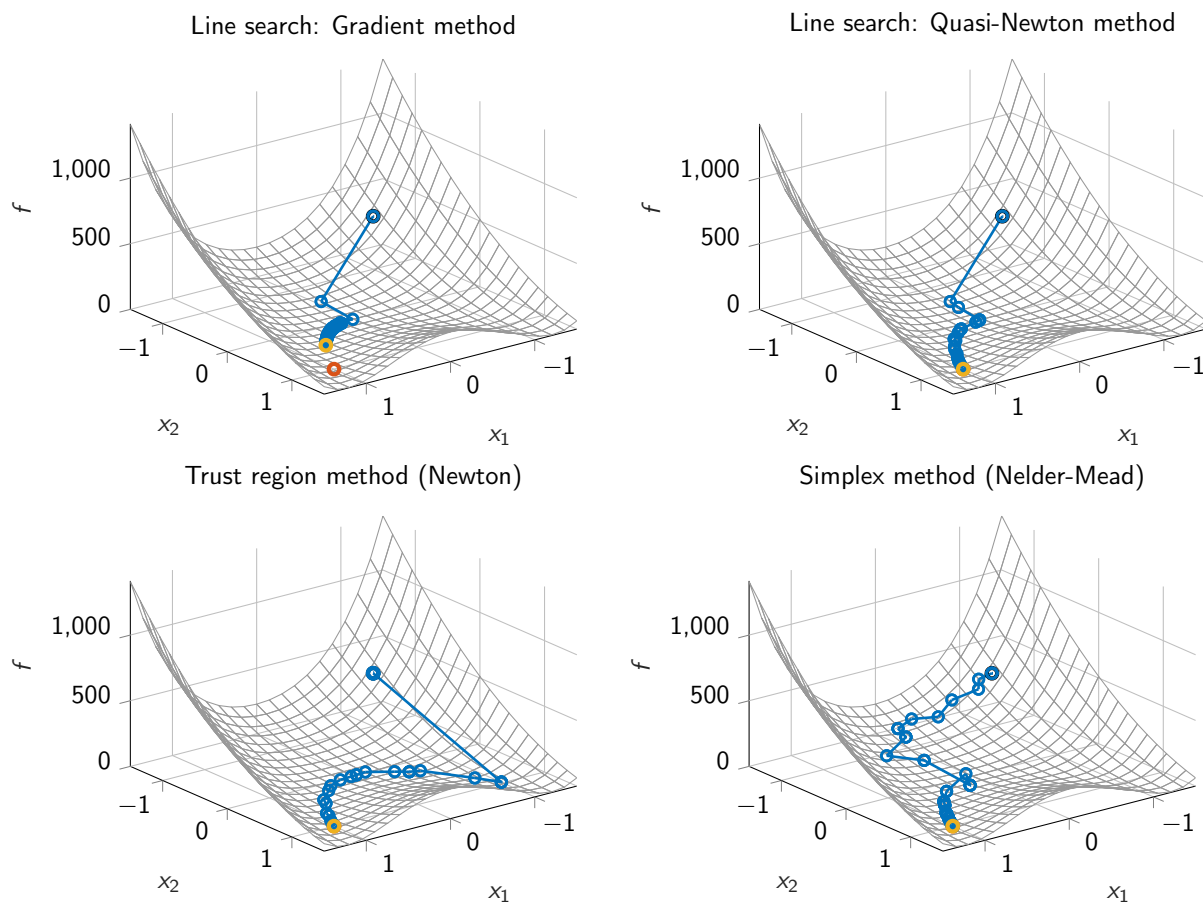


Figure 3.10: Iteration profiles for Rosenbrock's problem.

direction are perpendicular to the respective contour lines and become smaller over the iterations. The slow convergence has already been addressed in Theorem 3.4, also see Figure 3.6, and will be examined in more detail in the following exercise.

Exercise 3.5 Determine the convergence rate of the gradient method in the neighborhood of the minimum $\mathbf{x}^* = [1, 1]^T$ of Rosenbrock's problem (3.49).

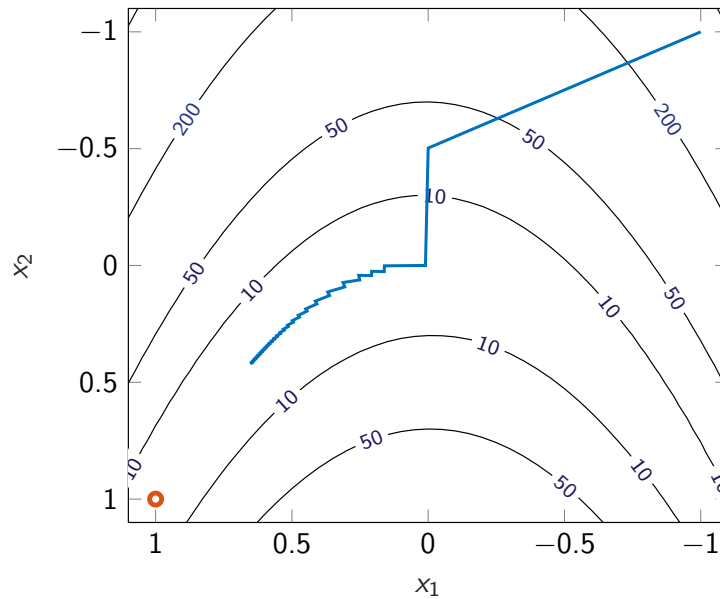


Figure 3.11: Iterations of the gradient method.

The quasi-Newton method in Figure 3.10 converges significantly better than the gradient method. The Newton/trust region method in Figure 3.10 initially starts in the “wrong” direction, because the quadratic approximation (3.45) at the initial point $\mathbf{x}^0 = [-1, 1]^T$ has its minimum close to $\mathbf{x} \approx [-1, 1]^T$. However, the individual steps along the valley of the Rosenbrock function are much larger since Newton’s method uses the Hessian matrix explicitly and does not rely on an approximation as in the case of the quasi-Newton method.

Table 3.6 and Figure 3.10 additionally show the results for the simplex method that are obtained with the MATLAB function `fminsearch`. The graphical output under `fminsearch` does not provide the possibility to display the individual simplices (or simplexes). The next exercise, therefore, examines the simplex method in more detail to get an impression of the simplex operations and its robustness.

Exercise 3.6 Write a MATLAB function that solves Rosenbrock’s problem (3.49) with the simplex method of Nelder and Mead. Construct the first simplex with the edges

$$\mathbf{x}^{0,1} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \quad \mathbf{x}^{0,2} = \mathbf{x}^{0,1} + s \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{x}^{0,3} = \mathbf{x}^{0,1} + s \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (3.50)$$

and the side length $s = 1$. Use the tolerance $\varepsilon_{\mathbf{x}} = \varepsilon_f = 10^{-9}$ for the convergence criterion in Table 3.5 and compare your results with the `fminsearch` results in Table 3.6. Plot the simplices graphically for the single iterations. Investigate the robustness and convergence behavior for different values of the side length s of the initial simplex and different initial points $\mathbf{x}^{0,1}$.

Exercise 3.7 (optional) Write a MATLAB function that solves Rosenbrock’s problem (3.49) with the Newton line search method. Use backtracking to determine the step size α^k with the default $\bar{\alpha} = 1$ and the Armijo condition (3.21) as stopping criterion. Compare the convergence results with Table 3.6. Plot the single iterates graphically and vary the initial point $\mathbf{x}^0$.

```

function Xopt = rosenbrock_problem(Xinit,methodQ)
% -----
% Xinit:    initial point
% methodQ: 1 - line search: gradient method
%           2 - line search: quasi-Newton
%           3 - trust region method (Newton)
%           4 - Nelder-Mead simplex method
global old
old = [Xinit; rosenbrock(Xinit)];

% options for optimization
opt1 = optimoptions('fminunc','Display','iter','PlotFcns',@plot_iterates);
opt2 = optimset('Display','iter','PlotFcns',@plot_iterates);

switch methodQ
case 1,      % line search: gradient method
    opt1 = optimoptions(opt1,'HessUpdate','steepdesc','GradObj','on');
case 2,      % line search: quasi-Newton
    opt1 = optimoptions(opt1,'Algorithm','quasi-newton','GradObj','on');
case 3,      % trust region method (Newton)
    opt1 = optimoptions(opt1,'Algorithm','trust-region','Hessian','on','GradObj','on');
end

if methodQ<4,    Xopt = fminunc(@rosenbrock,Xinit,opt1);      % numeric solution with
else            Xopt = fminsearch(@rosenbrock,Xinit,opt2);    % fminunc or fminsearch
end

function [f, grad, H] = rosenbrock(x)
% -----
grad = {}; H = {};
f = 100*(x(2)-x(1)^2)^2 + (x(1)-1)^2;      % Rosenbrock function
if nargin>1,                               % if gradient is requested
    grad = [ -400*(x(2)-x(1)^2)*x(1)+2*(x(1)-1);
             200*(x(2)-x(1)^2) ];
end
if nargin>2,                               % if Hessian is requested
    H = [ -400*(x(2)-3*x(1)^2)+2,    -400*x(1);
          -400*x(1),                200 ];
end

function stop = plot_iterates(x,info,state)
% -----
global old
f = rosenbrock(x);
switch state
case 'init',                               % graphical output:
    plot_surface(x,f);                     % initialization
case 'iter',                               % iterations
    plot3([old(1),x(1)],[old(2),x(2)],[old(3),f],'b-o','LineWidth',1);
case 'done',                               % after last iterations
    plot3(x(1),x(2),f,'go','LineWidth',5);
end
stop = false;                             % no stopping criterion
old = [x;f];

function plot_surface(x,f)
% -----
[X1,X2] = meshgrid(-1.5:0.15:1.5);
F = 100*(X2-X1.^2).^2 + (X1-1).^2;          % 3D profile of
                                            % Rosenbrock function
h = surf(X1,X2,F,'EdgeColor',0.6*[1,1,1],'FaceColor','none');
hold on; axis tight;
plot3(x(1),x(2),f,'ko','LineWidth',5);    % initial point
plot3(1,1,0,'ro','LineWidth',5);          % optimal solution
xlabel('x_1'); ylabel('x_2'); zlabel('f')
set(gcf,'ToolBar','figure');              % figure settings
set(gca,'Xdir','reverse','Ydir','reverse');

```

Figure 3.12: MATLAB code for Rosenbrock's problem (fminunc, fminsearch).

3.6 References

- [1] S. Boyd and L. Vandenberghe. *Convex Optimization*. http://www.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf. Cambridge: Cambridge University Press, 2004.
- [2] C.T. Kelley. *Iterative Methods for Optimization*. Society for Industrial and Applied Mathematics (SIAM), 1999.
- [3] J. Nocedal and S.J. Wright. *Numerical Optimization*. New York: Springer, 2006.
- [4] M. Papageorgiou, M. Leibold, and M. Buss. *Optimierung*. Berlin: Springer, 2012.
- [5] Mark Schmidt. *minFunc (MATLAB-Funktion zur Lösung unbeschränkter Optimierungsprobleme)*. University of British Columbia, Vancouver, Kanada. Available at <https://www.cs.ubc.ca/~schmidtm/Software/minFunc.html>.

4 Constrained optimization

This chapter extends the consideration of unconstrained (parameter) optimization problems to constrained ones of the form

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (4.1a)$$

$$\text{s.t. } g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \quad (4.1b)$$

$$h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q \quad (4.1c)$$

with the equality and inequality constraints (4.1b) and (4.1c). As it will turn out, the necessary and sufficient optimality conditions as well as the numerical algorithms to solve constrained optimization problems differ significantly from the unconstrained case (Chapter 3).

4.1 Illustrative examples

We recycle Example 1.2 from the introduction and successively augment the problem by the constraints (1.10)-(1.12) to highlight the extension of the unconstrained case as well as the difference between the handling of equality and inequality constraints.

4.1.1 One equality constraint

Example 4.1 We consider the optimization problem (1.9) with the equality constraint (1.10)

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2 \quad (4.2a)$$

$$\text{s.t. } g(\mathbf{x}) = x_2 - 2x_1 = 0. \quad (4.2b)$$

Figure 4.1 shows the constraint and the contour lines of $f(\mathbf{x})$. Obviously, an optimal point $\mathbf{x}^*$ must lie on the straight line $g(\mathbf{x}) = 0$. If we consider, for instance, the contour line $f(\mathbf{x}) = 4$, then two intersection points with $g(\mathbf{x}) = 0$ exist that move closer together for decreasing contour lines $f(\mathbf{x}) < 4$ and eventually lead to the minimum point

$$\mathbf{x}^* = \begin{bmatrix} 0.8 \\ 1.6 \end{bmatrix} \quad \text{with } f(\mathbf{x}^*) = 1.8. \quad (4.3)$$

An important role in this regard play the gradients of the functions $f(\mathbf{x})$ and $g(\mathbf{x})$

$$\nabla f(\mathbf{x}) = \begin{bmatrix} 2(x_1 - 2) \\ 2(x_2 - 1) \end{bmatrix}, \quad \nabla g(\mathbf{x}) = \begin{bmatrix} -2 \\ 1 \end{bmatrix}. \quad (4.4)$$

For all non-optimal points on the constraint $g(\mathbf{x}) = 0$, the gradients ∇f and ∇g are not collinear, i.e. there exists a component of $-\nabla f$ (steepest descent) in direction of $g(\mathbf{x}) = 0$

that leads to a decrease of the cost function f while staying on the constraint $g(\mathbf{x}) = 0$. Vice versa, the gradients at the optimal point $\mathbf{x}^*$

$$\nabla f(\mathbf{x}^*) = \begin{bmatrix} -2.4 \\ 1.2 \end{bmatrix}, \quad \nabla g(\mathbf{x}^*) = \begin{bmatrix} -2 \\ 1 \end{bmatrix} \quad (4.5)$$

must be collinear, i.e. $\nabla f(\mathbf{x}^*) \parallel \nabla g(\mathbf{x}^*)$, such that the cost cannot be further reduced without leaving the constraint. Mathematically speaking, the optimality condition is

$$0 = \nabla f(\mathbf{x}^*) + \lambda^* \nabla g(\mathbf{x}^*) \quad (4.6)$$

for some $\lambda^* \in \mathbb{R}$, in our case $\lambda^* = -1.2$.

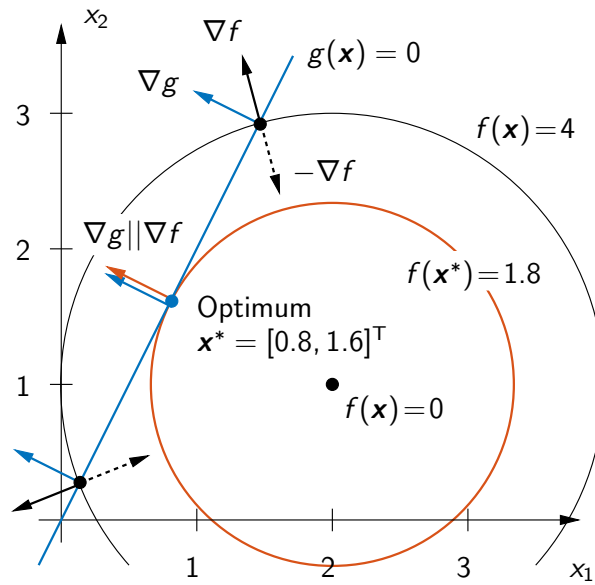


Figure 4.1: Illustration of the equality-constrained optimization problem (4.2).

The result of the last example can be generalized with the *Lagrangian function*

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x}), \quad (4.7)$$

where the equality constraint is added to the cost via the *Lagrange multiplier* λ . The collinearity of the gradients ∇f and ∇g at an optimal point $\mathbf{x}^*$ corresponds to the *stationarity condition*

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \lambda^*) = \nabla f(\mathbf{x}^*) + \lambda^* \nabla g(\mathbf{x}^*) = \mathbf{0}. \quad (4.8)$$

Together with the equality constraint $g(\mathbf{x}^*) = 0$, we have a set of equations of order $n + 1$ for the $n + 1$ unknown variables $\mathbf{x}^* \in \mathbb{R}^n$ and $\lambda^* \in \mathbb{R}$. It is, however, important to note that (4.8) is only a *necessary* and not a *sufficient* condition for optimality.

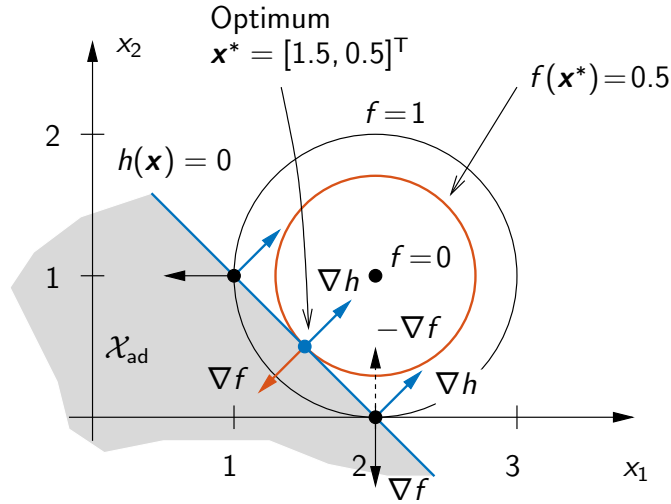


Figure 4.2: Illustration of inequality-constrained optimization problem (4.9).

4.1.2 One inequality constraint

Example 4.2 The equality constraint (1.10) of the previous example is now replaced by the inequality constraint (1.11), i.e.

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2 \quad (4.9a)$$

$$\text{s.t. } h(\mathbf{x}) = x_1 + x_2 - 2 \leq 0. \quad (4.9b)$$

Figure 4.2 shows the optimal point $\mathbf{x}^* = [1.5, 0.5]^T$ that lies on the boundary of the admissible set¹

$$\mathcal{X}_{ad} = \{\mathbf{x} \in \mathbb{R}^2 : h(\mathbf{x}) \leq 0\}.$$

The constraint (4.9b) is active, i.e. $h(\mathbf{x}^*) = 0$, and the gradients $\nabla f(\mathbf{x}^*) = [-1, -1]^T$, $\nabla h(\mathbf{x}^*) = [1, 1]^T$ are collinear at the optimal point $\mathbf{x}^*$. For all other points where the constraint (4.9b) is active, there exists a component of the gradient $-\nabla f$ in direction of $h(\mathbf{x}) = 0$ that leads to a reduction of the cost f while keeping the inequality constraint (4.9b) active, see Figure 4.2.

Similar to the equality-constrained case, an inequality constraint $h(\mathbf{x}) \leq 0$ can be added to the Lagrangian function

$$L(\mathbf{x}, \mu) = f(\mathbf{x}) + \mu h(\mathbf{x}) \quad (4.10)$$

with a Lagrange multiplier μ . The difference now is that we have to distinguish whether the constraint $h(\mathbf{x}) \leq 0$ is active or not at the optimal point $\mathbf{x}^*$. If $h(\mathbf{x}^*) < 0$ strictly holds, then $\mathbf{x}^*$ corresponds to the unconstrained case and we have

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \mu^*) = \nabla f(\mathbf{x}^*) = 0 \quad \text{with} \quad \mu^* = \mathbf{0}. \quad (4.11)$$

In case of an active constraint, $h(\mathbf{x}) = 0$ can be considered as equality constraints, i.e. the gradients ∇f and ∇h must be collinear at the optimal point $\mathbf{x}^*$. This can be expressed with the Lagrangian function (4.10) as

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \mu^*) = \nabla f(\mathbf{x}^*) + \mu^* \nabla h(\mathbf{x}^*) = 0. \quad (4.12)$$

¹ A useful geometric interpretation of an active inequality constraint $h(\mathbf{x}) \leq 0$ is that the negative gradient $-\nabla h(\mathbf{x})$ always points in the direction of the admissible set $\mathcal{X}_{ad}$.

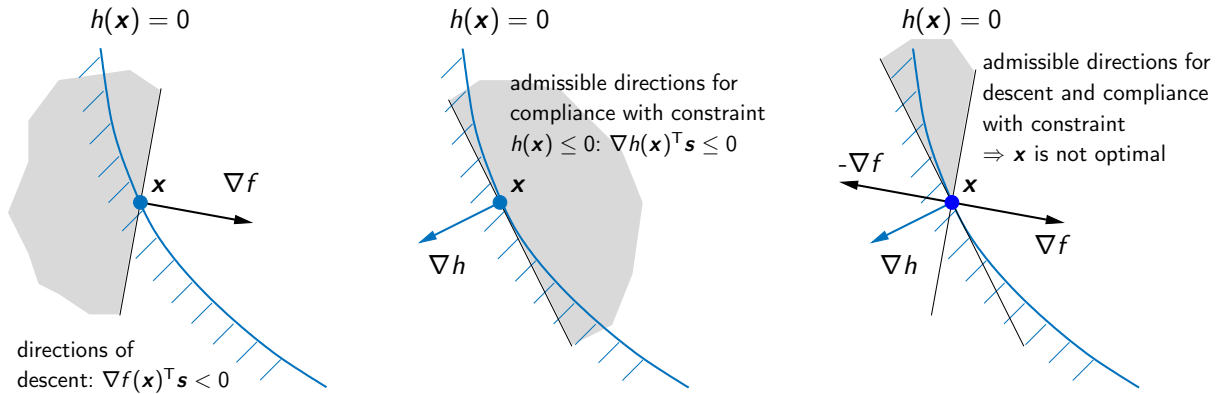


Figure 4.3: Geometric interpretation of the conditions (4.13) and (4.14) for an active inequality constraint.

However, in contrast to the equality-constrained case, the sign of the multiplier μ^* now plays a crucial role. To illustrate this further, the function $h(\mathbf{x})$ is expanded in a first-order Taylor series

$$h(\mathbf{x} + \mathbf{s}) = h(\mathbf{x}) + \nabla h(\mathbf{x})^T \mathbf{s} + \mathcal{O}(\|\mathbf{s}\|^2).$$

In case of an active constraint $h(\mathbf{x}) = 0$, all (sufficiently small) admissible directions $\mathbf{s}$ in the sense of $h(\mathbf{x} + \mathbf{s}) \leq 0$ must satisfy

$$\nabla h(\mathbf{x})^T \mathbf{s} \leq 0. \quad (4.13)$$

In addition, the cost function $f(\mathbf{x})$ is expanded in a Taylor series

$$f(\mathbf{x} + \mathbf{s}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{s} + \mathcal{O}(\|\mathbf{s}\|^2).$$

If the point $\mathbf{x}$ is *no* minimum, there exists an admissible descent direction $\mathbf{s}$ with $f(\mathbf{x} + \mathbf{s}) < f(\mathbf{x})$ if

$$\nabla f(\mathbf{x})^T \mathbf{s} < 0 \quad (4.14)$$

and (4.13) are satisfied at the same time. Vice versa, if $\mathbf{x}^*$ is a minimum point at the boundary $h(\mathbf{x}^*) = 0$, then no direction $\mathbf{s}$ is allowed to exist such that *both* conditions (4.13), (4.14) are satisfied.

This reasoning is illustrated in Figure 4.3 for a non-optimal point $\mathbf{x}$. Obviously, no admissible direction $\mathbf{s}$ exists that satisfies both inequalities (4.13) and (4.14) if $-\nabla f$ and ∇h point in the same direction, i.e.

$$-\nabla f(\mathbf{x}^*) = \mu^* \nabla h(\mathbf{x}^*) \quad \text{with} \quad \mu^* \geq 0. \quad (4.15)$$

In other words, if (4.15) holds then there exists no admissible direction $\mathbf{s}$ that leads to a reduction of the cost function $f(\mathbf{x}^* + \mathbf{s})$ without violating the active constraint $h(\mathbf{x}^*) = 0$. The discussion shows that the sign of μ^* is important. If the multiplier were negative, i.e. $\mu^* < 0$, then ∇f and ∇h would point into the same direction and a half plane of admissible directions would exist for a decrease in the cost f while satisfying the constraint $h(\mathbf{x}) \leq 0$.

The conditions for a minimum at the point $\mathbf{x}^*$ can be summarized as follows:

$$\exists \mu^* \geq 0: \quad \nabla_{\mathbf{x}} L(\mathbf{x}^*, \mu^*) = 0, \quad \mu^* h(\mathbf{x}^*) = 0. \quad (4.16)$$

The equation $\mu^* h(\mathbf{x}^*) = 0$ is called *complementary condition*. It is trivially satisfied in case of an active constraint $h(\mathbf{x}^*) = 0$ and implies $\mu^* = 0$ in case of $h(\mathbf{x}^*) < 0$.

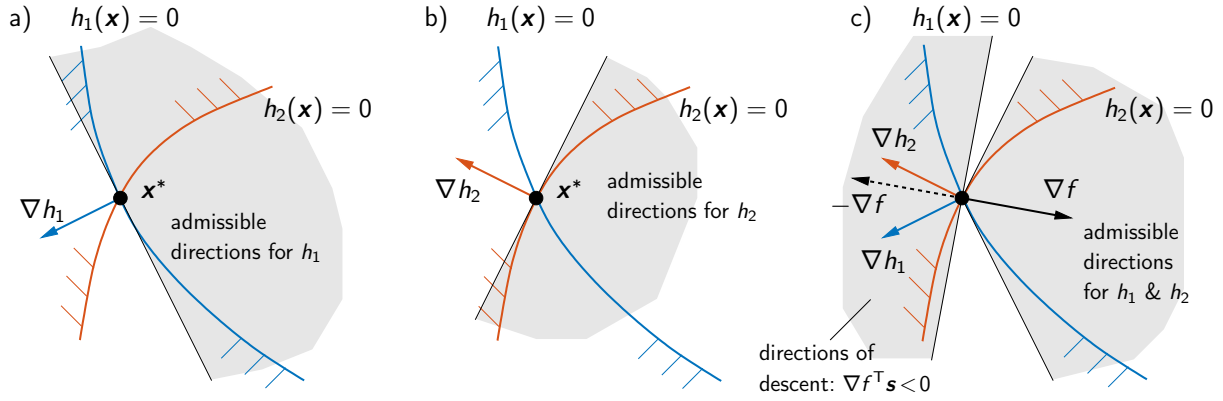


Figure 4.4: Geometric interpretation of the conditions (4.18) and (4.19) for two active inequality constraints.

4.1.3 Two inequality constraints

To further generalize the previous considerations, we consider an optimization problem with two inequality constraints

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (4.17a)$$

$$\text{s.t. } h_1(\mathbf{x}) \leq 0 \quad (4.17b)$$

$$h_2(\mathbf{x}) \leq 0 \quad (4.17c)$$

and follow the same reasoning as in the previous example of one inequality constraint.

If we consider a point $\mathbf{x}$ where both constraints are active, i.e. $h_1(\mathbf{x}) = 0$ and $h_2(\mathbf{x}) = 0$, then an admissible (sufficiently small) direction $\mathbf{s}$ must satisfy the condition (4.13) for both constraints

$$\nabla h_1(\mathbf{x})^T \mathbf{s} \leq 0, \quad \nabla h_2(\mathbf{x})^T \mathbf{s} \leq 0. \quad (4.18)$$

These conditions can be interpreted in the $\mathbb{R}^2$ -plane: all admissible directions $\mathbf{s}$ must lie in the cone defined by the tangents at the curves $h_1(\mathbf{x}) = 0$ and $h_2(\mathbf{x}) = 0$, see Figure 4.4.

If the point $\mathbf{x}$ is no minimum point, then there exist directions $\mathbf{s}$ that satisfy the descent condition (4.14)

$$\nabla f(\mathbf{x})^T \mathbf{s} < 0 \quad (4.19)$$

while complying with the constraints according to (4.18). Vice versa, at a minimum point $\mathbf{x}^*$ no direction $\mathbf{s}$ exists that satisfies both (4.18) and (4.19). This is the case if the negative gradient $-\nabla f(\mathbf{x}^*)$ points in the cone that is spanned by the gradients $\nabla h_1(\mathbf{x}^*)$ and $\nabla h_2(\mathbf{x}^*)$, see Figure 4.4c.

In other words, the negative gradient $-\nabla f(\mathbf{x}^*)$ at a minimum point $\mathbf{x}^*$ with active inequality constraints must be a *positive* linear combination of the gradients of the constraint functions, i.e.

$$\exists \mu_1^*, \mu_2^* \geq 0 : \quad -\nabla f(\mathbf{x}^*) = \mu_1^* \nabla h_1(\mathbf{x}^*) + \mu_2^* \nabla h_2(\mathbf{x}^*). \quad (4.20)$$

Similar to before, the Lagrangian function

$$L(\mathbf{x}, \boldsymbol{\mu}) = f(\mathbf{x}) + \mu_1 h_1(\mathbf{x}) + \mu_2 h_2(\mathbf{x}), \quad \boldsymbol{\mu} = [\mu_1, \mu_2]^T \quad (4.21)$$

can be used to formulate the necessary optimality conditions for a local minimum

$$\exists \mu_1^*, \mu_2^* \geq 0 : \quad \nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\mu}^*) = 0, \quad \mu_1^* h_1(\mathbf{x}^*) = 0, \quad \mu_2^* h_2(\mathbf{x}^*) = 0. \quad (4.22)$$

Both complementary conditions in (4.22) are satisfied, if the respective constraint $h_i(\mathbf{x}^*) \leq 0$ is active or if the multiplier is $\mu_i^* = 0$.

Exercise 4.1 *The optimization problem of Example 4.2 is extended by the second inequality constraint (1.12)*

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2 \quad (4.23a)$$

$$\text{s.t.} \quad h_1(\mathbf{x}) = x_1 + x_2 - 2 \leq 0 \quad (4.23b)$$

$$h_2(\mathbf{x}) = x_1^2 - x_2 \leq 0. \quad (4.23c)$$

Compute the Lagrange multipliers for the optimal solution $\mathbf{x}^* = [1, 1]^T$ and verify the sign condition (4.20) graphically.

4.2 Optimality conditions

The derivations from the previous section can be generalized to the constrained optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (4.24a)$$

$$\text{s.t.} \quad g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \quad (4.24b)$$

$$h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q \quad (4.24c)$$

with p equality constraints and q inequality constraints.

4.2.1 Constraint qualification

The equality-constrained example in Section 4.1.1 already showed that the $n + 1$ equations

$$g(\mathbf{x}) = 0, \quad \nabla_{\mathbf{x}} L(\mathbf{x}, \lambda) = \nabla f(\mathbf{x}) + \lambda \nabla g(\mathbf{x}) = \mathbf{0}$$

must be solved together for the unknowns $\mathbf{x} \in \mathbb{R}^n$ and $\lambda \in \mathbb{R}$. The same is true in the case of an active inequality constraint $h(\mathbf{x}) = 0$ in order to determine $\mathbf{x} \in \mathbb{R}^n$ and $\mu \in \mathbb{R}$ from the set of equations

$$h(\mathbf{x}) = 0, \quad \nabla_{\mathbf{x}} L(\mathbf{x}, \mu) = \nabla f(\mathbf{x}) + \mu \nabla h(\mathbf{x}) = \mathbf{0}.$$

For the general optimization problem (4.24) with p equality constraints and q inequality constraints, an additional assumption for active constraints is necessary to ensure the solvability of the optimality conditions. To this end, we require the following definition.

Definition 4.1 (Active constraint set) *The active constraint set of the optimization problem (4.24) at a point $\mathbf{x}$ is defined by*

$$\mathcal{A}(\mathbf{x}) = \{i \in \{1, \dots, q\} \mid h_i(\mathbf{x}) = 0\}. \quad (4.25)$$

The so-called *constraint qualification* requires the gradients of the equality constraints and active inequality constraints to be linearly independent, i.e.

$$\text{rank} \begin{bmatrix} \nabla \mathbf{g}(\mathbf{x}) \\ \nabla \mathbf{h}_{\mathcal{A}(\mathbf{x})}(\mathbf{x}) \end{bmatrix} = p + |\mathcal{A}(\mathbf{x})|. \quad (4.26)$$

The vector gradient notation $\nabla \mathbf{g}(\mathbf{x})$ is defined as $\nabla \mathbf{g}(\mathbf{x}) = [\nabla g_1(\mathbf{x}), \dots, \nabla g_p(\mathbf{x})]^\top$. The second term $\nabla \mathbf{h}_{\mathcal{A}(\mathbf{x})}(\mathbf{x})$ is similarly defined with the vector function $\mathbf{h}_{\mathcal{A}(\mathbf{x})}$ consisting of the active inequality constraints at point $\mathbf{x}$

$$\mathbf{h}_{\mathcal{A}(\mathbf{x})}(\mathbf{x}) = [h_i(\mathbf{x})]_{i \in \mathcal{A}(\mathbf{x})}. \quad (4.27)$$

The rank condition (4.26) also means that at most $n - p$ inequality constraints can be active at point $\mathbf{x}$. Otherwise, the set of equations consisting of $\mathbf{g}(\mathbf{x}) = 0$ and $\mathbf{h}_{\mathcal{A}(\mathbf{x})}(\mathbf{x}) = 0$ would be overdetermined.

The constraint qualification (4.26) is the most common one in optimization. However, note that (4.26) is only a *sufficient* condition and might be too conservative in some cases. Further constraint qualifications can be found in the literature.

4.2.2 First-order optimality conditions

The Lagrangian function was already introduced for the examples in Section 4.1. We now extend the concept to the general optimization problem (4.24).

Definition 4.2 (Lagrangian function) *The Lagrangian function of the optimization problem (4.24) is defined as*

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathbf{x}) + \sum_{i=1}^p \lambda_i g_i(\mathbf{x}) + \sum_{i=1}^q \mu_i h_i(\mathbf{x}) \quad (4.28)$$

with the Lagrange multipliers $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_p]^\top$ and $\boldsymbol{\mu} = [\mu_1, \dots, \mu_q]^\top$.

We can now state the necessary first-order optimality conditions for the optimization problem (4.24) as follows:

Theorem 4.1 (Necessary first-order optimality conditions) *Let $\mathbf{x}^*$ be a local solution of the optimization problem (4.24) and let the constraint qualification (4.26) be satisfied at point $\mathbf{x}^*$. If the functions f , g_i , and h_i are continuously differentiable, then there exist Lagrange multipliers $\boldsymbol{\lambda}^* = [\lambda_1^*, \dots, \lambda_p^*]^\top$ and $\boldsymbol{\mu}^* = [\mu_1^*, \dots, \mu_q^*]^\top$ such that the following conditions are satisfied*

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) = \mathbf{0} \quad (4.29a)$$

$$g_i(\mathbf{x}^*) = 0, \quad i = 1, \dots, p \quad (4.29b)$$

$$h_i(\mathbf{x}^*) \leq 0, \quad i = 1, \dots, q \quad (4.29c)$$

$$\mu_i^* \geq 0, \quad i = 1, \dots, q \quad (4.29d)$$

$$\mu_i^* h_i(\mathbf{x}^*) = 0, \quad i = 1, \dots, q. \quad (4.29e)$$

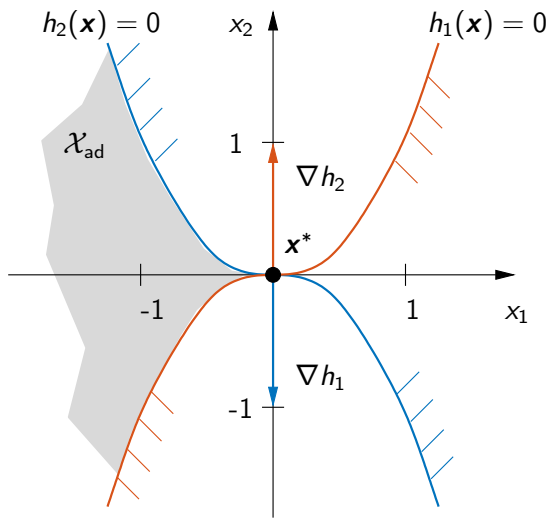


Figure 4.5: Illustration of problem (4.30) in Example 4.3.

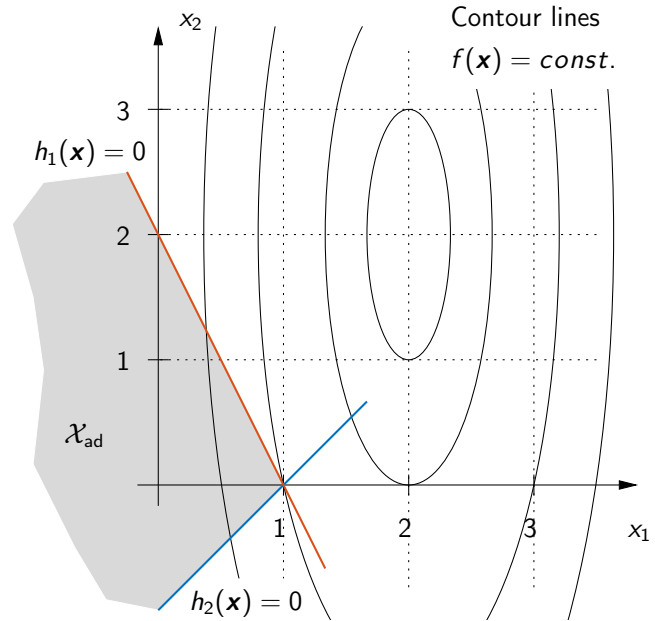


Figure 4.6: Contour lines and admissible set in Exercise 4.2.

The conditions (4.29) are known as *Karush-Kuhn-Tucker conditions* or simply *KKT conditions*. The *complementary condition* implies that the i -th inequality constraint is either active, i.e. $h_i(\mathbf{x}^*) = 0$, or $\mu_i^* = 0$ holds. The optimality conditions in Theorem 4.1 are only valid if the constraint qualification (4.26) is satisfied at $\mathbf{x}^*$. Otherwise, the KKT conditions (4.29) do not apply as the following example shows.

Example 4.3 Consider the problem

$$\min_{\mathbf{x} \in \mathbb{R}^2} -x_1 \quad (4.30a)$$

$$\text{s.t. } h_1(\mathbf{x}) = x_1^3 - x_2 \leq 0 \quad (4.30b)$$

$$h_2(\mathbf{x}) = x_1^3 + x_2 \leq 0. \quad (4.30c)$$

Figure 4.5 shows the admissible set $\mathcal{X}_{ad}$ at the optimal point $\mathbf{x}^* = [0, 0]^T$ at which both constraints are active. The gradients of the cost function f and the constraints at point $\mathbf{x}^*$ are

$$\nabla f(\mathbf{x}^*) = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad \nabla h_1(\mathbf{x}^*) = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \nabla h_2(\mathbf{x}^*) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (4.31)$$

The KKT conditions (4.29a) for this problem read

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\mu}^*) = \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \mu_1^* \begin{bmatrix} 0 \\ -1 \end{bmatrix} + \mu_2^* \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \mathbf{0}. \quad (4.32)$$

This set of equations has no solution because the constraint qualification (4.26) is violated at the optimal point $\mathbf{x}^*$.

Exercise 4.2 Compute the optimal solution $\mathbf{x}^*$ for the optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) = (x_1 - 2)^2 + \left(\frac{x_2 - 2}{3} \right)^2 \quad (4.33a)$$

$$\text{s.t. } h_1(\mathbf{x}) = 2x_1 + x_2 - 2 \leq 0 \quad (4.33b)$$

$$h_2(\mathbf{x}) = x_1 - x_2 - 1 \leq 0 \quad (4.33c)$$

by investigating the KKT conditions (4.29) for the different cases of active/inactive constraints. Mark the different candidate points in Figure 4.6.

Exercise 4.3 The following optimization problem is given in dependence of the parameter s

$$\min_{\mathbf{x} \in \mathbb{R}^n} \left(x_1 - \frac{3}{2} \right)^2 + (x_2 - s)^4 \quad \text{s.t.} \quad \begin{bmatrix} 1 - x_1 - x_2 \\ 1 - x_1 + x_2 \\ 1 + x_1 - x_2 \\ 1 + x_1 + x_2 \end{bmatrix} \geq \mathbf{0}. \quad (4.34a)$$

Determine the values of s for which the point $\mathbf{x}^* = [1, 0]^T$ satisfies the KKT conditions (4.29).

4.2.3 Second-order optimality conditions

Theorem 4.2 (Necessary second-order optimality conditions) Let $\mathbf{x}^*$ be a local solution of the optimization problem (4.24) and let the constraint qualification (4.26) be satisfied at point $\mathbf{x}^*$ such that the KKT conditions (4.29) hold with associated Lagrange multipliers $\boldsymbol{\lambda}^*$ and $\boldsymbol{\mu}^*$. If f , g_i , and h_i are twice continuously differentiable, then

$$\mathbf{s}^T \nabla_{\mathbf{x}\mathbf{x}}^2 L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) \mathbf{s} \geq 0 \quad \forall \mathbf{s} \in \mathcal{C}(\mathbf{x}^*, \boldsymbol{\mu}^*) \quad (4.35)$$

with

$$\begin{aligned} \mathcal{C}(\mathbf{x}^*, \boldsymbol{\mu}^*) = \{ \mathbf{s} \in \mathbb{R}^n : & \nabla g_i(\mathbf{x}^*)^T \mathbf{s} = 0, \quad i = 1, \dots, p \\ & \nabla h_i(\mathbf{x}^*)^T \mathbf{s} = 0, \quad i \in \mathcal{A}(\mathbf{x}^*), \mu_i^* > 0 \\ & \nabla h_i(\mathbf{x}^*)^T \mathbf{s} \leq 0, \quad i \in \mathcal{A}(\mathbf{x}^*), \mu_i^* = 0 \}. \end{aligned} \quad (4.36)$$

The condition (4.35) has similarity with the unconstrained case (Theorem 3.1), where the Hessian $\nabla^2 f(\mathbf{x}^*)$ at point $\mathbf{x}^*$ has to be positive semidefinite, i.e. $\mathbf{s}^T \nabla^2 f(\mathbf{x}^*) \mathbf{s} \geq 0$ for all $\mathbf{s} \in \mathbb{R}^n$, $\mathbf{s} \neq \mathbf{0}$. In the constrained case, the set $\mathcal{C}(\mathbf{x}^*, \boldsymbol{\mu}^*)$ only contains admissible directions $\mathbf{s}$ that comply with the constraints (4.24b) and (4.24c). In particular, only directions $\mathbf{s}$ are considered for which the gradients do not provide enough information whether the cost decreases or increases in direction of $\mathbf{s}$.²

² For active inequality constraints with $\mu_i^* > 0$, all admissible and sufficiently small directions $\mathbf{s}$ with $\nabla h_i(\mathbf{x}^*)^T \mathbf{s} < 0$ lead to an increase of the cost $f(\mathbf{x}^* + \mathbf{s})$, see Section 4.2.4.

Theorem 4.3 (Sufficient optimality conditions) *Let $\mathbf{x}^*$ be a feasible point of the optimization problem (4.24) with associated Lagrange multipliers $\boldsymbol{\lambda}^*$ and $\boldsymbol{\mu}^*$ such that the KKT conditions (4.29) are satisfied. If f , g_i , and h_i are twice continuously differentiable and*

$$\mathbf{s}^\top \nabla_{\mathbf{x}\mathbf{x}}^2 L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) \mathbf{s} > 0 \quad \forall \mathbf{s} \in \mathcal{C}(\mathbf{x}^*, \boldsymbol{\mu}^*), \mathbf{s} \neq \mathbf{0} \quad (4.37)$$

with $\mathcal{C}(\mathbf{x}^, \boldsymbol{\mu}^*)$ in (4.36), then $\mathbf{x}^*$ is a strict local minimum point.*

If the Hessian $\nabla_{\mathbf{x}\mathbf{x}}^2 L$ is positive definite, i.e.

$$\mathbf{s}^\top \nabla_{\mathbf{x}\mathbf{x}}^2 L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) \mathbf{s} > 0 \quad \forall \mathbf{s} \in \mathbb{R}^n, \mathbf{s} \neq \mathbf{0} \quad (4.38)$$

then the sufficient condition (4.37) is always satisfied without the necessity to compute the set $\mathcal{C}(\mathbf{x}^*, \boldsymbol{\mu}^*)$. This is in particular the case for quadratic problems with positive definite Hessian $\nabla^2 f(\mathbf{x})$ of the cost function.

For convex problems, the evaluation of the second-order optimality conditions can be avoided, because the KKT conditions (4.29) are necessary and sufficient in this case.

Theorem 4.4 (Sufficient optimality conditions for convex problems) *Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \dots, q$ are convex and continuously differentiable functions and that $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, \dots, p$ is continuously differentiable and linear. If a point $\mathbf{x}^*$ with Lagrange multipliers $\boldsymbol{\lambda}^*$ and $\boldsymbol{\mu}^*$ satisfies the KKT conditions (4.29), then $\mathbf{x}^*$ is a global minimum point of the optimization problem (4.24).*

Exercise 4.4 Verify the sufficient optimality conditions of Theorem 4.3 and Theorem 4.4 for the problem (4.33).

4.2.4 Interpretation of Lagrange multipliers

The meaning of the Lagrange multipliers was motivated by the introductory examples in Section 4.1. There exists another important interpretation that is often used in post-optimization analysis. To this end, we consider an optimization problem with a single equality constraint

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g(\mathbf{x}) = 0. \end{aligned}$$

The stationarity condition for an optimal solution $\mathbf{x}^*$ reads

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \lambda^*) = \nabla f(\mathbf{x}^*) + \lambda^* \nabla g(\mathbf{x}^*) = \mathbf{0}. \quad (4.39)$$

To investigate the relation between the Lagrange multiplier and the optimal solution $\mathbf{x}^*$, we consider a small perturbation of the constraint

$$g(\mathbf{x}) = \varepsilon \quad \Rightarrow \quad \bar{g}(\mathbf{x}(\varepsilon), \varepsilon) = g(\mathbf{x}(\varepsilon)) - \varepsilon = 0. \quad (4.40)$$

The variables $\mathbf{x}(\varepsilon)$ now depend on ε with the original solution $\mathbf{x}^* = \mathbf{x}(0)$ for $\varepsilon = 0$. Since $\bar{g}(\mathbf{x}(\varepsilon), \varepsilon) = 0$ must hold for all ε , we can take the total derivative

$$\frac{\partial \bar{g}}{\partial \varepsilon} + \nabla_{\mathbf{x}} \bar{g}(\mathbf{x}(\varepsilon), \varepsilon)^T \frac{d\mathbf{x}}{d\varepsilon} = 0 \quad \Rightarrow \quad \nabla_{\mathbf{x}} \bar{g}(\mathbf{x}(\varepsilon), \varepsilon)^T \frac{d\mathbf{x}}{d\varepsilon} = 1 \quad (4.41)$$

with $\partial \bar{g} / \partial \varepsilon = -1$. Of particular interest is the effect of ε on the minimal cost $f(\mathbf{x}^*)$, which can be expressed with $\bar{f}(\varepsilon) = f(\mathbf{x}(\varepsilon))$ as follows

$$\left. \frac{d\bar{f}}{d\varepsilon} \right|_{\varepsilon=0} = \nabla f(\mathbf{x}^*)^T \left. \frac{d\mathbf{x}}{d\varepsilon} \right|_{\varepsilon=0} \stackrel{(4.39)}{=} -\lambda^* \nabla g(\mathbf{x}^*)^T \left. \frac{d\mathbf{x}}{d\varepsilon} \right|_{\varepsilon=0} \stackrel{(4.41)}{\Rightarrow} \boxed{\left. \frac{d\bar{f}}{d\varepsilon} \right|_{\varepsilon=0} = -\lambda^*}. \quad (4.42)$$

Thus, the multiplier λ^* is a *direct measure for the rate of change (sensitivity)* of the optimal cost w.r.t. a variation of the equality constraint $g(\mathbf{x}^*) = 0$. Similar reasoning can be applied to an active inequality constraint

$$h(\mathbf{x}) \leq \varepsilon \quad \Rightarrow \quad \boxed{\left. \frac{d\bar{f}}{d\varepsilon} \right|_{\varepsilon=0} = -\mu^*} \quad (4.43)$$

with the optimal multiplier μ^* for $h(\mathbf{x}^*) = 0$. The interpretation is the same as in case of the equality constraint: the Lagrange multiplier μ^* of an active inequality constraint $h(\mathbf{x}) \leq 0$ at the optimal point $\mathbf{x}^*$ is the sensitivity of the optimal cost $f(\mathbf{x}^*)$ w.r.t. this constraint.

A large value of λ^* or μ^* thus indicates that a relaxation of this constraint can lead to a significant decrease of the cost function. This interpretation of the multipliers can be directly extended to the general case (4.24) with p equality constraints and q inequality constraints.

The equation (4.43) also helps to explain the sign condition (4.29d) of the KKT conditions. If a multiplier at an optimal point were negative, i.e. $\mu^* < 0$, this would imply

$$\left. \frac{d\bar{f}}{d\varepsilon} \right|_{\varepsilon=0} > 0$$

and thus a reduction of the cost function $f(\mathbf{x}^*)$ could be achieved by leaving the active constraint into the interior of the admissible set (i.e. for $\varepsilon < 0$). In this case, however, $\mathbf{x}^*$ would not be the optimal solution, which implies that the multiplier must be non-negative ($\mu^* \geq 0$).

Exercise 4.5 Interpret the multiplier values graphically for the problem (4.33).

4.3 Linear optimization

Linear optimization (also known as *linear programming*) is a special case of the general non-linear problem (4.1) with linear cost function and constraints. Linear programming is not only popular because of its wide range of applications such as production planning or transportation problems but also due to its simplicity and the existence of reliable algorithms. In particular, linear problems are always convex, see Theorem 2.5 and 4.4. Hence, every local solution is also a global one.

4.3.1 Standard form

Most numerical algorithms of linear programming are based on the standard form of a linear optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \quad (4.44a)$$

$$\text{s.t. } \mathbf{Ax} = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{p \times n}, \quad \mathbf{b} \in \mathbb{R}^p \quad (4.44b)$$

$$\mathbf{x} \geq \mathbf{0}. \quad (4.44c)$$

Every linear optimization problem can be transformed into the standard form. For instance, consider the linear problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \quad (4.45a)$$

$$\text{s.t. } \mathbf{Ax} \leq \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{q \times n}, \quad \mathbf{b} \in \mathbb{R}^q. \quad (4.45b)$$

The inequality constraint is transformed into equality constraints and additional non-negativity constraints of the form (4.44c) by means of *slack variables* $\mathbf{z}$

$$\mathbf{Ax} \leq \mathbf{b} \Rightarrow \mathbf{Ax} + \mathbf{z} = \mathbf{b}, \quad \mathbf{z} \geq \mathbf{0}. \quad (4.46)$$

Since the variables $\mathbf{x}$ so far are not constrained to be non-negative, we split $\mathbf{x}$ in non-negative and non-positive parts

$$\mathbf{x} = \mathbf{x}^+ - \mathbf{x}^-, \quad \mathbf{x}^-, \mathbf{x}^+ \geq \mathbf{0}. \quad (4.47)$$

In summary, the problem (4.45) is transformed in the standard form (4.44)

$$\min_{\bar{\mathbf{x}} \in \mathbb{R}^{2n+q}} [\mathbf{c}^T, -\mathbf{c}^T, \mathbf{0}^T] \bar{\mathbf{x}} \quad (4.48a)$$

$$\text{s.t. } [\mathbf{A}, -\mathbf{A}, \mathbf{I}] \bar{\mathbf{x}} = \mathbf{b}, \quad \bar{\mathbf{x}} = \begin{bmatrix} \mathbf{x}^+ \\ \mathbf{x}^- \\ \mathbf{z} \end{bmatrix} \quad (4.48b)$$

$$\bar{\mathbf{x}} \geq \mathbf{0} \quad (4.48c)$$

with the extended optimization vector $\bar{\mathbf{x}}$.

Exercise 4.6 Transform the following linear problem into the standard form (4.44)

$$\min_{\mathbf{x} \in \mathbb{R}^3} -x_1 + 2x_2 + x_3 \quad (4.49a)$$

$$\text{s.t. } x_1 + x_3 = 1 \quad (4.49b)$$

$$-x_1 + x_2 + x_3 \leq 1 \quad (4.49c)$$

$$x_2, x_3 \geq 0. \quad (4.49d)$$

4.3.2 Optimality conditions

In what follows, we assume that the optimization problem is already given in the standard form (4.44). With the Lagrangian function

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{Ax} - \mathbf{b}) + \boldsymbol{\mu}^T (-\mathbf{x})$$

with $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_p]^\top$, $\boldsymbol{\mu} = [\mu_1, \dots, \mu_n]^\top$ and its gradient

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda} - \boldsymbol{\mu},$$

the optimality conditions for the linear problem (4.44) are a special case of the KKT conditions (4.29)

$$\mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda}^* - \boldsymbol{\mu}^* = \mathbf{0} \quad (4.50a)$$

$$\mathbf{A}\mathbf{x}^* - \mathbf{b} = \mathbf{0} \quad (4.50b)$$

$$\mathbf{x}^* \geq \mathbf{0} \quad (4.50c)$$

$$\mu_i^* \geq 0, \quad i = 1, \dots, n \quad (4.50d)$$

$$\mu_i^*(-x_i^*) = \mu_i^* x_i^* = 0, \quad i = 1, \dots, n. \quad (4.50e)$$

Due to the convexity of the linear problem (4.44), the optimality conditions (4.50) are necessary and sufficient for a global minimum. This also means that the second-order optimality conditions do not need to be evaluated, which do not provide further insight anyway, since the Hessian matrix $\nabla_{\mathbf{x}\mathbf{x}} L = \mathbf{0}$ is the zero matrix.

4.3.3 Simplex method

The *simplex method* was developed by *George B. Danzig* in 1948 and marked the start of modern numerical optimization together with the advent of digital computers. The simplex algorithm exploits the geometric property that the admissible set

$$\mathcal{X}_{ad} = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\} \quad (4.51)$$

of linear optimization problems forms a convex *polyhedron* with a finite number of vertices, see Figure 4.7. The optimal solution $\mathbf{x}^*$ – if it exists – always lies on the boundary of $\mathcal{X}_{ad}$ and, in particular, on a vertex of the polyhedron if the solution $\mathbf{x}^*$ is unique. The simplex method starts from an initial vertex of the polyhedron and then proceeds along the edges towards the optimal solution $\mathbf{x}^*$.

Definition 4.3 (Simplex method – Notations)

Basis and basis matrix: Let $\mathcal{B}$ be a subset of $\{1, \dots, n\}$ with p elements. If the $(p \times p)$ -matrix $\mathbf{B} = [\mathbf{A}_i]_{i \in \mathcal{B}}$ is regular, then $\mathcal{B}$ is a basis and $\mathbf{B}$ is the basis matrix.³

Basic variables: $\mathbf{x}_B = [x_i]_{i \in \mathcal{B}} = \mathbf{B}^{-1} \mathbf{b}$

Non-basic variables: $\mathbf{x}_N = [x_i]_{i \in \mathcal{N}} = \mathbf{0}$ with $\mathcal{N} = \{1, \dots, n\} \setminus \mathcal{B}$

Basic feasible solution: $\mathbf{x}$ with $\mathbf{x}_B = \mathbf{B}^{-1} \mathbf{b} \geq \mathbf{0}$ and $\mathbf{x}_N = \mathbf{0}$.

Non-degenerate

basic feasible solution: $\mathbf{x}$ with $\mathbf{x}_B = \mathbf{B}^{-1} \mathbf{b} > \mathbf{0}$ and $\mathbf{x}_N = \mathbf{0}$.

The equality constraints (4.44b) with $p < n$ form an underdetermined set of equations from which p elements, the basic variables $\mathbf{x}_B$, can be computed. The remaining $n - p$ elements are the non-basic variables that are set to zero, i.e. $\mathbf{x}_N = \mathbf{0}$.

³ The vector $\mathbf{A}_i$ denotes the i -th column of the matrix $\mathbf{A}$.

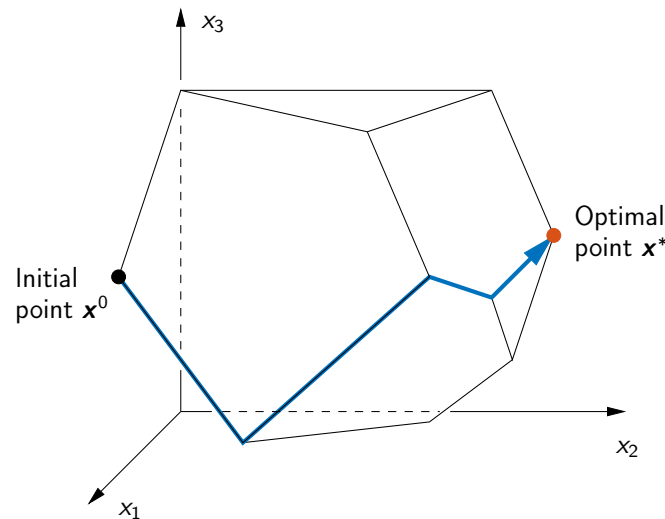


Figure 4.7: Illustration of the simplex method.

Under the assumption that the $(p \times n)$ -matrix $\mathbf{A}$ has row rank p , it is always possible to determine a basis $\mathcal{B}$ such that the associated basis matrix $\mathbf{B} = [\mathbf{A}_i]_{i \in \mathcal{B}}$ is regular. The equality constraints (4.44b) can then be split in $\mathbf{x}_B$ and $\mathbf{x}_N$

$$\mathbf{b} = \mathbf{B}\mathbf{x}_B + \mathbf{N}\mathbf{x}_N, \quad \mathbf{N} = [\mathbf{A}_i]_{i \in \mathcal{N}}, \quad (4.52)$$

in order to compute $\mathbf{x}_B$ by means of the regular basis matrix $\mathbf{B}$

$$\mathbf{x}_B = \mathbf{B}^{-1}(\mathbf{b} - \mathbf{N}\mathbf{x}_N) = \mathbf{B}^{-1}\mathbf{b} \quad \text{with} \quad \mathbf{x}_N = \mathbf{0}. \quad (4.53)$$

If $\mathbf{x}_B \geq \mathbf{0}$, then the basic variables $\mathbf{x}_B$ and $\mathbf{x}_N = \mathbf{0}$ form a basic feasible solution. The initialization phase of the simplex method consists in finding a basic solution with $\mathbf{x}_B \geq \mathbf{0}$, which is addressed at the end of the section. Once this initial point $\mathbf{x}$ is determined, the subsequent iterations of the simplex method always produce basic feasible solutions.

Example 4.4 We consider the example

$$\min_{\mathbf{x} \in \mathbb{R}^4} \quad \mathbf{c}^T \mathbf{x}, \quad \mathbf{c}^T = [1, 2, 3, 4] \quad (4.54a)$$

$$\text{s.t.} \quad \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & -3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ \frac{1}{2} \end{bmatrix} \quad (4.54b)$$

$$\mathbf{x} \geq \mathbf{0}. \quad (4.54c)$$

The following table shows various bases $\mathcal{B}$ with no basic feasible solution existing in the last case. The choice $\{1, 3\}$ does not form a basis because $\mathbf{B}$ would be singular.

Basis $\mathcal{B}$	Basis matrix $\mathbf{B}$	$\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$	$\mathbf{x}_N = \mathbf{0}$	Basic feasible solution?
$\mathcal{B} = \{1, 2\}$	$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$	$x_1 = 0.5$ $x_2 = 0.5$	$x_3 = 0$ $x_4 = 0$	yes
$\mathcal{B} = \{1, 4\}$	$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 1 & -3 \end{bmatrix}$	$x_1 = 0.875$ $x_4 = 0.125$	$x_2 = 0$ $x_3 = 0$	yes
$\mathcal{B} = \{2, 4\}$	$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & -3 \end{bmatrix}$	$x_2 = 7/6$ $x_4 = -1/6$	$x_1 = 0$ $x_3 = 0$	no

The principle approach of the simplex method consists in iteratively verifying the optimality conditions (4.50) and the subsequent determination of a new basic feasible solution $\mathbf{x}^+$ with $f(\mathbf{x}^+) < f(\mathbf{x})$.

Step 1: Verification of optimality conditions (4.50)

A basic feasible solution (4.53), i.e. $\mathbf{x}_B \geq \mathbf{0}$ and $\mathbf{x}_N = \mathbf{0}$, already satisfies the KKT condition (4.50b) and (4.50c). The stationarity condition (4.50a) is separated in terms of the basis $\mathcal{B}$ and non-basis $\mathcal{N}$

$$\begin{bmatrix} \mathbf{c}_B \\ \mathbf{c}_N \end{bmatrix} + \begin{bmatrix} \mathbf{B}^T \\ \mathbf{N}^T \end{bmatrix} \boldsymbol{\lambda} = \begin{bmatrix} \boldsymbol{\mu}_B \\ \boldsymbol{\mu}_N \end{bmatrix}. \quad (4.55)$$

The complementarity condition is already satisfied for $\boldsymbol{\mu}_N$ (due to $\mathbf{x}_N = \mathbf{0}$) and for the basic variables we set the multipliers to $\boldsymbol{\mu}_B = \mathbf{0}$. The remaining vectors $\boldsymbol{\lambda}$ and $\boldsymbol{\mu}_N$ in (4.55) become

$$\boldsymbol{\lambda} = -(\mathbf{B}^T)^{-1} \mathbf{c}_B, \quad \boldsymbol{\mu}_N = \mathbf{c}_N + \mathbf{N}^T \boldsymbol{\lambda}. \quad (4.56)$$

The last remaining KKT condition is $\boldsymbol{\mu}_N \geq \mathbf{0}$. If this condition is satisfied by (4.56), the basic solution $(\mathbf{x}_B, \mathbf{x}_N)$ is the optimal solution $\mathbf{x}^*$ and the algorithm terminates.

Step 2: Determination of new basic variable

If one or more multipliers $\mu_i, i \in \mathcal{N}$ are negative, then $\mathbf{x}$ is not the optimal solution. In this case, the simplex method constructs a new basic feasible solution $(\mathbf{x}_B^+, \mathbf{x}_N^+)$ by swapping two elements of $\mathbf{x}_B$ and $\mathbf{x}_N$ with the indices $r \in \mathcal{B}$ and $s \in \mathcal{N}$. We set $x_r^+ = 0$ and allow x_s^+ to rise in value such that

$$\mathbf{x}_B^+ = [\dots, x_r^+ = 0, \dots]^T, \quad \mathbf{x}_N^+ = [\dots, x_s^+ \geq 0, \dots]^T. \quad (4.57)$$

At first, an appropriate element x_s with $s \in \mathcal{N}$ and $\mu_s < 0$ must be determined. A reasonable choice is to pick the element with the most negative multiplier μ_i corresponding to the strongest violation of the optimality conditions $\boldsymbol{\mu}_N \geq \mathbf{0}$, i.e.

$$\mu_s = \min_{i \in \mathcal{N} | \mu_i < 0} \mu_i \quad \text{with associated index } s \in \mathcal{N}. \quad (4.58)$$

Step 3: Determination of new non-basic variable

In order to choose the index r of the new non-basic variable x_r^+ from the previous basic variables $\mathbf{x}_B$, we have to determine the value of x_s^+ . A starting point in this regard is the equality constraint (4.52) that must be satisfied for $\mathbf{x}_B^+$ and $\mathbf{x}_N^+$. With (4.53) we get for the new basic variable

$$\begin{aligned} \mathbf{x}_B^+ &= \mathbf{B}^{-1}(\mathbf{b} - \mathbf{N}\mathbf{x}_N^+) \\ &= \mathbf{x}_B - \mathbf{B}^{-1}\mathbf{A}_s x_s^+ \quad \text{with} \quad \mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b} \\ &= \mathbf{x}_B - \mathbf{d} x_s^+ \quad \mathbf{d} = \mathbf{B}^{-1}\mathbf{A}_s, \end{aligned} \quad (4.59)$$

where $\mathbf{A}_s$ denotes the s -th column of the matrix $\mathbf{A}$. The vector $\mathbf{d}$ corresponds to the direction of change of the previous basic variable $\mathbf{x}_B$ if x_s^+ is increased.

In particular, if $d_i > 0$ for an element of $\mathbf{d}$, then an increase of x_s^+ leads to a reduction of the respective element $x_{B,i}^+$ of $\mathbf{x}_B^+$. This means that x_s^+ can only be increased until an element of

Example 4.5 We consider again problem (4.54). The simplex method is initialized with

$$\begin{aligned}\mathcal{B} &= \{1, 2\}, \quad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{x}_B = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}, \quad \mathbf{c}_B = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \\ \mathcal{N} &= \{3, 4\}, \quad \mathbf{N} = \begin{bmatrix} 1 & 1 \\ 1 & -3 \end{bmatrix}, \quad \mathbf{x}_N = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{c}_N = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.\end{aligned}$$

The evaluation of the KKT conditions according to (4.56) yields

$$\boldsymbol{\lambda} = -(\mathbf{B}^\top)^{-1} \mathbf{c}_B = \begin{bmatrix} -2 \\ 1 \end{bmatrix}, \quad \boldsymbol{\mu}_N = \mathbf{c}_N + \mathbf{N}^\top \boldsymbol{\lambda} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}.$$

Obviously, the point is not optimal due to $\mu_{N,2} = \mu_4 < 0$. One step of the simplex method leads to

$$(4.58): \quad s = 4, \quad \mu_s = -1, \quad \mathbf{d} = \mathbf{B}^{-1} \mathbf{A}_4 = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -3 \end{bmatrix} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$$

$$(4.60): \quad r = 2, \quad x_s^+ = \frac{x_{B,2}}{d_2} = 0.125$$

$$(4.59): \quad \mathbf{x}_B^+ = \mathbf{x}_B - \mathbf{d} x_s^+ = \begin{bmatrix} 0.875 \\ 0 \end{bmatrix} = \begin{bmatrix} x_1^+ \\ x_2^+ \end{bmatrix}, \quad \mathbf{x}_N^+ = \begin{bmatrix} 0 \\ x_s^+ \end{bmatrix} = \begin{bmatrix} 0 \\ 0.125 \end{bmatrix} = \begin{bmatrix} x_3^+ \\ x_4^+ \end{bmatrix}.$$

For the next simplex iteration, the elements of $\mathcal{B}$ and $\mathcal{N}$ with the indices $r \in \mathcal{B}$ and $s \in \mathcal{N}$ are swapped, leading to the new basic feasible solution

$$\begin{aligned}\mathcal{B} &= \{1, 4\}, \quad \mathbf{B} = \begin{bmatrix} 1 & 1 \\ 1 & -3 \end{bmatrix}, \quad \mathbf{x}_B = \begin{bmatrix} 0.875 \\ 0.125 \end{bmatrix}, \quad \mathbf{c}_B = \begin{bmatrix} 1 \\ 4 \end{bmatrix} \\ \mathcal{N} &= \{3, 2\}, \quad \mathbf{N} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{x}_N = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{c}_N = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.\end{aligned}$$

The entries in $\mathcal{N}$, $\mathbf{N}$, $\mathbf{x}_B$, and $\mathbf{c}_N$ can be reordered in ascending order if desired. The multipliers are again computed with (4.56)

$$\boldsymbol{\lambda} = -(\mathbf{B}^\top)^{-1} \mathbf{c}_B = \begin{bmatrix} -1.75 \\ 0.75 \end{bmatrix}, \quad \boldsymbol{\mu}_N = \mathbf{c}_N + \mathbf{N}^\top \boldsymbol{\lambda} = \begin{bmatrix} 2 \\ 0.25 \end{bmatrix} > \mathbf{0}.$$

Both entries of $\boldsymbol{\mu}_N = [\mu_{N,1}, \mu_{N,2}]^\top = [\mu_3, \mu_2]^\top$ are positive, which shows that the point $\mathbf{x}^* = [0.875, 0, 0, 0.125]^\top$ is the optimal solution.

Initialization phase

If a basic feasible initial point cannot easily be found, the original problem (4.44) can be modified by introducing additional variables $\mathbf{z}$ along with the auxiliary optimization problem

$$\begin{aligned}\min_{(\mathbf{x}, \mathbf{z}) \in \mathbb{R}^{n+p}} \quad & \sum_{i=1}^p z_i \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} + \mathbf{E}\mathbf{z} = \mathbf{b} \quad \text{with} \quad \mathbf{E} = \text{diag}(\mathbf{E}_{11}, \dots, \mathbf{E}_{pp}), \quad \mathbf{E}_{ii} = \begin{cases} 1 & \text{if } b_i \geq 0 \\ -1 & \text{if } b_i < 0 \end{cases} \\ & \mathbf{x} \geq \mathbf{0}, \quad \mathbf{z} \geq \mathbf{0}. \end{aligned} \quad (4.61)$$

A basic feasible solution for this problem is

$$\mathbf{x} = \mathbf{0}, \quad z_i = |b_i|, \quad i = 1, \dots, p, \quad (4.62)$$

which shows that the simplex method can always be applied to the modified problem (4.61) with the initialization (4.62).

If the simplex method terminates with an optimal solution $\mathbf{x}^*$ and $\mathbf{z}^* = \mathbf{0}$, then $\mathbf{x}^*$ is a basic feasible solution for the original problem (4.44). The basis $\mathcal{B}^*$ and the basis matrix $\mathbf{B}^*$ of the last iteration can be used to initialize the first simplex iteration for the original problem (4.44). If not all elements z_i^* of $\mathbf{z}^*$ are identical zero, then the original problem (4.44) has no solution.

4.3.4 Interior point method

A drawback of the simplex method is that the maximum number of iterations grows exponentially with the problem size. Although the performance is usually better in practice, the number of iterations nevertheless can grow significantly for complex or high-dimensional problems. An alternative are *interior point methods* that basically go back to *Karmarkar's projection algorithm* from 1984. Interior point methods have a polynomial complexity w.r.t. the problem size n in contrast to the exponential complexity of the simplex method, which makes them particularly favorable for large-scale problems.

General approach

In principle, the KKT conditions (4.50) could be solved directly with a Newton method to determine an optimal point $\mathbf{x}^*$. This idea, however, would result in a very slow convergence as the step size of the Newton method would tend to very small values in order to satisfy the constraints $x_i \geq 0$ and $\mu_i \geq 0$ in each iteration.

The basic idea of interior point methods is to relax the complementary condition (4.50e) by a *barrier parameter* τ , i.e. $x_i \mu_i = \tau$. The reason for this designation will become clear later on. In vector notation, we get the nonlinear set of equations

$$\mathbf{F}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \begin{bmatrix} \mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda} - \boldsymbol{\mu} \\ \mathbf{A}\mathbf{x} - \mathbf{b} \\ \mathbf{X}\mathbf{M}\mathbf{e} \end{bmatrix} - \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \tau \mathbf{e} \end{bmatrix} = \mathbf{0}, \quad (\mathbf{x}, \boldsymbol{\mu}) \geq \mathbf{0} \quad (4.63)$$

with the matrices $\mathbf{X} = \text{diag}(\mathbf{x})$, $\mathbf{M} = \text{diag}(\boldsymbol{\mu})$, and $\mathbf{e} = [1, \dots, 1]^\top$. It can be shown that the optimal solution $(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ is obtained for $\tau \rightarrow 0$

$$\lim_{\tau \rightarrow 0} (\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = (\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*). \quad (4.64)$$

In many practical implementations of the interior point method that solve (4.63) in an iterative manner, the so-called *duality measure*

$$\bar{\mu} = \frac{1}{n} \sum_{i=1}^n x_i \mu_i = \frac{1}{n} \mathbf{x}^\top \boldsymbol{\mu} \quad (4.65)$$

is used to adapt the barrier parameter in each iteration k in dependence of a reduction factor σ^k

$$\tau^k = \sigma^k \bar{\mu}^k, \quad \sigma^k \in [0, 1]. \quad (4.66)$$

The iteration scheme for solving the nonlinear set of equations (4.63), (4.66) reads

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \boldsymbol{\lambda}^{k+1} \\ \boldsymbol{\mu}^{k+1} \end{bmatrix} = \begin{bmatrix} \mathbf{x}^k \\ \boldsymbol{\lambda}^k \\ \boldsymbol{\mu}^k \end{bmatrix} + \alpha^k \mathbf{s}^k \quad \text{with} \quad \mathbf{s}^k = \begin{bmatrix} \Delta \mathbf{x}^k \\ \Delta \boldsymbol{\lambda}^k \\ \Delta \boldsymbol{\mu}^k \end{bmatrix}. \quad (4.67)$$

The search direction $\mathbf{s}^k$ is computed with Newton's method as the solution of the vector equation⁴

$$\underbrace{\begin{bmatrix} \mathbf{0} & \mathbf{A}^\top & -\mathbf{I} \\ \mathbf{A} & \mathbf{0} & \mathbf{0} \\ \mathbf{M}^k & \mathbf{0} & \mathbf{X}^k \end{bmatrix}}_{\nabla \mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k)} \begin{bmatrix} \Delta \mathbf{x}^k \\ \Delta \boldsymbol{\lambda}^k \\ \Delta \boldsymbol{\mu}^k \end{bmatrix} = \underbrace{\begin{bmatrix} -\mathbf{c} - \mathbf{A}^\top \boldsymbol{\lambda}^k + \boldsymbol{\mu}^k \\ -\mathbf{A} \mathbf{x}^k + \mathbf{b} \\ -\mathbf{X}^k \mathbf{M}^k \mathbf{e} + \sigma^k \bar{\boldsymbol{\mu}}^k \mathbf{e} \end{bmatrix}}_{-\mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k)} \quad (4.68)$$

with $\bar{\boldsymbol{\mu}}^k$ from (4.65). In addition, the step size α^k in (4.67) must be determined by a suitable line search method that satisfies the strict constraints

$$\mathbf{x}^k > \mathbf{0}, \quad \boldsymbol{\mu}^k > \mathbf{0} \quad (4.69)$$

in each iteration k . This also explains the name *interior point method*. The reduction factor $\sigma^k \in [0, 1]$ also affects the step size α^k . Larger values of σ^k , i.e. $\sigma^k \approx 1$, typically imply a larger step α^k in the Newton direction $\mathbf{s}^k$ before the constraints $(\mathbf{x}^k, \boldsymbol{\mu}^k) > \mathbf{0}$ are violated.

There exist several more or less complex methods to compute α^k and σ^k , e.g. predictor-corrector schemes, that are not further detailed here. A critical point is the choice of the initial solution $(\mathbf{x}^0, \boldsymbol{\lambda}^0, \boldsymbol{\mu}^0)$ that must strictly satisfy the constraints $(\mathbf{x}^0, \boldsymbol{\mu}^0) > \mathbf{0}$ and the equations (4.63) at least with a not too large residual.

Interior point methods are well suited for large-scale problems. There are efficient algorithms that exploit the sparse structure of the linear set of equations (4.68). More details on this can e.g. be found in the textbooks [17, 14].

Relation to barrier methods

Interior point methods are closely related to barrier methods that use logarithmic penalty terms in the cost function to account for the constraints $\mathbf{x} \geq \mathbf{0}$, i.e.

$$\min_{\mathbf{x} \in \mathbb{R}^n} \quad \mathbf{c}^\top \mathbf{x} - \tau \sum_{i=1}^n \log x_i, \quad \tau > 0 \quad (4.70a)$$

$$\text{s.t.} \quad \mathbf{A} \mathbf{x} = \mathbf{b}. \quad (4.70b)$$

A solution of the problem (4.70) must strictly satisfy the constraints $\mathbf{x} > \mathbf{0}$. Otherwise, the cost function (4.70a) would approach infinity due to the logarithmic functions. The penalty term τ affects the form of the barrier function as illustrated in Figure 4.8. For $\tau \rightarrow 0$, the log-terms approach a discontinuous behavior, whereas the penalty for interior points $\mathbf{x} > 0$ becomes negligible.

⁴ Equation (4.68) results from the approximation $\mathbf{m}_F^k(\mathbf{s}) = \mathbf{0}$ of (4.63) with the linear model $\mathbf{m}_F^k(\mathbf{s}) = \mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k) + \nabla \mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k) \mathbf{s}$. The computation of the Newton direction $\mathbf{s}$ is similar to the approach in Section 3.3.4, where a quadratic model (3.32) was derived for the cost function and the stationarity condition $\nabla m^k(\mathbf{s}) = \mathbf{0}$ for its minimum was a linear model in $\mathbf{s}$.

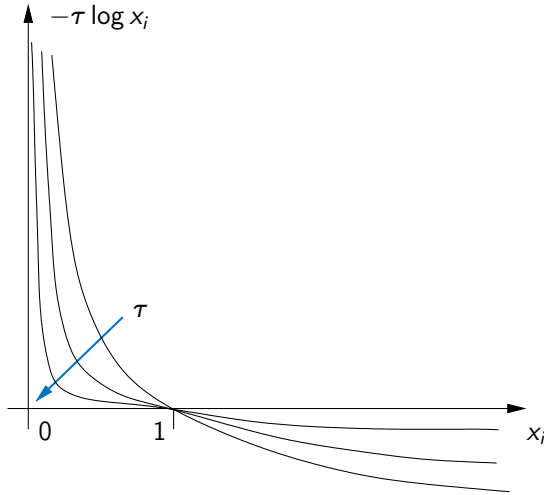


Figure 4.8: Logarithmic barrier function for different values of τ .

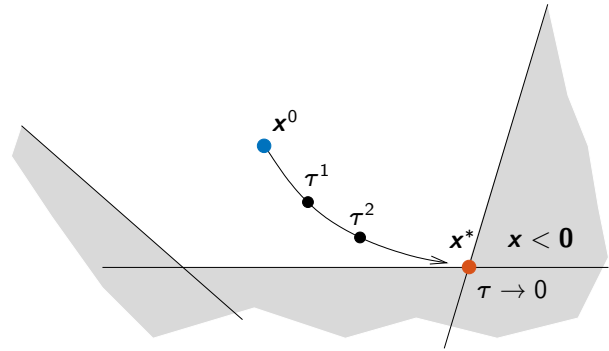


Figure 4.9: Central path for decreasing τ^k .

With the Lagrangian function

$$L(\mathbf{x}, \boldsymbol{\lambda}) = \mathbf{c}^T \mathbf{x} - \tau \sum_{i=1}^n \log x_i + \boldsymbol{\lambda}^T (\mathbf{A}\mathbf{x} - \mathbf{b}),$$

the KKT conditions (4.50a), (4.50b) for the barrier problem become

$$\begin{aligned} c_i - \frac{\tau}{x_i} + \mathbf{A}_i^T \boldsymbol{\lambda} &= 0, \quad i = 1, \dots, n \\ \mathbf{A}\mathbf{x} &= \mathbf{b} \end{aligned} \quad (4.71)$$

with $\mathbf{A}_i$ denoting the i -th column of matrix $\mathbf{A}$. When we introduce the new variables $\mu_i = \tau/x_i$, we obtain the same set of equations as in (4.63)

$$\mathbf{F}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \begin{bmatrix} \mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda} - \boldsymbol{\mu} \\ \mathbf{A}\mathbf{x} - \mathbf{b} \\ \mathbf{x} \mathbf{M} \mathbf{e} \end{bmatrix} - \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \tau \mathbf{e} \end{bmatrix} = \mathbf{0}, \quad (\mathbf{x}, \boldsymbol{\mu}) \geq \mathbf{0}.$$

This shows that the optimality conditions of the interior point method and the logarithmic barrier method are equivalent via the introduction of the new variables $\mu_i = \tau/x_i$.

Pure barrier methods as in (4.70) are numerically sensitive, because the terms in the KKT condition (4.71) can become very large, if $\mathbf{x}$ approaches the boundary of $\mathbf{x} \geq \mathbf{0}$ in the limit $\tau \rightarrow 0$. The formulation of the interior point methods is favorable in this regard as it can be shown that the set of equations (4.63) is better conditioned for $\tau \rightarrow 0$ than the optimality conditions (4.71) of barrier methods.

Nevertheless, the behavior of the barrier method can be used to illustrate the interior point method. Each value $\tau^k > 0$ defines a solution $(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k)$ and the set of all solutions parametrized by the barrier parameter τ describes the *central path* that approaches the optimal solution $(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ of the original problem (4.44) for a sequence $\{\tau^k\}$ with $\tau^{k+1} < \tau^k$ and $\lim_{k \rightarrow \infty} \tau^k = 0$, see Figure 4.9. Interior point methods have the same behavior and therefore approach the optimal solution along the central path as the barrier parameter τ^k is successively reduced.

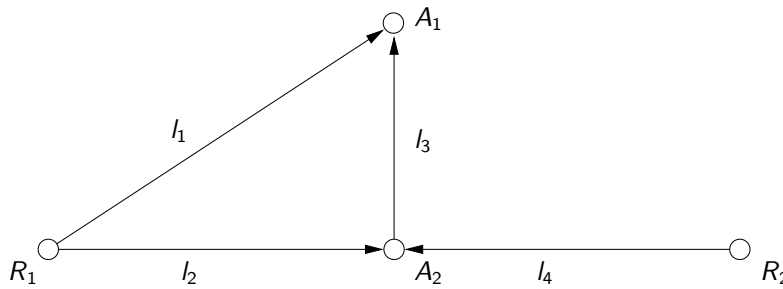


Figure 4.10: Transport network of the refinery example.

4.3.5 Example: Transport network

Figure 4.10 shows an abstract transport network of two refineries R_1 and R_2 . The produced fuel is transported to two consumption areas A_1 , A_2 , (e.g. industry or towns) via four pipelines $l_1, \dots, l_4$.⁵ We assume that the consumption areas A_1 , A_2 have the constant consumption rate R_1 , R_2 and that the transport costs for each pipeline is proportional to its flow rate x_i with the proportionality factor K_i .

The goal is to find the optimal distribution of flows x_i for the minimization of the transport costs while maintaining the consumption rates R_1 , R_2 and accounting for the maximum flow capacity C_i of each pipeline. Moreover, the refinery R_1 is supposed to have a maximum production rate P_1 .

The following (dimension-free) parameter values are considered:

$$\begin{aligned} K_1 &= 40, & C_1 &= 30, & R_1 &= 50, & P_1 &= 45 \\ K_2 &= 20, & C_2 &= 20, & R_2 &= 40 \\ K_3 &= 10, & C_3 &= 30 \\ K_4 &= 10, & C_4 &= 60. \end{aligned}$$

Exercise 4.7 Formulate the optimization problem of the transport network. Reduce the problem to two dimensions by substituting the equality constraints.

Exercise 4.8 Draw the admissible set as well as the contour lines of the cost function and determine the optimal solution graphically.

The Optimization Toolbox of MATLAB provides the function

$$\text{linprog}(f, A, b, A_{eq}, b_{eq}, LB, UB, options)$$

for solving linear optimization problems of the form

$$\min_{x \in \mathbb{R}^n} f^T x \quad \text{s.t.} \quad A_{eq} x = b_{eq}, \quad Ax \leq b, \quad LB \leq x \leq UB.$$

The simplex method and the interior point method can be selected with the following options:

Simplex method: `opt = optimoptions('linprog','Algorithm','dual-simplex')`
 Interior point method: `opt = optimoptions('linprog','Algorithm','interior-point')`

⁵ This is a modified version of the refinery example in [15].

Note that MATLAB employs a simplex method for the *dual problem* of (4.44), see Section 4.6 and Example 4.8, which is the reason for the setting 'dual-simplex'.

Exercise 4.9 Solve the transport network problem with MATLAB (see Figure 4.11) and compare the simplex method with the interior point method in terms of accuracy and number of iterations.

```
function transport_problem(methodQ)
% -----
% methodQ: 1 - Simplex method
%           2 - Interior point method

opt = optimoptions('linprog','Display','iter'); % options for output

switch methodQ
case 1, % simplex method
    opt = optimoptions(opt,'Algorithm','dual-simplex');
case 2, % interior point method
    opt = optimoptions(opt,'Algorithm','interior-point');
end

K = [40; 20; 10; 10]; % parameters for
C = [30; 20; 30; 60]; % transport problem
R = [50; 40];

xopt = [20;10;30;60]; % optimal point

c = K; % cost vector
Aeq = [1,0,1,0; 0,1,-1,1]; beq = R; % equality constraints
LB = [0;0;0;0]; UB = C; % constraints on x
A = [1,1,0,0]; b = 45; % general constraints Ax <= b

% call of linprog
% -----
[x,f] = linprog(c,A,b,Aeq,beq,LB,UB,opt);
fprintf('\nComputed solution: [%g, %g, %g, %g] with f(x) = %g',x,f);
fprintf('\nDistance to optimum: [%2.2g, %2.2g, %2.2g, %2.2g]\n',x-xopt);
```

Figure 4.11: MATLAB code for the transport network problem (linprog).

4.4 Quadratic optimization

Another special case is quadratic optimization – also known as *quadratic programming* – with quadratic cost function and linear constraints

$$\min_{\mathbf{x} \in \mathbb{R}^n} \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x} + \mathbf{c}^T \mathbf{x}, \quad \mathbf{G} \in \mathbb{R}^{n \times n} \quad (4.72a)$$

$$\text{s.t. } \mathbf{A} \mathbf{x} = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{p \times n}, \quad \mathbf{b} \in \mathbb{R}^p \quad (4.72b)$$

$$\mathbf{C} \mathbf{x} \leq \mathbf{d}, \quad \mathbf{C} \in \mathbb{R}^{q \times n}, \quad \mathbf{d} \in \mathbb{R}^q. \quad (4.72c)$$

The weighting matrix $\mathbf{G} \in \mathbb{R}^{n \times n}$ is supposed to be symmetric. If it is positive (semi)definite, then the quadratic problem is (strictly) convex.

Similar to linear optimization (Section 4.3), this problem class is of special importance. An example is portfolio optimization, where an optimal compromise is sought between profit maximization and risk minimization (covariance). Moreover, many numerical methods for solving general nonlinear optimization problems (Section 4.5) are based on the sequential solution of quadratic problems.

4.4.1 Optimality conditions

The general nonlinear KKT conditions (4.29) have a simplified structure for quadratic problems. With the Lagrangian function

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x} + \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A} \mathbf{x} - \mathbf{b}) + \boldsymbol{\mu}^T (\mathbf{C} \mathbf{x} - \mathbf{d})$$

and the Lagrange multipliers $\boldsymbol{\lambda} \in \mathbb{R}^p$, $\boldsymbol{\mu} \in \mathbb{R}^q$, the KKT conditions for an optimal solution read

$$\mathbf{G} \mathbf{x}^* + \mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}^* + \mathbf{C}^T \boldsymbol{\mu}^* = \mathbf{0} \quad (4.73a)$$

$$\mathbf{A} \mathbf{x}^* - \mathbf{b} = \mathbf{0} \quad (4.73b)$$

$$\mathbf{C} \mathbf{x}^* - \mathbf{d} \leq \mathbf{0} \quad (4.73c)$$

$$\mu_i^* \geq 0, \quad i = 1, \dots, n \quad (4.73d)$$

$$\mu_i^* (\mathbf{C}_i^T \mathbf{x}^* - d_i) = 0, \quad i = 1, \dots, n \quad (4.73e)$$

with $\mathbf{C}_i^T$ denoting the i -th row of the constraint matrix $\mathbf{C}$. In the convex case, i.e. if $\mathbf{G}$ is positive semidefinite, a solution $\mathbf{x}^*$ (if it exists) is always a global solution of the quadratic problem and the KKT conditions (4.29) are necessary and sufficient for the existence of a global minimum point $\mathbf{x}^*$. Non-convex quadratic problems are usually more difficult to solve since multiple stationary points and minima may exist.

4.4.2 Equality-constrained quadratic problems

An important subclass are equality-constrained quadratic problems

$$\min_{\mathbf{x} \in \mathbb{R}^n} \quad \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x} + \mathbf{c}^T \mathbf{x} \quad (4.74a)$$

$$\text{s.t.} \quad \mathbf{A} \mathbf{x} = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{p \times n}, \quad \mathbf{b} \in \mathbb{R}^p, \quad (4.74b)$$

for which the KKT conditions (4.73) reduce to the linear set of equations

$$\begin{bmatrix} \mathbf{G} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x}^* \\ \boldsymbol{\lambda}^* \end{bmatrix} = \begin{bmatrix} -\mathbf{c} \\ \mathbf{b} \end{bmatrix} \quad (4.75)$$

that consists of $n + p$ equations for the $n + p$ unknowns $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$. The block matrix in (4.75) is known as *Karush-Kuhn-Tucker matrix* or simply *KKT matrix*.

There exist different methods for solving (4.75) that are shortly summarized here. More information can be found, e.g. in [14].

- *Direct methods* solve the set of equations (4.75) by means of factorization, e.g. the *null-space method*. Direct methods are often used for small and mid-sized problems, but there are also efficient factorization methods for sparse large-scale problems.
- *Iterative methods* are well suited for large-scale problems and also allow for parallelization. A popular class of iterative methods are *conjugate gradient methods*.

Equality-constrained quadratic problems are important in optimization due to the linearity of the reduced KKT conditions (4.75) and their efficient solution by direct or iterative methods. Moreover, inequality-constrained problems as well as the solution of nonlinear constrained problems by means of *sequential quadratic programming* (see Section 4.5.1) break down to solving equality-constrained subproblems of the form (4.74).

4.4.3 Active set method

A popular method for solving general quadratic problems (4.72) is the *active set method*. Similar to the simplex method in linear optimization, the active set method identifies active constraints in each iteration and solves an equality-constrained problem of the form (4.74).

The basic idea of the active set method is to determine a number of active constraints from $\mathbf{C}\mathbf{x} \leq \mathbf{d}$ in each iteration k . The corresponding indices are captured in the *working set* $\mathcal{A}^k$, i.e.

$$\mathbf{C}_i^T \mathbf{x}^k = d_i, \quad i \in \mathcal{A}^k.$$

The working set $\mathcal{A}^k$ does not necessarily consider all constraints that are active at point $\mathbf{x}^k$, but it is iteratively completed over the iterations until the active constraint set (see Definition 4.1) is reached at the optimal point $\mathbf{x}^*$

$$\mathcal{A}(\mathbf{x}^*) = \{i \in \{1, \dots, q\} \mid \mathbf{C}_i^T \mathbf{x}^* = d_i\}.$$

The active set method typically assumes that the constraint qualification (4.26) is satisfied at each iteration point $\mathbf{x}^k$, i.e.

$$\text{rank} \begin{bmatrix} \mathbf{A} \\ [\mathbf{C}_i^T]_{i \in \mathcal{A}^k} \end{bmatrix} = p + |\mathcal{A}^k| \quad \forall k \quad (4.76)$$

and that the matrix $\mathbf{G}$ is positive definite.

In each iteration k , the active set method verifies if the point $\mathbf{x}^k$ with the working set $\mathcal{A}^k$ is the optimal solution of the quadratic problem. If this is not the case, a direction $\mathbf{s}$ is computed by solving an equality-constrained problem that accounts for all active constraints according to the working set $\mathcal{A}^k$. This problem is derived from the original one (4.72) by replacing $\mathbf{x} = \mathbf{x}^k + \mathbf{s}$ and using the current working set⁶

$$\min_{\mathbf{s} \in \mathbb{R}^n} \quad \frac{1}{2} \mathbf{s}^T \mathbf{G} \mathbf{s} + (\mathbf{G} \mathbf{x}^k + \mathbf{c})^T \mathbf{s} \quad (4.77a)$$

$$\text{s.t.} \quad \mathbf{A} \mathbf{s} = \mathbf{0}, \quad \mathbf{C}_i^T \mathbf{s} = 0, \quad i \in \mathcal{A}^k. \quad (4.77b)$$

Since we assume that $\mathbf{G}$ is positive definite, the problem (4.77) can be solved with the numerical methods mentioned in Section 4.4.2.

If the solution of (4.77) satisfies $\mathbf{s}^k \neq \mathbf{0}$ and $\mathbf{x}^k + \mathbf{s}^k$ is feasible in terms of the constraints $\mathbf{C}(\mathbf{x}^k + \mathbf{s}^k) \leq \mathbf{d}$, then $\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{s}^k$ is defined as new iteration point. If $\mathbf{x}^k + \mathbf{s}^k$ is not feasible, the step size $\alpha^k < 1$ for

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{s}^k \quad (4.78)$$

must be reduced until all constraints are satisfied. An important property in this regard is that each active constraint $i \in \mathcal{A}^k$ remains active in direction $\mathbf{s}^k$, because

$$\mathbf{C}_i^T (\mathbf{x}^k + \alpha \mathbf{s}^k) = \mathbf{C}_i^T \mathbf{x}^k = d_i, \quad \forall i \in \mathcal{A}^k \quad \forall \alpha \geq 0. \quad (4.79)$$

⁶ When substituting $\mathbf{x} = \mathbf{x}^k + \mathbf{s}$ in the cost function (4.72a), an additional term $\frac{1}{2}(\mathbf{x}^k)^T \mathbf{G} \mathbf{x}^k + \mathbf{c}^T \mathbf{x}^k$ appears that is independent of $\mathbf{s}$ and therefore omitted in the new cost function (4.77a).

It is therefore sufficient to consider those constraints that are not yet included in $\mathcal{A}^k$, especially those with $\mathbf{C}_i^\top \mathbf{s}^k > 0$. In this case, the original constraint $\mathbf{C}_i^\top (\mathbf{x}_k + \alpha^k \mathbf{s}^k) \leq d_i$ is only satisfied if

$$\alpha^k \leq \frac{d_i - \mathbf{C}_i^\top \mathbf{x}^k}{\mathbf{C}_i^\top \mathbf{s}^k}. \quad (4.80)$$

This shows that the largest possible step size α^k is given by

$$\alpha^k = \min \left\{ 1, \min_{i \notin \mathcal{A}^k, \mathbf{C}_i^\top \mathbf{s}^k > 0} \frac{d_i - \mathbf{C}_i^\top \mathbf{x}^k}{\mathbf{C}_i^\top \mathbf{s}^k} \right\}. \quad (4.81)$$

If $\alpha^k < 1$ follows from (4.81), a new constraint becomes active and its index is added to the new working set $\mathcal{A}^{k+1}$.

This procedure is iterated until $\mathbf{s}^k = \mathbf{0}$, which implies that point $\mathbf{x}^k$ minimizes the quadratic cost function (4.77a). To verify the KKT conditions (4.73) at this point $\mathbf{x}^k$, the Lagrange multipliers associated with the active constraints must be non-negative, i.e. $\mu_i^k \geq 0$, $i \in \mathcal{A}^k$. If this is the case, then $\mathbf{x}^k$ is the optimal and indeed global solution $\mathbf{x}^*$ due to the positive definiteness of $\mathbf{G}$. Otherwise, if at least one multiplier μ_i^k , $i \in \mathcal{A}^k$ is negative, the cost function (4.72a) can be further reduced if the corresponding constraint is removed from the working set, see Section 4.1.2 and 4.1.3. The reduction of $\mathcal{A}^k$ by one element $i \in \mathcal{A}^k$ with $\mu_i^k < 0$ and the new solution of the problem (4.77) then generates a new feasible direction $\mathbf{s}^{k+1}$.

The algorithm of the active set method is summarized in Table 4.2. For strictly convex quadratic problems, it can be shown that the active set method converges in a finite number of iterations.

Initialization:	$\mathbf{x}^0$ & $\mathcal{A}^0$	Initial point & working set
for	$k = 0, 1, 2, \dots$	do
	$\mathbf{s}^k$ & $\mu_i^k, i \in \mathcal{A}^k \leftarrow (4.77) \text{ \& } (4.73)$	Equality-constrained problem (4.77)
	if $\mathbf{s}^k = \mathbf{0}$	then Minimum found ($\mathbf{s}^k = \mathbf{0}$)?
	if $\mu_i^k \geq 0 \forall i \in \mathcal{A}^k$	then Optimal solution
	return $\mathbf{x}^* = \mathbf{x}^k$	
	else	
	$i^k \leftarrow \arg \min_{\mathcal{A}^k} \mu_i^k$	
	$\mathcal{A}^{k+1} \leftarrow \mathcal{A}^k \setminus \{i^k\}, \mathbf{x}^{k+1} \leftarrow \mathbf{x}^k$	Remove active constraint i^k
	end	
	else	Minimum not found ($\mathbf{s}^k \neq \mathbf{0}$)
	α^k & $i^k \leftarrow (4.81)$	Step size computation
	if $\alpha^k < 1$	then
	$\mathcal{A}^{k+1} \leftarrow \mathcal{A}^k \cup \{i^k\}$	Add new active constraint i^k
	else	
	$\mathcal{A}^{k+1} \leftarrow \mathcal{A}^k$	
	end	
	$\mathbf{x}^{k+1} \leftarrow \mathbf{x}^k + \alpha^k \mathbf{s}^k$	Next iteration point
	end	
	end	

Table 4.2: Active set method.

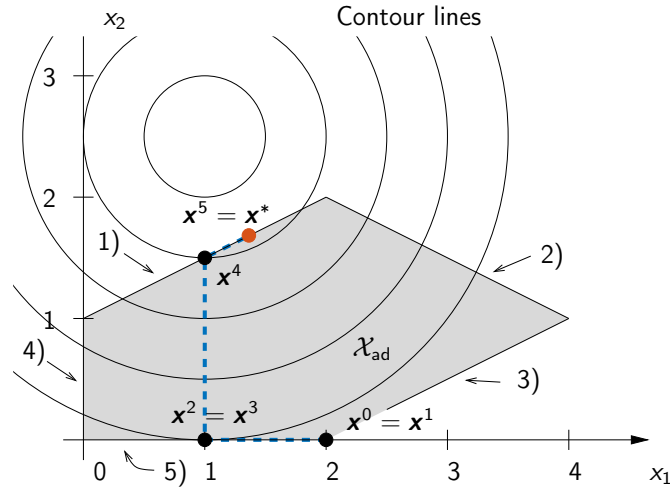


Figure 4.12: Iterations of the active set method in Example 4.6.

Example 4.6 ([14]) We consider the quadratic problem

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^2} \quad & (x_1 - 1)^2 + (x_2 - 2.5)^2 \\ \text{s.t.} \quad & 1) \quad -x_1 + 2x_2 - 2 \leq 0 \\ & 2) \quad x_1 + 2x_2 - 6 \leq 0 \\ & 3) \quad x_1 - 2x_2 - 2 \leq 0 \\ & 4) \quad x_1 \geq 0 \\ & 5) \quad x_2 \geq 0. \end{aligned}$$

The admissible set $\mathcal{X}_{ad}$ and the contour lines of the cost function are plotted in Figure 4.12. The constraints are numbered with the indices 1 bis 5.

Start: The initial point for the active set method is chosen as $\mathbf{x}^0 = [2, 0]^T$. Since the constraints 3 and 5 are active at $\mathbf{x}^0$, the working set is initialized with $\mathcal{A}^0 = \{3, 5\}$. The solution of problem (4.77) is $\mathbf{s}^0 = \mathbf{0}$ and the KKT condition (4.73a) with $\mu_i^0 = 0$, $i = 1, 2, 4$ leads to the multipliers $(\mu_3^0, \mu_5^0) = (-2, -1)$ of the active constraints 3 and 5.

Iteration 1: Constraint 3 is removed from the working set $\mathcal{A}^0$ because of $\mu_3^0 < \mu_5^0 < 0$, i.e. $\mathcal{A}^1 = \{5\}$, and the next solution of (4.77) is $\mathbf{s}^1 = [-1, 0]^T$. The step size $\alpha^k = 1$ follows from (4.81), leading to a “full” step $\mathbf{x}^2 = \mathbf{x}^1 + \mathbf{s}^1 = [1, 0]^T$ without a further constraint becoming active. The working set therefore remains unchanged with $\mathcal{A}^2 = \mathcal{A}^1$, see Figure 4.12.

Iteration 2: The solution of (4.77) now gives $\mathbf{s}^2 = \mathbf{0}$. The evaluation of the KKT conditions (4.73a) yields $\mu_5^2 = -5$ for the remaining active constraint in $\mathcal{A}^2 = \{5\}$. After removing the constraint, we get the empty working set $\mathcal{A}^3 = \{\}$.

Iteration 3: The problem (4.77) is now unconstrained and leads to $\mathbf{s}^3 = [0, 2.5]^T$ with the step size $\alpha^3 = 0.6$ and the new iteration point $\mathbf{x}^4 = [1, 1.5]^T$. The constraint 1 blocks a full step of the algorithm and becomes active. The working set for the next iteration is therefore updated to $\mathcal{A}^4 = \{1\}$.

Iteration 4: We obtain $\mathbf{s}^4 = [0.4, 0.2]^T$ with $\alpha^4 = 1$ and $\mathbf{x}^5 = [1.4, 1.7]^T$, see Figure 4.12. The working set remains unchanged $\mathcal{A}^5 = \mathcal{A}^4 = \{1\}$ since no new constraint becomes active.

Iteration 5: In the last iteration, (4.77) gives $\mathbf{s}^5 = \mathbf{0}$ with the Lagrange multiplier $\mu_1^5 = 0.8 > 0$ from (4.73a). Thus, $\mathbf{x}^5 = \mathbf{x}^*$ is the optimal solution and the algorithm terminates.

4.4.4 Further numerical methods

The active set method is well suited for low-dimensional or mid-sized problems but can show slow convergence properties for large-scale problems due to the fact that the working set $\mathcal{A}^k$ is only updated by one element in each iteration. An alternative for large-scale problems are *projected gradient methods* as they allow for a rapid adaptation of the working set $\mathcal{A}^k$.

A further class of methods for solving quadratic problems are interior point methods that were already introduced in terms of linear programming, see Section 4.3.4. The inequality constraints (4.72c) can be transformed into the standard form (4.44b)-(4.44c) by means of *slack variables*.

More information on projected gradient methods and interior points methods for quadratic problems can be found in the textbooks at the end of the chapter.

Exercise 4.10 Verify the solution of the quadratic problem in Example 4.6 numerically by using the MATLAB function `quadprog` of the Optimization Toolbox, also see Figure 4.13.

```
% Matlab solution of a quadratic example problem
% -----
opt = optimoptions('quadprog','Display','iter','Algorithm','interior-point-convex');

G = [2,0; 0,2]; % cost function: 0.5*x'*G*x + c'*x
c = [-2;-5];
C = [-1,2; 1,2; 1,-2; -1,0; 0,-1]; % constraints: C*x <= d
d = [ 2; 6; 2; 0; 0];

x0 = [2.0;0.0]; % initial guess

[x,f] = quadprog(G,c,C,d,[],[],[],[],[],opt); % solution with quadprog
fprintf('\nComputed solution: [%g, %g] mit f(x) = %g\n',x,f);
```

Figure 4.13: MATLAB code for Exercise 4.10 (quadprog).

4.5 Nonlinear optimization

After the special cases of linear and quadratic optimization, we focus on the solution of general nonlinear problems (4.1) that are repeated here for the sake of completeness:

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (4.82a)$$

$$\text{s.t. } g_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \quad (4.82b)$$

$$h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, q. \quad (4.82c)$$

4.5.1 Sequential quadratic programming (SQP)

An efficient class of algorithms for nonlinear problems is *sequential quadratic programming* (SQP) that solves a sequence of linear-quadratic approximations of the constrained problem (4.82). SQP methods therefore rely on methods from quadratic programming that are iteratively applied to the nonlinear problem.

General idea and algorithm

To illustrate the working principle of SQP, we consider an equality-constrained problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad (4.83a)$$

$$\text{s.t. } \mathbf{g}(\mathbf{x}) = \mathbf{0} \quad (4.83b)$$

with $\mathbf{g} = [g_1, \dots, g_p]^T$. The functions $f(\mathbf{x})$ and $\mathbf{g}(\mathbf{x})$ are supposed to be continuously differentiable. The idea of SQP methods is to approximate the nonlinear problem (4.83) by a quadratic problem at the current iteration point $\mathbf{x}^k$ and to use its solution to compute the next iterate $\mathbf{x}^{k+1}$.

The quadratic problem to be solved can be derived by looking at the optimality conditions for (4.83). With the Lagrangian function $L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x})$, the KKT conditions (4.29) simplify to the equations

$$\mathbf{F}(\mathbf{x}, \boldsymbol{\lambda}) = \begin{bmatrix} \nabla f(\mathbf{x}) + \mathbf{A}(\mathbf{x})^T \boldsymbol{\lambda} \\ \mathbf{g}(\mathbf{x}) \end{bmatrix} = \mathbf{0} \quad \text{with} \quad \mathbf{A}(\mathbf{x}) = [\nabla g_1(\mathbf{x}), \dots, \nabla g_p(\mathbf{x})]^T \quad (4.84)$$

of order $n + p$ for the $n + p$ unknowns $(\mathbf{x}, \boldsymbol{\lambda})$. Newton's method for solving the equations is based on the iteration scheme

$$\begin{bmatrix} \mathbf{x}^{k+1} \\ \boldsymbol{\lambda}^{k+1} \end{bmatrix} = \begin{bmatrix} \mathbf{x}^k \\ \boldsymbol{\lambda}^k \end{bmatrix} + \mathbf{s}^k, \quad \mathbf{s}^k = \begin{bmatrix} \Delta \mathbf{x}^k \\ \Delta \boldsymbol{\lambda}^k \end{bmatrix} \quad (4.85)$$

with the Newton direction $\mathbf{s}^k$ as the solution of

$$\underbrace{\begin{bmatrix} \nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) & \mathbf{A}(\mathbf{x}^k)^T \\ \mathbf{A}(\mathbf{x}^k) & \mathbf{0} \end{bmatrix}}_{\nabla \mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k)} \begin{bmatrix} \Delta \mathbf{x}^k \\ \Delta \boldsymbol{\lambda}^k \end{bmatrix} = \underbrace{\begin{bmatrix} -\nabla f(\mathbf{x}^k) - \mathbf{A}(\mathbf{x}^k)^T \boldsymbol{\lambda}^k \\ -\mathbf{g}(\mathbf{x}^k) \end{bmatrix}}_{-\mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k)}. \quad (4.86)$$

The matrix $\nabla \mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k)$ represents the Jacobian of $\mathbf{F}(\mathbf{x}^k, \boldsymbol{\lambda}^k)$ w.r.t. $(\mathbf{x}, \boldsymbol{\lambda})$.

An alternative view on the Newton iteration (4.85), (4.86) is obtained if we consider the quadratic problem

$$\min_{\mathbf{s} \in \mathbb{R}^n} f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^T \mathbf{s} + \frac{1}{2} \mathbf{s}^T \nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) \mathbf{s} \quad (4.87a)$$

$$\text{s.t. } \mathbf{A}(\mathbf{x}^k) \mathbf{s} + \mathbf{g}(\mathbf{x}^k) = \mathbf{0} \quad (4.87b)$$

with the optimality conditions

$$\nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) \mathbf{s}^k + \nabla f(\mathbf{x}^k) + \mathbf{A}(\mathbf{x}^k)^T \boldsymbol{\lambda}_s^k = \mathbf{0} \quad (4.88a)$$

$$\mathbf{A}(\mathbf{x}^k) \mathbf{s}^k + \mathbf{g}(\mathbf{x}^k) = \mathbf{0} \quad (4.88b)$$

for the solution $(\mathbf{s}^k, \boldsymbol{\lambda}_s^k)$. Adding the term $\mathbf{A}(\mathbf{x}^k)^T \boldsymbol{\lambda}^k$ on both sides of the first equation in (4.86) and with $\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + \Delta \boldsymbol{\lambda}^k$ we obtain

$$\begin{bmatrix} \nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) & \mathbf{A}(\mathbf{x}^k)^T \\ \mathbf{A}(\mathbf{x}^k) & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x}^k \\ \boldsymbol{\lambda}^{k+1} \end{bmatrix} = \begin{bmatrix} -\nabla f(\mathbf{x}^k) \\ -\mathbf{g}(\mathbf{x}^k) \end{bmatrix}. \quad (4.89)$$

Initialization:

$k \leftarrow 0$	Iteration counter
$\mathbf{x}^0, \mathbf{H}^0$	Initial point and Hessian approx. (pos. def.)
ε	Convergence tolerance

repeat

$\mathbf{s}^k \leftarrow$ Solution of (4.90)	
$\alpha^k \leftarrow$ e.g. Armijo condition (3.21)	Step size α^k for descent in penalty function $P(\mathbf{x}^k + \alpha^k \mathbf{s}^k, \sigma) < P(\mathbf{x}^k, \sigma)$, e.g. with (4.91)
$\mathbf{x}^{k+1} \leftarrow \mathbf{x}^k + \alpha^k \mathbf{s}^k$	Next iteration point
$k \leftarrow k + 1$	

until $\|\nabla_x L(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k)\| \leq \varepsilon$ Or other tolerance criterion

Table 4.3: Structure of SQP methods.

It is straightforward to see that (4.89) and the optimality conditions (4.88) for the quadratic problem (4.87) are equivalent for $\Delta \mathbf{x}^k = \mathbf{s}^k$ and $\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}_s^k$. The iteration variables $(\mathbf{x}^{k+1}, \boldsymbol{\lambda}^{k+1})$ therefore can be computed via the Newton iteration (4.85), (4.86) or as the solution of the quadratic problem (4.87) with $\mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{s}^k$ and the multipliers $\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}_s^k$.

This interpretation can be extended to inequality-constrained problems as the equality constraints (4.87b) of the quadratic problem represent a linearization of $\mathbf{g}(\mathbf{x})$ at point $\mathbf{x}^k$. In the case of the general nonlinear problem (4.82), SQP methods consider the following quadratic problem in each iteration k

$$\min_{\mathbf{s} \in \mathbb{R}^n} \quad \frac{1}{2} \mathbf{s}^T \nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k) \mathbf{s} + \nabla f(\mathbf{x}^k)^T \mathbf{s} \quad (4.90a)$$

$$\text{s.t.} \quad \nabla g_i(\mathbf{x}^k)^T \mathbf{s} + g_i(\mathbf{x}^k) = 0, \quad i = 1, \dots, p \quad (4.90b)$$

$$\nabla h_i(\mathbf{x}^k)^T \mathbf{s} + h_i(\mathbf{x}^k) \leq 0, \quad i = 1, \dots, q, \quad (4.90c)$$

whereby the term $f(\mathbf{x}^k)$ is independent of $\mathbf{s}$ and therefore omitted in the cost function. The equations (4.90) are again solved with the methods described in Section 4.4. The Hessian matrix $\nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\mu}^k)$ of the Lagrangian function can be determined directly (i.e. either analytically, numerically, or by using automatic differentiation) or by a quasi-Newton approximation, see e.g. [14].

The next iteration point is $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha^k \mathbf{s}^k$ with the solution $\mathbf{s}^k$ of (4.90). The step size α^k must be chosen as a compromise between a sufficient decrease of the cost function $f(\mathbf{x}^{k+1}) < f(\mathbf{x}^k)$ while complying with the inequality constraints (4.82c) within a certain tolerance. This point will be touched upon in the following lines. The general algorithm of an SQP method is listed in Table 4.3.

Numerical methods for solving (4.90) can be classified as follows :

- *EQP methods* (from “equality-constrained QP”) choose a working set of active constraints at each iteration point $\mathbf{x}^k$ and reduce (4.90) to an equality-constrained problem, where all constraints that are not part of the working set are discarded. The working set is updated in each iteration. This method has the advantage that the solution of high-dimensional equality-constrained quadratic problems requires less computational effort than the solution of the general problem (4.90).

- *IQP methods* (from “inequality-constrained QP”) use the active constraint set of (4.90) as the estimate of $\mathcal{A}(\mathbf{x}^*)$ at the optimal point $\mathbf{x}^*$. If the set $\mathcal{A}(\mathbf{x}^*)$ is determined correctly, the SQP method behaves similar to Newton’s method for equality-constrained problems and converges rapidly to the optimal point $\mathbf{x}^*$. This approach has proven successful in practice. The disadvantage is that the solution of (4.90) is computationally demanding for large-scale problems, although information from previous iterations can be used as warm-start strategy in the neighborhood of the optimum in order to start with a good estimate of the solution in the current iteration.

Step size computation

It is not sufficient to choose the step size α^k in view of the maximum descent of the cost function $f(\mathbf{x}^k + \alpha^k \mathbf{s}^k) < f(\mathbf{x}^k)$ because the compliance with the nonlinear constraints $g_i(\mathbf{x}) = 0$ and $h_i(\mathbf{x}) \leq 0$ must be taken into account as well. Both aspects can be combined in a *penalty function*. In particular, the l_1 -penalty function

$$P(\mathbf{x}, \sigma) = f(\mathbf{x}) + \sigma \left[\sum_{i=1}^p |g_i(\mathbf{x})| + \sum_{i=1}^q \max\{0, h_i(\mathbf{x})\} \right] \quad (4.91)$$

has the property that each solution $\mathbf{x}^*$ of the nonlinear problem (4.82) is also a minimizing point of P under the assumption that $(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ satisfies the sufficient optimality conditions (4.37) and that the penalty parameter σ is larger than the absolute value of the multipliers, i.e. $\sigma > |\lambda_i^*|$, $i = 1, \dots, p$ and $\sigma > |\mu_i^*|$, $i = 1, \dots, q$. In this case, the function (4.91) is also called *exact penalty function*. The step size α^k can be determined with the line search methods from Section 3.3.1, e.g. the Armijo condition (3.21) to satisfy $P(\mathbf{x}^k + \alpha^k \mathbf{s}^k, \sigma) < P(\mathbf{x}^k, \sigma)$.

Example 4.7 (Optimized controller parameters) We consider the optimization of controller parameters for a discrete-time system with delay

$$x(k) = ax(k-1) + bu(k-10) \quad (4.92)$$

and the system parameters $a = b = 0.5$ [15]. The control law is given by

$$u(k) = c_1 u(k-1) + c_2 e(k) + c_3 e(k-1), \quad (4.93)$$

where $e(k) = 1 - x(k)$ denotes the output error for a unit step as reference signal. The controller parameters (c_1, c_2, c_3) are to be determined such that the discrete-time control surface

$$f(\mathbf{c}) = \sum_{k=1}^N |e(k)| \quad (4.94)$$

or the time-weighted control surface

$$f(\mathbf{c}) = \sum_{k=1}^N \frac{k}{N} |e(k)| \quad (4.95)$$

are minimized. The latter case puts more weight on control errors over the time horizon. In addition, overshoot in the step response shall be bounded in form of an inequality constraint.

The Optimization Toolbox of MATLAB offers the function `fmincon` for the solution of nonlinear optimization problems. The SQP method is chosen with the option `optimset('Algorithm','active-set')` or `optimset('Algorithm','sqp')`.

The MATLAB code to compute the optimal controller parameters with `fmincon` is shown in Figure 4.14.⁷ The limitation of the overshoot is implemented as nonlinear constraint. The initial guess for the controller parameters is $\mathbf{c}^0 = [1, 0.5, -0.5]^T$.

The optimized step response for the cases (4.94) and (4.95) with 10% and 0% overshoot are shown in Figure 4.15 and 4.16. The corresponding controller parameters are listed in Table 4.4. Note that the optimization returns $c_1 = 1$ in all cases, which shows that the controller (4.93) has integral action. Figure 4.16 also shows that the time-dependent weighting in the cost function (4.95) leads to a reduction of the oscillations after the setpoint is reached.

```

function controllerparameters(caseQ)
% -----
global x cold
switch caseQ
    case 1, p.costQ=1; p.ov=0.1; % control surface, max. overshoot 10%
    case 2, p.costQ=1; p.ov=0; % control surface, max. overshoot 0%
    case 3, p.costQ=2; p.ov=0.1; % time-weighted control surface, max. overshoot 10%
    case 4, p.costQ=2; p.ov=0; % time-weighted control surface, max. overshoot 0%
end

p.N = 70; % number of steps
p.a = 0.5; p.b = 0.5; % system parameters
c0 = [1;0.5;-0.5]; % initial guess

opt = optimoptions('fmincon','Algorithm','sqp', ... % SQP method for fmincon
                  'Display','iter','MaxFunEvals',500);

[c,f] = fmincon(@costfct,c0,[],[],[],[],[],[],@nlconstr,opt,p);
fprintf('\nOptimal parameters c=[%2.3g,%2.3g,%2.3g] with f(c)=%2.3g\n',c,f);

x = stepresponse(c,p); % optimized step response

figure(1); clf % plotting
h(1) = stairs(ones(1,p.N),'r--'); hold on
h(2) = stairs(x,'k');
h(3) = stairs(1-x,'m'); xlabel('Time steps');
axis([0, p.N, -0.1, 1.1+p.ov]);
legend(h,'Reference','State x','Error e')

function x = stepresponse(c,p)
% -----
z = tf('z');
G0 = (c(2)+c(3)*z^(-1))/(1-c(1)*z^(-1)) * p.b/(z^10-p.a*z^9); % z-transformation:
x = step( G0/(1+G0), p.N-1 ); % Gcontrol * Gsystem
% step response

function f = costfct(c,p)
% -----
global x cold
x = stepresponse(c,p); % step response
if p.costQ==1, f = sum( abs(1-x) ); % cost function
else f = 1/p.N*sum( [1:p.N]*abs(1-x) ); end
cold = c;

function [C,Ceq] = nlconstr(c,p)
% -----
global x cold
if any(c~=cold), x=stepresponse(c,p); end % if new c-values
C = max(x-(1+p.ov)); Ceq = []; % nonlin. inequ. constr.

```

Figure 4.14: MATLAB code for optimizing the controller parameters (`fmincon`).

⁷ The control loop (4.92), (4.93) is implemented as z-transfer function. The step response is calculated with the function `step`.

Cost fct.	max. overshoot	c_1^*	c_2^*	c_3^*	$f(\mathbf{c}^*)$
(4.94)	10%	1.000	0.439	-0.369	14.6
(4.94)	0%	1.000	0.439	-0.372	14.8
(4.95)	10%	1.000	0.367	-0.297	1.93
(4.95)	0%	1.000	0.400	-0.333	2.06

Table 4.4: Optimized controller parameters for the control loop (4.92)-(4.93).

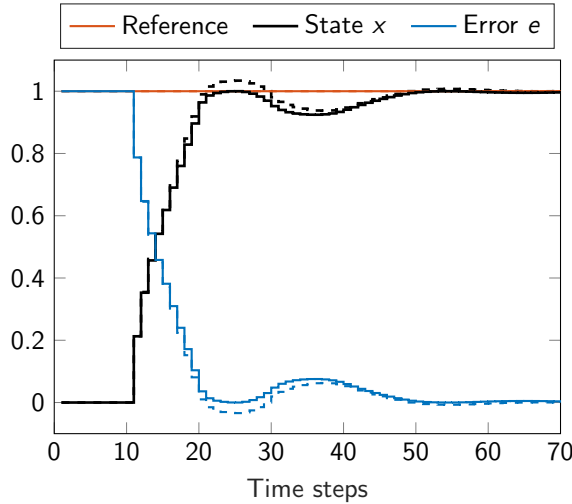


Figure 4.15: Step response of the optimized control loop with cost function (4.94) (overshoot 10%: - -, 0%: -).

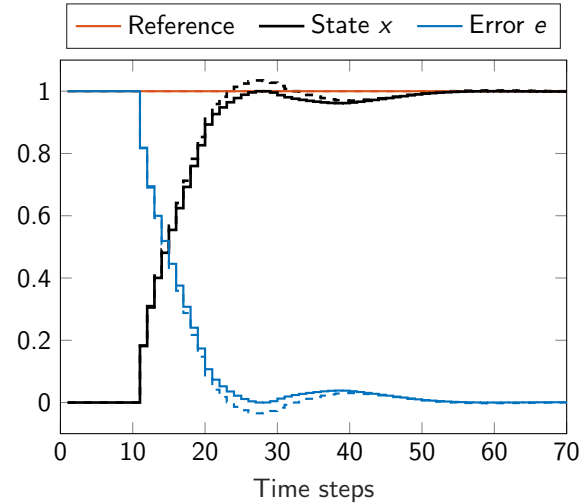


Figure 4.16: Step response of the optimized control loop with cost function (4.95) (overshoot 10%: - -, 0%: -).

4.5.2 Further numerical methods

An alternative to SQP is to replace the original problem (4.82) by one or several unconstrained problems that are solved in a successive manner. This section will give a brief introduction for selected methods, more information can be found in the textbooks at the end of the chapter.

External penalty functions

A straightforward approach is to replace the constrained problem (4.82) by an unconstrained one by adding the constraints (4.82b) und (4.82c) as penalty terms to the cost function

$$\min_{\mathbf{x} \in \mathbb{R}^n} P(\mathbf{x}, \tau) = f(\mathbf{x}) + \frac{1}{\tau} \left[\sum_{i=1}^p \phi(g_i(\mathbf{x})) + \sum_{i=1}^q \psi(h_i(\mathbf{x})) \right]. \quad (4.96)$$

The functions ϕ and ψ are *external penalty functions* that penalize a violation of the constraints. The penalty function ϕ for the equality constraints $g_i(\mathbf{x}) = 0$ is often designed as a power function $\phi(g_i(\mathbf{x})) = |g_i(\mathbf{x})|^r$ with $r \geq 1$, whereas ψ is typically defined as

$$\psi(h_i(\mathbf{x})) = (\max\{0, h_i(\mathbf{x})\})^r, \quad r \geq 1. \quad (4.97)$$

Figure 4.17a illustrates the behavior of ψ and the influence of the penalty parameter τ .

The solution $\mathbf{x}(\tau)$ of the new unconstrained problem (4.96) will differ from the optimal solution $\mathbf{x}^*$ of the original problem (4.82) because the penalty term in (4.96) alters the original cost function (4.82a).

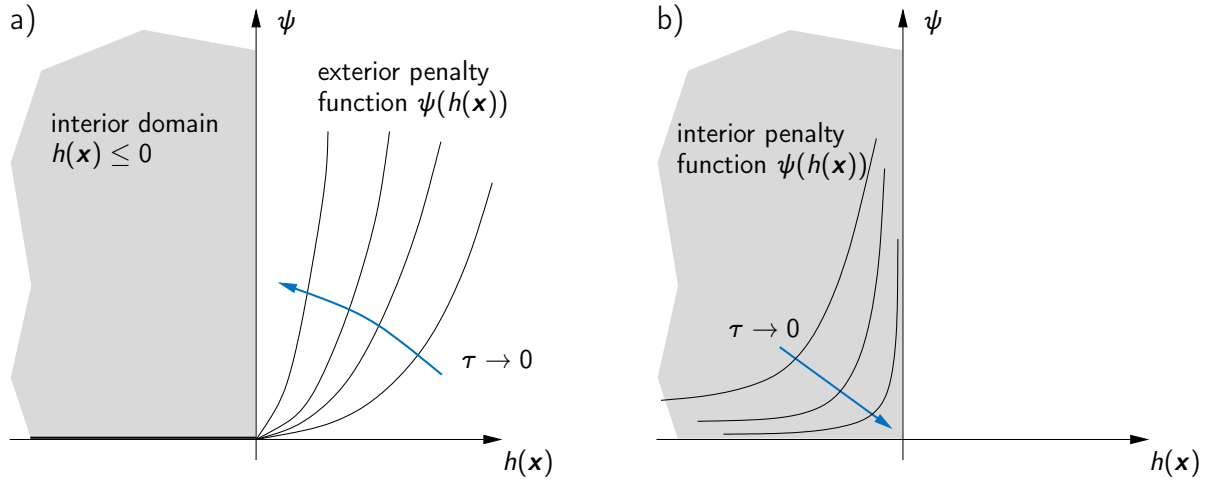


Figure 4.17: Illustration of external and interior penalty function $\psi(h(\mathbf{x}))$.

The problem (4.96) is therefore solved in a successive manner for a decreasing sequence $\{\tau^k\}$ of penalty parameters corresponding to an increasingly restrictive penalization of the constraint violation, see Figure 4.17a. Under certain assumptions, it can be shown that the sequence $\{\tau^k\}$ with $\lim_{k \rightarrow \infty} \tau^k = 0$ will approach the optimal solution in the limit, i.e.

$$\lim_{k \rightarrow \infty} \mathbf{x}(\tau^k) = \mathbf{x}^*. \quad (4.98)$$

A well-known drawback is that problem (4.96) is increasingly ill-conditioned for $\tau \rightarrow 0$, which makes it difficult in practice to achieve an accurate approximation of the optimal solution $\mathbf{x}^*$.

Interior penalty functions (barrier functions)

Interior penalty functions or *barrier functions* penalize the constraints $h_i(\mathbf{x}) \leq 0$ from the interior of the admissible set and approach infinity if the boundary $h_i(\mathbf{x}) = 0$ is touched, see Figure 4.17b as well as Figure 4.8. Examples of interior penalty functions are

$$\psi(h_i(\mathbf{x})) = \frac{-1}{h_i(\mathbf{x})} \quad \text{or} \quad \psi(h_i(\mathbf{x})) = -\log(-h_i(\mathbf{x})). \quad (4.99)$$

Since barrier functions are defined on the interior of the admissible set, only inequality constraints (4.82) can be considered within the unconstrained problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} P(\mathbf{x}, \tau) = f(\mathbf{x}) + \tau \sum_{i=1}^q \psi(h_i(\mathbf{x})). \quad (4.100)$$

The successive reduction of the penalty parameter τ reduces the penalization of the admissible set for $h_i(\mathbf{x}) < 0$, whereas ψ remains unbounded for $h_i(\mathbf{x}) \nearrow 0$, see Figure 4.17b. It can be shown under certain assumptions that the optimal solution $\mathbf{x}^*$ is reached in the limit (4.98) for a decreasing sequence $\{\tau^k\}$ with $\lim_{k \rightarrow \infty} \tau^k = 0$. For barrier methods, it is important to find a feasible starting point $\mathbf{x}^0$ that lies in the interior of the constraints $h_i(\mathbf{x}) \leq 0$.

Interior point methods

Interior point methods were already introduced in connection with linear programming in Section 4.3.4. Their extension to nonlinear problems is therefore only touched upon. Interior

point methods rely on the idea of perturbing the complementary condition (4.29e) by a parameter $\tau > 0$, in order to ease the numerical solution of the KKT conditions (4.29). As already mentioned in Section 4.3.4, there exists a relation between interior point methods and logarithmic barrier functions in (4.99). However, interior point methods are numerically better conditioned and belong to the most powerful methods in nonlinear optimization in particular for large-scale problems. Further information and details can be found in the comprehensive survey paper [9] and the textbooks [17, 14].

4.6 Lagrangian duality

The previous sections focused on constrained optimization problems from the perspective of the KKT conditions and their numerical or algorithmic solution. In contrast to this, the Lagrangian duality theory considers an alternative optimization problem that is *dual* to the original one. The dual problem has several interesting characteristics that makes the dual problem in some cases easier to solve than the original constrained optimization problem.

This section gives an introduction to Lagrangian duality and the dual formulation and afterwards presents a popular numerical method for solving dual problems, the so-called *augmented Lagrangian method*.

4.6.1 Dual formulation

We consider the nonlinear optimization problem

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad (4.101a)$$

$$\text{s.t.} \quad \mathbf{g}(\mathbf{x}) = \mathbf{0} \quad (4.101b)$$

$$\mathbf{h}(\mathbf{x}) \leq \mathbf{0} \quad (4.101c)$$

similar to (4.82) with the exception that the constraint functions are written in vector notation $\mathbf{g} = [g_1, \dots, g_p]^\top$ and $\mathbf{h} = [h_1, \dots, h_q]^\top$ for compactness reasons and the domain of definition $\mathcal{X} \subseteq \mathbb{R}^n$ is introduced for the optimization variables $\mathbf{x}$.

According to Definition 4.2, the Lagrangian for the problem (4.101) reads

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{x}) + \boldsymbol{\mu}^\top \mathbf{h}(\mathbf{x}) \quad (4.102)$$

with the Lagrange multipliers $\boldsymbol{\lambda} \in \mathbb{R}^p$ and $\boldsymbol{\mu} \in \mathbb{R}^q$. The so-called *dual function* is defined as the infimum⁸ of the Lagrangian w.r.t. $\mathbf{x}$.

Definition 4.4 (Dual function) *The Lagrangian dual function or simply dual function is defined as*

$$\theta(\boldsymbol{\lambda}, \boldsymbol{\mu}) = \inf_{\mathbf{x} \in \mathcal{X}} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}). \quad (4.103)$$

⁸ The infimum $\inf \mathcal{X}$ of a nonlinear set $\mathcal{X} \subset \mathbb{R}$ denotes the largest lower bound of $\mathcal{X}$. This means that there exists some ξ such that $x \geq \xi$ for all $x \in \mathcal{X}$ and, moreover, for all $\xi' > \xi$ there exists a $x \in \mathcal{X}$ such that $x < \xi'$. The infimum of $\mathcal{X}$ is not necessarily contained in the set $\mathcal{X}$, see e.g. $\inf \mathcal{X} = 0$ with $\mathcal{X} = (0, 1)$ and $0 \notin \mathcal{X}$. Applied to a function $f : \mathcal{U} \rightarrow \mathcal{V}$ we have $\inf_{u \in \mathcal{U}} f(u) = \inf \mathcal{V}$. The supremum $\sup \mathcal{X}$ as the smallest upper bound of a set $\mathcal{X}$ is defined accordingly.

Depending on the problem at hand, the dual function $\theta(\boldsymbol{\lambda}, \boldsymbol{\mu})$ can take on the value $-\infty$, e.g. if L is linear in $\mathbf{x}$ and $\mathcal{X} = \mathbb{R}^n$, see Example 4.8 below. For this reason, we define the domain on which θ is finite

$$D = \{(\boldsymbol{\lambda}, \boldsymbol{\mu}) \in \mathbb{R}^{p \times q} \mid \theta(\boldsymbol{\lambda}, \boldsymbol{\mu}) > -\infty\}. \quad (4.104)$$

The dual function has some interesting structural properties on D irrespective of the convexity properties of the cost $f(\mathbf{x})$ and constraint functions $\mathbf{g}(\mathbf{x})$, $\mathbf{h}(\mathbf{x})$ of the original problem (4.82).

Theorem 4.5 *The set D is convex and the dual function θ is concave on D .*

Proof: For each $\mathbf{x}$, $(\boldsymbol{\lambda}, \boldsymbol{\mu})$, $(\boldsymbol{\lambda}', \boldsymbol{\mu}')$, and $\alpha \in [0, 1]$, the Lagrangian satisfies

$$L(\mathbf{x}, \alpha\boldsymbol{\lambda} + (1 - \alpha)\boldsymbol{\lambda}', \alpha\boldsymbol{\mu} + (1 - \alpha)\boldsymbol{\mu}') = \alpha L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) + (1 - \alpha)L(\mathbf{x}, \boldsymbol{\lambda}', \boldsymbol{\mu}').$$

If we consider the infimum, we can derive the inequality

$$\begin{aligned} \inf_{\mathbf{x} \in \mathbb{R}^n} L(\mathbf{x}, \alpha\boldsymbol{\lambda} + (1 - \alpha)\boldsymbol{\lambda}', \alpha\boldsymbol{\mu} + (1 - \alpha)\boldsymbol{\mu}') &= \inf_{\mathbf{x} \in \mathbb{R}^n} (\alpha L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) + (1 - \alpha)L(\mathbf{x}, \boldsymbol{\lambda}', \boldsymbol{\mu}')) \\ &\geq \alpha \inf_{\mathbf{x} \in \mathbb{R}^n} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) + (1 - \alpha) \inf_{\mathbf{x} \in \mathbb{R}^n} L(\mathbf{x}, \boldsymbol{\lambda}', \boldsymbol{\mu}') \end{aligned}$$

or in terms of the dual function

$$\theta(\alpha\boldsymbol{\lambda} + (1 - \alpha)\boldsymbol{\lambda}', \alpha\boldsymbol{\mu} + (1 - \alpha)\boldsymbol{\mu}') \geq \alpha\theta(\boldsymbol{\lambda}, \boldsymbol{\mu}) + (1 - \alpha)\theta(\boldsymbol{\lambda}', \boldsymbol{\mu}'),$$

which proves the concavity of θ . If $(\boldsymbol{\lambda}, \boldsymbol{\mu})$, $(\boldsymbol{\lambda}', \boldsymbol{\mu}')$ belong to D , then the same holds for $\alpha\boldsymbol{\lambda} + (1 - \alpha)\boldsymbol{\lambda}' \in D$ due to $\theta(\alpha\boldsymbol{\lambda} + (1 - \alpha)\boldsymbol{\lambda}', \alpha\boldsymbol{\mu} + (1 - \alpha)\boldsymbol{\mu}') > -\infty$, which shows that D is convex. ■

Definition 4.5 (Dual problem) *The (Lagrangian) dual problem of (4.101) is defined by*

$$\max_{\boldsymbol{\lambda}, \boldsymbol{\mu}} \theta(\boldsymbol{\lambda}, \boldsymbol{\mu}) \quad (4.105a)$$

$$\text{s.t. } \boldsymbol{\mu} \geq \mathbf{0} \quad (4.105b)$$

with the dual variables $(\boldsymbol{\lambda}, \boldsymbol{\mu})$.

In the sense of the duality theory, the original optimization problem (4.101) is denoted as the *primal problem* with the *primal variables* $\mathbf{x}$.

The dual problem is always convex, even if the primal problem (4.101) is non-convex. Another advantage is that the dual problem (4.105) has only linear inequality constraints in contrast to the primal problem (4.82). Hence, it might be advantageous to solve the dual problem (4.105) instead of the primal one. The *augmented Lagrangian method* is a suitable algorithm in this regard that is presented in Section 4.6.2.

As mentioned before, the dual function $\theta(\boldsymbol{\lambda}, \boldsymbol{\mu})$ might evaluate to $-\infty$. In many cases, however, the set D in (4.104) can be determined explicitly and incorporated in the dual formulation as the following example shows.

Example 4.8 (Dual standard form of linear optimization problems) We consider the linear problem in standard form (also see Section 4.3)

$$\min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} \quad (4.106a)$$

$$\text{s.t. } \mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0} \quad (4.106b)$$

with the Lagrangian function

$$L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) + \boldsymbol{\mu}^T (-\mathbf{x}) = (\mathbf{A}^T \boldsymbol{\lambda} - \boldsymbol{\mu} + \mathbf{c})^T \mathbf{x} - \boldsymbol{\lambda}^T \mathbf{b}.$$

The dual problem (4.105) then reads

$$\max_{\boldsymbol{\lambda}, \boldsymbol{\mu}} \theta(\boldsymbol{\lambda}, \boldsymbol{\mu}) = \begin{cases} -\boldsymbol{\lambda}^T \mathbf{b} & \text{if } \mathbf{A}^T \boldsymbol{\lambda} - \boldsymbol{\mu} + \mathbf{c} = \mathbf{0} \\ -\infty & \text{else} \end{cases} \quad (4.107a)$$

$$\text{s.t. } \boldsymbol{\mu} \geq \mathbf{0}. \quad (4.107b)$$

The dual function $\theta(\boldsymbol{\lambda}, \boldsymbol{\mu})$ follows from Definition (4.103) and is only finite for $\mathbf{A}^T \boldsymbol{\lambda} - \boldsymbol{\mu} + \mathbf{c} = \mathbf{0}$. This condition can be considered in the formulation of the dual problem (4.107)

$$\max_{\boldsymbol{\lambda}} -\boldsymbol{\lambda}^T \mathbf{b} \quad (4.108a)$$

$$\text{s.t. } \mathbf{A}^T \boldsymbol{\lambda} + \mathbf{c} \geq \mathbf{0}, \quad (4.108b)$$

where the inequality constraint (4.108b) results from combining $\mathbf{A}^T \boldsymbol{\lambda} - \boldsymbol{\mu} + \mathbf{c} = \mathbf{0}$ and $\boldsymbol{\mu} \geq \mathbf{0}$. The formulation (4.108) is often referred to as Lagrangian dual standard form or in short dual standard form in linear optimization.

Exercise 4.11 Consider the linear problem (4.106) with the difference that the inequality constraint $\mathbf{x} \geq \mathbf{0}$ is integrated in the domain of definition $\mathbf{x} \in \mathcal{X}$ with $\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{x} \geq \mathbf{0}\}$. Determine the dual function (4.103) and show that the resulting dual problem is equivalent to (4.108).

The following theorem establishes the correspondence between the KKT conditions (4.29) of the primal problem (4.101) and the solution of the dual problem – at least for the case without equality constraints (4.101b) and under the assumption of convexity.

Theorem 4.6 ([14]) Suppose that the primal (inequality-constrained) optimization problem (4.101a), (4.101c) has a solution $\mathbf{x}^*$ and that the functions f and h_i , $i = 1, \dots, q$ are convex on $\mathbb{R}^n$ and differentiable at point $\mathbf{x}^*$. Then, every multiplier $\boldsymbol{\mu}^*$ satisfying the corresponding KKT conditions at point $\mathbf{x}^*$ is a solution of the dual problem.

If the dual formulation is used for the solution of an optimization problem, then the inverse question related to Theorem 4.6 is when does the solution of (4.105) correspond to the optimal solution of (4.101)? This fundamental question is closely related to the property of *weak and strong duality*.

Theorem 4.7 (Weak duality) For each primal-dual feasible point $\mathbf{x} \in \mathcal{X}_{ad} = \{\mathbf{x} \in \mathcal{X} \mid \mathbf{g}(\mathbf{x}) = \mathbf{0}, \mathbf{h}(\mathbf{x}) \leq \mathbf{0}\}$ and $(\boldsymbol{\lambda}, \boldsymbol{\mu})$ with $\boldsymbol{\mu} \geq \mathbf{0}$, the dual function is a lower bound of the cost function, i.e. $\theta(\boldsymbol{\lambda}, \boldsymbol{\mu}) \leq f(\mathbf{x})$.

In particular, weak duality implies that the maximum $\theta^* = \max_{\boldsymbol{\lambda}, \boldsymbol{\mu} \geq \mathbf{0}} \theta(\boldsymbol{\lambda}, \boldsymbol{\mu})$ of the dual function is always a lower bound for the optimal cost value $f^* = f(\mathbf{x}^*)$ of the primal optimization problem (4.82), i.e.

$$\theta^* \leq f^*.$$

The difference $f^* - \theta^*$ between both values is the so-called *duality gap*. If the duality gap is zero, we speak of *strong duality* with

$$\theta^* = f^*.$$

If $\mathbf{x}^*$ and $(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ is a primal-dual optimal solution and strong duality holds, then the Lagrangian satisfies the *saddle point condition*

$$L(\mathbf{x}^*, \boldsymbol{\lambda}, \boldsymbol{\mu}) \leq L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) \leq L(\mathbf{x}, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) \quad \forall \mathbf{x} \in \mathcal{X}, \boldsymbol{\lambda}, \boldsymbol{\mu} \geq \mathbf{0}. \quad (4.109)$$

Thus, an optimal solution minimizes the Lagrangian w.r.t. the primal variables and maximizes it w.r.t. the dual variables. Vice versa, if $(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ satisfies the saddle point condition (4.109), then $\mathbf{x}^*$ is primal optimal, $(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ is dual optimal and the duality gap is zero.

Theorem 4.8 (Saddle point) A primal-dual point $(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ with $\boldsymbol{\mu}^* \geq \mathbf{0}$ is optimal if and only if the saddle point condition (4.109) is satisfied for all $\mathbf{x} \in \mathcal{X}$ and $(\boldsymbol{\lambda}, \boldsymbol{\mu}) \in \mathbb{R}^{p \times q}$ with $\boldsymbol{\mu} \geq \mathbf{0}$.

Example 4.9 We consider the non-convex optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}} f(\mathbf{x}) = 2x - \frac{1}{2}x^2 - \frac{2}{3}x^3 + \frac{1}{4}x^4 \quad (4.110a)$$

$$\text{s.t. } h(\mathbf{x}) = 4x^2 - 1 \leq 0 \quad (4.110b)$$

with the Lagrangian $L(x, \mu) = 2x - \frac{1}{2}x^2 - \frac{2}{3}x^3 + \frac{1}{4}x^4 + \mu(4x^2 - 1)$. The left side of Figure 4.18 shows $f(x)$ and $h(x)$ as well as the admissible set $\mathcal{X}_{ad} = \{x \in \mathbb{R} \mid -\frac{1}{2} \leq x \leq \frac{1}{2}\}$ as the grey shaded area. In addition, $L(\cdot, \mu)$ is plotted for various values of $\mu \in [0, 3]$ along with the respective minima that correspond to the profile of the dual function $\theta(\mu)$. In accordance with Theorem 4.7, $\theta(\mu)$ is smaller than the cost function $f(x)$ in the admissible set $\mathcal{X}_{ad}$.

The right side of Figure 4.18 plots the dual function $\theta(\mu)$ over μ as well as the optimal value f^* of the cost function as the solution of (4.110). The maximum $\theta^* = \max_{\mu \geq 0} \theta(\mu)$ equals the optimal cost f^* , which shows that the duality gap is zero and strong duality holds. In addition, Figure 4.19 shows a surface plot of the Lagrangian $L(x, \lambda)$. The red lines illustrate that the saddle point condition (4.109) is satisfied at the primal-dual optimal point (x^*, μ^*) .

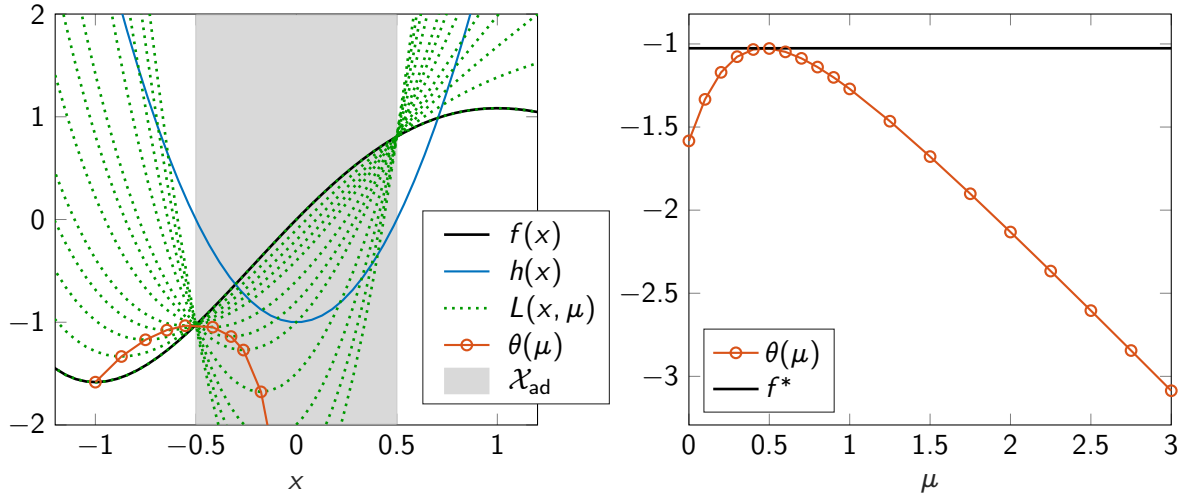


Figure 4.18: Functions $f(x)$, $h(x)$, and $L(x, \mu)$ for discrete values of $\mu \in [0, 3]$ as well as the dual function $\theta(\mu)$ as lower bound on the optimal cost f^* (Example 4.9).

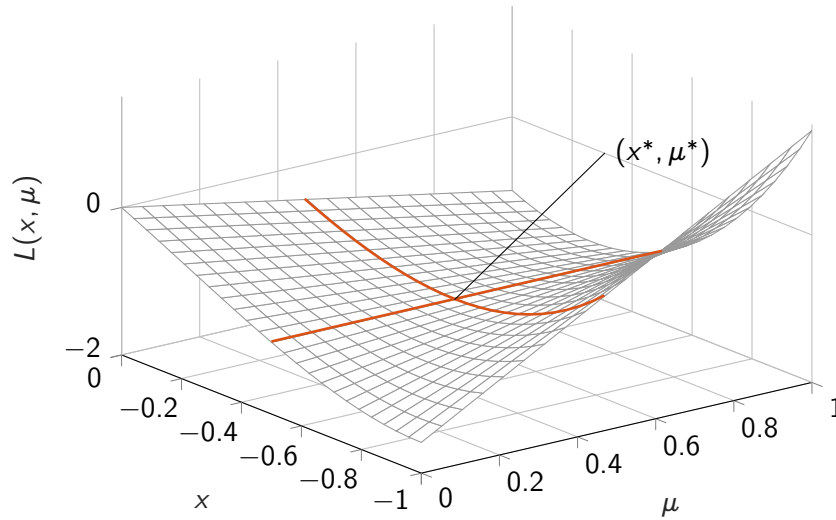


Figure 4.19: Graphical validation of the saddle point condition (4.109) for Example 4.9.

The proof of strong duality typically relies on convexity, although Example 4.9 shows that strong duality can also hold for non-convex problems. Remember that a nonlinear optimization problem with linear equality constraints

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad (4.111a)$$

$$\text{s.t. } \mathbf{Ax} = \mathbf{b} \quad (4.111b)$$

$$\mathbf{h}(\mathbf{x}) \leq \mathbf{0} \quad (4.111c)$$

is convex if the functions f and h_i , $i = 1, \dots, q$ are convex and if $\mathcal{X}$ is convex as well, see Definition 2.7 and Theorem 2.5. A further assumption that is often used in relation with strong duality is the constraint qualification of Slater

$$\exists \mathbf{x} \in \text{relint}(\mathcal{X}) : \mathbf{Ax} = \mathbf{b}, \quad h_i(\mathbf{x}) < 0, \quad i = 1, \dots, q \quad (4.112)$$

that implies that at least one interior point $\mathbf{x}$ of the primal problem must exist.⁹

⁹ If $\mathcal{X}$ contains at least one interior point, then $\text{relint}(\mathcal{X}) = \text{int}(\mathcal{X})$.

Theorem 4.9 (Strong duality) *Let the primal optimization problem (4.111) with $\mathbf{A} \in \mathbb{R}^{p \times n}$ and $\text{rank}(\mathbf{A}) = p$ be convex for $\mathbf{x} \in \mathbb{R}^n$ and feasible. Moreover, suppose that the minimum $f^* = f(\mathbf{x}^*)$ is finite and the constraint qualification (4.112) is satisfied. Then, the following statements are true:*

- *There exists no duality gap, i.e. $\theta^* = f^*$.*
- *There exists at least one pair of dual variables with $\theta(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) = \theta^*$.*
- *The saddle point condition (4.109) is satisfied.*
- *If $\mathbf{x}^*$ is a global primal solution, then $(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ is a global solution of the dual problem (4.105).*

The assumptions in Theorem 4.9 do not automatically imply the uniqueness of the primal solution $\mathbf{x}^*$. This can be achieved by additionally assuming strict convexity of the Lagrangian $L(\cdot, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ [14]. Moreover, convexity alone is not sufficient to guarantee strong duality. The following example illustrates this point.

Example 4.10 (Convex problem with duality gap [3]) *Consider the problem*

$$\min_{\mathbf{x} \in \mathcal{X}} e^{-\sqrt{x_1 x_2}} \quad (4.113a)$$

$$\text{s.t. } x_1 = 0 \quad (4.113b)$$

with the convex cost function $f(\mathbf{x}) = e^{-\sqrt{x_1 x_2}}$ on the domain of definition $\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^2 \mid \mathbf{x} \geq \mathbf{0}\}$. The minimum point is $\mathbf{x}^ = [0, x_2^*]^T$ with $f^* = f(\mathbf{x}^*) = 1$. With the Lagrangian $L(\mathbf{x}, \lambda) = e^{-\sqrt{x_1 x_2}} + \lambda x_1$, the dual function is defined by*

$$\theta(\lambda) = \inf_{\mathbf{x} \in \mathcal{X}} (e^{-\sqrt{x_1 x_2}} + \lambda x_1) = \begin{cases} 0 & \text{if } \lambda \geq 0 \\ -\infty & \text{else.} \end{cases} \quad (4.114)$$

The first case implies $x_1 x_2 \rightarrow \infty$ and in addition $x_1 \rightarrow 0$ if $\lambda > 0$. This becomes clear in view of $\mathbf{x} = [1/\xi, \xi^2]^T$ and

$$\lim_{\xi \rightarrow \infty} L(\mathbf{x}, \lambda \geq 0) = \lim_{\xi \rightarrow \infty} e^{-\sqrt{\xi}} + \frac{\lambda}{\xi} = 0.$$

We thus get $\theta^ = \max_{\lambda} \theta(\lambda) = 0$ and $f^* - \theta^* = 1 > 0$, which shows that a duality gap exists although the problem is convex. The reason for this is that Slater's constraint qualification (4.112) is not satisfied.*

Exercise 4.12 *Investigate strong duality for the optimization problems*

$$\begin{array}{ll} \min_{\mathbf{x} \in \mathbb{R}} x^4 & \min_{\mathbf{x} \in \mathbb{R}} x^2 \\ \text{s.t. } x^2 \geq 1 & \text{s.t. } x^4 \geq 1. \end{array}$$

If strong duality holds, the dual formulation can be used for the solution of the primal optimization problem (4.101) or (4.111). Formally, the computation of an optimal solution $\mathbf{x}^*$ requires to determine the dual function as the solution of (4.103) and its maximization as the dual problem (4.105) to obtain $(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$. The optimal primal variables $\mathbf{x}^*$ then follow as the minimizing arguments of (4.103). The augmented Lagrangian method solves this max-min-problem in an iterative manner.

4.6.2 Augmented Lagrangian method

The augmented Lagrangian method can be applied to equality-constrained nonlinear optimization problems

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad (4.115a)$$

$$\text{s.t. } \mathbf{g}(\mathbf{x}) = \mathbf{0} \quad (4.115b)$$

with the constraint functions $\mathbf{g} = [g_1 \dots g_p]^T$. The extension to inequality constraints is described in Section 4.6.3. The *augmented Lagrangian function* is defined by

$$L_c(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}) + \frac{c}{2} \|\mathbf{g}(\mathbf{x})\|^2 \quad (4.116)$$

with the Lagrange multipliers $\boldsymbol{\lambda}^T \in \mathbb{R}^p$ and an additional quadratic penalty term with parameter $c \geq 0$ that penalizes the deviation of the constraints $\mathbf{g}(\mathbf{x}) = \mathbf{0}$. The dual problem based on the augmented Lagrangian function (4.116) reads

$$\max_{\boldsymbol{\lambda}} \inf_{\mathbf{x} \in \mathcal{X}} L_c(\mathbf{x}, \boldsymbol{\lambda}). \quad (4.117)$$

If strong duality holds and $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ is a primal-dual solution, then the augmented Lagrangian function satisfies the saddle point condition

$$L_c(\mathbf{x}^*, \boldsymbol{\lambda}) \leq L_c(\mathbf{x}^*, \boldsymbol{\lambda}^*) \leq L_c(\mathbf{x}, \boldsymbol{\lambda}^*) \quad \forall \mathbf{x}, \boldsymbol{\lambda}. \quad (4.118)$$

The inverse statement is particularly important for the augmented Lagrangian method: if (4.118) is satisfied, then $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ is a primal-dual solution and the duality gap is zero.

The additional quadratic penalty of the constraint in the Lagrangian function (4.116) has a positive effect on the numerical conditioning and also affects the duality of the problem. In particular, it can be shown under certain assumptions that strong duality holds even for non-convex problems if the penalty parameter $c > 0$ is sufficiently large [16]. The following theorem [14] provides the interesting result that the penalty term does not need to be chosen unnecessarily large.

Theorem 4.10 *Let $\mathbf{x}^*$ be a local solution of (4.115), for which the constraint qualification (4.26) as well as the KKT conditions (4.29) and the sufficient optimality conditions (4.37) for $\boldsymbol{\lambda} = \boldsymbol{\lambda}^*$ are satisfied. Then, there exists a lower bound $\bar{c} > 0$ such that $L_c(\mathbf{x}, \boldsymbol{\lambda}^*)$ has a strict local minimum at $\mathbf{x} = \mathbf{x}^*$ for all $c \geq \bar{c}$.*

The augmented Lagrangian algorithm is based on the idea to compute $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ by iteratively approaching the saddle point (4.118). Table 4.5 summarizes the basic algorithm of the augmented Lagrangian method. If $\mathbf{x}$ is unconstrained, i.e. $\mathcal{X} = \mathbb{R}^n$, numerical methods of unconstrained optimization can be used for the solution, see Chapter 3. Box constraints of the form $\mathcal{X} = [\mathbf{x}^-, \mathbf{x}^+]$ can be considered via projection onto the respective boundary. The minimization problem is often solved approximately, e.g. by using a convergence criterion of the form

$$\|\nabla_{\mathbf{x}} L_{c^k}(\mathbf{x}^k, \boldsymbol{\lambda}^k)\| \leq \varepsilon_L^k, \quad (4.119)$$

where $\varepsilon_L^k > 0$ is reduced over the iterations k .

Initialization:

$k_{\max}$	Max. number of iterations
$\boldsymbol{\lambda}^1$	Initial dual point
$c^1 > 0$	Initial penalty parameter
$\varepsilon_g > 0$	Convergence tolerance
for $k = 1, \dots, k_{\max}$ do	
$\mathbf{x}^k = \arg \min_{\mathbf{x} \in \mathcal{X}} L_{c^k}(\mathbf{x}, \boldsymbol{\lambda}^k)$	Update of primal variables (minimization)
if $\ \mathbf{g}^k\ \leq \varepsilon_g$ or $k = k_{\max}$ then	Convergence check with $\mathbf{g}^k = \mathbf{g}(\mathbf{x}^k)$
return $\mathbf{x}^k$	
end	
$\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + c^k \mathbf{g}^k$	Update of dual variables (steepest ascent)
if $k > 1$ then	
$c^{k+1} = \psi_c(c^k, \ \mathbf{g}^k\ , \ \mathbf{g}^{k-1}\)$	Update of penalty parameter
end	
end	

Table 4.5: Structure of augmented Lagrangian algorithm.

The dual variables $\boldsymbol{\lambda}$ in Table 4.5 are updated in the direction of steepest ascent with $\nabla_{\boldsymbol{\lambda}} L_c = \mathbf{g}(\mathbf{x}^k)$. The step size is not computed via a line search because L_c is linear in $\boldsymbol{\lambda}$ which implies that no finite maximum point exists. Instead, the step size corresponds to the penalty parameter c , which can be illustrated by comparing the stationarity conditions for the minimizer $\mathbf{x}^k$ of the augmented Lagrangian (4.116)

$$\nabla_{\mathbf{x}} L_{c^k}(\mathbf{x}^k, \boldsymbol{\lambda}^k) = \nabla f(\mathbf{x}^k) + \sum_{i=1}^p (\lambda_i^k + c^k g_i(\mathbf{x}^k)) \nabla g_i(\mathbf{x}^k) = \mathbf{0} \quad (4.120)$$

with the optimal solution of (4.115)

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \nabla f(\mathbf{x}^*) + \sum_{i=1}^p \lambda_i^* \nabla g_i(\mathbf{x}^*) = \mathbf{0}. \quad (4.121)$$

This shows that the update law $\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + c^k \mathbf{g}(\mathbf{x}^k)$ in Table 4.5 corresponds to the optimal multiplier $\boldsymbol{\lambda}^*$. Another illustration for the choice of c as step size follows from considering the *proximal maximization problem*

$$\boldsymbol{\lambda}^{k+1} = \arg \max_{\boldsymbol{\lambda}} \left(L_{c^k}(\mathbf{x}^k, \boldsymbol{\lambda}) - \frac{1}{2c^k} \|\boldsymbol{\lambda} - \boldsymbol{\lambda}^k\|^2 \right) \quad (4.122)$$

that penalizes the distance from the previous iteration point as a trade-off to the maximization of L_c . The solution of this strictly convex problem corresponds to the update law in Table 4.5.

The update of the penalty parameter is often done heuristically. If the constraint violation $\|\mathbf{g}^k\| = \|\mathbf{g}(\mathbf{x}^k)\|$ is not reduced sufficiently fast, it is reasonable to increase c^k . A simple update rule therefore is

$$c^{k+1} = \psi_c(c^k, \|\mathbf{g}^k\|, \|\mathbf{g}^{k-1}\|) = \begin{cases} \beta c^k & \text{if } \|\mathbf{g}^k\| > \gamma \|\mathbf{g}^{k-1}\| \\ c^k & \text{else} \end{cases} \quad (4.123)$$

with $\gamma < 1$ and $\beta > 1$. Recommended values are $\gamma = 0.25$ and $\beta \in [5, 10]$, see [3]. As already mentioned, the minimization problem $\min_{\mathbf{x} \in \mathcal{X}} L_{c^k}(\mathbf{x}, \boldsymbol{\lambda}^k)$ in Table 4.5 is often solved approximately. In this case, an adaptive update of the penalty parameter c^k in both directions that also allows a temporary reduction of $c^{k+1} < c^k$ has shown to be advantageous in practice, see e.g. LANCELOT [5] or the MPC toolbox GRAMPC [6]. In particular, a decrease of c^k makes sense if the constraints already satisfy the convergence criterion $\|\mathbf{g}^k\| \leq \varepsilon_g$ but the minimization of $L_{c^k}(\mathbf{x}, \boldsymbol{\lambda}^k)$ is not yet sufficiently accurate in terms of the criterion (4.119).

The following theorem [3, 14] proves that the augmented Lagrangian algorithm has linear convergence properties at least locally around an optimal solution $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ and for sufficiently large penalty parameters.

Theorem 4.11 (Augmented Lagrangian algorithm) *Let the assumptions of Theorem 4.10 be satisfied at the point $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ and suppose that $\bar{c} > 0$ is chosen according to Theorem 4.10. Then, there exist constants $\delta_c, \delta_x, C > 0$ such that for all $\boldsymbol{\lambda}^k$ and c^k with $\|\boldsymbol{\lambda}^k - \boldsymbol{\lambda}^*\| \leq \delta_c \bar{c}$ and $c^k \geq \bar{c}$ the problem*

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{X}} \quad & L_{c^k}(\mathbf{x}, \boldsymbol{\lambda}^k) \\ \text{s.t.} \quad & \|\mathbf{x} - \mathbf{x}^*\| \leq \delta_x \end{aligned}$$

has a unique solution $\mathbf{x}^k$. Moreover, $\mathbf{x}^k$ and $\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + c^k \mathbf{g}(\mathbf{x}^k)$ satisfy

$$\|\mathbf{x}^k - \mathbf{x}^*\| \leq C \|\boldsymbol{\lambda}^k - \boldsymbol{\lambda}^*\| / c^k, \quad \|\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^*\| \leq C \|\boldsymbol{\lambda}^k - \boldsymbol{\lambda}^*\| / c^k. \quad (4.124)$$

Exercise 4.13 ([3]) *Investigate the solution behavior of the augmented Lagrangian algorithm for the example problems*

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^2} \quad & \frac{1}{2}(x_1^2 + x_2^2) & \min_{\mathbf{x} \in \mathbb{R}^2} \quad & \frac{1}{2}(-x_1^2 + x_2^2) \\ \text{s.t.} \quad & x_1 = 1 & \text{s.t.} \quad & x_1 = 1 \end{aligned}$$

and determine the lower bound $\bar{c} > 0$ of the penalty parameter $c^k \geq \bar{c}$ for the convergence of the algorithm.

4.6.3 Consideration of inequality constraints

In the last section, the augmented Lagrangian method was introduced for equality-constrained problems (4.115). Inequality constraints

$$\mathbf{h}(\mathbf{x}) \leq \mathbf{0} \quad (4.125)$$

with $\mathbf{h} = [h_1, \dots, h_q]^\top$ can be added to the equality constraints (4.115b) by introducing slack variables $\mathbf{s} \geq \mathbf{0}$

$$\hat{\mathbf{g}}(\mathbf{x}, \mathbf{s}) = \begin{bmatrix} \mathbf{g}(\mathbf{x}) \\ \mathbf{h}(\mathbf{x}) + \mathbf{s} \end{bmatrix} = \mathbf{0}. \quad (4.126)$$

The augmented Lagrangian function then reads

$$L_c(\mathbf{x}, \mathbf{s}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \hat{\mathbf{g}}(\mathbf{x}, \mathbf{s}) + \frac{c}{2} \|\hat{\mathbf{g}}(\mathbf{x}, \mathbf{s})\|^2 \quad (4.127)$$

with the multipliers $\boldsymbol{\lambda}^\top = [\boldsymbol{\lambda}_g^\top, \boldsymbol{\lambda}_h^\top] \in \mathbb{R}^{p+q}$ and the penalty parameter $c > 0$. The corresponding dual problem (4.117) is defined by

$$\max_{\boldsymbol{\lambda}} \inf_{\mathbf{x} \in \mathcal{X}, \mathbf{s} \geq \mathbf{0}} L_c(\mathbf{x}, \mathbf{s}, \boldsymbol{\lambda}). \quad (4.128)$$

The inner minimization w.r.t. $\mathbf{s} \geq \mathbf{0}$ reduces to the strictly convex problem ($c > 0$)

$$\min_{\mathbf{s} \geq \mathbf{0}} \left(\boldsymbol{\lambda}_h^\top (\mathbf{h}(\mathbf{x}) + \mathbf{s}) + \frac{c}{2} \|\mathbf{h}(\mathbf{x}) + \mathbf{s}\|^2 \right). \quad (4.129)$$

Its solution follows from the stationarity condition and projection onto the admissible set $\mathbf{s} \geq \mathbf{0}$

$$\mathbf{s} = \mathbf{max} \{ \mathbf{0}, -\boldsymbol{\lambda}_h/c - \mathbf{h}(\mathbf{x}) \}, \quad (4.130)$$

where the vector-valued **max**-function is to be understood elementwise. Inserting (4.130) into (4.126) leads to the new form of the equality constraints

$$\bar{\mathbf{g}}_c(\mathbf{x}, \boldsymbol{\lambda}) = \begin{bmatrix} \mathbf{g}(\mathbf{x}) \\ \mathbf{max} \{ \mathbf{h}(\mathbf{x}), -\boldsymbol{\lambda}_h/c \} \end{bmatrix} = \mathbf{0} \quad (4.131)$$

as well as the adapted augmented Lagrangian function

$$\bar{L}_c(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \bar{\mathbf{g}}_c(\mathbf{x}, \boldsymbol{\lambda}) + \frac{c}{2} \|\bar{\mathbf{g}}_c(\mathbf{x}, \boldsymbol{\lambda})\|^2 \quad (4.132)$$

for the dual problem

$$\max_{\boldsymbol{\lambda}} \inf_{\mathbf{x} \in \mathcal{X}} \bar{L}_c(\mathbf{x}, \boldsymbol{\lambda}). \quad (4.133)$$

In summary, inequality constraints can be incorporated into the augmented Lagrangian algorithm by replacing $\mathbf{g}$ und L_c in Table 4.5 with (4.131) and (4.132) and adapting the dimension of $\boldsymbol{\lambda}$ accordingly. Note that the new equality constraints (4.131) now depend on the multipliers $\boldsymbol{\lambda}$ and the penalty parameter c .

4.6.4 Remarks

The advantage of the augmented Lagrangian method is that the inner minimization problem

$$\inf_{\mathbf{x} \in \mathcal{X}} L_c(\mathbf{x}, \boldsymbol{\lambda})$$

is often easier to solve than the original constrained optimization problem due to the fact that it is only constrained to the set $\mathcal{X}$. Over the last years, the dual formulation became popular in connection with variants of the gradient method, especially the accelerated gradient method [13]. A survey over different primal-dual solution methods is given in [11].

The rather simple structure of the augmented Lagrangian method also offers advantages for the implementation on embedded systems or hardware with limited resources such as electronic control units (ECU) in the automotive sector or programmable logic controllers (PLC) in automation systems. Embedded systems require algorithms that allow for a fast computation of single iterations (at the price of suboptimality) with numerical robustness despite limited numerical accuracy [7, 8].

The duality theory is also important for distributed optimization problems and networked systems as it enables the decoupling of central optimization problem and their distributed or parallelized solution. A popular method in this regard is the *Alternating Direction Method of Multipliers (ADMM)* [10, 4] for optimization problems of the form

$$\min_{\mathbf{x}, \mathbf{y}} f_1(\mathbf{x}) + f_2(\mathbf{y}) \quad (4.134a)$$

$$\text{s.t.} \quad \underbrace{\mathbf{Ax} + \mathbf{By} - \mathbf{b}}_{\mathbf{g}(\mathbf{x}, \mathbf{y})} = \mathbf{0} \quad (4.134b)$$

with the optimization variables $(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{n_1 \times n_2}$, the cost functions $f_1(\mathbf{x})$ and $f_2(\mathbf{y})$ and the coupling conditions (4.134b). With the augmented Lagrangian function

$$L_c(\mathbf{x}, \mathbf{y}, \boldsymbol{\lambda}) = f_1(\mathbf{x}) + f_2(\mathbf{y}) + \boldsymbol{\lambda}^T \mathbf{g}(\mathbf{x}, \mathbf{y}) + \frac{c}{2} \|\mathbf{g}(\mathbf{x}, \mathbf{y})\|^2,$$

the ADMM iteration scheme is defined by

$$\mathbf{x}^{k+1} = \arg \min_{\mathbf{x}} L_{c^k}(\mathbf{x}, \mathbf{y}^k, \boldsymbol{\lambda}^k) \quad (4.135a)$$

$$\mathbf{y}^{k+1} = \arg \min_{\mathbf{y}} L_{c^k}(\mathbf{x}^{k+1}, \mathbf{y}, \boldsymbol{\lambda}^k) \quad (4.135b)$$

$$\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + c^k \mathbf{g}^{k+1} \quad (4.135c)$$

$$c^{k+1} = \psi_c(c^k, \|\mathbf{g}^k\|, \|\mathbf{g}^{k+1}\|) \quad (4.135d)$$

with $\mathbf{g}^k = \mathbf{g}(\mathbf{x}^k, \mathbf{y}^k)$. The sequential solution of the optimization problems for $\mathbf{x}$ and $\mathbf{y}$ (“alternating directions”) enables the decomposition of the overall problem. ADMM has become popular in the field of distributed model predictive control of networked systems or in image processing and digital communication for solving high-dimensional optimization problems in a distributed manner.

4.7 Software overview

The following lines give a selection of software tools for solving linear, quadratic, and nonlinear optimization problems.

Linear optimization

- linprog: MATLAB Optimization Toolbox (commercial)
- CPLEX (commercial)
<http://www.ilog.com/products/cplex>
- GLPK: “GNU Linear Programming Kit” (free)
<http://www.gnu.org/software/glpk>
- lp_solve: Mixed-Integer Linear Optimierung (free)
<http://lpsolve.sourceforge.net>
- CVXGEN (commercial, academic license available)
<https://cvxgen.com>

Quadratic optimization

- quadprog: MATLAB Optimization Toolbox (commercial)
- qpOASES (free)
<http://github.com/coin-or/qpOASES>
- OOQP (free)
<http://pages.cs.wisc.edu/~swright/ooqp>
- CPLEX (commercial)
<http://www.ilog.com/products/cplex>
- LOQO (commercial)
<http://vanderbei.princeton.edu/loqomenu.html>
- CVXGEN (commercial, academic license available)
<https://cvxgen.com>

Nonlinear optimization

- fmincon: MATLAB Optimization Toolbox (commercial)
- IPOPT (free)
<https://github.com/coin-or/Ipopt>
- SNOPT (commercial, student license available)
<http://ccom.ucsd.edu/~optimizers>
- LOQO (commercial)
<http://vanderbei.princeton.edu>
- MINOS (commercial)
http://www.sbsi-sol-optimize.com/asp/sol_product_minos.htm

Modeling languages & automatic code generation

Most of the mentioned optimization tools have an interface for MATLAB or for the modeling language AMPL (e.g. LOQO, GLPK, IPOPT) or GAMS (e.g. MINOS). These languages offer a symbolic syntax to formulate optimization problems. Moreover, automatic code generation as e.g. offered by CasADi is of growing importance in parameter and dynamic optimization.

- AMPL: "A Mathematical Programming Language"
<http://www.ampl.com>
- GAMS: "General Algebraic Modeling System"
<http://www.gams.com>
- CasADi: Code generation including automatic differentiation
<https://web.casadi.org>

4.8 References

- [1] G. Allaire. *Numerical Analysis and Optimization*. Oxford: Oxford University Press, 2007.
- [2] J.A.E. Andersson, J. Gillis, G. Horn, J.B. Rawlings, and M. Diehl. “CasADi – A software framework for nonlinear optimization and optimal control”. In: *Mathematical Programming Computation* (2018). Online verfügbar.
- [3] D.P. Bertsekas. *Nonlinear Programming*. Belmont. Athena Scientific, 1999.
- [4] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. “Distributed optimization and statistical learning via the alternating direction method of multipliers”. In: *Foundations and Trends in Machine Learning* 3.1 (2011), pp. 1–122.
- [5] A. R. Conn, G.I.M. Gould, and P. L. Toint. *LANCELOT: A Fortran Package for Large-Scale Nonlinear Optimization (Release A)*. Berlin, Germany: Springer-Verlag, 1992.
- [6] T. Englert, A. Völz, F. Mesmer, S. Rhein, and K. Graichen. “A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach (GRAMPC)”. In: *Optimization and Engineering* 20.3 (2019), pp. 769–809. DOI: 10.1007/s11081-018-9417-2.
- [7] H.J. Ferreau, S. Almér, R. Verschueren, M. Diehl, D. Frick, A. Domahidi, J.L. Jerez, G. Stathopoulos, and C. Jones. “Embedded optimization methods for industrial automatic control”. In: *Proc. 20th IFAC World Congress*. 2017, pp. 13736–13751.
- [8] R. Findeisen, K. Graichen, and M. Mönnigmann. “Eingebettete Optimierung in der Regelungstechnik – Grundlagen und Herausforderungen”. In: *at – Automatisierungstechnik* 66.11 (2018), pp. 877–902.
- [9] A. Forsgren, P.E. Gill, and M.H. Wright. “Interior methods for nonlinear optimization”. In: *SIAM Review* 44.4 (2002), pp. 525–597.
- [10] M. Fortin and R. Glowinski, eds. *Augmented Lagrangian Methods: Applications to the Solution of Boundary-Value Problems*. Amsterdam: North-Holland, 1983.
- [11] D. Kouzoupis, H.J. Ferreau, H. Peyrl, and M. Diehl. “First-order methods in embedded nonlinear model predictive control”. In: *Proc. 2015 European Control Conference (ECC)*. 2015, pp. 2617–2622.
- [12] J. Mattingley and S. Boyd. “CVXGEN: A code generator for embedded convex optimization”. In: *Optimization and Engineering* 13.1 (2012), pp. 1–27.
- [13] Y. Nesterov. “Introductory lectures on convex optimization: A basic course”. In: *Applied Optimization*. Ed. by P.M. Pardalos and D.W. Hearn. Vol. 87. Applied Optimization. Norwell, MA, USA: Kluwer Academic Publishers, 2004.
- [14] J. Nocedal and S.J. Wright. *Numerical Optimization*. New York: Springer, 2006.
- [15] M. Papageorgiou. *Optimierung*. München: Oldenbourg, 1991.
- [16] R.T. Rockafellar. “Augmented Lagrange multiplier functions and duality in nonconvex programming”. In: *SIAM Journal on Control* 12.2 (1974), pp. 268–285.
- [17] S.J. Wright. *Primal-Dual Interior-Point Methods*. Society for Industrial and Applied Mathematics (SIAM), 1997.

5 Dynamic optimization

The previous chapters focused on parameter optimization problems with the optimization variables $\mathbf{x}$ being elements of the Euclidean space $\mathbb{R}^n$. In what follows, we focus on dynamic optimization problems, where a *functional* is minimized w.r.t. a function of an independent variable, in most cases a function of time. The most common application of dynamic optimization is the computation of an optimal control trajectory $\mathbf{u}^*(t)$ for a dynamical system based on the solution of an *optimal control problem*.

5.1 Problem statement

A quite general class of optimal control problems is represented by

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{u}) = V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt \quad (5.1a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (5.1b)$$

$$\mathbf{g}(\mathbf{x}(t_f), t_f) = \mathbf{0} \quad (5.1c)$$

$$\mathbf{h}(\mathbf{x}(t), \mathbf{u}(t), t) \leq \mathbf{0} \quad \forall t \in [t_0, t_f]. \quad (5.1d)$$

The problem is characterized by the cost functional (5.1a), the system dynamics (5.1b) with state $\mathbf{x} \in \mathbb{R}^n$ and control $\mathbf{u} \in \mathbb{R}^m$, the terminal condition (5.1c), the constraints (5.1d) as well as the end time t_f . A standing assumption we have to impose – similar to the parameter optimization case – is that all functions are at least continuously differentiable in their arguments.

Note that the cost functional (5.1a) formally depends on the control and the state. However, $J(\mathbf{u})$ is usually written in sole dependency of the control, if it is assumed that the state trajectory $\mathbf{x}(t)$ is uniquely determined by the dynamics (5.1b) for a given control trajectory $\mathbf{u}(t)$, $t \in [0, T]$.

Terminal condition (5.1c)

The dynamics and boundary conditions in (5.1b) and (5.1c) define a *transition problem* from the initial state $\mathbf{x}(t_0) = \mathbf{x}_0$ to the terminal condition (5.1c) at the end time $t = t_f$. The general nonlinear form (5.1c)¹ often breaks down to partial terminal conditions of the form

$$x_i(t_f) = x_{f,i}, \quad i \in \mathcal{I}_f. \quad (5.2)$$

The set $\mathcal{I}_f$ includes the indices of the state variables x_i that are fixed at the end time t_f . If the full state vector $\mathbf{x}(t_f) = \mathbf{x}_f$ is fixed at t_f , we have $\mathcal{I}_f = \{1, \dots, n\}$. The other special

¹ An example of a nonlinear terminal condition is the optimization of the flight trajectory of a space launcher rocket that brings a satellite to a geostationary orbit.

case is a free terminal state with $\mathcal{I}_f = \{\}$. In this chapter, we will mainly focus on the partial terminal condition (5.2). The general form (5.1c) is briefly addressed in Section 5.3.6.

Cost functional (5.1a)

In general, the transition problem (5.1b), (5.1c) has infinitely many solutions $\mathbf{u}(t)$, $t \in [t_0, t_f]$, from which an *optimal control* $\mathbf{u}^*(t)$ is sought that minimizes the cost functional (5.1a). The general form (5.1a) includes a *terminal cost* or *Mayer term* $V(\mathbf{x}(t_f), t_f)$ and an *integral cost* $l(\mathbf{x}, \mathbf{u}, t)$. Their combination leads to the following cases

$$J(\mathbf{u}) = \begin{cases} V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt & \text{Bolza form} \\ \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt & \text{Lagrange form} \\ V(\mathbf{x}(t_f), t_f) & \text{Mayer form.} \end{cases} \quad (5.3)$$

The Bolza and Lagrange forms can always be expressed in Mayer form by introducing the integral term as new state variable

$$\dot{x}_{n+1} = l(\mathbf{x}, \mathbf{u}, t), \quad x_{n+1}(t_0) = 0 \quad (5.4)$$

and adding $x_{n+1}(t_f)$ to the terminal cost $V(\mathbf{x}(t_f), t_f)$. The most common examples of the integral cost are

$J(\mathbf{u}) = \frac{1}{2} \int_{t_0}^{t_f} \mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) dt$	quadratic in the state ($\mathbf{Q}$ symmetric & $\mathbf{Q} > \mathbf{0}$)
$J(\mathbf{u}) = \frac{1}{2} \int_{t_0}^{t_f} \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t) dt$	energy optimal criterion ($\mathbf{R}$ symmetric & $\mathbf{R} > \mathbf{0}$)
$J(\mathbf{u}) = \frac{1}{2} \int_{t_0}^{t_f} \mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) + \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t) dt$	general quadratic form ($\mathbf{Q}, \mathbf{R}$ symmetric & $\mathbf{Q} \geq \mathbf{0}, \mathbf{R} > \mathbf{0}$)
$J(\mathbf{u}) = \int_{t_0}^{t_f} 1 dt = t_f - t_0$	time optimality
$J(\mathbf{u}) = \int_{t_0}^{t_f} 1 + \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t) dt$	combination of time/energy optimality.

The denomination *energy optimality* for a quadratic control term $\mathbf{u}^T \mathbf{R} \mathbf{u}$ stems from interpreting the control as an electrical signal.

Example 5.1 Several examples of optimal control problems were considered in Chapter 1. In case of the Goddard rocket (Example 1.4), the terminal cost $V(\mathbf{x}(t_f), t_f) = -h(t_f)$ maximizes the altitude. For the economic model in Example 1.5, $V(\mathbf{x}(t_f), t_f) = -K^\kappa(t_f)$ describes the capital of inheritance. The combination of time/energy optimality was considered for the inverted pendulum in Example 1.3.

End time t_f

The end time or optimization horizon t_f is either pre-defined (fixed) or unspecified (free). In the free end time case, t_f is part of the solution of the problem (5.1). The end time t_f plays a major role in this regard and often decides upon whether a solution to an optimal control problem exists or not (also see Example 1.3).

Constraints (5.1d)

Constraints are present in many practical problems. In most cases, the general nonlinear form (5.1d) can be reduced to input constraints

$$\mathbf{u}(t) \in U \subseteq \mathbb{R}^m \quad \forall t \in [t_0, t_f] \quad (5.5)$$

and/or state constraints

$$\mathbf{x}(t) \in X \subseteq \mathbb{R}^n \quad \forall t \in [t_0, t_f]. \quad (5.6)$$

Examples of input constraints are voltage or current limits or a maximum force or torque of an actuator. This chapter focuses on the unconstrained and the input-constrained case (5.5). The rigorous mathematical treatment of state constraints (5.6) or general constraints of the form (5.1d) is significantly harder than the input-constrained case and is only briefly addressed in Section 5.5.5.

5.2 Calculus of variations

The consideration of dynamic optimization problems is more complex than in the parameter optimization case. An elegant framework for solving dynamic optimization problems is the *calculus of variations* that was developed at the end of the 17th and at the beginning of the 18th century. The most prominent names that have contributed to the calculus of variations are *Bernoulli*, *Euler*, and *Lagrange*.

For a general introduction into the calculus of variations, we consider a general functional to be minimized before the results are applied to unconstrained optimal control problems (Section 5.3).

5.2.1 Fundamentals

The previous parameter or finite-dimensional optimization was concerned with the minimization of a function $f(\mathbf{x})$ subject to optional constraints, where the optimization variables $\mathbf{x}$ are elements of the Euclidean space $\mathbb{R}^n$. Contrary to this, dynamic optimization minimizes a *functional*

$$J(\mathbf{x}^*) = \min_{\mathbf{x} \in \mathcal{X}} J(\mathbf{x}) \quad (5.7)$$

with $J : \mathcal{X} \rightarrow \mathbb{R}$, where $\mathcal{X}$ is a suitable *function space*. An example is the functional

$$J(\mathbf{x}) = \int_{t_0}^{t_f} \phi(\mathbf{x}(t), \dot{\mathbf{x}}(t), t) dt \quad (5.8)$$

with the optimal time function $\mathbf{x}^*(t)$, $t \in [t_0, t_f]$ that minimizes $J(\mathbf{x})$. The functional (5.8) is also known as *Lagrange problem* in the calculus of variations and will be investigated in Section 5.2.2.

In the spirit of dynamic optimization, the time function $\mathbf{x}(t)$ is also called *trajectory*. The set $\mathcal{X}_{ad}$ denotes the respective admissible set. For instance, if the minimization of the functional (5.7) is subject to the boundary conditions

$$\mathbf{x}(t_0) = \mathbf{x}_0, \quad \mathbf{x}(t_f) = \mathbf{x}_f, \quad (5.9)$$

the admissible set is defined by

$$\mathcal{X}_{ad} = \{\mathbf{x} \in \mathcal{X} \mid \mathbf{x}(t_0) = \mathbf{x}_0, \mathbf{x}(t_f) = \mathbf{x}_f\}. \quad (5.10)$$

A point $\mathbf{x}^* = \mathbf{x}^*(t)$ (i.e. trajectory) is a global minimum point of a functional $J(\mathbf{x})$ if

$$J(\mathbf{x}^*) \leq J(\mathbf{x}) \quad \forall \mathbf{x} \in \mathcal{X}_{ad}. \quad (5.11)$$

The definition of a *local* minimum, however, is based on a suitable norm $\|\cdot\|$ of $\mathcal{X}$

$$\exists \alpha > 0 : \quad J(\mathbf{x}^*) \leq J(\mathbf{x}) \quad \forall \mathbf{x} \in \{\mathbf{x} \in \mathcal{X}_{ad} \mid \|\mathbf{x} - \mathbf{x}^*\| \leq \alpha\}. \quad (5.12)$$

The calculus of variations often considers the class of continuous (piecewise) differentiable functions as function space with

- $\mathcal{C}^k[t_0, t_f]$: Class of k -times continuously differentiable functions on $[t_0, t_f]$
- $\bar{\mathcal{C}}^k[t_0, t_f]$: Class of k -times piecewise continuously differentiable functions on $[t_0, t_f]$
- $\mathcal{C}_n^k[t_0, t_f]$: Class of functions $\mathbf{x} = [x_1, \dots, x_n]^\top$ with $x_i \in \mathcal{C}^k[t_0, t_f]$, $i = 1, \dots, n$
- $\bar{\mathcal{C}}_n^k[t_0, t_f]$: Class of functions $\mathbf{x} = [x_1, \dots, x_n]^\top$ with $x_i \in \bar{\mathcal{C}}^k[t_0, t_f]$, $i = 1, \dots, n$.

A suitable function norm to equip $\mathcal{X}$ with is the supremum norm

$$\|\mathbf{x}\|_\infty = \|\mathbf{x}(\cdot)\|_\infty = \max_{t \in [t_0, t_f]} \|\mathbf{x}(t)\| \quad \text{or} \quad \|\mathbf{x}\|_\infty = \|\mathbf{x}(\cdot)\|_\infty = \max_{t \in [t_0, t_f]} \|\mathbf{x}(t)\| \quad (5.13)$$

with an associated vector norm $\|\mathbf{x}(t)\|$ in $\mathbb{R}^n$.

The generalization of the gradient $\nabla f(\mathbf{x})$ of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ to the function space setting is the *Gâteaux derivative*, also known as *first variation* of the functional $J(\mathbf{x})$.

Definition 5.1 (First variation / Gâteaux derivative) The first variation or Gâteaux derivative of the functional $J : \mathcal{X} \rightarrow \mathbb{R}$ at point $\mathbf{x} \in \mathcal{X}$ in direction of $\mathbf{y} \in \mathcal{X}$ is defined by

$$\delta J(\mathbf{x}; \mathbf{y}) = \lim_{\varepsilon \rightarrow 0} \frac{J(\mathbf{x} + \varepsilon \mathbf{y}) - J(\mathbf{x})}{\varepsilon} = \left. \frac{dJ(\mathbf{x} + \varepsilon \mathbf{y})}{d\varepsilon} \right|_{\varepsilon=0}. \quad (5.14)$$

If $\delta J(\mathbf{x}; \mathbf{y})$ exists for all $\mathbf{y} \in \mathcal{X}$, then J is Gâteaux differentiable at point $\mathbf{x} \in \mathcal{X}$.

Example 5.2 For the functional $J(x) = \int_{t_0}^{t_f} x(t)^2 + x(t) dt$ with $x \in \mathcal{C}^1[t_0, t_f]$, the Gâteaux derivative in direction $y \in \mathcal{C}^1[t_0, t_f]$ is

$$\begin{aligned} \delta J(x; y) &= \lim_{\varepsilon \rightarrow 0} \frac{J(x + \varepsilon y) - J(x)}{\varepsilon} \\ &= \lim_{\varepsilon \rightarrow 0} \left[\frac{1}{\varepsilon} \int_{t_0}^{t_f} (x(t) + \varepsilon y(t))^2 + x(t) + \varepsilon y(t) - x(t)^2 - x(t) dt \right] \\ &= \lim_{\varepsilon \rightarrow 0} \left[\int_{t_0}^{t_f} 2x(t)y(t) + y(t) + \varepsilon y(t)^2 dt \right] \\ &= \int_{t_0}^{t_f} 2x(t)y(t) + y(t) dt. \end{aligned} \quad (5.15)$$

The same result is obtained by directly differentiating J w.r.t. ε

$$\begin{aligned}
 \delta J(\mathbf{x}; \mathbf{y}) &= \left. \frac{dJ(\mathbf{x} + \varepsilon \mathbf{y})}{d\varepsilon} \right|_{\varepsilon=0} \\
 &= \int_{t_0}^{t_f} \frac{d}{d\varepsilon} [(x(t) + \varepsilon y(t))^2 + x(t) + \varepsilon y(t)] dt \Big|_{\varepsilon=0} \\
 &= \int_{t_0}^{t_f} 2(x(t) + \varepsilon y(t))y(t) + y(t) dt \Big|_{\varepsilon=0} \\
 &= \int_{t_0}^{t_f} 2x(t)y(t) + y(t) dt.
 \end{aligned} \tag{5.16}$$

This shows that the functional J is Gâteaux differentiable in all points $\mathbf{y} \in \mathcal{C}^1[t_0, t_f]$.

According to Definition (5.14), the existence of the Gâteaux derivative $\delta J(\mathbf{x}; \mathbf{y})$ requires that the functionals $J(\mathbf{x})$ and $J(\mathbf{x} + \varepsilon \mathbf{y})$ are defined for all sufficiently small ε and that the derivative of $J(\mathbf{x} + \varepsilon \mathbf{y})$ w.r.t. ε must exist. The following example illustrates the necessity behind this.

Example 5.3 (Non-existence of Gâteaux derivative [5]) The functional $J(\mathbf{x}) = \int_0^1 |x(t)| dt$ is defined and bounded for all $\mathbf{x} \in \mathcal{C}^1[0, 1]$. For $x(t) = 0$ and $y(t) = t$ we get

$$\frac{J(\mathbf{x} + \varepsilon \mathbf{y}) - J(\mathbf{x})}{\varepsilon} = \frac{1}{\varepsilon} \int_0^1 |\varepsilon t| dt = \text{sign}(\varepsilon) \int_0^1 t dt = \begin{cases} \frac{1}{2} & \text{for } \varepsilon > 0 \\ -\frac{1}{2} & \text{for } \varepsilon < 0. \end{cases} \tag{5.17}$$

Thus, the limit value for $\varepsilon \rightarrow 0$ is not unique and therefore the Gâteaux derivative at point $\mathbf{x}(t) = 0$ in direction of $\mathbf{y}(t) = t$ does not exist.

If the Gâteaux derivative of a functional $J(\mathbf{x})$ at a point $\mathbf{x} \in \mathcal{X}$ exists and $\delta J(\mathbf{x}; \mathbf{y}) < 0$ holds, then $\mathbf{y} \in \mathcal{X}$ is a descent direction, i.e.

$$J(\mathbf{x} + \alpha \mathbf{y}) < J(\mathbf{x})$$

for a sufficiently small step size $\alpha > 0$. Hence, the condition $\delta J(\mathbf{x}; \mathbf{y}) < 0$ is a generalization of the descent condition (3.18) in parameter optimization. A necessary condition for a minimum $J(\mathbf{x}^*)$ at point $\mathbf{x}^*$ is that no direction $\mathbf{y}$ is allowed to exist with $\delta J(\mathbf{x}^*; \mathbf{y}) < 0$. This leads to the concept of $\mathcal{X}_{ad}$ -admissible directions and the necessary first-order optimality conditions.

Definition 5.2 ($\mathcal{X}_{ad}$ -admissible direction) Consider the functional $J : \mathcal{X} \rightarrow \mathbb{R}$ with the admissible set $\mathcal{X}_{ad} \subseteq \mathcal{X}$. A direction $\mathbf{y} \in \mathcal{X}$, $\mathbf{y} \neq \mathbf{0}$ is called $\mathcal{X}_{ad}$ -admissible at point $\mathbf{x} \in \mathcal{X}$ if $\delta J(\mathbf{x}; \mathbf{y})$ exists and $\mathbf{x} + \varepsilon \mathbf{y} \in \mathcal{X}_{ad}$ for sufficiently small values of $\varepsilon \in \mathbb{R}$.

Theorem 5.1 (Necessary first-order optimality conditions) Suppose that the functional $J : \mathcal{X} \rightarrow \mathbb{R}$ has a minimum at point $\mathbf{x}^* \in \mathcal{X}_{ad} \subseteq \mathcal{X}$. Then,

$$\delta J(\mathbf{x}^*; \delta \mathbf{x}) = 0 \tag{5.18}$$

for all $\mathcal{X}_{ad}$ -admissible directions $\delta \mathbf{x} \in \mathcal{X}$ at point $\mathbf{x}^*$.

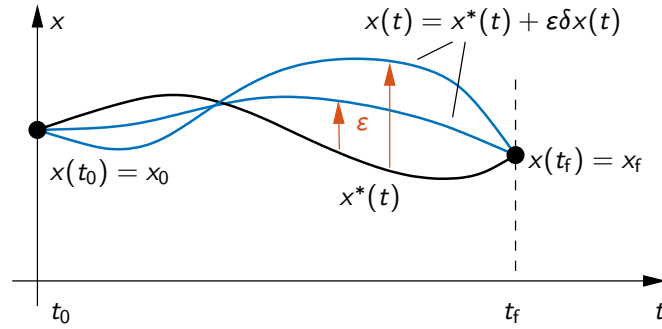


Figure 5.1: Illustration of $\mathcal{X}_{ad}$ -admissible directions $\delta \mathbf{x}$ at $\mathbf{x}^* \in \mathcal{X}_{ad}$.

The concept of $\mathcal{X}_{ad}$ -admissible directions $\delta \mathbf{x}$ is relevant in connection with additional side constraints that are accounted for in the admissible set $\mathcal{X}_{ad}$. For instance, if boundary conditions (5.9) are given, also see (5.10), then $\mathcal{X}_{ad}$ -admissible directions $\delta \mathbf{x}(t)$ must satisfy the homogeneous boundary conditions $\delta \mathbf{x}(t_0) = \mathbf{0}$ and $\delta \mathbf{x}(t_f) = \mathbf{0}$, such that $\mathbf{x} = \mathbf{x}^* + \epsilon \delta \mathbf{x}$ as the argument in the first variation

$$\delta J(\mathbf{x}^*; \delta \mathbf{x}) = \left. \frac{dJ(\mathbf{x}^* + \epsilon \delta \mathbf{x})}{d\epsilon} \right|_{\epsilon=0}$$

satisfies $\mathbf{x} \in \mathcal{X}_{ad}$ (similar to $\mathbf{x}^* \in \mathcal{X}_{ad}$), see Figure 5.1. In this light, Theorem 5.1 states that no $\mathcal{X}_{ad}$ -admissible direction $\delta \mathbf{x}(t)$ is allowed to exist that leads to a reduction of the cost functional $J(\mathbf{x})$.

In order to make the optimality condition (5.18) more explicit, we invoke the *fundamental lemma of the calculus of variations*. Its proof is omitted here but can e.g. be found in [2]. As it will turn out in the next section, the fundamental lemma is essential to derive the famous *Euler-Lagrange equations*.

Theorem 5.2 (Fundamental lemma of the calculus of variations) Suppose that a continuous function $\mathbf{h} \in \mathcal{C}_n^0[t_0, t_f]$ satisfies

$$\int_{t_0}^{t_f} \mathbf{h}(t)^\top \delta \mathbf{x}(t) dt = 0 \quad (5.19)$$

for all continuous functions $\delta \mathbf{x} \in \mathcal{C}_n^0[t_0, t_f]$. Then, $\mathbf{h}(t) = \mathbf{0}$ for all $t \in [t_0, t_f]$.

5.2.2 Euler-Lagrange equations

We now focus on the Lagrange problem (5.8) with the boundary conditions (5.9) and repeat it for the sake of completeness

$$J(\mathbf{x}) = \int_{t_0}^{t_f} \phi(\mathbf{x}(t), \dot{\mathbf{x}}(t), t) dt \quad (5.20a)$$

with

$$\mathbf{x} \in \mathcal{X}_{ad} = \{\mathbf{x} \in \mathcal{X} \mid \mathbf{x}(t_0) = \mathbf{x}_0, \mathbf{x}(t_f) = \mathbf{x}_f\}. \quad (5.20b)$$

The end time t_f is supposed to be fixed and we assume that an optimal solution $\mathbf{x}^* \in \mathcal{X}_{ad} \subset \mathcal{X} = \mathcal{C}_n^1[t_0, t_f]$ exists that minimizes the functional (5.20a). The necessary condition

(5.18) requires that the Gâteaux derivative (or first variation) vanishes for all $\mathcal{X}_{ad}$ -admissible variations $\delta \mathbf{x} \in \mathcal{C}_n^1[t_0, t_f]$ with the homogeneous boundary conditions

$$\delta \mathbf{x}(t_0) = \mathbf{0}, \quad \delta \mathbf{x}(t_f) = \mathbf{0}. \quad (5.21)$$

The Gâteaux derivative of J at the optimal point $\mathbf{x}^*$ in direction of $\delta \mathbf{x}$ is computed by differentiation

$$\begin{aligned} \delta J(\mathbf{x}^*; \delta \mathbf{x}) &= \left. \frac{dJ(\mathbf{x}^* + \varepsilon \delta \mathbf{x})}{d\varepsilon} \right|_{\varepsilon=0} \\ &= \int_{t_0}^{t_f} \left. \frac{d}{d\varepsilon} \phi(\mathbf{x}^*(t) + \varepsilon \delta \mathbf{x}(t), \dot{\mathbf{x}}^*(t) + \varepsilon \delta \dot{\mathbf{x}}(t), t) \right|_{\varepsilon=0} dt \\ &= \int_{t_0}^{t_f} \phi_{\mathbf{x}}(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t))^T \delta \mathbf{x}(t) + \phi_{\dot{\mathbf{x}}}(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t))^T \delta \dot{\mathbf{x}}(t) dt. \end{aligned} \quad (5.22)$$

The functions $\phi_{\mathbf{x}}$ and $\phi_{\dot{\mathbf{x}}}$ denote the (transposed) partial derivatives $\phi_{\mathbf{x}} = (\frac{\partial \phi}{\partial \mathbf{x}})^T$ and $\phi_{\dot{\mathbf{x}}} = (\frac{\partial \phi}{\partial \dot{\mathbf{x}}})^T$. The integrand of (5.22) is continuous because the function ϕ is continuously differentiable by assumption and $\delta \mathbf{x} \in \mathcal{C}_n^1[t_0, t_f]$. This implies that $J(\mathbf{x})$ is Gâteaux differentiable at all points $\mathbf{x} \in \mathcal{X} = \mathcal{C}_n^1[t_0, t_f]$.

Partial integration of the second term in (5.22) leads to² (neglecting arguments for readability)

$$\int_{t_0}^{t_f} \phi_{\dot{\mathbf{x}}}(\mathbf{x}^*, \dot{\mathbf{x}}^*, t)^T \delta \dot{\mathbf{x}} dt = [\phi_{\dot{\mathbf{x}}}(\mathbf{x}^*, \dot{\mathbf{x}}^*, t)^T \delta \mathbf{x}]_{t_0}^{t_f} - \int_{t_0}^{t_f} \frac{d}{dt} \phi_{\dot{\mathbf{x}}}(\mathbf{x}^*, \dot{\mathbf{x}}^*, t)^T \delta \mathbf{x} dt. \quad (5.23)$$

The boundary terms vanish due to the fact that only $\mathcal{X}_{ad}$ -admissible variations with $\delta \mathbf{x}(t_0) = \delta \mathbf{x}(t_f) = \mathbf{0}$ are considered. Hence, the Gâteaux derivative (5.22) simplifies to

$$\delta J(\mathbf{x}^*; \delta \mathbf{x}) = \int_{t_0}^{t_f} \left[\phi_{\mathbf{x}}(\mathbf{x}^*, \dot{\mathbf{x}}^*, t) - \frac{d}{dt} \phi_{\dot{\mathbf{x}}}(\mathbf{x}^*, \dot{\mathbf{x}}^*, t) \right]^T \delta \mathbf{x} dt \quad (5.24)$$

in dependence of $\delta \mathbf{x}(t)$. For an optimal trajectory $\mathbf{x}^*(t)$, the Gâteaux derivative $\delta J(\mathbf{x}^*; \delta \mathbf{x})$ must vanish for all $\mathcal{X}_{ad}$ -admissible variations $\delta \mathbf{x}(t)$, see Theorem 5.1. Thus, the fundamental lemma of the calculus of variations (Theorem 5.2) yields

$$\phi_{\mathbf{x}}(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t), t) - \frac{d}{dt} \phi_{\dot{\mathbf{x}}}(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t), t) = \mathbf{0}.$$

(5.25)

The differential equations (5.25) are known as *Euler-Lagrange equations*. They are a *necessary condition* for an optimal solution $\mathbf{x}^*(t)$ to minimize the functional (5.20a) subject to the boundary conditions in (5.20b).

With the *second variation* of J

$$\delta^2 J(\mathbf{x}^*; \delta \mathbf{x}) = \left. \frac{d^2 J(\mathbf{x}^* + \varepsilon \delta \mathbf{x})}{d\varepsilon^2} \right|_{\varepsilon=0} \geq 0, \quad (5.26)$$

it can be shown that the Hessian matrix of $\phi(\mathbf{x}, \dot{\mathbf{x}})$ w.r.t. $\dot{\mathbf{x}}$ must be positive semidefinite, i.e.

$$\phi_{\dot{\mathbf{x}}\dot{\mathbf{x}}}(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t), t) \geq 0 \quad \forall t \in [t_0, t_f]. \quad (5.27)$$

This so-called *Legendre condition* is a necessary second-order optimality condition for an optimal solution $\mathbf{x}^*(t)$, $t \in [t_0, t_f]$ in addition to the Euler-Lagrange equations (5.25). The derivation of (5.27) is similar to the Gâteaux derivative or first variation (5.14) by means of partial integration and is not further detailed here.

² Remember that $\int \mathbf{u}^T \dot{\mathbf{v}} dt = \mathbf{u}^T \mathbf{v} - \int \dot{\mathbf{u}}^T \mathbf{v} dt$.

There are two special cases of the Euler-Lagrange equations (5.25) that are worth mentioning:

- If ϕ does not explicitly depend on $\mathbf{x}$, i.e. $\phi = \phi(\dot{\mathbf{x}}, t)$, a first integration of (5.25) shows that

$$\phi_{\dot{\mathbf{x}}}(\dot{\mathbf{x}}^*(t), t) = \text{const.} \quad (5.28)$$

- If ϕ does not explicitly depend on t , i.e. $\phi = \phi(\mathbf{x}, \dot{\mathbf{x}})$, it can be shown that the Euler-Lagrange equations (5.25) simplify to

$$\phi(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t)) - \dot{\mathbf{x}}^*(t)^T \phi_{\dot{\mathbf{x}}}(\mathbf{x}^*(t), \dot{\mathbf{x}}^*(t)) = \text{const.} \quad (5.29)$$

Exercise 5.1 Compute the minimum distance between two points A and B in the (x, y) -plane. To this end, determine the cost functional to be minimized in dependence of the arc length s and use the relation

$$ds = \sqrt{dx^2 + dy^2} = \sqrt{1 + y'(x)^2} dx. \quad (5.30)$$

5.2.3 Example: Galileo's hanging chain

In 1683, Galileo formulated the question how the form of a hanging chain that is fixed at two points can be mathematically described. Galileo (wrongly) postulated that the form is described by a parabola. The problem was solved in 1690 by *Jakob and Johannes Bernoulli* and *Leibnitz* using the calculus of variations.

We consider a chain of length L with both ends being fixed at two points A and B , see Figure 5.2. Hamilton's principle in mechanics states that the potential energy of the chain in stationary conditions must be minimal. The potential energy can be formulated with the arc length s and the relation (5.30)

$$V = \rho g \int_0^L y ds = \rho g \int_{x_A}^{x_B} y \sqrt{1 + y'(x)^2} dx, \quad (5.31)$$

where g denotes the acceleration due to gravity and ρ is the mass per chain length. Since the minimum of the potential energy V is sought, the constant factor ρg can be discarded, leading to the cost functional along with the boundary conditions

$$J(y) = \int_{x_A}^{x_B} y \sqrt{1 + y'(x)^2} dx, \quad y(x_A) = y_A, \quad y(x_B) = y_B. \quad (5.32)$$

Due to the fact that the spatial variable x does not explicitly appear in $\phi(y, y') = y \sqrt{1 + y'(x)^2}$, we can refer to the special case (5.29) of the Euler-Lagrange equations (again omitting arguments)

$$\begin{aligned} \phi(y, y') - y' \phi_{y'}(y, y') &= y \sqrt{1 + y'^2} - \frac{y y'^2}{\sqrt{1 + y'^2}} \\ &= \frac{y}{\sqrt{1 + y'^2}} = c_1 = \text{const.} \end{aligned} \quad (5.33)$$

This equation can be written as

$$y' = \sqrt{\frac{y^2}{c_1^2} - 1} \quad (5.34)$$

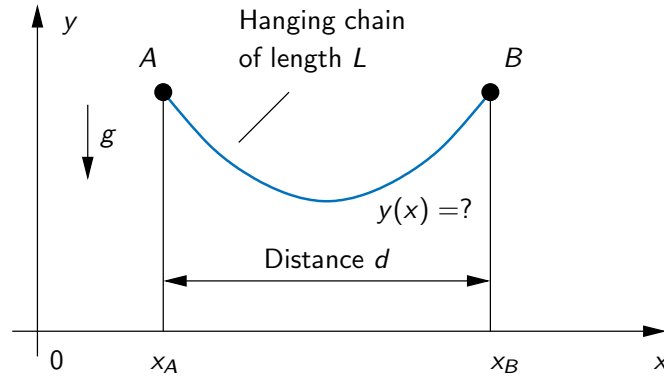


Figure 5.2: Galileo's problem: Determination of the form $y(x)$ of a hanging chain.

if we assume that $c_1 \neq 0$. With $y' = \frac{dy}{dx}$ and using integration, the spatial coordinate x can be expressed as function of y

$$x = \int \frac{1}{\sqrt{y^2/c_1^2 - 1}} dy = c_1 \ln \left(\frac{y + \sqrt{y^2 - c_1^2}}{c_1} \right) + c_2$$

with the integration constant c_2 . By transforming and squaring this equation

$$\left(c_1 e^{(x-c_2)/c_1} - y \right)^2 = y^2 - c_1^2,$$

we can express y as

$$y = \frac{1}{2} c_1 \frac{e^{2(x-c_2)/c_1} + 1}{e^{(x-c_2)/c_1}} = \frac{1}{2} c_1 \left(e^{(x-c_2)/c_1} + e^{-(x-c_2)/c_1} \right).$$

With $\cosh b = \frac{1}{2}(e^b + e^{-b})$, we eventually obtain the famous *catenary* of the hanging chain

$$y(x) = c_1 \cosh \left(\frac{x - c_2}{c_1} \right). \quad (5.35)$$

Optionally, a third constant c_3 can be added to (5.35) in order to compute the catenary for a chain of length L hanging between the points A and B by (numerically) solving the equations

$$y(x_A) = y_A, \quad y(x_B) = y_B, \quad \int_{x_A}^{x_B} \sqrt{1 + y'(x)^2} dx = L. \quad (5.36)$$

Figure 5.3 shows the catenary $y(x)$ with length $L = 1.5$ for different hang points compared to Galileo's postulated parabola.

5.3 Unconstrained optimal control problems

In the next step, the calculus of variations is applied to unconstrained optimal control problems

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{u}) = V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt \quad (5.37a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (5.37b)$$

$$x_i(t_f) = x_{f,i}, \quad i \in \mathcal{I}_f \quad (5.37c)$$

to derive necessary conditions for an optimal solution.

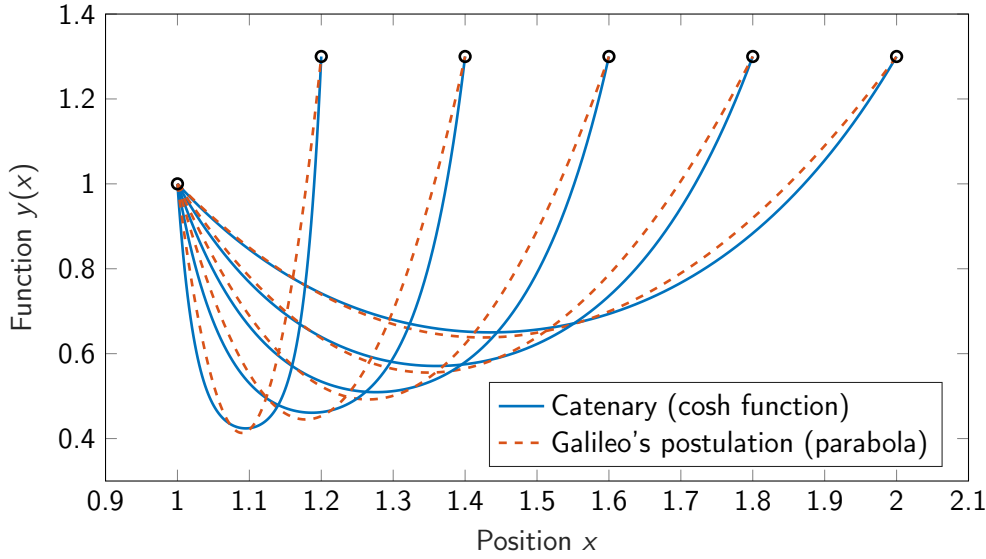


Figure 5.3: Hanging chain of length $L = 1.5$ for different hang points.

5.3.1 Application of the calculus of variations

The considerations for the Euler-Lagrange equations are successively extended to account for the system dynamics (5.37b) and the partial terminal conditions (5.37c). The end time t_f is supposed to be fixed, i.e. $V = V(\mathbf{x}(t_f))$. The free end time case will be considered afterwards.

Similar to the constrained optimization case in Chapter 4, the system dynamics (5.37b) are treated as (dynamic) equality constraints that are added to the cost functional via Lagrange multipliers

$$\bar{J}(\mathbf{u}, \mathbf{x}) = V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) + \boldsymbol{\lambda}^\top (\mathbf{f}(\mathbf{x}, \mathbf{u}, t) - \dot{\mathbf{x}}) dt \quad (5.38)$$

with the optimization variables $\mathbf{u} \in \mathcal{U} = \mathcal{C}_m^0[t_0, t_f]$ and $\mathbf{x} \in \mathcal{X} = \mathcal{C}_n^1[t_0, t_f]$. The boundary conditions of (5.37) are incorporated in the admissible set

$$\mathcal{X}_{ad} = \{\mathbf{x} \in \mathcal{X} \mid \mathbf{x}(t_0) = \mathbf{x}_0, \mathbf{x}_i(t_f) = x_{f,i}, i \in \mathcal{I}_f\}. \quad (5.39)$$

Note that the multipliers $\boldsymbol{\lambda}(t), t \in [t_0, t_f]$ are functions of time in contrast to the finite-dimensional (parameter optimization) case, where the multipliers $\boldsymbol{\lambda}$ for equality constraints are constant vectors.

Following the procedure of the Euler-Lagrange equations, we compute the Gâteaux derivative of $\bar{J}$ at an optimal point $\mathbf{u}^* \in \mathcal{U}$ and $\mathbf{x}^* \in \mathcal{X}_{ad}$ in direction of $\delta \mathbf{u}$ and $\delta \mathbf{x}$, i.e. (without time arguments)

$$\begin{aligned} \delta \bar{J}(\mathbf{u}^*, \mathbf{x}^*; \delta \mathbf{u}, \delta \mathbf{x}) &= \left. \frac{d\bar{J}(\mathbf{u}^* + \varepsilon \delta \mathbf{u}, \mathbf{x}^* + \varepsilon \delta \mathbf{x})}{d\varepsilon} \right|_{\varepsilon=0} \\ &= V_{\mathbf{x}}(\mathbf{x}^*(t_f), t_f)^\top \delta \mathbf{x}(t_f) + \int_{t_0}^{t_f} [l_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}]^\top \delta \mathbf{x} \\ &\quad + [l_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}]^\top \delta \mathbf{u} - \boldsymbol{\lambda}^\top \delta \dot{\mathbf{x}} dt. \end{aligned} \quad (5.40)$$

Partial integration of the latter integral term gives

$$\begin{aligned} \delta \bar{J}(\mathbf{u}^*, \mathbf{x}^*; \delta \mathbf{u}, \delta \mathbf{x}) &= V_{\mathbf{x}}(\mathbf{x}^*(t_f), t_f)^T \delta \mathbf{x}(t_f) - [\boldsymbol{\lambda}^T \delta \mathbf{x}]_{t_0}^{t_f} \\ &+ \int_{t_0}^{t_f} [l_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t)^T \boldsymbol{\lambda} + \dot{\boldsymbol{\lambda}}]^T \delta \mathbf{x} + [l_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t)^T \boldsymbol{\lambda}]^T \delta \mathbf{u} dt. \end{aligned}$$

The boundary terms can be simplified according to

$$[\boldsymbol{\lambda}^T \delta \mathbf{x}]_{t_0}^{t_f} = \boldsymbol{\lambda}(t_f)^T \delta \mathbf{x}(t_f) - \boldsymbol{\lambda}(t_0)^T \delta \mathbf{x}(t_0) = \boldsymbol{\lambda}(t_f)^T \delta \mathbf{x}(t_f)$$

because $\mathcal{X}_{ad}$ -admissible directions must satisfy $\delta \mathbf{x}(t_0) = \mathbf{0}$.

A necessary optimality condition is $\delta \bar{J}(\mathbf{u}^*, \mathbf{x}^*; \delta \mathbf{u}, \delta \mathbf{x}) = 0$. The fundamental lemma of the calculus of variations implies that the following equations must be satisfied

$$\dot{\boldsymbol{\lambda}}^* = -l_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t) - \mathbf{f}_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t)^T \boldsymbol{\lambda}^*, \quad t \in (t_0, t_f) \quad (5.41a)$$

$$\mathbf{0} = l_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t)^T \boldsymbol{\lambda}^*, \quad t \in (t_0, t_f) \quad (5.41b)$$

$$0 = (V_{\mathbf{x}}(\mathbf{x}^*(t_f)) - \boldsymbol{\lambda}^*(t_f))^T \delta \mathbf{x}(t_f) = \sum_{i=1}^n (V_{x_i}(\mathbf{x}^*(t_f)) - \lambda_i^*(t_f)) \delta x_i(t_f). \quad (5.41c)$$

In particular, a Lagrange multiplier $\boldsymbol{\lambda}^*(t)$, $t \in [t_0, t_f]$ corresponding to $\mathbf{x}^*(t)$, $\mathbf{u}^*(t)$ must satisfy the differential equation (5.41a). In addition, (5.41b) is an algebraic equation that must be satisfied over the whole time interval.

The terminal condition (5.41c) must hold for all $\mathcal{X}_{ad}$ -admissible variations $\delta \mathbf{x}(t_f)$. They can be further simplified in light of the partial terminal conditions (5.37c). For the states x_i , $i \in \mathcal{I}_f$ that are fixed at the end time $t = t_f$, see (5.37c), an admissible variation must satisfy $\delta x_i(t_f) = 0$, whereas for all free terminal states $x_i(t_f)$, $i \notin \mathcal{I}_f$ the corresponding variation at the end time is arbitrary. Hence, if

$$\lambda_i^*(t_f) = V_{x_i}(\mathbf{x}^*(t_f)), \quad i \notin \mathcal{I}_f, \quad (5.42)$$

holds, then (5.41c) is satisfied for all admissible variations at $t = t_f$.

Consideration of free end time

If the end time t_f of the optimal control problem (5.1) is unspecified, we additionally have to consider the variation around the optimal end time t_f^* . The Gâteaux derivative at the optimal point $(\mathbf{u}^*, \mathbf{x}^*, t_f^*)$ in direction of $(\delta \mathbf{u}, \delta \mathbf{x}, \delta t_f)$ is an extension of (5.40) and reads (again omitting time arguments)

$$\begin{aligned} \delta \bar{J}(\mathbf{u}^*, \mathbf{x}^*, t_f^*; \delta \mathbf{u}, \delta \mathbf{x}, \delta t_f) &= \frac{d \bar{J}(\mathbf{u}^* + \varepsilon \delta \mathbf{u}, \mathbf{x}^* + \varepsilon \delta \mathbf{x}, t_f^* + \varepsilon \delta t_f)}{d\varepsilon} \Big|_{\varepsilon=0} \\ &= \underbrace{\Delta \mathbf{x}_f}_{= \boldsymbol{\lambda}^*(t_f^*)^T \delta \mathbf{x}(t_f^*)} \\ &= V_{\mathbf{x}}(\mathbf{x}^*(t_f^*), t_f^*)^T (\delta \mathbf{x}(t_f^*) + \dot{\mathbf{x}}^*(t_f^*) \delta t_f) + V_t(\mathbf{x}^*(t_f^*), t_f^*) \delta t_f \\ &\quad + (l_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t) + \boldsymbol{\lambda}^T(\mathbf{f}(\mathbf{x}^*, \mathbf{u}^*, t) - \dot{\mathbf{x}}))_{t=t_f^*}^T \delta t_f \\ &\quad + \int_{t_0}^{t_f^*} [l_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t)^T \boldsymbol{\lambda}]^T \delta \mathbf{x} \\ &\quad + [l_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t)^T \boldsymbol{\lambda}]^T \delta \mathbf{u} - \boldsymbol{\lambda}^T \delta \dot{\mathbf{x}} dt. \quad (5.43) \end{aligned}$$

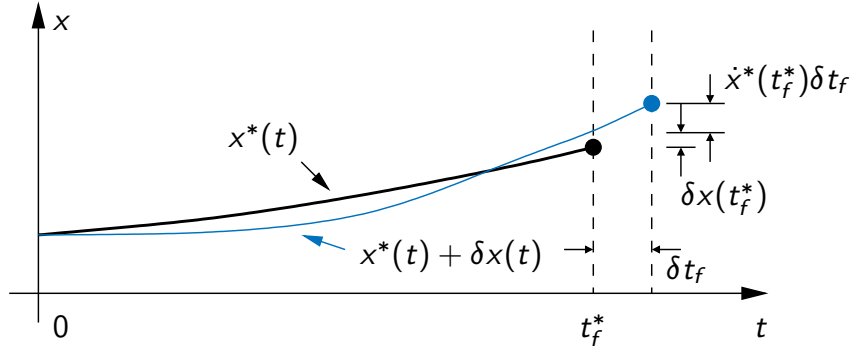


Figure 5.4: Total variation $\Delta \mathbf{x}_f = \delta \mathbf{x}(t_f^*) + \dot{\mathbf{x}}^*(t_f^*) \delta t_f$ of the terminal state for $\mathbf{x} \in \mathbb{R}$.

The *total variation* $\Delta \mathbf{x}_f$ accounts for the additional influence of the end time variation δt_f on the change of the terminal state, see Figure 5.4. The variation of the integral term due to a small variation of the end time δt_f results in the term $(l(\mathbf{x}^*, \mathbf{u}^*, t) + \boldsymbol{\lambda}^\top (\mathbf{f}(\mathbf{x}^*, \mathbf{u}^*, t) - \dot{\mathbf{x}}))_{t=t_f^*} \delta t_f$ in (5.43). Partial integration of (5.43) with $\boldsymbol{\lambda}(t_0)^\top \delta \mathbf{x}(t_0) = 0$ yields

$$\begin{aligned} \delta \bar{J}(\mathbf{u}^*, \mathbf{x}^*, t_f^*; \delta \mathbf{u}, \delta \mathbf{x}, \delta t_f) &= V_x(\mathbf{x}^*(t_f^*), t_f^*)^\top \overbrace{\Delta \mathbf{x}_f}^{= \Delta \mathbf{x}_f} - \boldsymbol{\lambda}(t_f^*)^\top (\delta \mathbf{x}(t_f^*) + \dot{\mathbf{x}}^*(t_f^*) \delta t_f) \\ &\quad + \left(V_t(\mathbf{x}^*, t) + l(\mathbf{x}^*, \mathbf{u}^*, t) + \boldsymbol{\lambda}^\top \mathbf{f}(\mathbf{x}^*, \mathbf{u}^*, t) \right)_{t=t_f^*} \delta t_f \\ &\quad + \int_{t_0}^{t_f^*} [l_x(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_x(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda} + \dot{\boldsymbol{\lambda}}]^\top \delta \mathbf{x} \\ &\quad + [l_u(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_u(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}]^\top \delta \mathbf{u} \, dt. \end{aligned}$$

If $\mathbf{x}^*(t)$, $\mathbf{u}^*(t)$, t_f^* is an optimal solution, then $\delta \bar{J} = 0$ must hold for all admissible variations $\delta \mathbf{x}(t)$, $\delta \mathbf{u}(t)$, δt_f . The fundamental lemma of the calculus of variations thus yields

$$\dot{\boldsymbol{\lambda}}^* = -l_x(\mathbf{x}^*, \mathbf{u}^*, t) - \mathbf{f}_x(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}^*, \quad t \in (t_0, t_f) \quad (5.44a)$$

$$\mathbf{0} = l_u(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_u(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}^*, \quad t \in (t_0, t_f) \quad (5.44b)$$

$$0 = (V_x(\mathbf{x}^*(t_f^*), t_f^*) - \boldsymbol{\lambda}^*(t_f))^\top \Delta \mathbf{x}_f \quad (5.44c)$$

$$0 = \left(V_t(\mathbf{x}^*, t) + l(\mathbf{x}^*, \mathbf{u}^*, t) + \boldsymbol{\lambda}^\top \mathbf{f}(\mathbf{x}^*, \mathbf{u}^*, t) \right)_{t=t_f^*}. \quad (5.44d)$$

In analogy to the previous fixed end time case, the condition (5.44c) is satisfied for (5.42). Moreover, the free end time t_f leads to the additional (scalar) terminal condition (5.44d).

5.3.2 Optimality conditions

The relations of the last section are necessary conditions for a solution of the optimal control problem (5.37). These conditions can be formalized by introducing the *Hamiltonian function* or in short *Hamiltonian*.

Definition 5.3 (Hamiltonian function) *The Hamiltonian (function) for the optimal control problem (5.37) is defined by*

$$H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}, t) = l(\mathbf{x}, \mathbf{u}, t) + \boldsymbol{\lambda}^\top \mathbf{f}(\mathbf{x}, \mathbf{u}, t) \quad (5.45)$$

with the Lagrange multipliers $\boldsymbol{\lambda}(t) \in \mathbb{R}^n$.

The Hamiltonian allows to express the necessary first-order optimality conditions for (5.37) in a compact form as follows.

Theorem 5.3 (Necessary first-order optimality conditions) *Suppose that the functions l and $\mathbf{f}$ of the optimal control problem (5.37) are continuously differentiable in $(\mathbf{x}, \mathbf{u})$ and continuous in t . Suppose that $\mathbf{u}^* \in C_m^0[t_0, t_f]$ is a (local) optimal solution of (5.37) with associated state trajectory $\mathbf{x}^* \in C_n^1[t_0, t_f]$. Then, there exist time functions $\boldsymbol{\lambda}^* \in C_n^1[t_0, t_f]$ with $\boldsymbol{\lambda}^* = [\lambda_1^*, \dots, \lambda_n^*]^\top$ such that the following conditions are satisfied:*

a) $(\mathbf{u}^*, \mathbf{x}^*, \boldsymbol{\lambda}^*)$ satisfy

$$\dot{\mathbf{x}}^* = H_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, \boldsymbol{\lambda}^*, t) = \mathbf{f}(\mathbf{x}^*, \mathbf{u}^*, t) \quad (5.46a)$$

$$\dot{\boldsymbol{\lambda}}^* = -H_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, \boldsymbol{\lambda}^*, t) = -l_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t) - \mathbf{f}_{\mathbf{x}}(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}^* \quad (5.46b)$$

$$\mathbf{0} = H_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, \boldsymbol{\lambda}^*, t) = l_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_{\mathbf{u}}(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}^* \quad (5.46c)$$

for $t \in [t_0, t_f]$ with the boundary conditions

$$\mathbf{x}^*(t_0) = \mathbf{x}_0 \quad (5.47a)$$

$$x_i^*(t_f) = x_{f,i}, \quad i \in \mathcal{I}_f \quad (5.47b)$$

and the transversality condition

$$\lambda_i^*(t_f) = V_{x_i}(\mathbf{x}(t_f), t_f), \quad i \notin \mathcal{I}_f. \quad (5.47c)$$

b) If the end time t_f is unspecified and t_f^* is (locally) optimal, then (5.46) is satisfied for $t \in [t_0, t_f^*]$ and (5.47b), (5.47c) for $t_f = t_f^*$. In addition, the Hamiltonian satisfies the transversality condition

$$H(\mathbf{x}^*(t_f^*), \mathbf{u}^*(t_f^*), \boldsymbol{\lambda}^*(t_f^*), t_f^*) = -V_t(\mathbf{x}^*(t_f^*), t_f^*). \quad (5.48)$$

The differential equations (5.46a), (5.46b) together are known as *canonical equations* and (5.46c) is sometimes called *stationarity condition*. The dynamics (5.46b) are called the *adjoint system* with the *adjoint state* $\boldsymbol{\lambda}$. The optimality conditions define a *two-point boundary value problem* (BVP) for the optimal states $\mathbf{x}^*(t)$ and $\boldsymbol{\lambda}^*(t)$ as well as for the optimal control $\mathbf{u}^*(t)$.

If t_f is fixed, the BVP (5.46), (5.47) has $2n$ dynamical equations with $2n$ boundary conditions and m algebraic equations for the $2n + m$ unknowns $\mathbf{x}^*(t)$, $\boldsymbol{\lambda}^*(t)$, and $\mathbf{u}^*(t)$. If t_f is free, the transversality condition (5.48) has to be satisfied as well, i.e. one scalar equation is added for one additional unknown variable t_f^* . Thus, the optimality conditions are well defined in terms of dimensions. The terminal conditions in (5.47) include the following special cases:

- **Fixed terminal state** $\mathbf{x}(t_f) = \mathbf{x}_f$, i.e. $\mathcal{I}_f = \{1, \dots, n\}$: The full state $\mathbf{x}$ is fixed at the initial time t_0 and final time t_f , the adjoint terminal state $\boldsymbol{\lambda}(t_f)$ is free.
- **Free terminal state** $\mathbf{x}(t_f)$, i.e. $\mathcal{I}_f = \{\}$: In this case, the full adjoint state is fixed at $t = t_f$, i.e. $\boldsymbol{\lambda}(t_f) = \mathbf{V}_x(\mathbf{x}(t_f), t_f)$. Thus, the boundary conditions separate into initial conditions for $\mathbf{x}(t_0)$ and terminal conditions for $\boldsymbol{\lambda}(t_f)$.
- **Free end time** t_f with mayer term $V = 0$ or $V = V(\mathbf{x}(t_f))$: In this case, the transversality condition (5.48) is homogeneous, i.e. $H|_{t=t_f} = 0$.

The extension of the second-order Legendre condition (5.27) to the optimal control problem (5.1) requires the Hessian of the Hamiltonian w.r.t. the control $\mathbf{u}$ to be positive semidefinite:

$$H_{uu}(\mathbf{x}^*(t), \mathbf{u}^*(t), \boldsymbol{\lambda}^*(t), t) \geq 0 \quad \forall t \in [t_0, t_f]. \quad (5.49)$$

The function H_{uu} thereby denotes the second partial derivative $\frac{\partial^2 H}{\partial \mathbf{u}^2}$ of the Hamiltonian.

An interesting property of the Hamiltonian (5.45) is its behavior along an optimal solution:

$$\begin{aligned} \frac{d}{dt} H(\mathbf{x}^*(t), \mathbf{u}^*(t), \boldsymbol{\lambda}^*(t), t) &= H_t^* + \underbrace{(H_x^*)^\top \dot{\mathbf{x}}^*}_{\mathbf{0}} + \underbrace{(H_u^*)^\top \dot{\mathbf{u}}^*}_{\mathbf{0}} + \underbrace{-(f^*)^\top H_x^*}_{\mathbf{0}} \\ &= H_t(\mathbf{x}^*(t), \mathbf{u}^*(t), \boldsymbol{\lambda}^*(t), t), \end{aligned} \quad (5.50)$$

where the notation $()^*$ denotes the evaluation of the respective function along the optimal solution. In particular, for time-invariant problems (5.37), i.e. l as well as $\mathbf{f}$ are independent of t , the Hamiltonian H is constant along the optimal trajectory.

Exercise 5.2 (Fixed end time & terminal condition) Consider the problem

$$\begin{aligned} \min_{u(\cdot)} \quad & \int_0^1 \frac{1}{2} u^2 + \frac{a}{2} x^2 dt, \quad a > 0 \\ \text{s.t.} \quad & \dot{x} = u, \quad x(0) = 1, \quad x(1) = 0. \end{aligned}$$

Show that the solution is given by

$$x^*(t) = \frac{1}{1 - e^{2\sqrt{a}}} \left(e^{\sqrt{a}t} - e^{\sqrt{a}(2-t)} \right), \quad u^*(t) = \frac{\sqrt{a}}{1 - e^{2\sqrt{a}}} \left(e^{\sqrt{a}t} + e^{\sqrt{a}(2-t)} \right) \quad (5.51)$$

in dependence of the parameter $a \geq 0$. Interpret the results shown in Figure 5.5.

Exercise 5.3 (Fixed end time & partial terminal condition) Consider the problem

$$\begin{aligned} \min_{u(\cdot)} \quad & \frac{a}{2} x_2(1)^2 + \int_0^1 \frac{1}{2} u^2 dt, \quad a \geq 0 \\ \text{s.t.} \quad & \dot{x}_1 = x_2, \quad x_1(0) = 1, \quad x_1(1) = 0 \\ & \dot{x}_2 = u, \quad x_2(0) = 0 \end{aligned}$$

Show that the (free) terminal state is $x_2^*(1) = -6/(4 + a)$ and that the optimal solution is given by

$$x_1^*(t) = \frac{2(1+a)}{4+a} t^3 - \frac{3(2+a)}{a+4} t^2 + 1, \quad u^*(t) = \frac{12(1+a)}{4+a} t - \frac{6(2+a)}{a+4} \quad (5.52)$$

in dependence of the parameter $a \geq 0$. Interpret the results shown in Figure 5.6.

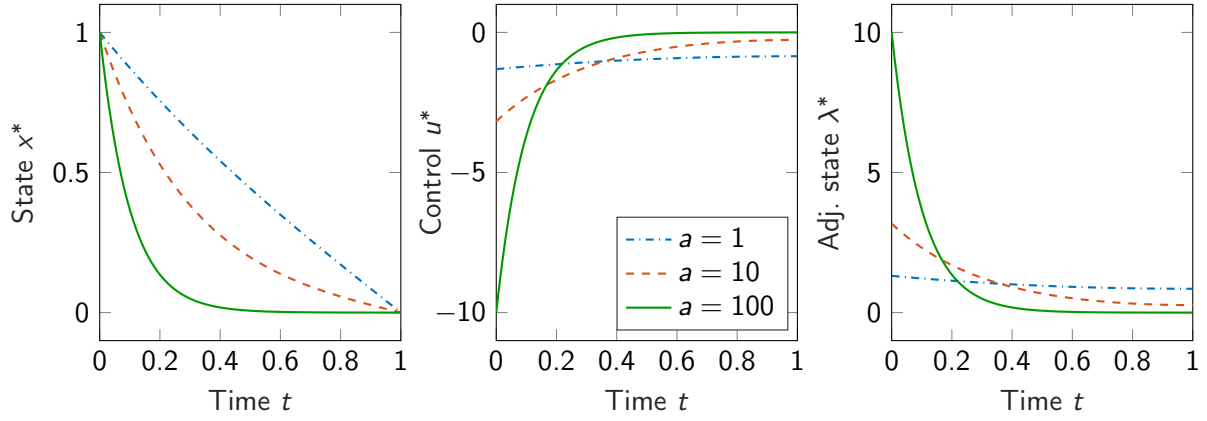


Figure 5.5: Optimal trajectories in Exercise 5.2.

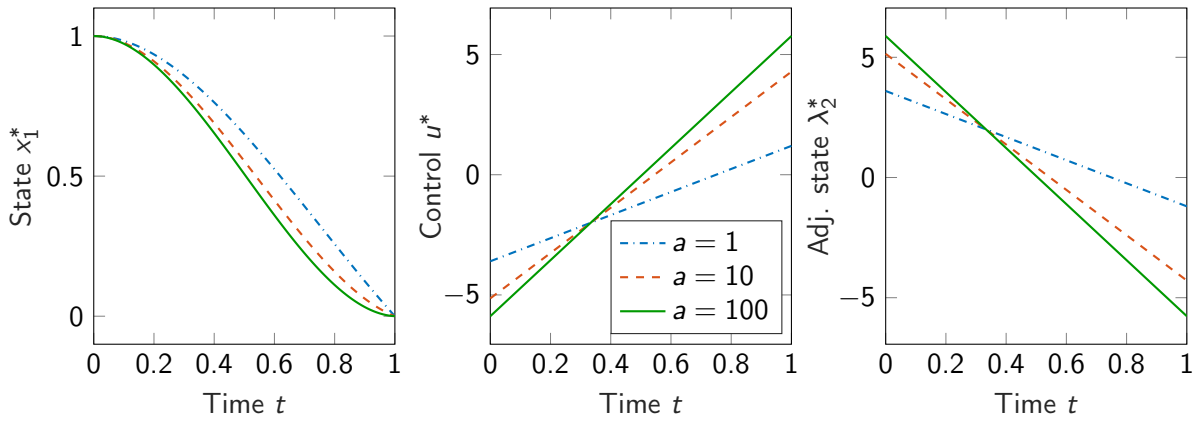


Figure 5.6: Optimal trajectories in Exercise 5.3.

5.3.3 Example with free end time

We consider a particle with mass m in the (x, y) -plane with the constant thrust force $F = ma$. The control input u is the angle between the thrust and the x -axis, see Figure 5.7. The goal is to steer the particle in minimum time t_f to the target point (x_f, y_f) . Under the assumption that no other forces act on the particle besides the thrust, the optimal control problem can be formulated as follows

$$\min_{u(\cdot)} t_f \quad (5.53a)$$

$$\text{s.t. } \dot{x} = v, \quad x(0) = x_0, \quad x(t_f) = x_f \quad (5.53b)$$

$$\dot{v} = a \cos(u), \quad v(0) = v_0 \quad (5.53c)$$

$$\dot{y} = w, \quad y(0) = y_0, \quad y(t_f) = y_f \quad (5.53d)$$

$$\dot{w} = a \sin(u), \quad w(0) = w_0. \quad (5.53e)$$

The terminal condition is only defined for the position (x, y) and not for the velocity. In analogy to the partial terminal conditions (5.37c), the index set is $\mathcal{I}_f = \{x, y\}$. The Hamiltonian (5.45) of the problem is

$$H(\mathbf{x}, u, \boldsymbol{\lambda}) = \lambda_x v + \lambda_v a \cos(u) + \lambda_y w + \lambda_w a \sin(u) \quad (5.54)$$

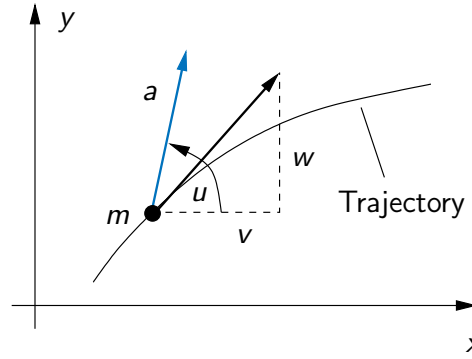


Figure 5.7: Motion of a particle with mass m in the (x, y) -plane.

with $\mathbf{x} = [x, v, y, w]^T$ and the adjoint states $\boldsymbol{\lambda} = [\lambda_x, \lambda_v, \lambda_y, \lambda_w]^T$. The adjoint system (5.46b) with the terminal conditions (5.47c) reads

$$\dot{\lambda}_x = -\frac{\partial H}{\partial x} = 0 \quad (5.55a)$$

$$\dot{\lambda}_v = -\frac{\partial H}{\partial v} = -\lambda_x, \quad \lambda_v(t_f) = 0 \quad (5.55b)$$

$$\dot{\lambda}_y = -\frac{\partial H}{\partial y} = 0 \quad (5.55c)$$

$$\dot{\lambda}_w = -\frac{\partial H}{\partial w} = -\lambda_y, \quad \lambda_w(t_f) = 0 \quad (5.55d)$$

with the solution

$$\lambda_x = \text{const.}, \quad \lambda_v = \lambda_x(t_f - t), \quad \lambda_y = \text{const.}, \quad \lambda_w = \lambda_y(t_f - t). \quad (5.56)$$

Moreover, the stationarity condition (5.46c) w.r.t. the control must be satisfied

$$\frac{\partial H}{\partial u} = -\lambda_v a \sin(u) + \lambda_w a \cos(u) = 0,$$

which can be solved for u

$$\tan(u) = \frac{\lambda_w}{\lambda_v} \stackrel{(5.56)}{=} \frac{\lambda_y(t_f - t)}{\lambda_x(t_f - t)} = \frac{\lambda_y}{\lambda_x} \Rightarrow u = \arctan\left(\frac{\lambda_y}{\lambda_x}\right) \text{ for } -\frac{\pi}{2} < u < \frac{\pi}{2}. \quad (5.57)$$

This shows that the optimal control, i.e. the thrust angle, is constant over the whole time interval $[t_0 = 0, t_f]$. As a consequence, the states $x(t)$ and $y(t)$ follow from integrating the dynamics (5.53c), (5.53e) twice and inserting the initial conditions³

$$x(t) = g_x(\lambda_x, \lambda_y, t) = x_0 + v_0 t + \frac{1}{2} a \cos(u) t^2, \quad \cos(u) \stackrel{(5.57)}{=} \frac{1}{\sqrt{1 + \lambda_y^2 / \lambda_x^2}} \quad (5.58a)$$

$$v(t) = g_v(\lambda_x, \lambda_y, t) = v_0 + a \cos(u) t \quad (5.58b)$$

$$y(t) = g_y(\lambda_x, \lambda_y, t) = y_0 + w_0 t + \frac{1}{2} a \sin(u) t^2, \quad \sin(u) \stackrel{(5.57)}{=} \frac{\lambda_y}{\lambda_x \sqrt{1 + \lambda_y^2 / \lambda_x^2}} \quad (5.58c)$$

$$w(t) = g_w(\lambda_x, \lambda_y, t) = w_0 + a \sin(u) t. \quad (5.58d)$$

³ Remember that $\sin(\arctan(b)) = b / \sqrt{1 + b^2}$, $\cos(\arctan(b)) = 1 / \sqrt{1 + b^2}$.

In addition, the transversality condition (5.48) must be satisfied to account for the free end time t_f , whereby we can take advantage of the terminal conditions for the adjoint states $\lambda_v(t_f) = \lambda_w(t_f) = 0$

$$H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda})|_{t=t_f} = -1 \quad \Rightarrow \quad \lambda_x g_v(\lambda_x, \lambda_y, t_f) + \lambda_y g_w(\lambda_x, \lambda_y, t_f) = -1. \quad (5.59)$$

The desired terminal conditions $x(t_f) = x_f$, $y(t_f) = y_f$ for the target point and the end time t_f must be computed from the equations (5.58a), (5.58c), (5.59). This results in a set of (nonlinear) equations for the three unknowns $(\lambda_x, \lambda_y, t_f)$

$$\begin{bmatrix} g_x(\lambda_x, \lambda_y, t_f) \\ g_y(\lambda_x, \lambda_y, t_f) \\ \lambda_x g_v(\lambda_x, \lambda_y, t_f) + \lambda_y g_w(\lambda_x, \lambda_y, t_f) \end{bmatrix} - \begin{bmatrix} x_f \\ y_f \\ -1 \end{bmatrix} = \mathbf{0} \quad (5.60)$$

that must be solved numerically.

A suitable MATLAB function for the solution of the nonlinear equations is `fsolve` from the Optimization Toolbox. The function `fsolve` uses the trust region method by default (see Section 3.4.1) to solve a set of equations in residual form $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ as minimization problem in $\mathbf{x}$.

Figure 5.8 lists the MATLAB code for the particle problem as an example for the usage of `fsolve` to solve (5.60). It is assumed that the particle starts at the point $(x_0, y_0) = (0, 0)$ with the velocity $(v_0, w_0) = (0, 1)$ in vertical direction. Figure 5.9 shows the optimal trajectories $x^*(t)$, $y^*(t)$ of the particle in the (x, y) -plane for different target points (x_f, y_f) . The arrows illustrate the (constant) direction $u = \arctan(\lambda_y/\lambda_x)$ of the thrust force $F = ma$.

5.3.4 General procedure and singular case

The previous examples illustrate how the optimality conditions (5.46a)-(5.48) can be solved in simple cases using analytical considerations. The general procedure for solving the optimality conditions consists of the following steps:

1. Set-up of the Hamiltonian $H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}, t) = l(\mathbf{x}, \mathbf{u}, t) + \boldsymbol{\lambda}^\top \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$.
2. Computation of $\mathbf{u}$ from (5.46c) such that $\mathbf{u}$ can be expressed as function of $(\mathbf{x}, \boldsymbol{\lambda}, t)$

$$\mathbf{u} = \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}, t). \quad (5.61)$$

3. Inserting the control function (5.61) into the canonical equations (5.46a)-(5.46b) leads to the boundary value problem

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}, t), t), \quad \dot{\boldsymbol{\lambda}} = -H_x(\mathbf{x}, \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}, t), \boldsymbol{\lambda}, t), \quad \text{BCs} \quad (5.47)$$

that is now independent of the control $\mathbf{u}$. If the end time t_f is free, then the transversality condition (5.48) extends the boundary conditions (BCs) (5.47).

4. The solution of the BVP leads to the trajectories $\mathbf{x}^*(t)$, $\boldsymbol{\lambda}^*(t)$, $t \in [t_0, t_f]$. The optimal control trajectory $\mathbf{u}^*(t)$ then follows from (5.61). In most cases, the BVP must be solved numerically (see Section 5.6).

```

function [t,x,y,p] = particle(xf,yf)
% -----
% (xf,yf): desired target point
% (t,x,y): trajectory of the particle
% p:      parameter structure

p.a = 1;                                     % parameters
p.x0=0; p.v0=0; p.y0=0; p.w0=1;             % initial conditions
p.xf=xf; p.yf=yf;                           % terminal conditions

opt = optimset('Display','iter');            % options
X0 = [-1,0,1];                              % initial guess for fsolve
Xopt = fsolve(@eqns,X0,opt,p);               % numerical solution with fsolve
p.lx=Xopt(1); p.ly=Xopt(2); p.tf=Xopt(3);    % solution

t = linspace(0,p.tf,100);                   % trajectories
x = xfct(p.lx,p.ly,t,p);
y = yfct(p.lx,p.ly,t,p);
% -----
function res = eqns(X,p)                    % equations in residual form
lx=X(1); ly=X(2); tf=X(3);
res = [ xfct(lx,ly,tf,p) - p.xf;
        yfct(lx,ly,tf,p) - p.yf;
        lx*vfct(lx,ly,tf,p) + ly*wfct(lx,ly,tf,p) + 1 ];
% -----
function x = xfct(lx,ly,t,p)                % functions for x and v
cosu = 1/sqrt(1+(ly/lx)^2);
x = p.x0 + p.v0*t + p.a/2*cosu*t.^2;        % 't.^2' = component-wise evaluation
function v = vfct(lx,ly,t,p)
cosu = 1/sqrt(1+(ly/lx)^2);
v = p.v0 + p.a*cosu*t;
% -----
function y = yfct(lx,ly,t,p)                % functions for y and w
sinu = ly/(lx*sqrt(1+(ly/lx)^2));
y = p.y0 + p.w0*t + p.a/2*sinu*t.^2;        % 't.^2' = component-wise evaluation
function w = wfct(lx,ly,t,p)
sinu = ly/(lx*sqrt(1+(ly/lx)^2));
w = p.w0 + p.a*sinu*t;

```

Figure 5.8: MATLAB code for the particle problem.

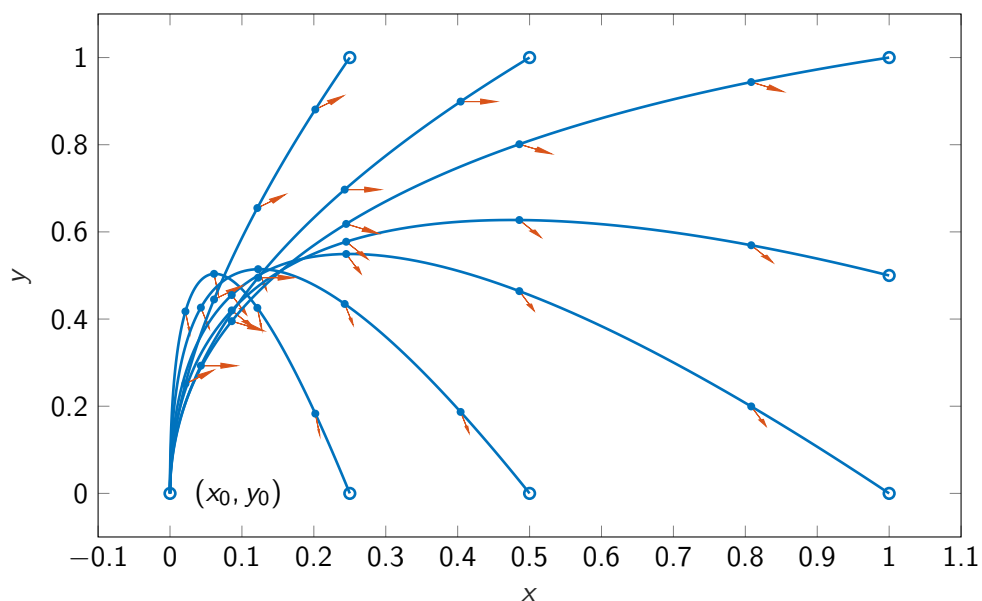


Figure 5.9: Time optimal control of a particle to different target points.

It is important to point out that the optimality conditions (5.46a)-(5.48) are necessary but not sufficient. Hence, the solution of the canonical equations does not automatically imply the optimality of the trajectories $\mathbf{x}^*(t)$, $\mathbf{u}^*(t)$. From a practical perspective, however, it is often assumed that a (unique) optimal solution exists for a given problem. Moreover, the plausability of the solution can often be verified by physical insight and process knowledge.

If the system function $\mathbf{f}(\mathbf{x}, \mathbf{u})$ or the integral cost $l(\mathbf{x}, \mathbf{u}, t)$ of the cost functional (5.37a) depend nonlinearly on the control $\mathbf{u}$, it might be difficult to determine the function (5.61) analytically. In many practical cases, however, the numerical solution of (5.61) can be avoided:

Example 5.4 (Input-affine system & energy optimal cost term) *The computation of the control function (5.61) is straightforward for input-affine systems*

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \mathbf{f}_0(\mathbf{x}) + \sum_{i=1}^m \mathbf{f}_i(\mathbf{x}) u_i \quad (5.62)$$

and an energy optimal term in the cost functional

$$J(\mathbf{u}) = V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l_0(\mathbf{x}) + \frac{1}{2} \sum_{i=1}^m r_i u_i^2 dt, \quad r_i > 0. \quad (5.63)$$

In this case, $\mathbf{u} = \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda})$ with $\boldsymbol{\psi} = [\psi_1, \dots, \psi_m]^T$ can be explicitly computed from the single conditions $\frac{\partial H}{\partial u_i} = r_i u_i + \boldsymbol{\lambda}^T \mathbf{f}_i(\mathbf{x}) = 0$, i.e.

$$u_i = \psi_i(\mathbf{x}, \boldsymbol{\lambda}) = -\frac{1}{r_i} \boldsymbol{\lambda}^T \mathbf{f}_i(\mathbf{x}), \quad i = 1, \dots, m.$$

A special case occurs if one or more elements u_i of the control vector $\mathbf{u}$ cannot be determined from the stationarity condition (5.46c). This can easily be demonstrated for an input-affine system (5.62) with scalar control u ($m = 1$). If the integral term of the cost functional (5.37a) is independent of u , i.e. $l = l(\mathbf{x}, t)$, the Hamiltonian and its derivative w.r.t. u read

$$H(\mathbf{x}, u, \boldsymbol{\lambda}, t) = l(\mathbf{x}, t) + \boldsymbol{\lambda}^T (\mathbf{f}_0(\mathbf{x}) + \mathbf{f}_1(\mathbf{x}) u), \quad H_u(\mathbf{x}, \boldsymbol{\lambda}) = \boldsymbol{\lambda}^T \mathbf{f}_1(\mathbf{x}) = 0.$$

This case is referred to as *singular problem* because the control u cannot be determined from the stationarity condition. Nevertheless, $H_u(\mathbf{x}^*(t), \boldsymbol{\lambda}^*(t)) = 0$ must be satisfied, although it does not provide further information about the optimal control itself. This also implies that its time derivatives $\frac{d^r}{dt^r} H_u(\cdot)$ must vanish over the horizon $t \in [t_0, t_f]$. The first time derivative r for which the control u appears explicitly, i.e.

$$\frac{d^r}{dt^r} H_u(\mathbf{x}^*(t), \boldsymbol{\lambda}^*(t)) = 0 \quad \text{with} \quad \frac{\partial}{\partial u} \left(\frac{d^r}{dt^r} H_u(\cdot) \right) \neq 0$$

is used for computing $u^*(t)$. The index r is the *degree of singularity* of the optimal control problem.

It is obvious that the treatment of singular problems can be delicate and is problem-specific. In practice, the singular case can be avoided by adding a *regularization term* $\sum_{i=1}^m r_i u_i^2$ to the integral cost function with sufficiently small weights $r_i > 0$. This ensures that u_i does not vanish in the single stationarity conditions $H_{u_i} = 0$, also see Example 5.4.

5.3.5 Interpretation of adjoint variables

In the parameter optimization case, it was shown that the Lagrange multipliers λ_i and μ_i correspond to the sensitivity of the cost function f at the optimum w.r.t. small changes of the corresponding constraints (see Section 4.2.4). A similar interpretation can be derived for the adjoint variables $\boldsymbol{\lambda}^*(t)$ of an optimal control problem. To this end, we consider a fixed end time problem without terminal constraints and terminal cost

$$\min_{\mathbf{u}(\cdot)} \int_{t_0}^{t_f} l(\mathbf{x}, \mathbf{u}, t) dt \quad (5.64a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0. \quad (5.64b)$$

We assume that an optimal and continuous solution $\mathbf{x}^*(t)$, $\mathbf{u}^*(t)$ exists and that the functions $\mathbf{f}$ and l are continuously differentiable.

In the following, we consider a small perturbation $\boldsymbol{\varepsilon}$ of the initial state $\mathbf{x}(t_0) = \mathbf{x}_0 + \boldsymbol{\varepsilon}$ with the associated optimal trajectories $\mathbf{x}^\varepsilon(t)$, $\mathbf{u}^\varepsilon(t)$ that satisfy the system dynamics (5.64b)

$$\dot{\mathbf{x}}^\varepsilon(t) = \mathbf{f}(\mathbf{x}^\varepsilon(t), \mathbf{u}^\varepsilon(t), t), \quad \mathbf{x}^\varepsilon(t_0) = \mathbf{x}_0 + \boldsymbol{\varepsilon} \quad (5.65)$$

and define the nominal optimal solution for $\boldsymbol{\varepsilon} = \mathbf{0}$

$$\mathbf{x}^*(t) = \mathbf{x}^0(t), \quad \mathbf{u}^*(t) = \mathbf{u}^0(t), \quad t \in [t_0, t_f]. \quad (5.66)$$

In the next step, we add the dynamics (5.65) to the cost functional (5.64a)

$$\begin{aligned} V(\mathbf{x}^\varepsilon(t_0), t_0) &= \int_{t_0}^{t_f} l(\mathbf{x}^\varepsilon(t), \mathbf{u}^\varepsilon(t), t) dt \\ &= \int_{t_0}^{t_f} l(\mathbf{x}^\varepsilon(t), \mathbf{u}^\varepsilon(t), t) + \boldsymbol{\lambda}^*(t)^\top [\mathbf{f}(\mathbf{x}^\varepsilon(t), \mathbf{u}^\varepsilon(t), t) - \dot{\mathbf{x}}^\varepsilon(t)] dt. \end{aligned}$$

where the adjoint variables $\boldsymbol{\lambda}^*(t)$ correspond to the nominal optimal solution $\mathbf{x}^*(t)$, $\mathbf{u}^*(t)$.

Under the assumption that $\mathbf{x}^\varepsilon(t)$ and $\mathbf{u}^\varepsilon(t)$ are continuously differentiable in $\boldsymbol{\varepsilon}$, we integrate by parts and differentiate afterwards w.r.t. $\boldsymbol{\varepsilon}$, which leads to (omitting arguments)

$$\begin{aligned} \frac{d}{d\boldsymbol{\varepsilon}} V(\mathbf{x}^\varepsilon(t_0), t_0) &= -[(\mathbf{x}_\varepsilon^\varepsilon)^\top \boldsymbol{\lambda}^*]_{t_0}^{t_f} + \int_{t_0}^{t_f} (\mathbf{x}_\varepsilon^\varepsilon)^\top [\mathbf{l}_x(\mathbf{x}^\varepsilon, \mathbf{u}^\varepsilon, t) + \mathbf{f}_x(\mathbf{x}^\varepsilon, \mathbf{u}^\varepsilon, t)^\top \boldsymbol{\lambda}^* + \dot{\boldsymbol{\lambda}}^*] \\ &\quad + (\mathbf{u}_\varepsilon^\varepsilon)^\top [\mathbf{l}_u(\mathbf{x}^\varepsilon, \mathbf{u}^\varepsilon, t) + \mathbf{f}_u(\mathbf{x}^\varepsilon, \mathbf{u}^\varepsilon, t)^\top \boldsymbol{\lambda}^*] dt. \end{aligned}$$

Note that the sensitivities $\mathbf{x}_\varepsilon^\varepsilon(t) = \frac{d}{d\boldsymbol{\varepsilon}} \mathbf{x}^\varepsilon(t) \in \mathbb{R}^{n \times n}$ and $\mathbf{u}_\varepsilon^\varepsilon(t) = \frac{d}{d\boldsymbol{\varepsilon}} \mathbf{u}^\varepsilon(t) \in \mathbb{R}^{m \times n}$ are time-varying matrices. The equation above is a vector equation, where the integral has to be understood component-wise. We recover the nominal optimal solution (5.66) for $\boldsymbol{\varepsilon} = \mathbf{0}$, i.e.

$$\begin{aligned} \frac{d}{d\boldsymbol{\varepsilon}} V(\mathbf{x}^\varepsilon(t_0), t_0) \Big|_{\boldsymbol{\varepsilon}=\mathbf{0}} &= -\mathbf{x}_\varepsilon^0(t_f)^\top \boldsymbol{\lambda}^*(t_f) + \mathbf{x}_\varepsilon^0(t_0)^\top \boldsymbol{\lambda}^*(t_0) \\ &+ \int_{t_0}^{t_f} \mathbf{x}_\varepsilon^0(t)^\top [\mathbf{l}_x(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_x(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}^* + \dot{\boldsymbol{\lambda}}^*] + \mathbf{u}_\varepsilon^0(t)^\top [\mathbf{l}_u(\mathbf{x}^*, \mathbf{u}^*, t) + \mathbf{f}_u(\mathbf{x}^*, \mathbf{u}^*, t)^\top \boldsymbol{\lambda}^*] dt. \end{aligned}$$

This equation can be drastically simplified because the triplet $(\mathbf{x}^*, \mathbf{u}^*, \boldsymbol{\lambda}^*)$ satisfies the necessary optimality conditions (5.46b)-(5.46c) with $\boldsymbol{\lambda}^*(t_f) = \mathbf{0}$, which leads to

$$\frac{d}{d\boldsymbol{\varepsilon}} V(\mathbf{x}^\varepsilon(t_0), t_0) \Big|_{\boldsymbol{\varepsilon}=\mathbf{0}} = \mathbf{x}_\varepsilon^0(t_0)^\top \boldsymbol{\lambda}^*(t_0) \quad \Rightarrow \quad V_x(\mathbf{x}_0, t_0) = \boldsymbol{\lambda}^*(t_0) \quad (5.67)$$

with $V_x = \frac{\partial V}{\partial x}$. In the last relation, we exploited the fact that the sensitivity $\mathbf{x}_\varepsilon^\varepsilon(t_0)$ at the initial time $t = t_0$ is exactly the $(n \times n)$ -unit matrix $\mathbf{I}$, which is easily seen from (5.65).

The relation (5.67) illustrates that the adjoint variables $\boldsymbol{\lambda}^*(t_0)$ at the initial time $t = t_0$ are a measure for the sensitivity of the value of the cost functional w.r.t. perturbations of the initial state $\mathbf{x}_0$, also see Figure 5.5 and 5.6 of Exercise 5.2 and 5.3. This important insight for the initial time point can be generalized to the time interval $[t_0, t_f]$ when applying the *principle of optimality*.

Theorem 5.4 (Principle of optimality) Suppose that $\mathbf{u}^*(t)$, $t \in [t_0, t_f]$ is an optimal control of the problem (5.64) with the associated state trajectory $\mathbf{x}^*(t)$. Then, for each subinterval $[t_1, t_f]$ with $t_0 \leq t_1 \leq t_f$, the control $\mathbf{u}^*(t)$, $t \in [t_1, t_f]$ is the optimal solution of the problem

$$\min_{\mathbf{u}(\cdot)} \int_{t_1}^{t_f} l(\mathbf{x}, \mathbf{u}, t) dt \quad (5.68a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_1) = \mathbf{x}^*(t_1). \quad (5.68b)$$

Thanks to the principle of optimality, the derivation of (5.67) can be transferred to the remainder problem (5.68), i.e. $V_x(\mathbf{x}^*(t_1), t_1) = \boldsymbol{\lambda}^*(t_1)$. Since the time point t_1 can be freely chosen within the interval $[t_0, t_f]$, we have

$$V_x(\mathbf{x}^*(t), t) = \boldsymbol{\lambda}^*(t), \quad t \in [t_0, t_f]. \quad (5.69)$$

Thus, the adjoint state $\boldsymbol{\lambda}^*(t)$ is a measure of how strongly the cost value would be affected by a small perturbation of the state $\mathbf{x}^*(t)$ at time t .

5.3.6 General terminal conditions

The case of partial terminal conditions is the most common one in practice and nicely reflects the duality between the terminal conditions (5.47b) and (5.47c) for the system state $\mathbf{x}$ and the adjoint state $\boldsymbol{\lambda}$. However, the derivation of the optimality conditions via the calculus of variations also allows for handling more general terminal conditions of the form

$$\mathbf{g}(\mathbf{x}(t_f), t_f) = \mathbf{0}, \quad \mathbf{g} : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^r. \quad (5.70)$$

Similar to the procedure in Section 5.3.1, we add the terminal conditions (5.70) to the extended cost functional (5.38) by introducing further multipliers $\boldsymbol{\nu} \in \mathbb{R}^r$

$$\bar{J} = V(\mathbf{x}(t_f)) + \boldsymbol{\nu}^\top \mathbf{g}(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) + \boldsymbol{\lambda}^\top (\mathbf{f}(\mathbf{x}, \mathbf{u}, t) - \dot{\mathbf{x}}) dt. \quad (5.71)$$

By considering the first variation of the cost functional and applying partial integration as well as the fundamental lemma of the calculus of variations, it can be shown that the transversality condition (5.47c) for an optimal solution is extended by

$$\boldsymbol{\lambda}^*(t_f) = V_x(\mathbf{x}^*(t_f), t_f) + \mathbf{g}_x(\mathbf{x}^*(t_f), t_f)^\top \boldsymbol{\nu}^*. \quad (5.72)$$

If the end time t_f is free, the transversality condition (5.48) for the Hamiltonian is modified accordingly

$$H(\mathbf{x}^*(t_f^*), \mathbf{u}^*(t_f^*), \boldsymbol{\lambda}^*(t_f^*), t_f^*) = -V_t(\mathbf{x}^*(t_f^*), t_f^*) - \mathbf{g}_t(\mathbf{x}^*(t_f^*), t_f^*)^\top \boldsymbol{\nu}^*. \quad (5.73)$$

In summary, the optimality conditions comprise $2n$ differential equations (5.46a), (5.46b) and $2n + r$ boundary conditions (5.47a), (5.70), (5.72) for the $2n$ states $\mathbf{x}^*(t)$, $\boldsymbol{\lambda}^*(t)$ and the r (constant) multipliers $\boldsymbol{\nu}^*$ as well as m algebraic equations (5.46c) for the control $\mathbf{u}^*(t)$. The scalar transversality condition (5.73) is added to the equations if the end time t_f^* is free.

5.4 Optimization of linear systems with quadratic cost functional

A large class of optimal control problems concern linear systems with quadratic cost functional

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{u}) = \frac{1}{2} \mathbf{x}(t_f)^\top \mathbf{S} \mathbf{x}(t_f) + \frac{1}{2} \int_{t_0}^{t_f} \mathbf{x}(t)^\top \mathbf{Q} \mathbf{x}(t) + \mathbf{u}(t)^\top \mathbf{R} \mathbf{u}(t) dt \quad (5.74a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \quad \mathbf{x}(t_0) = \mathbf{x}_0. \quad (5.74b)$$

The terminal state $\mathbf{x}(t_f)$ is free and we assume that the weighting matrices $\mathbf{S}$, $\mathbf{Q}$ are positive semidefinite and $\mathbf{R}$ is positive definite. Moreover, the system matrices $\mathbf{A}$ and $\mathbf{B}$ as well as the weighting matrices $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{S}$ are supposed to be time-invariant although the considerations in the next section directly apply to the time-varying case.

The optimal control problem (5.74) can be interpreted as follows: starting from $\mathbf{x}(t_0) = \mathbf{x}_0$, bring the system (5.74b) close to the origin $\mathbf{x}(t_f) \approx \mathbf{0}$ within the time interval $[t_0, t_f]$, whereby the weighting matrices $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{S}$ determine the state and input behavior of the system, also see Exercise 5.2 and 5.3.

5.4.1 Riccati differential equation

In the following, we derive the optimality conditions for the linear-quadratic problem (5.74). With the Hamiltonian

$$H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}, t) = \frac{1}{2} (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}) + \boldsymbol{\lambda}^\top (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}),$$

the optimality conditions (5.46a)-(5.47c) become

$$\dot{\mathbf{x}}^* = H_{\mathbf{x}} = \mathbf{A}\mathbf{x}^* + \mathbf{B}\mathbf{u}^*, \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (5.75a)$$

$$\dot{\boldsymbol{\lambda}}^* = -H_{\mathbf{x}} = -\mathbf{Q}\mathbf{x}^* - \mathbf{A}^\top \boldsymbol{\lambda}^*, \quad \boldsymbol{\lambda}^*(t_f) = \mathbf{S}\mathbf{x}^*(t_f) \quad (5.75b)$$

$$\mathbf{0} = H_{\mathbf{u}} = \mathbf{R}\mathbf{u}^* + \mathbf{B}^\top \boldsymbol{\lambda}^*. \quad (5.75c)$$

Solving (5.75c) for the optimal control

$$\mathbf{u}^*(t) = -\mathbf{R}^{-1} \mathbf{B}^\top \boldsymbol{\lambda}^*(t) \quad (5.76)$$

and inserting into the canonical equations (5.75a), (5.75b) yields the two-point BVP

$$\begin{bmatrix} \dot{\mathbf{x}}^* \\ \dot{\boldsymbol{\lambda}}^* \end{bmatrix} = \begin{bmatrix} \mathbf{A} & -\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top \\ -\mathbf{Q} & -\mathbf{A}^\top \end{bmatrix} \begin{bmatrix} \mathbf{x}^* \\ \boldsymbol{\lambda}^* \end{bmatrix}, \quad \begin{matrix} \mathbf{x}^*(t_0) = \mathbf{x}_0 \\ \boldsymbol{\lambda}^*(t_f) = \mathbf{S}\mathbf{x}^*(t_f) \end{matrix}. \quad (5.77)$$

It can be shown by means of the transition matrix of (5.77) that $\mathbf{x}^*(t)$ and $\boldsymbol{\lambda}^*(t)$ are coupled via a time-varying matrix $\mathbf{P}(t)$ with the terminal condition $\mathbf{P}(t_f) = \mathbf{S}$, i.e.

$$\boldsymbol{\lambda}^*(t) = \mathbf{P}(t)\mathbf{x}^*(t), \quad t \in [t_0, t_f]. \quad (5.78)$$

By differentiating (5.78), $\dot{\boldsymbol{\lambda}}^* = \dot{\mathbf{P}}\mathbf{x}^* + \mathbf{P}\dot{\mathbf{x}}^*$, and inserting it into the differential equations (5.77), we get

$$\left[\dot{\mathbf{P}} + \mathbf{P}\mathbf{A} + \mathbf{A}^\top \mathbf{P} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P} + \mathbf{Q} \right] \mathbf{x}^*(t) = \mathbf{0}.$$

In order for this relation to hold for all potential optimal trajectories $\mathbf{x}^*(t)$ and all times $t \in [t_0, t_f]$, it is not only sufficient but also necessary that the bracketed term vanishes. This leads to the so-called *Riccati differential equation*

$$\dot{\mathbf{P}} = -\mathbf{P}\mathbf{A} - \mathbf{A}^\top \mathbf{P} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P} - \mathbf{Q}, \quad \mathbf{P}(t_f) = \mathbf{S}. \quad (5.79)$$

Note that the Riccati differential equation is nonlinear because $\mathbf{P}(t)$ appears quadratically in the term $\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P}$. Inserting (5.78) into (5.76) gives

$$\mathbf{u}^*(t) = -\mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P}(t)\mathbf{x}^*(t). \quad (5.80)$$

Some important aspects in this context are:

- The Riccati differential equation (5.79) is typically solved backward in time to use the terminal condition $\mathbf{P}(t_f) = \mathbf{S}$ as initial condition.
- $\mathbf{P}(t)$ is symmetric and positive semidefinite. Thus, the Riccati differential equation of order n^2 is equivalent to $n(n+1)/2$ coupled differential equations.
- The derivation is the same for time-varying system matrices $\mathbf{A}(t)$, $\mathbf{B}(t)$ and/or weighting matrices $\mathbf{Q}(t)$, $\mathbf{R}(t)$.
- After computing $\mathbf{P}(t)$, the optimal trajectories for the state $\mathbf{x}^*(t)$ and control $\mathbf{u}^*(t)$ follow from integrating the system dynamics (5.75a), i.e. $\dot{\mathbf{x}}^* = (\mathbf{A} - \mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P}(t))\mathbf{x}^*$ with $\mathbf{x}^*(t_0) = \mathbf{x}_0$ and eventually evaluating (5.80).

The Riccati differential equation is also important in the context of *optimal feedback control*, because the relation (5.80) corresponds to the optimal feedback law

$$\mathbf{u}(t) = -\mathbf{K}(t)\mathbf{x}(t) \quad \text{with} \quad \mathbf{K}(t) = \mathbf{R}^{-1}\mathbf{B}^\top \mathbf{P}(t), \quad (5.81)$$

where $\mathbf{K}(t)$ is considered as time-varying feedback gain matrix.

5.4.2 Example: Swing-up of the pendulum on a cart

The linear-quadratic control design is particularly useful in the context of the *two-degrees-of-freedom control scheme* (2DOF) in Figure 5.10, which is often used in practice for trajectory tracking. The main idea behind the 2DOF structure is to separately design the control performance regarding reference tracking and disturbance rejection. A reference trajectory $\mathbf{u}_{ref}(t)$ is injected as *feedforward control* that allows the system to exactly track the reference state trajectory $\mathbf{x}_{ref}(t)$ in the nominal case. The feedback controller as the second degree-of-freedom

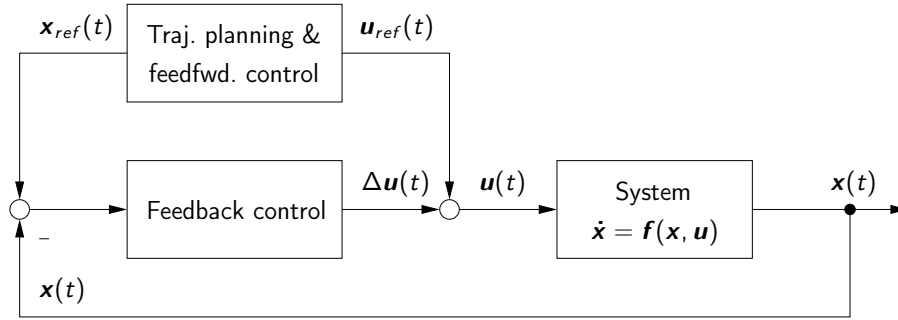


Figure 5.10: Two-degrees-of-freedom control scheme.

stabilizes the system along the reference trajectory $\mathbf{x}_{ref}(t)$ and compensates for model inaccuracies and disturbances.

To design the feedback controller, the nonlinear system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ can be linearized around the reference trajectories leading to the linear time-varying system

$$\Delta \dot{\mathbf{x}} = \mathbf{A}(t)\Delta \mathbf{x} + \mathbf{B}(t)\Delta \mathbf{u} \quad \text{with} \quad \mathbf{A}(t) = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{x}_{ref}(t), \mathbf{u}_{ref}(t)} \quad \text{and} \quad \mathbf{B}(t) = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\mathbf{x}_{ref}(t), \mathbf{u}_{ref}(t)}. \quad (5.82)$$

The state $\Delta \mathbf{x}(t) = \mathbf{x}(t) - \mathbf{x}_{ref}(t)$ and control $\Delta \mathbf{u}(t) = \mathbf{u}(t) - \mathbf{u}_{ref}(t)$ of the linearized system denote the deviations from the reference trajectories. The Riccati differential equation can be used to derive an optimal feedback law of the form (5.81) to stabilize the nonlinear system around the reference trajectory $\mathbf{x}_{ref}(t)$.

The concept is illustrated for the swing-up of a pendulum on a cart that is depicted in Figure 5.11. A simplified nonlinear model for the cart position y and the angle ϕ of the pendulum is⁴

$$\ddot{y} = u, \quad \ddot{\phi} = \frac{3}{2l}(g \sin(\phi) - \cos(\phi)u) \quad (5.83)$$

with the length l of the pendulum and the acceleration due to gravity g . The input u is the acceleration $\ddot{y}$ of the cart.

The control task is the swing-up of the pendulum from the lower to the upper (inverted) equilibrium, i.e.

$$\left. \begin{array}{l} y(0) = 0, \quad \dot{y}(0) = 0 \\ \phi(0) = -\pi, \quad \dot{\phi}(0) = 0 \end{array} \right\} \Rightarrow \left\{ \begin{array}{l} y(t_f) = 0, \quad \dot{y}(t_f) = 0 \\ \phi(t_f) = 0, \quad \dot{\phi}(t_f) = 0. \end{array} \right. \quad (5.84)$$

A suitable feedforward control trajectory for the swing-up of a pendulum with length $l = 1$ m in the time $t_f = 2$ s is

$$u_{ref}(t) = \sum_{i=1}^5 c_i t^i, \quad t \in [0, t_f = 2 \text{ s}] \quad (5.85)$$

with the coefficients $\mathbf{c} = [-135.128, 625.696, -888.598, 497.538, -96.3861]^T$ (neglecting SI units). This trajectory was computed with the methodology [8] which is not further detailed here.

⁴ The model can be derived with the Lagrange formalism and neglecting friction for the sake of simplicity. The moment of inertia of the pendulum is $J = \frac{1}{12}ml^2$ with the mass m . The pendulum dynamics (5.83) are independent of m because the mass m of the pendulum is cancelled out in the equations.

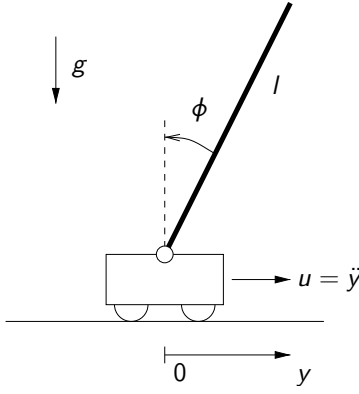


Figure 5.11: Pendulum on a cart.

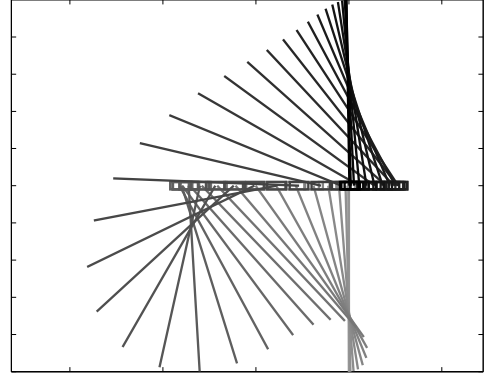


Figure 5.12: Snapshots of the swing-up.

Figure 5.12 illustrates the swing-up motion of the pendulum. The corresponding state trajectory $\mathbf{x}_{ref}(t)$ with $\mathbf{x} = [y, \dot{y}, \phi, \dot{\phi}]^T$ is shown in Figure 5.13 (---) and is computed by numerically integrating the dynamics (5.83).

Due to potential model inaccuracies and the instability of the inverted pendulum position, a feedback controller is essential to stabilize the pendulum during the swing-up maneuver. To this end, the nonlinear model (5.83) is linearized around the reference trajectories $\mathbf{x}_{ref}(t)$ and $\mathbf{u}_{ref}(t)$ leading to the linear time-varying system (5.82)

$$\Delta \dot{\mathbf{x}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{3}{2l}[g \cos \phi_{ref}(t) + \sin \phi_{ref}(t)u_{ref}(t)] & 0 \end{bmatrix}}_{=\mathbf{A}(t)} \Delta \mathbf{x} + \underbrace{\begin{bmatrix} 0 \\ 1 \\ 0 \\ -\frac{3}{2l} \cos \phi_{ref}(t) \end{bmatrix}}_{=\mathbf{b}(t)} \Delta u \quad (5.86)$$

with $\mathbf{x}(t) = \mathbf{x}_{ref}(t) + \Delta \mathbf{x}(t)$ and $u(t) = u_{ref}(t) + \Delta u(t)$. The Riccati control law (5.81) is computed for the cost functional (5.74a)

$$J(\Delta u) = \frac{1}{2} \Delta \mathbf{x}(t_f)^T \mathbf{S} \Delta \mathbf{x}(t_f) + \frac{1}{2} \int_0^{t_f} \Delta \mathbf{x}(t)^T \mathbf{Q} \Delta \mathbf{x}(t) + R \Delta u(t)^2 dt, \quad t_f = 2s \quad (5.87)$$

with the diagonal matrices $\mathbf{S} = \mathbf{Q} = \text{diag}(1, 1, 1, 1)$. The weight for the control is chosen as $R = 0.01$ and therefore relatively small compared to the states, which corresponds to a fast control action.

The Riccati differential equation (5.79) must be integrated numerically to obtain the matrix $\mathbf{P}(t)$ on the time interval $t \in [0, t_f]$. The optimal feedback law for the swing-up of the pendulum follows from (5.81)

$$\Delta u(t) = -\mathbf{k}(t)^T \Delta \mathbf{x}(t) = \frac{1}{R} \mathbf{b}(t)^T \mathbf{P}(t) (\mathbf{x}_{ref}(t) - \mathbf{x}(t)). \quad (5.88)$$

For simplicity, we assume that the full state vector $\mathbf{x} = [y, \dot{y}, \phi, \dot{\phi}]^T$ is available and no state observer is necessary. The 2DOF control structure in Figure 5.10 shows that the control u that is applied to the pendulum consists of the combination of the feedforward and feedback part

$$u(t) = u_{ref}(t) + \Delta u(t). \quad (5.89)$$

The simulated trajectories for the swing-up of the pendulum in Figure 5.13 show the open-loop case (only feedforward action without feedback) and the closed-loop case (feedforward and feedback) for a disturbance of the initial downward position. The open-loop trajectories illustrate that the pendulum fails to follow the reference trajectories and falls down before the upper unstable position is reached. In closed-loop, the pendulum is stabilized along the swing-up trajectories despite the initial disturbance. The plot of the control signal shows that the feedback correction $\Delta u(t)$ requires approximately 1.2 s to stabilize the initial disturbance, before the control signal is dominated by the feedforward part $u_{ref}(t)$.

Another interesting aspect is that the feedback gains $\mathbf{k}(t)$ in Figure 5.13 change sign which is due to the fact the pendulum is transferred from the lower stable position to the upper unstable one. The change of sign is an indicator that the linear time-varying system (5.86) temporarily loses controllability during the swing-up. For an experimental validation of the swing-up, it would be recommended to reduce the feedback gains $\mathbf{k}(t)$ to zero during these critical time intervals and to steer through them open-loop with the feedforward control $u_{ref}(t)$.

Figure 5.14 lists the MATLAB code for solving the Riccati differential equation (5.79), the computation of the feedback gains (5.88), and the simulation of the swing-up with the perturbed initial state (5.84). For simplicity, the Riccati differential equation (5.79) is numerically integrated for all elements of the matrix $\mathbf{P}(t)$ although $\mathbf{P}(t)$ is symmetric as pointed out before.

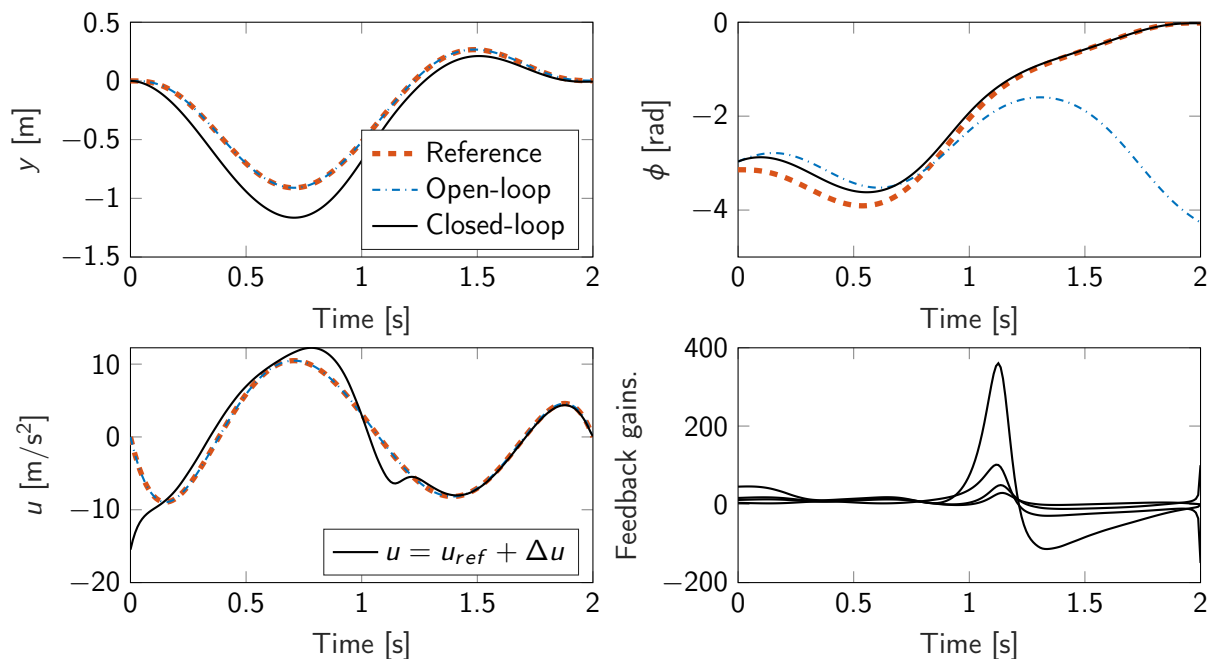


Figure 5.13: Simulation results for the pendulum swing-up.

5.4.3 Algebraic Riccati equation

The computation of the optimal feedback (5.81) is particularly simple if the time interval $[t_0, t_f]$ of the cost functional (5.74a) is extended to infinity with $t_f \rightarrow \infty$. In this case, the terminal cost $\mathbf{x}(t_f)^T \mathbf{S} \mathbf{x}(t_f)$ can be discarded since $\lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{0}$ has to hold anyway or the integral in (5.74a) would not be finite. We therefore set $\mathbf{S} = \mathbf{0}$.

```

function vec = pendulum
% -----
g = 9.81; l = 1;                                     % parameters
p.a1 = 3*g/(2*l); p.a2 = 3/(2*l); p.n=4; p.steps = 200;

p.tf = 2;                                             % reference trajectories
vec.t = linspace(0,p.tf,p.steps);                  % -----
c = [-135.128, 625.696, -888.598, 497.538, -96.3861]; % feedforward trajectory
vec.uref = c(1)*vec.t+c(2)*vec.t.^2+c(3)*vec.t.^3+c(4)*vec.t.^4+c(5)*vec.t.^5;
[tsim,xsim] = ode45(@pendulum_sim,vec.t,[0;0;-pi;0],[],vec,p, 0); % num. int. for state traj.
vec.xref = xsim';

p.Q = diag([1,1,1,1]); p.R = 0.01; p.S = diag([1,1,1,1]); % weighting matrices
S = reshape(p.S,p.n^2,1); % matrix -> vector
[tsim,Psim] = ode45(@riccati_ode,vec.t(end:-1:1),S,[],vec,p); % Riccati: backward integration
vec.P = Psim(end:-1:1,:)''; % -----

for i=1:length(vec.t), % time-varying feedback gain
    [A,B] = matrices(vec.t(i),vec,p); % -----
    P = reshape(vec.P(:,i),p.n,p.n);
    vec.k(:,i) = inv(p.R)*B'*P;
end

% simulation with initial error
X0 = [0;0;-pi+10*pi/180;2]; % -----
[tsim,x_ol] = ode45(@pendulum_sim,vec.t,X0,[],vec,p, 0); % open-loop
[tsim,x_cl] = ode45(@pendulum_sim,vec.t,X0,[],vec,p, 1); % closed-loop
vec.x_ol = x_ol'; vec.x_cl = x_cl';

for i=1:p.steps, % computation of feedback part
    vec.du(i) = vec.k(:,i)'*(vec.xref(:,i)-vec.x_cl(:,i));
end

function F = pendulum_sim(t,x,vec,p,control)
% -----
y=x(1:2); phi=x(3:4);
uref = interp1(vec.t,vec.uref,t); % feedforward control
if control==0, % simulation open-loop
    u = uref;
else % simulation closed-loop
    xref = interp1(vec.t,vec.xref',t)';
    k = interp1(vec.t,vec.k',t);
    u = uref + k*(xref-x);
end
F = [ y(2); u; phi(2); p.a1*sin(phi(1))-p.a2*cos(phi(1))*u ]; % nonlin. pendulum model

function Pp = riccati_ode(t,Pvec,vec,p)
% -----
[A,B] = matrices(t,vec,p); % time-varying matrices A and B
P = reshape(Pvec,p.n,p.n); % vector -> matrix
Pp = -P*A - A'*P + P*B*inv(p.R)*B'*P - p.Q; % Riccati diff. eqn.
Pp = reshape(Pp,p.n^2,1); % matrix -> vector

function [A,B] = matrices(t,vec,p)
% -----
phiref = interp1(vec.t,vec.xref(3:4,:),'',t)'; % current reference values
uref = interp1(vec.t,vec.uref,t);
A = [0,1,0,0; 0,0,0,0; 0,0,0,1; % time-varying matrices A and B
      0,0,p.a1*cos(phiref(1))+p.a2*sin(phiref(1))*uref,0];
B = [0; 1; 0; -p.a2*cos(phiref(1))];

```

Figure 5.14: MATLAB code to compute the optimal feedback law (5.88) and for simulating the pendulum swing-up.

It is also assumed that

- the matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{Q}$, and $\mathbf{R}$ in (5.74) are time-invariant,
- $\mathbf{Q}$ is positive semidefinite and $\mathbf{R}$ is positive definite,
- the system (5.74b) is completely controllable,
- the pair $[\mathbf{A}, \mathbf{C}]$ is completely observable, where the matrix $\mathbf{C}$ is defined by the decomposition⁵ $\mathbf{C}^T \mathbf{C} = \mathbf{Q}$ (e.g. via a Cholesky decomposition).

Under these assumptions, the solution of the Riccati differential equation (5.79) converges in backward time against a stationary positive definite solution, see e.g. [11, 12]. One option to compute the stationary solution $\mathbf{P}$ is to integrate (5.79) backward in time starting from $\mathbf{P}(t_f) = \mathbf{0}$ for a sufficiently large t_f until the variations of $\mathbf{P}(t)$ lie below a certain tolerance.

A more elegant method for computing $\mathbf{P}$ is the solution of the *algebraic Riccati equation*

$$\mathbf{P}\mathbf{A} + \mathbf{A}^T \mathbf{P} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T \mathbf{P} + \mathbf{Q} = \mathbf{0} \quad (5.90)$$

that directly follows from the stationary consideration of (5.79). The Riccati equation is nonlinear, which implies that multiple solutions can exist. Under the given assumptions, however, there is only *one positive definite* solution $\mathbf{P}$. There exist various methods for solving the algebraic Riccati equation (5.90), e.g. those due to *Kalman* or *Englar*, that are not detailed in the following.

The principle of optimality (Theorem 5.4) can be used to show that the time-invariant feedback law

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t) \quad \text{with} \quad \mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^T \mathbf{P}$$

is the optimal solution to the problem (5.74) on the infinite horizon $[t_0, \infty)$. This important result is summarized in the following theorem.

Theorem 5.5 (LQ control for linear time-invariant systems) *Consider the completely controllable linear time-invariant system*

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \quad \mathbf{x}(t_0) = \mathbf{x}_0$$

and the cost functional

$$J(\mathbf{u}) = \frac{1}{2} \int_{t_0}^{\infty} \mathbf{x}(t)^T \mathbf{Q} \mathbf{x}(t) + \mathbf{u}(t)^T \mathbf{R} \mathbf{u}(t) dt \quad (5.91)$$

with $\mathbf{Q}$ being positive semidefinite and $\mathbf{R}$ positive definite. Moreover, suppose that the pair $[\mathbf{A}, \mathbf{C}]$ with $\mathbf{Q} = \mathbf{C}^T \mathbf{C}$ is completely observable. Then, the minimum of (5.91) is attained by the optimal feedback law

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t) \quad \text{with} \quad \bar{\mathbf{K}} = \mathbf{R}^{-1}\mathbf{B}^T \mathbf{P}, \quad (5.92)$$

where $\mathbf{P}$ is the unique positive definite solution of the algebraic Riccati equation (5.90). Moreover, all eigenvalues of the matrix $(\mathbf{A} - \mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T \mathbf{P})$ have negative real part such that the closed-loop system

$$\dot{\mathbf{x}} = (\mathbf{A} - \mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T \mathbf{P}) \mathbf{x}, \quad \mathbf{x}(t_0) = \mathbf{x}_0$$

is asymptotically stable.

⁵ An illustrative explanation of the last two assumptions can be found in [12].

The optimal feedback law (5.92) is often called *Riccati controller*, *LQ controller* or simply *LQR* (“linear quadratic regulator”) and is often used in practice. In contrast to the concept of pole placement or eigenvalue assignment, the optimal controller is designed by choosing the weighting matrices $\mathbf{Q}$ and $\mathbf{R}$, usually in diagonal form, which allows a straightforward interpretation for the single states and control variables. In MATLAB, the optimal feedback gain $\mathbf{K} = \mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P}$ can be computed with the functions `lqr` or `lqrd/dlqr` in the discrete-time case.

5.5 Optimal control problems with input constraints

For the previous considerations and the calculus of variations in general, we assumed that the optimal control problem (5.37) is unconstrained. In the following, we extend these results to the case of input constraints

$$\mathbf{u}(t) \in U \subseteq \mathbb{R}^m \quad \forall t \in [t_0, t_f]. \quad (5.93)$$

Input constraints often occur in practice and can be incorporated into the structure of the optimality conditions by means of *Pontryagin’s maximum principle*. State constraints are significantly harder to account for and are therefore not considered at this stage.

5.5.1 Pontryagin’s maximum principle

The maximum principle was postulated by *L.S. Pontryagin* in 1956 and was proven by him and his colleagues *Boltjanski*, *Gamkrelidse*, and *Mischenko* in the following years for more and more general system classes.⁶ Similar to the calculus of variations, the maximum principle is typically formulated in terms of the Hamiltonian function and the canonical equations (5.46). However, the stationary condition $H_u = 0$ is not valid anymore due to the constraints (5.93).

To illustrate this aspect, Figure 5.15 plots exemplary profiles of the Hamiltonian $H(\mathbf{x}^*, u, \boldsymbol{\lambda}^*, t)$ for a scalar control variable u and some fixed (optimal) trajectories $\mathbf{x}^*(t)$, $\boldsymbol{\lambda}^*(t)$ evaluated at a specific time point t . In the unconstrained case a), the conditions (5.46c) and (5.49), i.e. $H_u = 0$ and positive semidefiniteness of H_{uu} , hold for a minimum of the Hamiltonian at every time point t . However, if u is constrained to an interval $U = [u^-, u^+]$, the minimum can lie on the boundary u^- or u^+ , where $H_u = 0$ is in general not satisfied as the cases b) and c) in Figure 5.15 illustrate.

The discussion shows that it is reasonable to replace the stationarity condition (5.46c) for the Hamiltonian by its minimization

$$H(\mathbf{x}^*, \mathbf{u}^*, \boldsymbol{\lambda}^*, t) = \min_{\mathbf{u} \in U} H(\mathbf{x}^*, \mathbf{u}, \boldsymbol{\lambda}^*, t).$$

This condition forms the basis of Pontryagin’s maximum principle that is summarized in the following theorem for time-invariant problems.

⁶ The name “maximum principle” is due to the fact that it was originally formulated for maximization problems. In this light, we should rather speak of minimum principle as it is also done in parts of the literature. However, to credit its importance in dynamic optimization and the contributions of *L.S. Pontryagin* in general, we stay with the original name “maximum principle”.

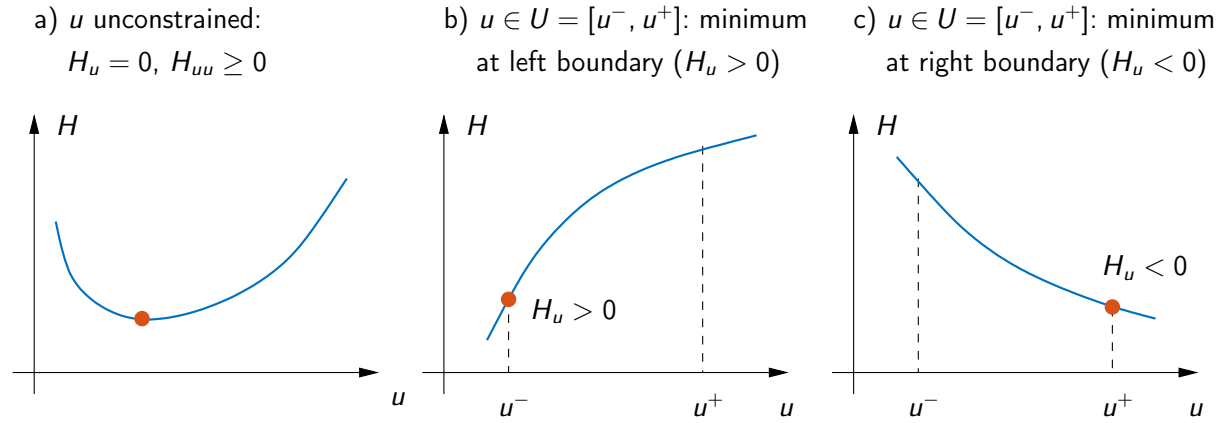


Figure 5.15: Minimum of the Hamiltonian in the unconstrained and constrained case.

Theorem 5.6 (Maximum principle for time-invariant systems) Consider the optimal control problem

$$\min_{u(\cdot)} J(u) = \int_{t_0}^{t_f} l(x(t), u(t)) dt \quad (5.94a)$$

$$\text{s.t. } \dot{x} = f(x, u) \quad (5.94b)$$

$$x(t_0) = x_0, \quad x(t_f) = x_f \quad (5.94c)$$

$$u(t) \in U, \quad \forall t \in [t_0, t_f] \quad (5.94d)$$

with the free end time t_f . Suppose that the functions l and f are continuous in (x, u) and continuously differentiable in x . If the problem (5.94) has an optimal solution $x^*(t)$, $u^*(t)$, $t \in [t_0, t_f^*]$, then there exists a non-negative constant $\lambda_0 \geq 0$ and a non-vanishing (piecewise continuously differentiable) time function $\lambda^* \in C_n^1[t_0, t_f^*]$ with $\lambda^* = [\lambda_1^*, \dots, \lambda_n^*]^T$ such that

a) $x^*(t)$ and $\lambda^*(t)$, $t \in [t_0, t_f^*]$ satisfy the canonical equations

$$\dot{x}^* = f(x^*, u^*), \quad \dot{\lambda}^* = -H_x(x^*, u^*, \lambda^*) \quad (5.95)$$

with the boundary conditions (5.94c) and the Hamiltonian

$$H(x, u, \lambda, \lambda_0) = \lambda_0 l(x, u) + \lambda^T f(x, u). \quad (5.96)$$

b) $H(x^*(t), u, \lambda^*(t), \lambda_0^*)$ as a function of u has a global minimum at the point $u = u^*(t) \in U$ for all $t \in [t_0, t_f^*]$, i.e.

$$H(x^*(t), u^*(t), \lambda^*(t), \lambda_0^*) = \min_{u \in U} H(x^*(t), u, \lambda^*(t), \lambda_0^*) \quad \forall t \in [t_0, t_f^*]. \quad (5.97)$$

c) $H(x^*(t), u^*(t), \lambda^*(t), \lambda_0^*)$ is constant for all $t \in [t_0, t_f^*]$. If the end time t_f^* is free, then the transversality condition

$$H(x^*(t_f^*), u^*(t_f^*), \lambda^*(t_f^*), \lambda_0^*) = 0 \quad (5.98)$$

is additionally satisfied.

In analogy to the unconstrained case, the maximum principle can be extended to problems with partial or general terminal conditions by adapting the boundary conditions in (5.94c) by (5.47b), (5.47c) or (5.70), (5.72), respectively.

In the time-varying case, i.e. $l = l(\mathbf{x}, \mathbf{u}, t)$ and $\mathbf{f} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$, the Hamiltonian is no longer constant along the optimal solution (statement in point c) of Theorem 5.6) but satisfies the differential equation (5.50)

$$\frac{d}{dt}H = \lambda_0^* l_t(\mathbf{x}^*(t), \mathbf{u}^*(t), t) + \boldsymbol{\lambda}^*(t)^\top \mathbf{f}_t(\mathbf{x}^*(t), \mathbf{u}^*(t), t) \quad \forall t \in [t_0, t_f^*]$$

with $l_t = \frac{\partial l}{\partial t}$ and $\mathbf{f}_t = \frac{\partial \mathbf{f}}{\partial t}$.

The additional parameter λ_0 in the Hamiltonian (5.96) accounts for *abnormal problems* for which $\lambda_0^* = 0$ holds. This case occurs, for instance, if the optimal solution is independent of the integrated cost function l in (5.94a). An abnormal problem is often ill-posed in the sense that its solution does not affect the value of the cost functional. For well-posed problems, λ_0^* is typically set to $\lambda_0^* = 1$ such that the same Hamiltonian results as in the unconstrained case.

Example 5.5 (Abnormal case) We consider the problem

$$\begin{aligned} \min_{u(\cdot)} \quad & \int_0^1 l(x, u) dt \\ \text{s.t.} \quad & \dot{x} = u, \quad x(0) = 0, \quad x(1) = 1 \\ & u(t) \in [0, 1]. \end{aligned}$$

There exists only one control $u^*(t) = 1$ that steers the state $x^*(t) = t$ from $x^*(0) = 0$ to $x^*(1) = 1$. This shows that the optimal solution is independent of the choice of the integral cost $l(x, u)$.

5.5.2 General procedure

The application of the maximum principle is similar to the unconstrained case in Section 5.3.4.⁷

1. Set-up of the Hamiltonian with $\lambda_0 = 1$: $H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = l(\mathbf{x}, \mathbf{u}) + \boldsymbol{\lambda}^\top \mathbf{f}(\mathbf{x}, \mathbf{u})$.
2. The solution of the minimization problem $\min_{\mathbf{u} \in U} H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda})$ for each potential combination $(\mathbf{x}, \boldsymbol{\lambda})$ defines the optimal control function

$$\mathbf{u} = \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}) = \arg \min_{\mathbf{u} \in U} H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) \quad (5.99)$$

in dependence of $(\mathbf{x}, \boldsymbol{\lambda})$. The construction of (5.99) is illustrated in Section 5.5.3 for selected cases.

3. Inserting the function (5.99) into the canonical equations (5.95) leads to the boundary value problem

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda})), \quad \dot{\boldsymbol{\lambda}} = -H_{\mathbf{x}}(\mathbf{x}, \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}), \boldsymbol{\lambda}), \quad \text{BCs} \quad (5.94c).$$

⁷ In analogy to Theorem 5.6, we consider the time-invariant case without loss of generality.

In case of partial or general nonlinear terminal conditions, $\mathbf{x}(t_0) = \mathbf{x}_0$ and $\mathbf{x}(t_f) = \mathbf{x}_f$ in (5.94c) are replaced by (5.47b), (5.47c) or (5.70), (5.72), respectively. If the end time is free, then (5.98) extends the boundary conditions (BCs).

4. The (numerical) solution of the boundary value problem consists of $\mathbf{x}^*(t)$, $\boldsymbol{\lambda}^*(t)$. The optimal control trajectory $\mathbf{u}^*(t)$ is obtained afterwards from evaluating (5.99).

Similar to the unconstrained optimality conditions, the maximum principle only defines necessary conditions for an optimal solution. For certain problem classes, however, the maximum principle is also sufficient, in particular for the time-optimal control of time-invariant linear systems.

5.5.3 Minimization of the Hamiltonian

The solution of the minimization problem (5.97) is illustrated for selected relevant cases. It is assumed that the nonlinear system (5.94b) is given in the input-affine form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \mathbf{f}_0(\mathbf{x}) + \sum_{i=1}^m \mathbf{f}_i(\mathbf{x}) u_i. \quad (5.100)$$

Moreover, we consider the common case that all components u_i of the control vector $\mathbf{u} = [u_1, \dots, u_m]^T$ are box-constrained, i.e.

$$\mathbf{u} \in U = [\mathbf{u}^-, \mathbf{u}^+] \quad \text{or} \quad u_i \in [u_i^-, u_i^+], \quad i = 1, \dots, m, \quad (5.101)$$

and derive the optimal control function (5.99) component-wise with $\boldsymbol{\psi} = [\psi_1, \dots, \psi_m]^T$.

Cost functional with consumption term

We consider the particular form of the cost functional (5.94a)

$$J(\mathbf{u}) = \int_{t_0}^{t_f} l_0(\mathbf{x}) + \sum_{i=1}^m r_i |u_i| dt, \quad r_i > 0. \quad (5.102)$$

If the state-dependent term $l_0(\mathbf{x})$ is zero, the term $\sum_{i=1}^m r_i |u_i|$ corresponds to the *control surface* over time or the *consumption* of the optimal control problem. For $l_0(\mathbf{x}) \neq 0$, a trade-off is achieved between both terms depending on the coefficients $r_i > 0$.

The Hamiltonian for the problem is defined as

$$H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = l_0(\mathbf{x}) + \sum_{i=1}^m r_i |u_i| + \boldsymbol{\lambda}^T \left(\mathbf{f}_0(\mathbf{x}) + \sum_{i=1}^m \mathbf{f}_i(\mathbf{x}) u_i \right).$$

We can omit the term $l_0(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{f}_0(\mathbf{x})$ in the minimization problem $\min_{\mathbf{u} \in U} H$ as it is independent of the control $\mathbf{u}$.⁸ In addition, the elements u_i of the control vector $\mathbf{u} = [u_1, \dots, u_m]^T$ enter linearly in H which allows one to consider the minimization of H separately for each u_i , i.e.

$$\min_{u_i \in [u_i^-, u_i^+]} H_i(u_i) = r_i |u_i| + q_i(\mathbf{x}, \boldsymbol{\lambda}) u_i \quad \text{with} \quad q_i(\mathbf{x}, \boldsymbol{\lambda}) = \boldsymbol{\lambda}^T \mathbf{f}_i(\mathbf{x}). \quad (5.103)$$

⁸ It is important to note that for the Hamiltonian minimization, $H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda})$ is understood as a function of $\mathbf{u}$ with $\mathbf{x}$, $\boldsymbol{\lambda}$ being fixed (more accurately: for the *optimal* $\mathbf{x}^*(t)$, $\boldsymbol{\lambda}^*(t)$ at each time point t) as formulated in the maximum principle (Theorem 5.6).

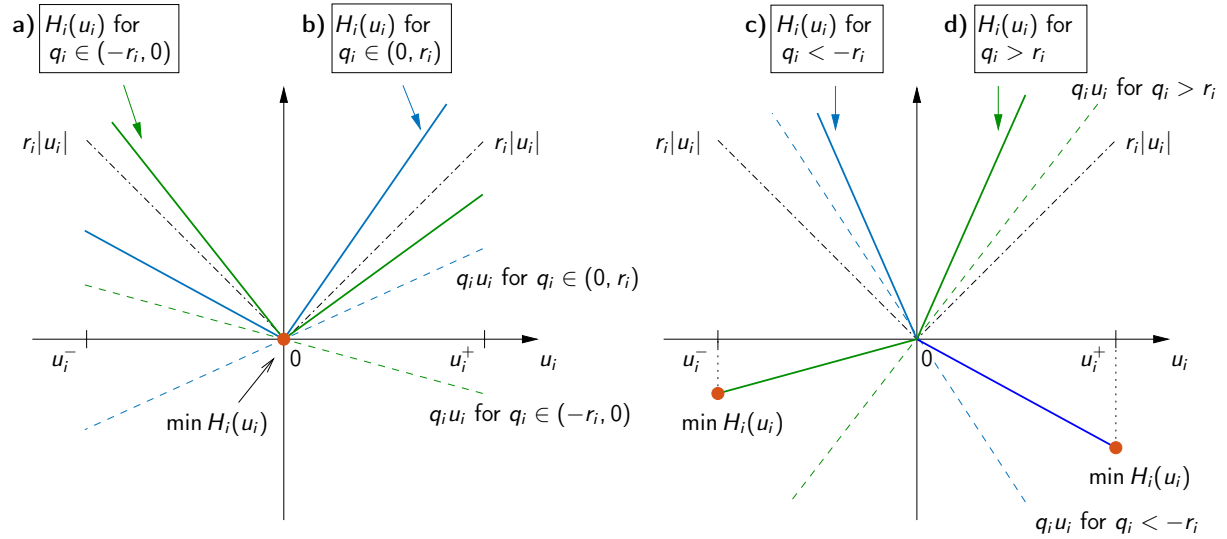


Figure 5.16: Illustration of (5.103) for the consumption optimal case.

The prefactor $q_i(\mathbf{x}, \boldsymbol{\lambda})$ plays an important role for solving the problem. Figure 5.16 illustrates the different cases a) to d) that are captured in the case-dependent and component-wise definition of the optimal control function (5.99)

$$u_i = \psi_i(\mathbf{x}, \boldsymbol{\lambda}) = \begin{cases} u_i^- & \text{if } q_i(\mathbf{x}, \boldsymbol{\lambda}) \geq r_i \\ 0 & \text{if } q_i(\mathbf{x}, \boldsymbol{\lambda}) \in (-r_i, r_i), \\ u_i^+ & \text{if } q_i(\mathbf{x}, \boldsymbol{\lambda}) \leq -r_i \end{cases} \quad i = 1, \dots, m. \quad (5.104)$$

Note that the control is piecewise continuous and not uniquely defined at the switching points $q_i(\mathbf{x}, \boldsymbol{\lambda}) = \pm r_i$. In practice, however, this is typically not an issue as the switching points are usually crossed at isolated time points.

Cost functional with energy term

The partial consideration of energy optimality (in the sense of the energy of the control signal $\mathbf{u}(t)$) in the cost functional (5.94a) leads to the formulation

$$J(\mathbf{u}) = \int_{t_0}^{t_f} l_0(\mathbf{x}) + \frac{1}{2} \sum_{i=1}^m r_i u_i^2 dt, \quad r_i > 0. \quad (5.105)$$

Similar to the previous case, the minimization of the Hamiltonian can be handled element-wise by defining the functions

$$\min_{u_i \in [u_i^-, u_i^+]} H_i(u_i) = \frac{1}{2} r_i u_i^2 + q_i(\mathbf{x}, \boldsymbol{\lambda}) u_i \quad \text{with} \quad q_i(\mathbf{x}, \boldsymbol{\lambda}) = \boldsymbol{\lambda}^\top \mathbf{f}_i(\mathbf{x}). \quad (5.106)$$

Thanks to the quadratic term with $r_i > 0$, the function $H_i(u_i)$ always has an unconstrained minimum at the point

$$u_i^0 = -\frac{1}{r_i} q_i(\mathbf{x}, \boldsymbol{\lambda}),$$

which directly follows from the condition $\frac{dH_i}{du_i} = r_i u_i + q_i(\mathbf{x}, \boldsymbol{\lambda}) = 0$. If u_i^0 lies inside the admissible interval $[u_i^-, u_i^+]$, then the optimal solution of (5.106) is defined by $u_i^* = u_i^0$. If

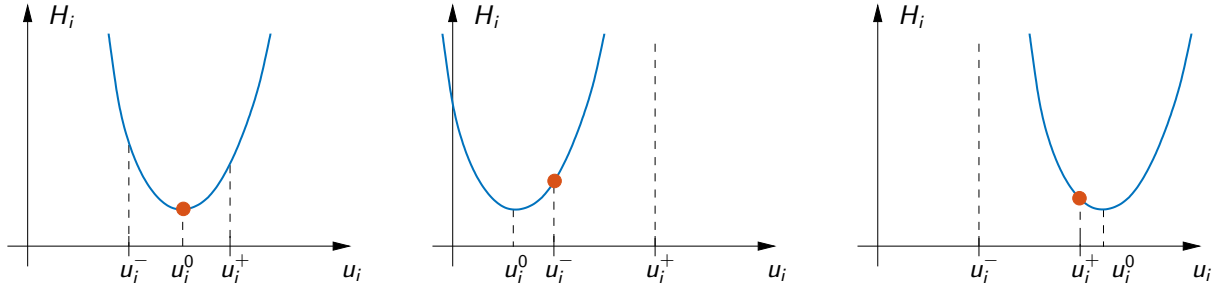


Figure 5.17: Illustration of (5.106) for the energy optimal case.

u_i^0 lies outside the interval $[u_i^-, u_i^+]$, the minimum of $H_i(u_i)$ lies either on the left or right boundary u_i^- or u_i^+ , because $H_i(u_i)$ is strictly monotonically increasing (decreasing) for $u_i^0 < u_i^-$ ($u_i^0 > u_i^+$), see Figure 5.17. In summary, the optimal control function (5.99) is defined component-wise by

$$u_i = \psi_i(\mathbf{x}, \boldsymbol{\lambda}) = \begin{cases} u_i^- & \text{if } u_i^0 \leq u_i^- \\ u_i^0 & \text{if } u_i^0 \in (u_i^-, u_i^+) \\ u_i^+ & \text{if } u_i^0 \geq u_i^+ \end{cases}, \quad i = 1, \dots, m. \quad (5.107)$$

Note that u_i is continuous in contrast to the previous consumption optimal case.

Time optimal cost functional

For time optimal problems, the cost functional (5.94a) is defined by

$$J(\mathbf{u}) = \int_{t_0}^{t_f} 1 \, dt = t_f - t_0 \quad (5.108)$$

with the respective Hamiltonian

$$H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = 1 + \boldsymbol{\lambda}^\top \left(\mathbf{f}_0(\mathbf{x}) + \sum_{i=1}^m \mathbf{f}_i(\mathbf{x}) u_i \right).$$

Again, the minimization problem (5.97) is treated component-wise

$$\min_{u_i \in [u_i^-, u_i^+]} H_i(u_i) = q_i(\mathbf{x}, \boldsymbol{\lambda}) u_i \quad \text{with} \quad q_i(\mathbf{x}, \boldsymbol{\lambda}) = \boldsymbol{\lambda}^\top \mathbf{f}_i(\mathbf{x}). \quad (5.109)$$

The problem is linear in u_i and the solution therefore depends on the sign of $q_i(\mathbf{x}, \boldsymbol{\lambda})$

$$u_i = \psi_i(\mathbf{x}, \boldsymbol{\lambda}) = \begin{cases} u_i^- & \text{if } q_i(\mathbf{x}, \boldsymbol{\lambda}) > 0 \\ u_i^+ & \text{if } q_i(\mathbf{x}, \boldsymbol{\lambda}) < 0 \end{cases}, \quad i = 1, \dots, m. \quad (5.110)$$

A critical case occurs if $q_i(\mathbf{x}(t), \boldsymbol{\lambda}(t))$ is zero on a subinterval $I_s \subseteq [t_0, t_f]$. In this case, the Hamiltonian H is independent of u_i and H is trivially minimal for every value of u_i . This *singular problem* was already described in Section 5.3.4 in the unconstrained case. In practice, the singularity issue is often avoided by adding a *regularization term*

$$J(\mathbf{u}) = \int_{t_0}^{t_f} 1 + \frac{1}{2} \sum_{i=1}^m r_i u_i^2 \, dt$$

with sufficiently small coefficients $r_i > 0$. This formulation corresponds to the case (5.105) such that $u_i = \psi_i(\mathbf{x}, \boldsymbol{\lambda})$ can be computed with (5.107).

5.5.4 Example: Double integrator

A popular example to illustrate the maximum principle is the time optimal transition of a double integrating system to the origin $\mathbf{x}(t_f) = \mathbf{0}$. The optimal control problem with the state $\mathbf{x} = [x_1, x_2]^T$ is formulated as

$$\min_{u(\cdot)} \int_{t_0=0}^{t_f} 1 \, dt = t_f \quad (5.111a)$$

$$\text{s.t. } \dot{x}_1 = x_2, \quad \dot{x}_2 = u \quad (5.111b)$$

$$\mathbf{x}(0) = \mathbf{x}_0, \quad \mathbf{x}(t_f) = \mathbf{0} \quad (5.111c)$$

$$|u(t)| \leq 1 \quad \forall t \in [0, t_f]. \quad (5.111d)$$

With the Hamiltonian $H(\mathbf{x}, u, \boldsymbol{\lambda}) = 1 + \lambda_1 x_2 + \lambda_2 u$, the adjoint dynamics for $\boldsymbol{\lambda} = [\lambda_1, \lambda_2]^T$ read

$$\dot{\lambda}_1 = 0 \quad \Rightarrow \quad \lambda_1(t) = c_1 \quad (5.112a)$$

$$\dot{\lambda}_2 = -\lambda_1 \quad \lambda_2(t) = -c_1 t + c_2 \quad (5.112b)$$

with the integration constants c_1 and c_2 . The condition (5.97) for the Hamiltonian leads to the optimal control function (5.110)

$$u(t) = \begin{cases} +1 & \text{if } \lambda_2 < 0 \\ -1 & \text{if } \lambda_2 > 0. \end{cases} \quad (5.113)$$

The singular case for $\lambda_2(t) = 0$ on a non-vanishing subinterval I_s cannot occur due to the fact that $\lambda_2(t)$ represents a straight line, see (5.112b), and therefore $\lambda_2(t) = 0$ can only hold for an isolated time point t_s (switching between the limits ± 1) or for the whole time interval $[0, t_f]$. This, however, would contradict the transversality condition (5.98) for the free end time t_f

$$H(\mathbf{x}, u, \boldsymbol{\lambda})|_{t=t_f} = 1 + \lambda_2(t_f) u(t_f) = 0.$$

In fact, it can be shown that the time-optimal control of linear time-invariant systems is always non-singular, see e.g. [12, 7].

Since $\lambda_2(t)$ is a straight line, there exists at most one switching event for the control in the time interval $[t_0, t_f]$ leading to the four possible control sequences $\{+1\}$, $\{-1\}$, $\{+1, -1\}$, $\{-1, +1\}$ for an optimal solution. This also implies that the trajectories in the (x_1, x_2) -plane are parabolas, which is addressed in the following exercise.

Exercise 5.4 Show that the solution of the double integrator dynamics (5.111b) for $u(t) = \pm 1 = \text{const.}$ satisfies the parabola equation

$$x_1 = \frac{x_2^2}{2u} + c, \quad u = \pm 1, \quad c \in \mathbb{R}.$$

The optimization goal is to reach the origin $\mathbf{x}(t_f) = \mathbf{0}$ as fast as possible from an arbitrary initial state $\mathbf{x}(0) = \mathbf{x}_0$. Figure 5.18 illustrates that the origin is only reachable via the parabola $x_1 = x_2^2/2$ for $u = 1$ or $x_1 = -x_2^2/2$ for $u = -1$, respectively. Both cases can be combined in the *switching curve*

$$S(x_2) = -\frac{1}{2}x_2|x_2|.$$

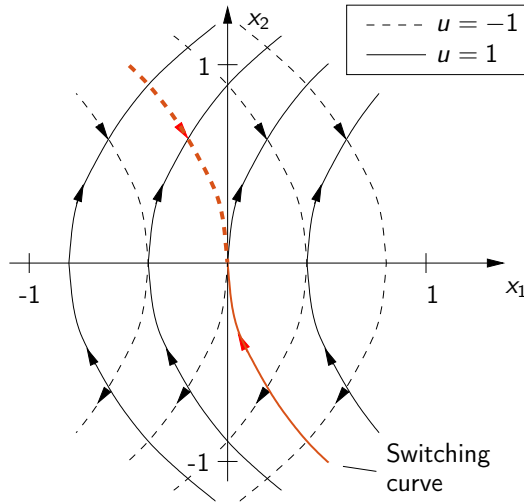


Figure 5.18: Trajectories of the double integrator in the (x_1, x_2) -plane.

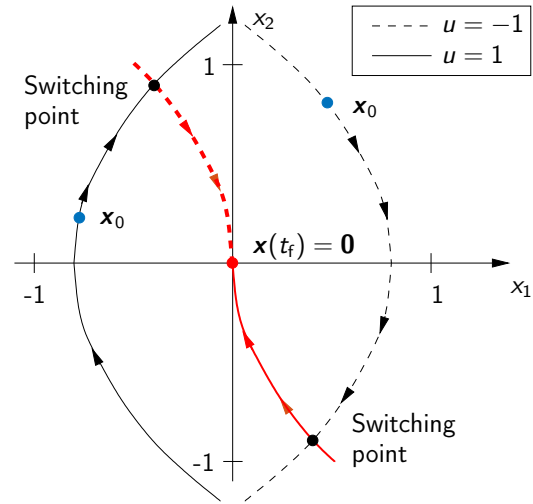


Figure 5.19: Optimal switching for different initial states $\mathbf{x}_0$.

In view of the fact that only four switching sequences $\{+1\}$, $\{-1\}$, $\{+1, -1\}$, $\{-1, +1\}$ are possible for u , there exists only one way to reach the origin $\mathbf{x}(t_f) = \mathbf{0}$ as fast as possible from $\mathbf{x}(0) = \mathbf{x}_0$.

- If $x_{1,0} = S(x_{2,0})$, no switching is required and $\mathbf{x}(t_f) = \mathbf{0}$ is directly reached via the switching curve with $u(t) = 1$ or $u(t) = -1$, see Figure 5.18.
- If $\mathbf{x}_0$ lies to the left or the right of the switching curve, i.e. $x_{1,0} < S(x_{2,0})$ or $x_{1,0} > S(x_{2,0})$, exactly one switching event is necessary to reach the curve $S(x_2)$ and follow it to the origin, see Figure 5.19.

These considerations lead to the optimal control law

$$u(t) = \begin{cases} +1 & \text{if } x_1 < S(x_2) \\ +1 & \text{if } x_1 = S(x_2) \text{ and } x_1 > 0 \\ -1 & \text{if } x_1 > S(x_2) \\ -1 & \text{if } x_1 = S(x_2) \text{ and } x_1 < 0. \end{cases} \quad (5.114)$$

Figure 5.20 shows the time optimal trajectories of the double integrator for various initial states $\mathbf{x}_0 = [x_{1,0}, x_{2,0}]^T$.

Exercise 5.5 Verify that the optimal switching point t_s and the end time t_f are case-dependently given by

$$t_s = \begin{cases} x_{2,0} + \sqrt{\frac{1}{2}x_{2,0}^2 + x_{1,0}} & \text{if } x_{1,0} > S(x_{2,0}) \\ -x_{2,0} + \sqrt{\frac{1}{2}x_{2,0}^2 - x_{1,0}} & \text{if } x_{1,0} < S(x_{2,0}) \end{cases} \quad (5.115)$$

$$t_f = \begin{cases} x_{2,0} + \sqrt{2x_{2,0}^2 + 4x_{1,0}} & \text{if } x_{1,0} > S(x_{2,0}) \\ -x_{2,0} + \sqrt{2x_{2,0}^2 - 4x_{1,0}} & \text{if } x_{1,0} < S(x_{2,0}) \\ |x_{2,0}| & \text{if } x_{1,0} = S(x_{2,0}). \end{cases} \quad (5.116)$$

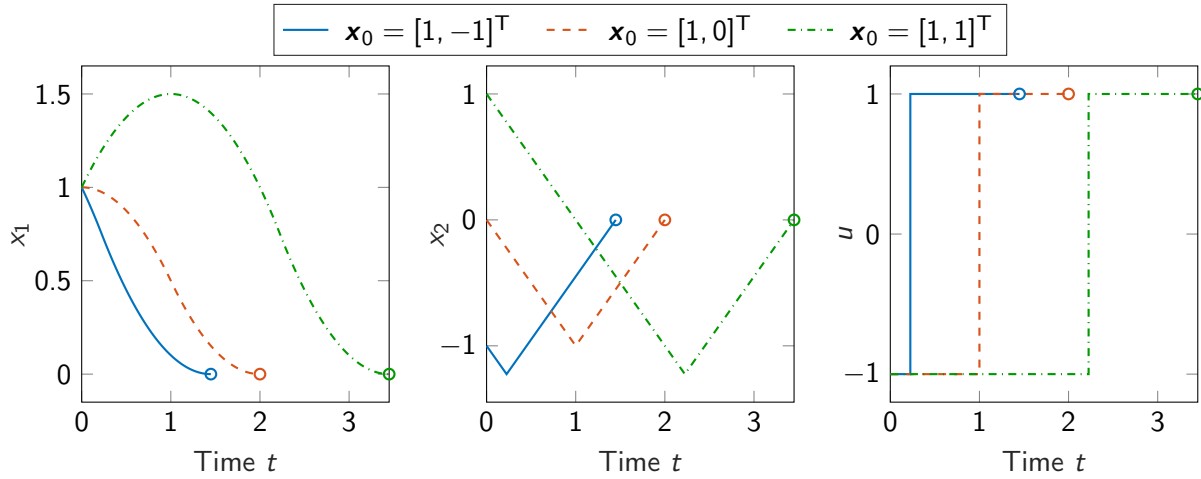


Figure 5.20: Time-optimal trajectories of the double integrator.

A compact way to express the case-dependent definition of the control law (5.114) is

$$u(t) = -\text{sign}\left(x_1 + \frac{1}{2}x_2|x_2|\right), \quad 0 = \text{sign}(0). \quad (5.117)$$

The sign function does not include the special case $x_1 = S(x_2)$ in (5.114), which means that the state $\mathbf{x}$ will always slightly cross the curve $S(x_2)$ before the switching occurs. This is not an issue from the numerical perspective because after the switching event the state $\mathbf{x}$ follows a parallel trajectory in close vicinity of the switching curve $S(x_2)$.

Exercise 5.6 Implement the system (5.111b) with the optimal control law (5.117) in MATLAB/SIMULINK and verify the results in Figure 5.20 for different initial states. Use t_f from (5.116) as the simulation time in MATLAB/SIMULINK.

5.5.5 State-dependent input constraints

The maximum principle (Theorem 5.6) can be extended to a more general class of input constraints

$$\mathbf{h}(\mathbf{x}(t), \mathbf{u}(t)) \leq \mathbf{0} \quad \forall t \in [t_0, t_f] \quad (5.118)$$

with the state/input-dependent constraint function $\mathbf{h} : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^q$. Note that the admissible set

$$U(\mathbf{x}) = \{\mathbf{u} \in \mathbb{R}^m : \mathbf{h}(\mathbf{x}, \mathbf{u}) \leq \mathbf{0}\} \quad (5.119)$$

now depends on the state $\mathbf{x}$, i.e. the control must satisfy $\mathbf{u}(t) \in U(\mathbf{x}(t)) \forall t \in [t_0, t_f]$ with the associated state trajectory $\mathbf{x}(t)$. We have to assume that all elements of $\mathbf{h} = [h_1, \dots, h_q]^T$ explicitly depend on $\mathbf{u}$ and that the constraint qualification⁹

$$\text{rank} \left[\frac{\partial \mathbf{h}}{\partial \mathbf{u}}, \text{diag}(\mathbf{h}) \right] = q \quad (5.120)$$

is satisfied. Similar to the procedure in the parameter optimization case (Chapter 4), the inequality constraints (5.118) are added to the Hamiltonian (5.96) with (time-varying) Lagrange

⁹ The constraint qualification (5.120) implies that the gradients of all constraints $h_i(\mathbf{x}, \mathbf{u}) \leq 0$ that are active at point $(\mathbf{x}, \mathbf{u})$ are linearly independent.

multipliers $\boldsymbol{\mu} = [\mu_1, \dots, \mu_q]^T$

$$H(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}, \lambda_0, \boldsymbol{\mu}) = \lambda_0 l(\mathbf{x}, \mathbf{u}) + \boldsymbol{\lambda}^T \mathbf{f}(\mathbf{x}, \mathbf{u}) + \boldsymbol{\mu}^T \mathbf{h}(\mathbf{x}, \mathbf{u}). \quad (5.121)$$

The conditions (5.95)-(5.98) of the maximum principle are adapted accordingly for the modified Hamiltonian (5.121) and the (state-dependent) admissible set (5.119). In addition, the *complementary and sign conditions*

$$\mu_i^*(t) h_i(\mathbf{x}^*(t), \mathbf{u}^*(t)) = 0, \quad \mu_i^*(t) \geq 0 \quad \forall t \in [t_0, t_f] \quad (5.122)$$

must be satisfied, similar to the KKT conditions (4.29d), (4.29e) in the parameter optimization case.

A significantly more complex case are pure state constraints

$$\mathbf{x}(t) \in X \subseteq \mathbb{R}^n \quad \text{or} \quad \mathbf{h}(\mathbf{x}(t)) \leq 0 \quad \forall t \in [t_0, t_f]$$

with $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^q$, which require a detailed look at *constrained and unconstrained arcs*, i.e. subintervals of $[t_0, t_f]$ where the constraints are active or inactive. Moreover, there are various ways of incorporating state constraints within the Hamiltonian function leading to different formulations of the optimality conditions. More information on this topic can be found in the survey [9] and the textbooks [4, 12].

5.6 Numerical solution of optimality conditions (indirect methods)

In most cases, dynamic optimization problems or optimal control problems must be solved by numerical methods. So-called *indirect methods* are based on the solution of the optimality conditions (5.46)-(5.48) or (5.95)-(5.98) in the input-constrained case. The most common methods for solving these kinds of two-point boundary value problems are *discretization*, *shooting* or the *gradient method* that are illustrated in the following.

5.6.1 Discretization method

The general optimality conditions (5.46)-(5.48) or (5.95)-(5.98) in the input-constrained case form a two-point boundary value problem for the states $(\mathbf{x}, \boldsymbol{\lambda})$ and the control $\mathbf{u}$. In the following, we assume that the optimal control can be expressed as the function $\mathbf{u} = \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda})$ of the states $\mathbf{x}$ and $\boldsymbol{\lambda}$, which renders the canonical equations independent of $\mathbf{u}$ (see Section 5.3.4 and 5.5.2)

$$\left. \begin{aligned} \dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}), t) \\ \dot{\boldsymbol{\lambda}} &= -H_{\mathbf{x}}(\mathbf{x}, \boldsymbol{\psi}(\mathbf{x}, \boldsymbol{\lambda}), \boldsymbol{\lambda}, t) \end{aligned} \right\} \quad \dot{\bar{\mathbf{x}}} = \mathbf{F}(\bar{\mathbf{x}}, t), \quad \bar{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\lambda} \end{bmatrix}. \quad (5.123)$$

In what follows, we consider a fixed end time t_f and the initial and partial terminal conditions

$$\mathbf{x}(t_0) = \mathbf{x}_0, \quad \mathbf{G}(\bar{\mathbf{x}}(t_f)) = \begin{bmatrix} [x_i(t_f) - x_{f,i}]_{i \in \mathcal{I}_f} \\ [\lambda_i(t_f) - V_{x_i}(\mathbf{x}(t_f), t_f)]_{i \notin \mathcal{I}_f} \end{bmatrix} = \mathbf{0} \quad (5.124)$$

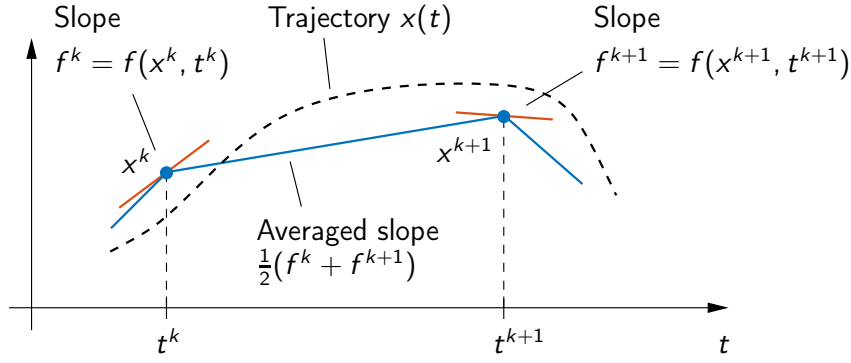


Figure 5.21: Illustration of the trapezoidal scheme.

with $\mathbf{G} : \mathbb{R}^{2n} \rightarrow \mathbb{R}^n$ such that $2n$ boundary conditions belong to the $2n$ differential equations. The handling of general terminal conditions and of a free end time t_f is addressed in Section 5.6.3.

A popular method for solving two-point boundary value problems is the discretization of the time interval $[t_0, t_f]$

$$t_0 = t^0 < t^1 < \dots < t^N = t_f,$$

in order to approximate the solution at the $N + 1$ discretized time points t^k , i.e.

$$\bar{\mathbf{x}}^k \approx \bar{\mathbf{x}}(t^k), \quad k = 0, 1, \dots, N. \quad (5.125)$$

A well known discretization formula is the *trapezoidal scheme*

$$\frac{\bar{\mathbf{x}}^{k+1} - \bar{\mathbf{x}}^k}{t^{k+1} - t^k} = \frac{1}{2} [\mathbf{F}(\bar{\mathbf{x}}^k, t^k) + \mathbf{F}(\bar{\mathbf{x}}^{k+1}, t^{k+1})], \quad k = 0, 1, \dots, N-1 \quad (5.126a)$$

that is illustrated in Figure 5.21. The discretizations (5.126) together with the $2n$ boundary conditions (5.124)

$$\mathbf{x}^0 = \mathbf{x}_0, \quad \mathbf{G}(\bar{\mathbf{x}}^N) = \mathbf{0} \quad (5.126b)$$

form a nonlinear set of equations (5.126) of order $2n(N+1)$ for the $2n(N+1)$ variables (5.125).

A related method for solving two-point BVPs is *collocation* that uses basis functions (e.g. polynomials) to approximate the state $\bar{\mathbf{x}}(t)$ on each interval $[t^k, t^{k+1}]$. The consistency with the differential equations (5.123) is enforced at additional *collocation points* within the intervals $[t^k, t^{k+1}]$. Continuity over the intervals $[t^k, t^{k+1}]$ is ensured by further consistency constraints at the interval bounds t^k . Overall, this leads to a set of nonlinear equations for the coefficients of the basis functions. Note that the collocation method is equivalent to certain discretization formulas depending on the choice of basis functions and collocation points.

Some characteristic properties of discretization methods are listed in the following:

- Discretization methods are often numerically more robust than shooting (Section 5.6.2) due to the simultaneous consideration of the discretized dynamics and boundary conditions in (5.126) and the possibility to use implicit integration schemes such as the trapezoidal scheme.
- A good initial guess of the adjoint variables is important and often decides upon whether and how fast the method converges. This is particularly true for full terminal constraints

$\mathbf{x}(t_f) = \mathbf{x}_f$ with the consequence that the adjoint dynamics are not supported by any boundary conditions, see (5.123).

- The discretization method requires a high implementation effort. However, the Jacobian of the set of equations (5.126) is sparsely occupied and is block-structured, which can be effectively exploited by *sparse numerics*.
- The number of discretization points $N + 1$ has a direct impact on the solution. Larger values of N enhance the accuracy but increase the overall dimension of the equations to be solved.
- The trapezoidal scheme is a second-order method, which means that the numerical solution $\bar{\mathbf{x}}^k \approx \bar{\mathbf{x}}(t^k)$ corresponds to the Taylor expansion of the exact solution up to the quadratic term. An alternative are e.g. higher-order Runge-Kutta schemes.

5.6.2 Shooting method

The discretization method constructs a high-dimensional set of equations for the differential equations (5.123) and boundary conditions (5.124). In contrast to this, the *shooting method* considers the initial value problem

$$\dot{\bar{\mathbf{x}}} = \mathbf{F}(\bar{\mathbf{x}}, t), \quad \bar{\mathbf{x}}(t_0) = \begin{bmatrix} \mathbf{x}_0 \\ \boldsymbol{\lambda}_0 \end{bmatrix} \quad (5.127)$$

with the unknown initial values $\boldsymbol{\lambda}_0$ of the adjoint states $\boldsymbol{\lambda}$. The solution of (5.127) can formally be written in dependence of $\boldsymbol{\lambda}_0$

$$\bar{\mathbf{x}}(t; \boldsymbol{\lambda}_0) = \begin{bmatrix} \mathbf{x}_0 \\ \boldsymbol{\lambda}_0 \end{bmatrix} + \int_{t_0}^t \mathbf{F}(\bar{\mathbf{x}}(\tau; \boldsymbol{\lambda}_0), \tau) d\tau. \quad (5.128)$$

The satisfaction of the terminal conditions (5.124) leads to the nonlinear equations

$$\mathbf{G}(\bar{\mathbf{x}}(t_f; \boldsymbol{\lambda}_0)) = \mathbf{0} \quad (5.129)$$

of order n for the initial adjoint values $\boldsymbol{\lambda}_0 \in \mathbb{R}^n$ of the same order.

The nonlinear equations (5.129) are in general solved iteratively, e.g. by means of Newton's method. To this end, the initial value problem (5.127) is numerically integrated to obtain the trajectory (5.128), which is also the reason for the name *shooting*.

Example 5.6 To illustrate the shooting method, we consider the boundary value problem

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = f(x_1, x_2), \quad x_1(t_0) = 0, \quad x_1(t_f) = 0.$$

If the problem is written as initial value problem, the solution $x_1(t; x_{2,0})$ depends on the initial value $x_2(t_0) = x_{2,0}$ of the second state, where the value $x_{2,0}$ corresponds to the initial slope of the trajectory $x_1(t; x_{2,0})$ that must be chosen such that the terminal condition $x_1(t_f; x_{2,0}) = 0$ is satisfied, see Figure 5.22.

Some important properties of the shooting method that are worth mentioning are:

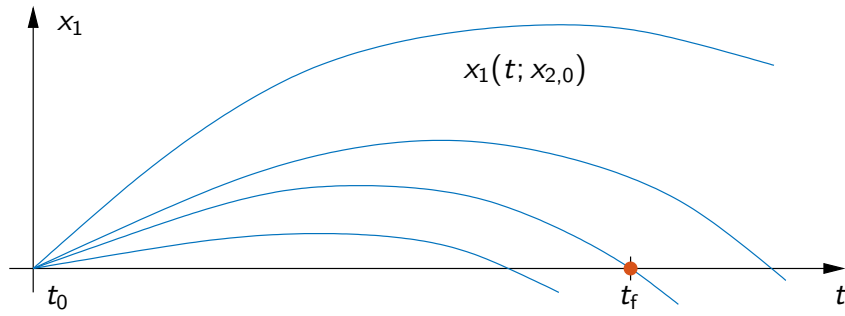


Figure 5.22: Illustration of the shooting method (Example 5.6).

- The implementation effort is low (at least compared to the discretization or the gradient method in Section 5.6.4) due to the sequential solution of the initial value problem (5.127) and the terminal constraints (5.129).
- Good initial values of λ_0 are important as the shooting method is very sensitive w.r.t. λ_0 . This is also closely related to the next point.
- The integration of the initial value problem (5.127) can be numerically critical for a larger time interval $[t_0, t_f]$. This is due to the fact that the canonical equations contain unstable modes.¹⁰
- The numerical sensitivity issues can be resolved or at least mitigated by *multiple shooting*, where the interval $[t_0, t_f]$ is further divided in subintervals [1, 13].

5.6.3 Handling of general terminal constraints and free end time

In the previous sections, a fixed end time and partial terminal conditions were assumed. In case of general terminal constraints (5.70), the function $\mathbf{G}$ in (5.124) is replaced by

$$\mathbf{G}(\bar{\mathbf{x}}(t_f), \boldsymbol{\nu}) = \begin{bmatrix} \mathbf{g}(\mathbf{x}(t_f), t_f) \\ \boldsymbol{\lambda}(t_f) - V_{\mathbf{x}}(\mathbf{x}(t_f), t_f) - \mathbf{g}_{\mathbf{x}}(\mathbf{x}(t_f), t_f)^T \boldsymbol{\nu} \end{bmatrix} = \mathbf{0} \quad (5.130)$$

with $\mathbf{G} : \mathbb{R}^{2n} \times \mathbb{R}^r \rightarrow \mathbb{R}^{n+r}$. The Lagrange multipliers $\boldsymbol{\nu} \in \mathbb{R}^r$ are additional parameters that account for the terminal conditions in the function $\mathbf{G}$.

If the end time t_f is unspecified, a time transformation

$$t = t_0 + \delta\tau \quad \text{with} \quad \tau \in [0, 1] \quad (5.131)$$

can be applied to transform the canonical equations to the normalized time interval $[0, 1]$ of the new time coordinate τ . The parameter δ is the stretching factor that determines the end time $t_f = t_0 + \delta$ (corresponding to $\tau = 1$). With the new states

$$\tilde{\mathbf{x}}(\tau) := \begin{bmatrix} \mathbf{x}(t_0 + \delta\tau) \\ \boldsymbol{\lambda}(t_0 + \delta\tau) \end{bmatrix}, \quad \tau \in [0, 1]$$

and the notation $(\cdot)' = \frac{d}{d\tau}(\cdot)$, the canonical equations (5.123) become

$$\tilde{\mathbf{x}}' = \delta \mathbf{F}(\tilde{\mathbf{x}}, t_0 + \delta\tau), \quad \tau \in (0, 1). \quad (5.132)$$

¹⁰ In the linear case, the eigenvalues of the canonical equations, see (5.77), are symmetric to the imaginary axis.

In addition, the initial and terminal conditions in (5.124) or (5.130), respectively, must be expressed in terms of the new time coordinate $\tau \in [0, 1]$ and are extended by the transversality condition (5.48) or (5.73).

The time stretch factor δ and/or multipliers ν are further unknown variables for the discretization or shooting method with the same number of additional equations in the boundary conditions $\mathbf{G}(\cdot) = \mathbf{0}$. The following table summarizes the different cases:

End time t_f	Terminal cond.	Add. parameters $\delta \in \mathbb{R}$	$\nu \in \mathbb{R}^r$	Number of initial/terminal cond.	
				$\mathbf{x}(t_0) = \mathbf{x}_0$	$\mathbf{G}(\cdot) = \mathbf{0}$
fixed	partial	-	-	n	n
fixed	general	-	✓	n	$n + r$
free	partial	✓	-	n	$n + 1$
free	general	✓	✓	n	$n + r + 1$

5.6.4 Gradient method

Another well known method for the solution of optimal control problems is the *gradient method* that follows the same spirit as the gradient method in the context of parameter optimization (see Section 3.3.2). The gradient method is mainly used for optimal control problems with fixed end time and free terminal state

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{u}) = V(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt \quad (5.133a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0. \quad (5.133b)$$

The essential question in this regard is how the gradient, i.e. the direction of steepest descent for the functional $J(\mathbf{u})$ can be computed.

Theorem 5.7 (Gâteaux derivative of $J(\mathbf{u})$) Suppose that the functional $J(\mathbf{u})$ is Gâteaux differentiable at point $\mathbf{u} \in \mathcal{C}_m^0[t_0, t_f]$ and let $\mathbf{x}, \boldsymbol{\lambda} \in \mathcal{C}_n^1[t_0, t_f]$ be the solution of the canonical equations

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (5.134a)$$

$$\dot{\boldsymbol{\lambda}} = -H_{\mathbf{x}}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}, t), \quad \boldsymbol{\lambda}(t_f) = V_{\mathbf{x}}(\mathbf{x}(t_f)). \quad (5.134b)$$

Then, the Gâteaux derivative in direction of $\mathbf{v} \in \mathcal{C}_m^0[t_0, t_f]$ is defined by

$$\delta J(\mathbf{u}; \mathbf{v}) = \int_{t_0}^{t_f} H_{\mathbf{u}}(\mathbf{x}(t), \mathbf{u}(t), \boldsymbol{\lambda}(t), t)^T \mathbf{v}(t) dt. \quad (5.135)$$

Exercise 5.7 (optional) Prove Theorem 5.7 by considering the control trajectory $\mathbf{w}^\varepsilon(t) = \mathbf{u}(t) + \varepsilon \mathbf{v}(t)$, $t \in [t_0, t_f]$ with the associated state trajectory $\mathbf{y}^\varepsilon(t)$ and $\mathbf{x}(t) = \mathbf{y}^0(t)$ for $\varepsilon = 0$. Derive $\delta J(\mathbf{u}, \mathbf{v}) = \left. \frac{dJ(\mathbf{w}^\varepsilon)}{d\varepsilon} \right|_{\varepsilon=0}$ by augmenting $J(\mathbf{w}^\varepsilon)$ with the integrand $\boldsymbol{\lambda}(t)^T (\mathbf{f}(\mathbf{y}^\varepsilon(t), \mathbf{w}^\varepsilon(t), t) - \dot{\mathbf{y}}^\varepsilon(t)) = 0$.

Initialization:

$j \leftarrow 0$	Iteration counter
ε	Convergence tolerance
$\mathbf{u}^0(t)$	Initial control trajectory
$\mathbf{x}^0(t)$	Integration of (5.134a) in forward time
repeat	
$\boldsymbol{\lambda}^j(t)$	Integration of (5.134b) in backward time
$\mathbf{g}^j(t) \leftarrow (5.136)$	Computation of gradient
$\alpha^j \leftarrow \arg \min_{\alpha > 0} J(\mathbf{u}^j - \alpha \mathbf{g}^j)$	Line search (often approximate solution)
$\mathbf{u}^{j+1}(t) = \mathbf{u}^j(t) - \alpha^j \mathbf{g}^j(t)$	New control trajectory with step size α^j
$\mathbf{x}^{j+1}(t)$	Integration of (5.134a) in forward time
$j \leftarrow j + 1$	
until $ J(\mathbf{u}^j) - J(\mathbf{u}^{j-1}) \leq \varepsilon$	Convergence criterion (or similar one)

Table 5.1: Algorithm of the gradient method.

If we define the gradient as the partial derivative of the Hamiltonian w.r.t. $\mathbf{u}$

$$\mathbf{g}(t) = H_{\mathbf{u}}(\mathbf{x}(t), \mathbf{u}(t), \boldsymbol{\lambda}(t), t), \quad t \in [t_0, t_f], \quad (5.136)$$

then the Gâteaux derivative (5.135) becomes $\delta J(\mathbf{u}; -\mathbf{g}) < 0$ and $-\mathbf{g}$ is the steepest descent direction at the point $\mathbf{u} \in \mathcal{C}_m^0[t_0, t_f]$. Given the current control trajectory $\mathbf{u}^j(t)$, $t \in [t_0, t_f]$ in iteration j , the gradient method exploits the decoupling of the boundary conditions for $\mathbf{x}(t_0)$ and $\boldsymbol{\lambda}(t_f)$ and sequentially solves the system and adjoint dynamics (5.134) in forward and backward time to compute $\mathbf{x}^j(t)$ and $\boldsymbol{\lambda}^j(t)$. Afterwards, the negative gradient $-\mathbf{g}^j(t)$ is used to achieve a decrease in the cost functional (5.133a). The basic algorithm of the gradient method is summarized in Table 5.1.

The underlying line search problem is usually solved approximately, in its simplest form based on the Armijo condition (see Section 3.3.1). Some variants of the gradient method use a constant step size α leading to the control update $\mathbf{u}^{j+1}(t) = \mathbf{u}^j(t) - \alpha \mathbf{g}^j(t)$ in all iterations. In this case, however, α must be chosen sufficiently small which can lead to a slow convergence speed. A good trade-off between accuracy and computation effort can be achieved with an explicit or adaptive line search strategy. In particular, an explicit step size can be computed from minimizing the difference between two control updates $\mathbf{u}^{j+1}(t)$ and $\mathbf{u}^j(t)$. An alternative is to approximately solve the line search problem on an interval $[\alpha^-, \alpha^+]$ that is dynamically adapted over the iterations. More details on these strategies can be found in [6].

Some important properties of the gradient method are listed in the following. More information can e.g. be found in [14, 4, 12].

- Input constraints $\mathbf{u} \in U$ can be considered via *projection* onto the admissible set U .
- An advantage of the gradient method in particular in case of general nonlinear problems is that the control $\mathbf{u}$ is not explicitly computed from $H_{\mathbf{u}} = \mathbf{0}$ or $\min_{\mathbf{u} \in U} H$ (see Sections 5.3.4 and 5.5.2).

- The gradient method is numerically more robust than shooting due to the fact that the adjoint system (5.134b) is not integrated in the unstable forward direction. A further advantage over shooting and also discretization methods is that no initial guess of the adjoint trajectory is required.
- The gradient method typically leads to a fast cost decrease in the first iterations but tends to converge slowly in the vicinity of the optimal solution.
- An alternative are *conjugate gradient methods* or *second-order gradient methods* with better convergence speed towards the optimal solution.
- The gradient method can be extended to handle partial or general terminal conditions and a free end time. This, however, impacts the robustness and convergence properties.

5.7 Numerical solution by discretization (direct methods)

Direct methods parametrize the control trajectory $\mathbf{u}(t)$, $t \in [t_0, t_f]$ in order to reduce the optimal control problem to a parameter optimization problem that can be solved with the methods from unconstrained and constrained optimization (see Chapter 3 and 4). The following table lists some advantages of direct/indirect methods.

<u>Direct methods:</u>	<u>Indirect methods:</u>
• Derivation of canonical equations is not necessary • Handling of state constraints is easier • Usually larger convergence domain • Powerful algorithms available	• Give insight into the structure of the optimal solution • Highly accurate solution (important e.g. in aerospace applications) • Adjoint variables can be used for sensitivity analysis and feedback control ("<i>neighbouring extremal control</i>")

Direct methods rely on *partial* or *full discretization*, which is detailed in the following. To this end, we consider a general constrained optimal control problem of the form

$$\min_{\mathbf{u}(\cdot)} J(\mathbf{u}) = V(\mathbf{x}(t_f)) + \int_{t_0}^{t_f} l(\mathbf{x}(t), \mathbf{u}(t), t) dt \quad (5.137a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (5.137b)$$

$$\mathbf{g}(\mathbf{x}(t_f)) = \mathbf{0} \quad (5.137c)$$

$$\mathbf{h}(\mathbf{x}(t), \mathbf{u}(t), t) \leq \mathbf{0} \quad \forall t \in [t_0, t_f]. \quad (5.137d)$$

The end time t_f is supposed to be fixed for simplicity. A free end time t_f can be considered by the time transformation (5.131) in order to transform the problem to a new time coordinate $\tau \in [0, 1]$.

5.7.1 Partial discretization

Direct solution methods discretize the time interval $[t_0, t_f]$

$$t_0 = t^0 < t^1 < \dots < t^N = t_f \quad (5.138)$$

and parametrize the control $\mathbf{u}$ on the single N subintervals $[t^k, t^{k+1}]$. In the simplest case, $\mathbf{u}$ is chosen constant on each subinterval

$$\mathbf{u}(t) = \boldsymbol{\psi}(t, \hat{\mathbf{u}}) = \mathbf{u}^k, \quad t \in [t^k, t^{k+1}) \quad \text{with} \quad \hat{\mathbf{u}} = \begin{bmatrix} \mathbf{u}^0 \\ \vdots \\ \mathbf{u}^{N-1} \end{bmatrix} \in \mathbb{R}^{mN}, \quad (5.139)$$

which results in a piecewise constant control trajectory $\mathbf{u}(t) = \boldsymbol{\psi}(t, \hat{\mathbf{u}})$ in dependence of the mN parameters $\hat{\mathbf{u}}$. An alternative is, for instance, a piecewise linear parametrization for $\boldsymbol{\psi}(t, \hat{\mathbf{u}})$, see Figure 5.23.

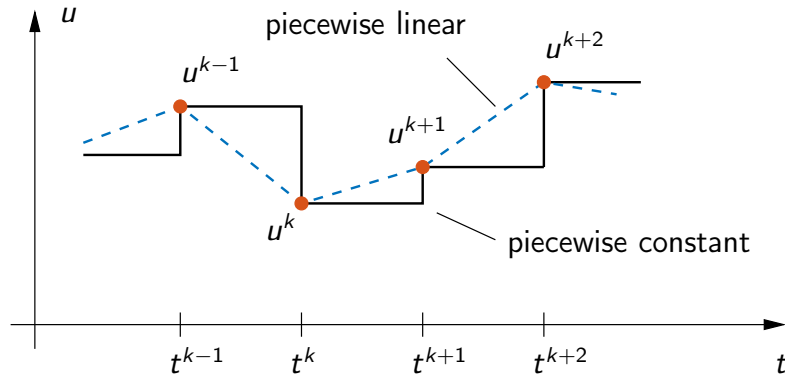


Figure 5.23: Piecewise constant and piecewise linear parametrization of the control ($m = 1$).

The parametrization of the control trajectory $\mathbf{u}(t) = \boldsymbol{\psi}(t, \hat{\mathbf{u}})$ transforms the optimal control problem (5.137) to a (finite-dimensional) parameter optimization problem

$$\min_{\hat{\mathbf{u}} \in \mathbb{R}^{mN}} \hat{J}(\hat{\mathbf{u}}) = V(\mathbf{x}(t_f)) + \sum_{k=0}^{N-1} \int_{t^k}^{t^{k+1}} l(\mathbf{x}(t), \boldsymbol{\psi}(t, \hat{\mathbf{u}}), t) dt \quad (5.140a)$$

$$\text{s.t. } \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\psi}(t, \hat{\mathbf{u}}), t), \quad t \in (t^k, t^{k+1}), \quad k = 0, 1, \dots, N-1 \quad (5.140b)$$

$$\mathbf{x}(t^0) = \mathbf{x}_0, \quad \mathbf{x}(t^k) = \lim_{t \nearrow t^k} \mathbf{x}(t), \quad k = 0, 1, \dots, N-1 \quad (5.140c)$$

$$\mathbf{g}(\mathbf{x}(t^N)) = \mathbf{0} \quad (5.140d)$$

$$\mathbf{h}(\mathbf{x}(t), \boldsymbol{\psi}(t, \hat{\mathbf{u}}), t) \leq \mathbf{0} \quad \forall t \in [t^k, t^{k+1}], \quad k = 0, 1, \dots, N-1. \quad (5.140e)$$

The solution of the nonlinear optimization problem (5.140) w.r.t. the parameters $\hat{\mathbf{u}}$ requires to numerically solve the dynamics (5.140b) on the single subintervals. The conditions (5.140c) ensure the continuity of the state $\mathbf{x}(t)$ at the interval boundary points t^k .

A problem of the formulation (5.140) is that formally the inequality constraints (5.140e) have to be satisfied on each interval $[t^k, t^{k+1}]$ and not only at the boundary points t^k . A practical approach in this regard is to replace (5.140e) by pointwise inequality constraints

$$\mathbf{h}(\mathbf{x}(t^{k,j}), \boldsymbol{\psi}(t^{k,j}, \hat{\mathbf{u}}), t^{k,j}) \leq \mathbf{0}$$

at a number of inner points $t^{k,j} \in [t^k, t^{k+1}]$. The problem (5.140) can then be solved with standard methods from nonlinear optimization, e.g. sequential quadratic programming (SQP).

Some characteristic properties of direct methods based on partial discretization are:

- If the conditions (5.140c) are implemented as initial conditions and not as equality constraints within the optimization method, the state trajectory $\mathbf{x}(t)$, $t \in [t_0, t_f]$ satisfies the differential equation (5.140b) (to the accuracy of the numerical integration scheme) during the single iterations of the optimization method.
- The computation of the gradient of the cost and constraints in (5.140), e.g. via sensitivity differential equations, is computationally complex.
- The partial discretization approach is a *direct sequential method* due to the underlying integration of the system equations (5.140b) during the optimization iterations.

5.7.2 Full discretization

The full discretization approach discretizes the control trajectory as well as the differential equations (5.137b) at the discretization points (5.138). In particular, if we use the trapezoidal scheme (5.126), see Section 5.6.1, and use a piecewise constant discretization for the control $\mathbf{u}$, we get the equations

$$\frac{\mathbf{x}^{k+1} - \mathbf{x}^k}{t^{k+1} - t^k} = \frac{1}{2} \left[\mathbf{f}(\mathbf{x}^k, \mathbf{u}^k, t^k) + \mathbf{f}(\mathbf{x}^{k+1}, \mathbf{u}^{k+1}, t^{k+1}) \right], \quad k = 0, 1, \dots, N-1.$$

The full discretization of the optimal control problem (5.137) thus yields the nonlinear optimization problem

$$\min_{\hat{\mathbf{x}}, \hat{\mathbf{u}}} \quad \hat{J}(\hat{\mathbf{x}}, \hat{\mathbf{u}}) = V(\mathbf{x}^N) + \sum_{k=0}^{N-1} \frac{t^{k+1} - t^k}{2} \left[l(\mathbf{x}^k, \mathbf{u}^k, t^k) + l(\mathbf{x}^{k+1}, \mathbf{u}^{k+1}, t^{k+1}) \right] \quad (5.141a)$$

$$\text{s.t.} \quad \frac{\mathbf{x}^{k+1} - \mathbf{x}^k}{t^{k+1} - t^k} = \frac{1}{2} \left[\mathbf{f}(\mathbf{x}^k, \mathbf{u}^k, t^k) + \mathbf{f}(\mathbf{x}^{k+1}, \mathbf{u}^{k+1}, t^{k+1}) \right], \quad k = 0, 1, \dots, N-1 \quad (5.141b)$$

$$\mathbf{x}^0 = \mathbf{x}_0, \quad \mathbf{g}(\mathbf{x}^N) = \mathbf{0} \quad (5.141c)$$

$$\mathbf{h}(\mathbf{x}^k, \mathbf{u}^k, t^k) \leq \mathbf{0} \quad k = 0, 1, \dots, N. \quad (5.141d)$$

The discretized system equations (5.141b) as well as the initial and terminal conditions (5.141c) are treated as equality constraints. The optimization variables consist of the discretized input and state variables

$$\hat{\mathbf{u}} = \begin{bmatrix} \mathbf{u}^0 \\ \vdots \\ \mathbf{u}^N \end{bmatrix} \in \mathbb{R}^{m(N+1)}, \quad \hat{\mathbf{x}} = \begin{bmatrix} \mathbf{x}^0 \\ \vdots \\ \mathbf{x}^N \end{bmatrix} \in \mathbb{R}^{n(N+1)} \quad (5.142)$$

of dimension $(m+n)(N+1)$. In contrast, the partial discretization of the control led to the optimization vector $\hat{\mathbf{u}}$ of dimension mN . On the other hand, the differential equations (5.137b) do not need to be integrated separately as they are handled as (discretized) equality constraints.

Some properties of the full discretization approach are:

- The optimization variables $\hat{\mathbf{x}}$ satisfy the discretized system equations (5.141b) only after convergence to the optimal solution $\hat{\mathbf{x}}^*$.
- The computations of the gradients of the cost function and constraints in (5.141) is simpler than in case of partial discretization. Moreover, the matrices are sparsely occupied and block-structured, which can be effectively exploited by sparse numerics.
- The inequality constraints (5.141d) are already given in discretized form in contrast to (5.140e) in the partial discretization case.
- The accuracy of the fully discretized optimal solution is directly related to the number of discretization points $N + 1$. The choice of N not only influences the resolution of the control trajectory as in case of partial discretization but also how accurately $\hat{\mathbf{x}}$ approximates the actual solution of the differential equations (5.137b).
- The full discretization approach is a *direct simultaneous method* because the solution of the dynamics and the actual optimization is done at the same time.

5.8 Software overview

Besides the option to discretize an optimal control problem manually and to solve it as nonlinear optimization problem, there exist various software packages to solve dynamic optimization problems that mainly rely on direct partial or full discretization and often link to external solvers (e.g. IPOPT or qpOASES). A selection of open source and commercial tools are listed in the following.

- acados (open source license)
<https://docs.acados.org>
- CasADi (open source license)
<https://web.casadi.org>
- GRAMPC (open source license)
<https://github.com/grampc/grampc>
- HQP (open source license)
<http://hqp.sourceforge.net>
- ICLOCS2 (open source, but external solver required)
<http://www.ee.ic.ac.uk/iclocs>
- TOMLAB/PROPT (commercial)
<http://tomdyn.com>

5.9 References

- [1] U.M. Ascher, R.M.M. Mattheij, and R.D. Russell. *Numerical solution of boundary value problems of ordinary differential equations*. Prentice Hall, 1988.
- [2] M. Athans and P.L. Falb. *Optimal Control*. New York: McGraw-Hill, 1966.

- [3] B. van Brunt. *The Calculus of Variations*. New York: Springer, 2004.
- [4] A.E. Bryson and Y.-C. Ho. *Applied Optimal Control*. New York: John Wiley & Sons, 1975.
- [5] B. Chachuat. *Nonlinear and Dynamic Optimization: From Theory to Practice*. <https://infoscience.epfl.ch/record/111939>, 2007.
- [6] T. Englert, A. Völz, F. Mesmer, S. Rhein, and K. Graichen. “A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach (GRAMPC)”. In: *Optimization and Engineering* 20.3 (2019), pp. 769–809. DOI: 10.1007/s11081-018-9417-2.
- [7] O. Föllinger. *Optimale Steuerung und Regelung*. München: Oldenbourg, 1994.
- [8] K. Graichen. *Feedforward Control Design for Finite-Time Transition Problems of Nonlinear Systems with Input and Output Constraints*. Dissertation, Universität Stuttgart. <http://dx.doi.org/10.18419/opus-4093>. Aachen: Shaker-Verlag, 2006.
- [9] R.F. Hartl, S.P. Sethi, and R.G. Vickson. “A survey of the maximum principles for optimal control problems with state constraints”. In: *SIAM Review* 37 (1995), pp. 181–218.
- [10] M.I. Kamien and N.L. Schwartz. *Dynamic Optimization: The Calculus of Variations and Optimal Control in Economics and Management*. 2nd edition. Amsterdam: North-Holland, Amsterdam, 1991.
- [11] H. Kwakernaak and R. Sivan. *Linear Optimal Control Systems*. New York: Wiley-Interscience, 1972.
- [12] M. Papageorgiou, M. Leibold, and M. Buss. *Optimierung*. Berlin: Springer, 2012.
- [13] H.J. Pesch. “Optimal and nearly optimal guidance by multiple shooting”. In: *Proc. Intern. Symp. “Mécanique Spatiale”*. Ed. by Centre d’Études Spatiales (CNES). Toulouse, 6.-10.11.1989: Cepadues Editions, 1990, pp. 761–771.
- [14] E. Polak. “An historical survey of computational methods in optimal control”. In: *SIAM Review* 15 (1973), pp. 553–584.

6 Model predictive control (MPC)

Model predictive control relies on the solution of a dynamic optimization problem on a moving horizon and is particularly suitable for controlling nonlinear multiple input systems subject to constraints. The chapter gives an introduction to MPC and different MPC formulations. Since MPC typically requires a high computational effort, we also focus on aspects of a real-time implementation. Further insights and detailed discussions on MPC can be found in the literature, e.g. [20, 2, 8, 5, 15].

6.1 Basic concept behind MPC

This section introduces the working principle of MPC for controlling nonlinear systems of the form

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(0) = \mathbf{x}_0 \quad (6.1)$$

with the state $\mathbf{x} \in \mathbb{R}^n$ and control input $\mathbf{u} \in \mathbb{R}^m$ that are subject to the constraints

$$\mathbf{u}(t) \in U, \quad \mathbf{x}(t) \in X, \quad t \geq 0. \quad (6.2)$$

Without loss of generality, we consider the control task of stabilizing the origin

$$\mathbf{0} = \mathbf{f}(\mathbf{0}, \mathbf{0}). \quad (6.3)$$

Note that any other equilibrium $(\mathbf{x}_e, \mathbf{u}_e)$ with $\mathbf{0} = \mathbf{f}(\mathbf{x}_e, \mathbf{u}_e)$ can be mapped to the origin by considering the simple state/input transformation $\tilde{\mathbf{x}} = \mathbf{x} - \mathbf{x}_e$ and $\tilde{\mathbf{u}} = \mathbf{u} - \mathbf{u}_e$. Although we focus on setpoint stabilization, MPC can be used for various control tasks such as path following MPC, economic MPC, stochastic MPC, robust MPC or even as moving horizon estimator (MHE) for state observation. We refer to the references above for more details.

6.1.1 Dynamic optimization on moving horizon

The goal of MPC is to act as a control law for a given system based on the solution of an optimal control problem

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = V(\bar{\mathbf{x}}(T)) + \int_0^T l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.4a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.4b)$$

$$\bar{\mathbf{x}}(\tau) \in X, \quad \bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, T] \quad (6.4c)$$

$$\bar{\mathbf{x}}(T) \in S \quad (6.4d)$$

with the current system state $\mathbf{x}_k$ at the time point t_k . The prediction horizon T defines the time interval $[0, T]$ on which the system trajectories are predicted. The bar indicates

MPC-internal variables with the prediction time coordinate $\tau \in [0, T]$ in contrast to the real system (6.1) with time $t \geq 0$. We also consider the following standing assumptions:

Standing assumptions:

- A.1 The system function $\mathbf{f}$ is continuously differentiable in its arguments and there exists a bounded solution of the system (6.1) for all $\mathbf{x}_0 \in X$ and all controls $\mathbf{u}(t) \in U$, $t \in [0, T]$.
- A.2 The constraint sets X and U with $\mathbf{0} \in X \subseteq \mathbb{R}^n$ and $\mathbf{0} \in U \subseteq \mathbb{R}^m$ are convex and U is additionally compact.
- A.3 The integral cost term $l(\mathbf{x}, \mathbf{u})$ and the (optional) terminal cost function $V(\mathbf{x})$ in the cost functional (6.4a) are continuously differentiable in their arguments. Moreover, they are positive definite with $l(\mathbf{0}, \mathbf{0}) = V(\mathbf{0}) = 0$ and satisfy the quadratic bounds

$$\begin{aligned} m_l(\|\mathbf{x}\|^2 + \|\mathbf{u}\|^2) &\leq l(\mathbf{x}, \mathbf{u}) \leq M_l(\|\mathbf{x}\|^2 + \|\mathbf{u}\|^2) \\ m_v\|\mathbf{x}\|^2 &\leq V(\mathbf{x}) \leq M_v\|\mathbf{x}\|^2 \end{aligned} \quad (6.5)$$

for some constants $m_l, M_l > 0$ and $m_v, M_v > 0$.

A common choice of $l(\mathbf{x}, \mathbf{u})$ and $V(\mathbf{x})$ are quadratic functions

$$V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}, \quad l(\mathbf{x}, \mathbf{u}) = \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} \quad (6.6)$$

with positive definite matrices $\mathbf{P}, \mathbf{Q}, \mathbf{R}$. Besides the input and state constraints (6.4c), some MPC formulations impose an additional terminal constraint (6.4d) with $\mathbf{0} \in S \subseteq \mathbb{R}^n$ or $S_\beta = \mathbf{0}$ (Section 6.2).

Under the assumption that the optimal control problem (6.4) is feasible¹, the optimal solution and the optimal cost functional are denoted by

$$\bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k), \quad \bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \quad \tau \in [0, T] \quad (6.7)$$

and

$$J_T^*(\mathbf{x}_k) := J_T(\mathbf{x}_k, \bar{\mathbf{u}}_k^*).$$

It is typically assumed in the literature that the optimal solution of (6.4) is determined in each time step t_k . In the current sampling interval $t \in [t_k, t_k + \Delta t)$ with the *sampling time* Δt , the first part of the optimal control trajectory $\bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)$ is used as input

$$\mathbf{u}(t_k + \tau) = \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k) =: \kappa_T(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k); \mathbf{x}_k), \quad \tau \in [0, \Delta t) \quad (6.8)$$

for the system (6.1), which can be interpreted as an *optimal control law* with $\kappa_T(\mathbf{0}; \mathbf{x}_k) = \mathbf{0}$. In the next sampling step $t_{k+1} = t_k + \Delta t$, the problem (6.4) is re-solved with the new initial state $\mathbf{x}_{k+1}$. Figure 6.1 illustrates the moving horizon concept of MPC.

If no model unaccuracies or disturbances occur, the system follows the predicted state trajectory to the next state $\mathbf{x}_{k+1} = \mathbf{x}_T^*(\Delta t; \mathbf{x}_k)$. The closed-loop trajectory in the nominal case therefore reads

$$\begin{aligned} \mathbf{x}(t) &= \mathbf{x}(t_k + \tau) = \bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \\ \mathbf{u}(t) &= \mathbf{u}(t_k + \tau) = \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k), \quad \tau \in [0, \Delta t), \quad k \in \mathbb{N}_0^+. \end{aligned} \quad (6.9)$$

¹ Existence results for solutions of certain classes of optimal control problems can e.g. be found in [18, 1].

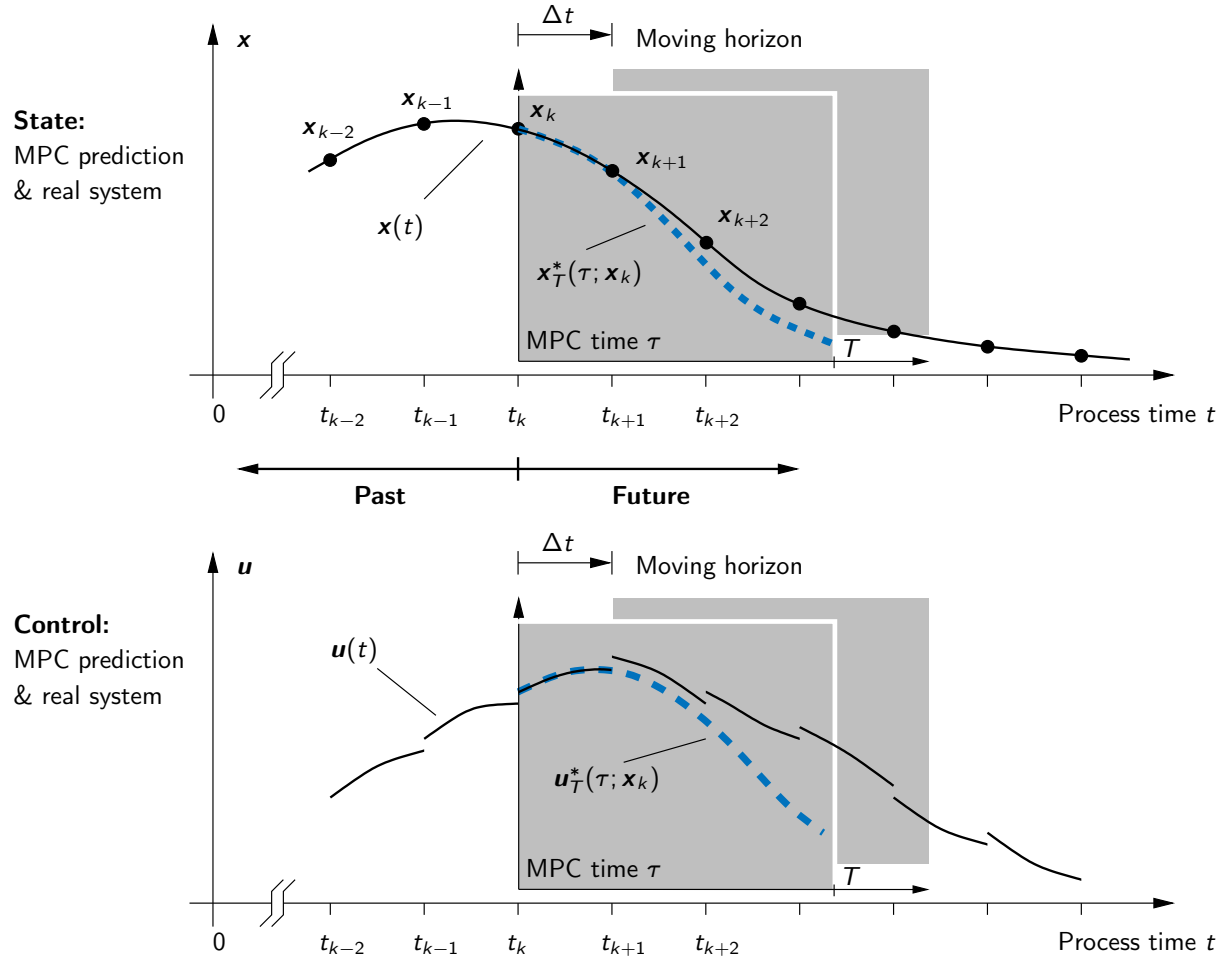


Figure 6.1: Moving horizon principle of MPC.

6.1.2 Continuous-time vs. discrete-time MPC

For highly dynamical systems with short sampling times, the piecewise continuous control input (6.8) is of rather theoretical nature. Instead, a constant input is often applied to the system on the current sampling interval, e.g. by setting $\mathbf{u}(t_k + \tau) = \bar{\mathbf{u}}_T^*(0; \mathbf{x}_k) = \text{const.}$ for $\tau \in [0, \Delta t)$, or by directly using a piecewise constant parametrization of the control trajectory for the solution of (6.4), see Figure 6.2 as well as Section 5.7.1.

If the control is kept constant over a sampling interval, i.e. $\mathbf{u}(t) = \mathbf{u}_k$ for $t \in [t_k, t_k + \Delta t)$, we formally obtain the state trajectory $\mathbf{x}(t) = \mathbf{x}(t; \mathbf{x}_k, \mathbf{u}_k)$ and, in particular, the state $\mathbf{x}_{k+1} = \mathbf{x}(t_{k+1}; \mathbf{x}_k, \mathbf{u}_k)$ at the next sampling instant $t_{k+1} = t_k + \Delta t$. Thus, the continuous-time system (6.1) can formally be written as a nonlinear discrete-time system

$$\mathbf{x}_{k+1} = \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k). \quad (6.10)$$

In fact, the majority of the MPC literature considers a discrete-time version of (6.4)

$$\min_{\bar{\mathbf{u}}} J_N(\mathbf{x}_k, \bar{\mathbf{u}}) = V(\bar{\mathbf{x}}_N) + \sum_{j=0}^{N-1} l(\bar{\mathbf{x}}_j, \bar{\mathbf{u}}_j) \quad (6.11a)$$

$$\text{s.t. } \bar{\mathbf{x}}_{j+1} = \mathbf{f}_d(\bar{\mathbf{x}}_j, \bar{\mathbf{u}}_j), \quad \bar{\mathbf{x}}_0 = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.11b)$$

$$\bar{\mathbf{x}}_j \in X, \quad \bar{\mathbf{u}}_j \in U, \quad j = 0, 1, \dots, N-1 \quad (6.11c)$$

$$\bar{\mathbf{x}}_N \in S \quad (6.11d)$$

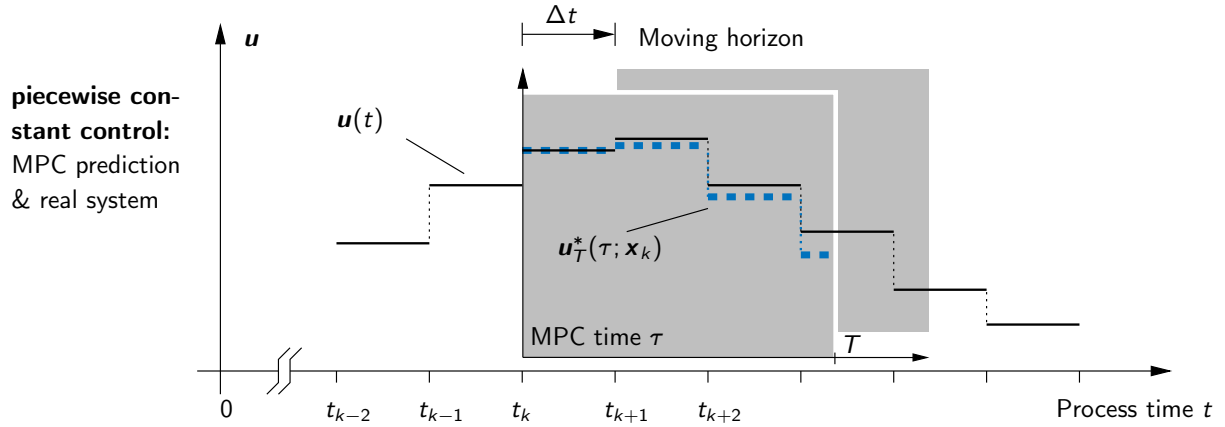


Figure 6.2: MPC with piecewise constant control.

with N prediction steps and the input parametrization $\hat{\mathbf{u}} = [\bar{\mathbf{u}}_0, \dots, \bar{\mathbf{u}}_{N-1}]^T$. The discrete-time approach has the advantage that it is mathematically easier to analyze and that weaker assumptions are required compared to the continuous-time case.

On the other hand, the discrete-time formulation (6.10) for a nonlinear continuous-time system (6.1) can in general only be derived approximately. In addition, the system theoretic properties of the original system (6.1) are usually better observed in continuous-time, which is the reason that we focus on the continuous-time MPC formulation from Section 6.1.1.

6.2 MPC formulations

Different MPC formulations exist as variants of the general optimal control problem (6.4). These different formulations are the result of the historical development of MPC but are also motivated by practical considerations such as computational efficiency for real-time implementations. The formulations therefore differ in their importance in theory and practice. For instance, terminal constraints (6.4d) are useful to prove stability in closed-loop. On the other hand, it is difficult to guarantee the existence of a solution for terminal-constrained problems and the numerical effort for its solution is significantly higher than in case of a free terminal state. This section gives an overview of different MPC formulations and their stability properties in closed-loop.

6.2.1 Infinite horizon

If we consider an infinitely long prediction horizon $T \rightarrow \infty$, the optimal control problem (6.4) can be written without terminal conditions (6.4d) and without terminal cost in (6.4a) because the optimal solution must satisfy $\lim_{t \rightarrow \infty} \bar{\mathbf{x}}_\infty^*(t; \mathbf{x}_k) = \mathbf{0}$ for the cost to be finite. This leads to the MPC formulation

$$\min_{\bar{\mathbf{u}}(\cdot)} J_\infty(\mathbf{x}_k, \bar{\mathbf{u}}) = \int_0^\infty l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.12a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.12b)$$

$$\bar{\mathbf{x}}(\tau) \in X, \quad \bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, \infty) \quad (6.12c)$$

and the following stability result:

Theorem 6.1 (MPC with infinite horizon) Suppose that the set $\Gamma \subseteq X \subseteq \mathbb{R}^n$ of initial states $\mathbf{x}_k$ for which (6.12) has a bounded solution with $J_\infty^*(\mathbf{x}_k) < \infty$ is non-empty. Then, the cost satisfies

$$J_\infty^*(\bar{\mathbf{x}}_\infty^*(\Delta t; \mathbf{x}_k)) = J_\infty^*(\mathbf{x}_k) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_\infty^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_\infty^*(\tau; \mathbf{x}_k)) d\tau \quad \forall \mathbf{x}_k \in \Gamma \quad (6.13)$$

and the origin of the system (6.1) in closed-loop is asymptotically stable in the sense of $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$ with Γ as the domain of attraction.

Proof: The equation (6.13) directly follows from Bellman's principle of optimality. If (6.12) is feasible for $t = t_0$ ($\mathbf{x}_0 \in \Gamma$), then the infinite horizon guarantees that (6.12) is also feasible for the next point $\mathbf{x}_{k+1} = \bar{\mathbf{x}}_\infty^*(\Delta t; \mathbf{x}_k)$, i.e. $\mathbf{x}_{k+1} \in \Gamma$. By the definition of the set Γ , the closed-loop trajectory $\mathbf{x}(t)$ in (6.9) is bounded. This also holds for the time derivative $\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t))$ with $\mathbf{u}(t) \in U$ since U is compact and $\mathbf{f}$ is continuous in $\mathbf{x}$ and $\mathbf{u}$. Thus, $\mathbf{x}(t)$ is *uniformly continuous*. Moreover, with inductive reasoning the cost satisfies

$$J_\infty^*(\mathbf{x}(t)) = J_\infty^*(\mathbf{x}_0) - \int_0^t l(\mathbf{x}(\tau), \mathbf{u}(\tau)) d\tau \stackrel{(6.5)}{\leq} J_\infty^*(\mathbf{x}_0) - \int_0^t m_l \|\mathbf{x}(\tau)\|^2 d\tau \quad (6.14)$$

along the closed-loop trajectory $\mathbf{x}(t)$. Since $J_\infty^*(\mathbf{x}_0)$, $\mathbf{x}_0 \in \Gamma$ is bounded and $J_\infty^*(\mathbf{x}(t)) \geq 0$, the integral in (6.14) is bounded as well. We now require *Barbalat's lemma* to proceed:

Lemma 6.1 (Barbalat's Lemma) Let $\phi : \mathbb{R} \rightarrow \mathbb{R}$ be a uniformly continuous function on the interval $[0, \infty)$. If the integral

$$\lim_{t \rightarrow \infty} \int_0^t \phi(\tau) d\tau$$

exists and is finite, then $\lim_{t \rightarrow \infty} \phi(t) = 0$.

Obviously, the assumptions of Lemma 6.1 are satisfied with $\phi(\tau) = \|\mathbf{x}(\tau)\|^2$, which implies that $\|\mathbf{x}(t)\| \rightarrow 0$ for $t \rightarrow \infty$. ■

Note that $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$ does not automatically imply asymptotic stability in the sense of *Lyapunov* (see e.g. the course "Nonlinear Control Systems"). This can be proved if in addition continuity of J_∞^* is assumed or shown.

Moreover, if J_∞^* is continuously differentiable on Γ and if the levels sets of J_∞^* are compact, then for $\Delta t \rightarrow 0$ the cost satisfies

$$\frac{d}{dt} J_\infty^*(\mathbf{x}(t)) = \frac{\partial J_\infty^*(\mathbf{x}(t))}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) = -l(\mathbf{x}(t), \mathbf{u}(t)) \stackrel{(6.5)}{\leq} -m_l \|\mathbf{x}(t)\|^2 \leq 0 \quad (6.15)$$

along the closed-loop trajectory (6.9). This shows that each compact subset

$$\Gamma_\alpha = \{\mathbf{x} \in \Gamma : J_\infty^*(\mathbf{x}) \leq \alpha\} \quad (6.16)$$

of the domain of attraction Γ is *positive invariant*. In fact, this property holds not only for the infinitesimal case ($\Delta t \rightarrow 0$) but also for the incremental one ($\Delta t > 0$). The optimal cost $J_\infty^*(\mathbf{x})$ then represents a *Lyapunov function*, which implies that the origin is asymptotically stable.

6.2.2 Finite horizon and terminal condition

The numerical solution of the problem (6.12) on an infinite horizon can only be solved approximately and requires a high computational effort. Over the history of MPC, several formulations have emerged that use a finite horizon $T < \infty$ in connection with additional conditions to maintain stability in closed-loop. The first variant concerns an MPC formulation with terminal condition:

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = \int_0^T l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.17a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.17b)$$

$$\bar{\mathbf{x}}(\tau) \in X, \quad \bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, T] \quad (6.17c)$$

$$\bar{\mathbf{x}}(T) = \mathbf{0}. \quad (6.17d)$$

The formulation (6.17) requires that the predicted state trajectory $\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k)$ exactly reaches the origin (i.e. the desired setpoint) at the end of the prediction horizon T . We have the following stability results for this MPC formulation with terminal condition.

Theorem 6.2 (MPC with terminal condition) *Suppose that the set $\Gamma \subseteq X \subseteq \mathbb{R}^n$ of initial states $\mathbf{x}_k$ for which (6.17) has a bounded solution with $J_T^*(\mathbf{x}_k) < \infty$ is non-empty. Then, the cost satisfies*

$$J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq J_T^*(\mathbf{x}_k) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \quad \forall \mathbf{x}_k \in \Gamma \quad (6.18)$$

and the origin of the system (6.1) in closed-loop is asymptotically stable in the sense of $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$ with Γ as the domain of attraction.

Proof: Since $\bar{\mathbf{x}}_T^*(T; \mathbf{x}_k) = \mathbf{0}$ holds for all $\mathbf{x}_k \in \Gamma$, the trajectory

$$\hat{\mathbf{x}}(\tau) = \begin{cases} \bar{\mathbf{x}}_T^*(\tau + \Delta t; \mathbf{x}_k) & \tau \in [0, T - \Delta t] \\ \mathbf{0} & \tau \in [T - \Delta t, T] \end{cases}, \quad \hat{\mathbf{u}}(\tau) = \begin{cases} \bar{\mathbf{u}}_T^*(\tau + \Delta t; \mathbf{x}_k) & \tau \in [0, T - \Delta t] \\ \mathbf{0} & \tau \in [T - \Delta t, T] \end{cases}$$

is feasible for the problem (6.17) in MPC step $k + 1$, i.e. for $\mathbf{x}_{k+1} = \bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)$. The trajectories $\hat{\mathbf{x}}(\tau)$ and $\hat{\mathbf{u}}(\tau)$ can be used to compute an upper bound on the cost in step $k + 1$

$$\begin{aligned} J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) &\leq \int_0^T l(\hat{\mathbf{x}}(\tau), \hat{\mathbf{u}}(\tau)) d\tau \\ &= \int_{\Delta t}^T l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau + \overbrace{\int_0^{\Delta t} l(\mathbf{0}, \mathbf{0}) d\tau}^{=0} \\ &= J_T^*(\mathbf{x}_k) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau, \end{aligned}$$

which corresponds to (6.18). The proof of stability ($\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$) then follows the lines of the proof of Theorem 6.1. ■

The practical implementation of the MPC formulation with terminal conditions (6.17d) has several deficiencies. On the one hand, the set Γ on which the problem (6.17) has a (bounded) solution can be small if the prediction horizon T is chosen too short. The problem might even be unsolvable in case of disturbances. On the other hand, the (numerically) exact satisfaction of a terminal condition requires a high numerical effort.

6.2.3 Finite horizon and terminal set

One option to relax the terminal condition $\bar{\mathbf{x}}(T) = \mathbf{0}$ in (6.17) is to replace it with a *terminal set* $S_\beta \subseteq X$ that contains the origin ($\mathbf{0} \in S_\beta$). This directly leads to the MPC formulation with terminal set

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = V(\bar{\mathbf{x}}(T)) + \int_0^T l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.19a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.19b)$$

$$\bar{\mathbf{x}}(\tau) \in X, \quad \bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, T] \quad (6.19c)$$

$$\bar{\mathbf{x}}(T) \in S_\beta. \quad (6.19d)$$

However, we require an additional design condition for the terminal cost function in (6.19a) to compensate for the relaxation of the terminal condition in the last formulation (6.17).

Theorem 6.3 (MPC with terminal set) Suppose that the set $\Gamma \subseteq X \subseteq \mathbb{R}^n$ of initial states $\mathbf{x}_k$ for which (6.19) has a bounded solution with $J_T^*(\mathbf{x}_k) < \infty$ is non-empty. Moreover, there exists a compact non-empty set $S_\beta = \{\mathbf{x} \in \mathbb{R}^n : V(\mathbf{x}) \leq \beta\} \subseteq \Gamma$ with $\beta > 0$ and a (local) control law $\mathbf{q}(\mathbf{x}) \in U$ for all $\mathbf{x} \in S_\beta$ such that

$$\frac{\partial V}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}, \mathbf{q}(\mathbf{x})) + l(\mathbf{x}, \mathbf{q}(\mathbf{x})) \leq 0 \quad \forall \mathbf{x} \in S_\beta. \quad (6.20)$$

Then, the cost satisfies

$$J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq J_T^*(\mathbf{x}_k) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \quad \forall \mathbf{x}_k \in \Gamma \quad (6.21)$$

and the origin of the system (6.1) in closed-loop is asymptotically stable in the sense of $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$ with Γ as the domain of attraction.

Proof: A solution of (6.3) satisfies $\bar{\mathbf{x}}_T^*(T; \mathbf{x}_k) \in S_\beta$ for all $\mathbf{x}_k \in \Gamma$. To derive an estimate of the cost in MPC step $k+1$, we consider the auxiliary feasible trajectory

$$\hat{\mathbf{x}}(\tau) = \begin{cases} \bar{\mathbf{x}}_T^*(\tau + \Delta t; \mathbf{x}_k), & \tau \in [0, T - \Delta t] \\ \bar{\mathbf{x}}^q(\tau - T + \Delta t), & \tau \in [T - \Delta t, T] \end{cases}, \quad \hat{\mathbf{u}}(\tau) = \begin{cases} \bar{\mathbf{u}}_T^*(\tau + \Delta t; \mathbf{x}_k), & \tau \in [0, T - \Delta t] \\ \bar{\mathbf{u}}^q(\tau - T + \Delta t), & \tau \in [T - \Delta t, T] \end{cases},$$

where $\bar{\mathbf{x}}^q(\tau)$ with $\bar{\mathbf{x}}^q(0) = \bar{\mathbf{x}}_T^*(T; \mathbf{x}_k)$ is the trajectory that results from applying the local control law $\bar{\mathbf{u}}^q(\tau) = \mathbf{q}(\bar{\mathbf{x}}^q(\tau))$. Note that $\bar{\mathbf{x}}^q(\tau) \in S_\beta$ for all $\tau \geq 0$, i.e. S_β is positive invariant due to the definition of S_β and the inequality (6.20) that can be expressed in the form

$$\frac{d}{d\tau} V(\bar{\mathbf{x}}^q(\tau)) \leq -l(\bar{\mathbf{x}}^q(\tau), \bar{\mathbf{u}}^q(\tau)). \quad (6.22)$$

Obviously, the trajectory $\hat{\mathbf{x}}(\tau)$, $\hat{\mathbf{u}}(\tau)$, $\tau \in [0, T]$ is feasible for the problem (6.19) in step $k+1$, i.e. for $\mathbf{x}_{k+1} = \bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)$, which demonstrates the problem's solvability in step $k+1$. We

can now derive an upper bound on the cost

$$\begin{aligned}
 J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) &\leq \int_0^T l(\hat{\mathbf{x}}(\tau), \hat{\mathbf{u}}(\tau)) d\tau + V(\hat{\mathbf{x}}(T)) \\
 &= J_T^*(\mathbf{x}_k) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \\
 &\quad + \underbrace{V(\bar{\mathbf{x}}^q(\Delta t)) - V(\bar{\mathbf{x}}^q(0)) + \int_0^{\Delta t} l(\bar{\mathbf{x}}^q(\tau), \bar{\mathbf{u}}^q(\tau)) d\tau}_{\leq 0}. \quad (6.23)
 \end{aligned}$$

The integration of the inequality (6.22) shows that the bracketed term in (6.23) is non-positive, which proves (6.21). The proof of stability in the sense of $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$ corresponds to the proof of Theorem 6.1. ■

The additional requirement in Theorem 6.3 concerns the existence of a (local) control law $\mathbf{q}(\mathbf{x})$ such that the terminal cost $V(\mathbf{x})$ satisfies the inequality (6.20). In nonlinear control theory, (6.20) represents a (strengthened) *control Lyapunov inequality* which implies that the terminal cost $V(\mathbf{x})$ represents a *control Lyapunov function* (CLF) and $\mathbf{q}(\mathbf{x})$ a CLF control law.

Exercise 6.1 (CLF in the linear unconstrained case) Consider the linear unconstrained MPC formulation

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = \bar{\mathbf{x}}^T(T) \mathbf{P} \bar{\mathbf{x}}(T) + \int_0^T \bar{\mathbf{x}}^T(\tau) \mathbf{Q} \bar{\mathbf{x}}(\tau) + \bar{\mathbf{u}}^T(\tau) \mathbf{R} \bar{\mathbf{u}}(\tau) d\tau \quad (6.24a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{A} \bar{\mathbf{x}}(\tau) + \mathbf{B} \bar{\mathbf{u}}(\tau), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.24b)$$

with $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{u} \in \mathbb{R}^m$ and the symmetric positive definite matrices $\mathbf{Q} \in \mathbb{R}^{n \times n}$ and $\mathbf{R} \in \mathbb{R}^{m \times m}$. Suppose that the system (6.24b) is stabilizable and the matrix $\mathbf{P}$ is the solution of the algebraic Riccati equation

$$\mathbf{P} \mathbf{A} + \mathbf{A}^T \mathbf{P} - \mathbf{P} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} + \mathbf{Q} = \mathbf{0}. \quad (6.25)$$

Show that the quadratic terminal cost and the control law

$$\mathbf{q}(\mathbf{x}) = -\mathbf{K} \mathbf{x} \quad \text{with} \quad \mathbf{K} = \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} \quad (6.26)$$

together satisfy the CLF inequality (6.20) on $S_\beta = \mathbb{R}^n$, which implies that the terminal set S_β can be omitted.

Obviously, the CLF inequality (6.20) can be globally satisfied in the linear unconstrained MPC case (6.24) without having to account for the terminal set S_β .

Exercise 6.2 (CLF in the nonlinear unconstrained case) Consider the nonlinear MPC formulation

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = \bar{\mathbf{x}}^T(T) \mathbf{P} \bar{\mathbf{x}}(T) + \int_0^T \gamma \bar{\mathbf{x}}^T(\tau) \mathbf{Q} \bar{\mathbf{x}}(\tau) + \bar{\mathbf{u}}^T(\tau) \mathbf{R} \bar{\mathbf{u}}(\tau) d\tau \quad (6.27a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.27b)$$

$$\bar{\mathbf{x}}(T) \in S_\beta. \quad (6.27c)$$

with $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{u} \in \mathbb{R}^m$, the positive definite matrices $\mathbf{Q} \in \mathbb{R}^{n \times n}$ and $\mathbf{R} \in \mathbb{R}^{m \times m}$ and some constant $\gamma \in (0, 1)$. Suppose that the linearization of the system (6.27b) around the origin

is stabilizable and let $\mathbf{P}$ be the solution of the algebraic Riccati equation (6.25). Show that for each $\gamma \in (0, 1)$ there exists a terminal set $S_\beta \ni \mathbf{0}$ on which the CLF inequality (6.20) is satisfied with the local control law (6.26).

In the nonlinear case (6.27), the additional parameter $\gamma \in (0, 1)$ is required to weaken the state-dependent integral part compared to the terminal cost to guarantee that a non-empty terminal set S_β exists.

In Exercise 6.1 and 6.2, the terminal cost function $V(\mathbf{x})$ is computed with the Riccati equation (6.25). There exist further approaches for constructing $V(\mathbf{x})$ such that the CLF inequality (6.20) holds. An algorithm for computing $V(\mathbf{x})$ and the terminal set S_β can e.g. be found in [4].

6.2.4 Finite horizon and terminal cost

The substitution of the terminal condition $\bar{\mathbf{x}}(T) = \mathbf{0}$ by the terminal set $\bar{\mathbf{x}}(T) \in S_\beta$ in the previous section already relaxes the computational aspects for solving the MPC problem. For an actual implementation, it would be even better to also drop the terminal set constraint $\bar{\mathbf{x}}(T) \in S_\beta$. This leads to the following MPC formulation with terminal cost and free terminal state

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = V(\bar{\mathbf{x}}(T)) + \int_0^T l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.28a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.28b)$$

$$\bar{\mathbf{x}}(\tau) \in X, \quad \bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, T]. \quad (6.28c)$$

The absence of a terminal set S_β leads to the question under which conditions the end point of the optimal trajectory $\bar{\mathbf{x}}_T^*(T; \mathbf{x}_k)$ reaches S_β on its own such that the CLF inequality (6.20) holds. The proof of the following lemma [14] is an adaptation of the discrete-time version in [19] and is omitted here for simplicity.

Lemma 6.2 (Reachability of the terminal set) Suppose that the set $\Gamma \subseteq X \subseteq \mathbb{R}^n$ of initial states $\mathbf{x}_k$ for which (6.28) has a bounded solution with $J_T^*(\mathbf{x}_k) < \infty$ is non-empty. Moreover, there exists a compact non-empty set $S_\beta = \{\mathbf{x} \in \mathbb{R}^n : V(\mathbf{x}) \leq \beta\} \subseteq \Gamma$ with $\beta > 0$ and a (local) control law $\mathbf{q}(\mathbf{x}) \in U$ for all $\mathbf{x} \in S_\beta$ such that

$$\frac{\partial V}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}, \mathbf{q}(\mathbf{x})) + l(\mathbf{x}, \mathbf{q}(\mathbf{x})) \leq 0 \quad \forall \mathbf{x} \in S_\beta \quad (6.29)$$

and consider the compact set

$$\Gamma_\alpha = \{\mathbf{x} \in \Gamma : J_T^*(\mathbf{x}) \leq \alpha\}, \quad \alpha = \beta \left(1 + \frac{m_l}{M_V} T\right) \quad (6.30)$$

with $m_l, M_V > 0$ from (6.5). Then, $\bar{\mathbf{x}}_T^*(T; \mathbf{x}_k) \in S_\beta$ for all $\mathbf{x}_k \in \Gamma_\alpha$ and $S_\beta \subseteq \Gamma_\alpha$.

This shows the interesting fact that the terminal set is automatically reached for all $\mathbf{x}_k \in \Gamma_\alpha$. The following theorem now states the stability properties.

Theorem 6.4 (MPC with terminal cost) *Let the assumptions of Lemma 6.2 hold. Then, the cost satisfies*

$$J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq J_T^*(\mathbf{x}_k) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \quad \forall \mathbf{x}_k \in \Gamma_\alpha \quad (6.31)$$

and the origin of the system (6.1) in closed-loop is asymptotically stable in the sense of $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$ with Γ_α lying in the domain of attraction.

Proof: Lemma 6.2 states that $\bar{\mathbf{x}}_T^*(T; \mathbf{x}_k) \in S_\beta$ for all $\mathbf{x}_k \in \Gamma_\alpha$. Thus, the derivation of the inequality (6.31) as well as the feasibility of (6.28) in step $k + 1$ follows the proof of Theorem 6.3. Moreover, $\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k) \in \Gamma_\alpha$ in view of (6.31) and the definition of Γ_α , which ensures that (6.31) also holds in the next step $k + 1$. The proof of stability ($\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$) in closed-loop corresponds again to the proof of Theorem 6.1. ■

The definition of the domain of attraction in (6.30) shows that Γ_α can be enlarged by choosing a longer prediction horizon T , see Figure 6.3. This intuitive property is also observed in practice.

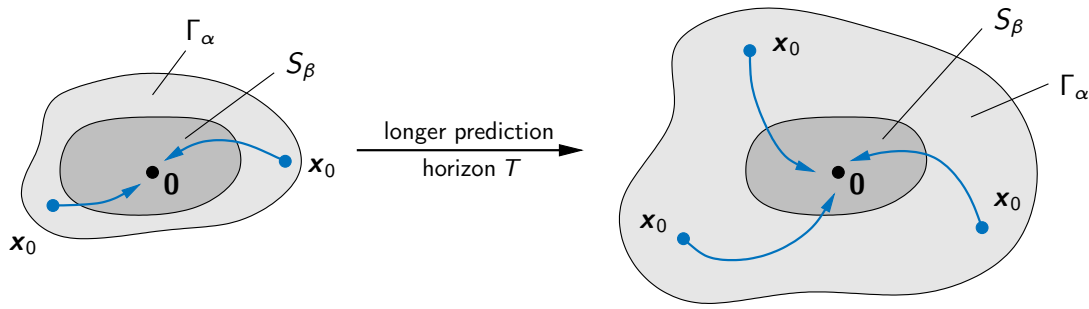


Figure 6.3: Domain of attraction for MPC with terminal cost.

Example 6.1 *We consider the nonlinear system [4]*

$$\begin{aligned} \dot{x}_1 &= x_2 + (\mu + (1 - \mu)x_1)u, & x_1(0) &= x_{10} \\ \dot{x}_2 &= x_1 + (\mu - 4(1 - \mu)x_2)u, & x_2(0) &= x_{20} \end{aligned} \quad (6.32)$$

with the state $\mathbf{x} = [x_1, x_2]^T$ and the constrained control input $u \in U = [-2, 2]$. The parameter $\mu \in (0, 1)$ is set to $\mu = 0.5$. The origin is an unstable equilibrium of the system (6.32) and the linearization around the origin is stabilizable. The integral term in the cost functional (6.28a) is defined by

$$l(\mathbf{x}, u) = 0.5x_1^2 + 0.5x_2^2 + u^2. \quad (6.33)$$

The CLF inequality (6.29) is satisfied on the set $S_\beta = \{\mathbf{x} \in \mathbb{R}^2 : V(\mathbf{x}) \leq \beta = 0.7\}$ with the terminal cost

$$V(\mathbf{x}) = \mathbf{x}^T \begin{bmatrix} 16.5926 & 11.5926 \\ 11.5926 & 16.5926 \end{bmatrix} \mathbf{x} \quad (6.34)$$

and the linear control law $\mathbf{q}(\mathbf{x}) = [2.118, 2.118]\mathbf{x}$. The left side of Figure 6.4 shows S_β as well as the set Γ_α for different prediction horizons T . For all initial values $\mathbf{x}_k \in \Gamma_\alpha$, the end point $\bar{\mathbf{x}}_T^*(T; \mathbf{x}_k)$ lies in the terminal region S_β . The single sets Γ_α are computed numerically and

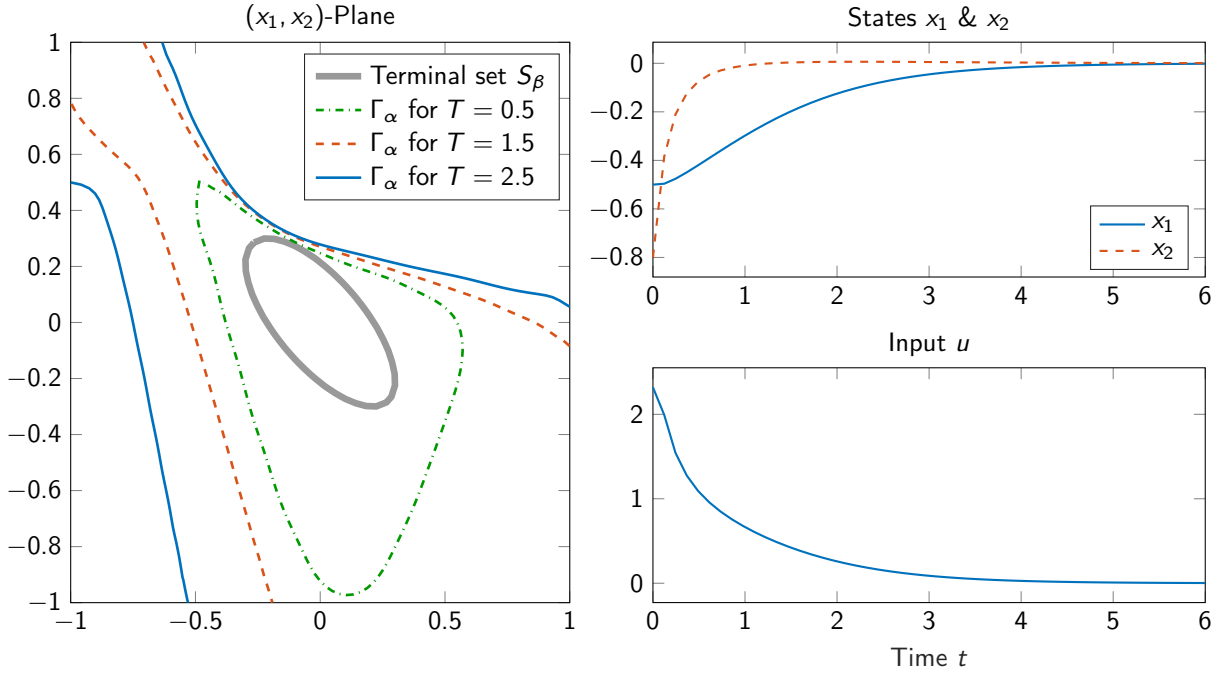


Figure 6.4: Terminal set S_β with $\beta = 0.7$ and domain of attraction Γ_α for different horizon lengths T (Example 6.1).

therefore are a better estimate of the domain of attraction than the conservative definition (6.30). As already pointed out in Lemma 6.2, the shape of S_β and Γ_α reveals that $S_\beta \subseteq \Gamma_\alpha$ holds for all T and that the domain of attraction Γ_α can be enlarged by increasing the prediction horizon. In addition, the right side of Figure 6.4 shows the simulated trajectories in closed-loop for the initial point $\mathbf{x}_0 = [-0.5, -0.8]^T$ and $T = 2.5$.

6.2.5 Sufficiently long horizon

Even if the previous MPC formulation does not require a terminal set constraint, the proof of stability nevertheless requires the terminal cost $V(\mathbf{x})$ to satisfy the CLF inequality (6.29). Depending on the horizon length T , this leads to a dominance within the cost functional and thus “falsifies” the qualitative control behavior that is usually tuned by the user via the weights in the integral term. This effect can e.g. be seen in Example 6.1.

On the other hand, we know from Section 6.2.1 that an infinite horizon guarantees stability without terminal cost or terminal constraint. It is therefore reasonable to ask whether a “sufficiently long” horizon can ensure asymptotic stability without having to impose the terminal cost $V(\mathbf{x})$ and the CLF inequality (6.29). To answer this question, we consider the following MPC formulation:

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = \int_0^T l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.35a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.35b)$$

$$\bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, T]. \quad (6.35c)$$

The investigation of this MPC formulation without terminal cost and terminal constraint is mathematically more complex than the previous cases and therefore requires stronger assump-

tions. This is also the reason for omitting the state constraints $\bar{\mathbf{x}}(\tau) \in X$. Moreover, we require the definition of (local) Lipschitz continuity.

Definition 6.1 (Local Lipschitz continuity) A function f is locally Lipschitz continuous in $\mathbf{x}$ if for every point $\mathbf{x}_0$ there exists a neighborhood $B_r(\mathbf{x}_0) = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x} - \mathbf{x}_0\| \leq r\}$ with radius $r > 0$ and a Lipschitz constant $L > 0$ such that

$$|f(\mathbf{x}_1) - f(\mathbf{x}_2)| \leq L\|\mathbf{x}_1 - \mathbf{x}_2\| \quad (6.36)$$

for all $\mathbf{x}_1, \mathbf{x}_2 \in B_r(\mathbf{x}_0)$.

The Lipschitz condition (6.36) can be extended to vector-valued functions $\mathbf{f}$ with $\|\mathbf{f}(\mathbf{x}_1) - \mathbf{f}(\mathbf{x}_2)\| \leq L\|\mathbf{x}_1 - \mathbf{x}_2\|$ and using a suitable vector norm. The following theorem states that under certain conditions a sufficiently long horizon T can be found such that the closed-loop system is stabilized.

Theorem 6.5 (MPC with sufficiently long horizon) Suppose that the problem (6.35) is feasible for all $\mathbf{x}_k \in \mathbb{R}^n$ and $T \in [\Delta t, \infty)$. Moreover, assume that

1. the linearization of (6.1) around the origin is stabilizable,
2. $J_T^*(\mathbf{x})$ is continuously differentiable in $\mathbf{x}$ and for all $T \in [\Delta t, \infty)$,
3. the optimal control law in (6.8) is locally Lipschitz continuous.

Then, for every compact set $\Gamma_\alpha = \{\mathbf{x} \in \mathbb{R}^n : J_\infty^*(\mathbf{x}) \leq \alpha\}$ with $\alpha < \infty$ and for each $\delta \in (0, 1)$, there exists some $T_\delta < \infty$ such that for all $T \geq T_\delta$ the cost satisfies

$$J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq J_T^*(\mathbf{x}_k) - \delta \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \quad \forall \mathbf{x}_k \in \Gamma_\alpha \quad (6.37)$$

and the origin of the system (6.1) in closed-loop is exponentially stable with Γ_α lying in the domain of attraction.

Proof: Bellman's principle of optimality allows us to split the optimal cost functional $J_T^*(\mathbf{x}_k)$ at point $\mathbf{x}_k$ into two parts

$$J_T^*(\mathbf{x}_k) = \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau + J_{T-\Delta t}^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)). \quad (6.38)$$

Moreover, the inequality

$$J_{T_1}^*(\mathbf{x}_k) \leq \int_0^{T_1} l(\bar{\mathbf{x}}_{T_2}^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_{T_2}^*(\tau; \mathbf{x}_k)) d\tau \leq J_{T_2}^*(\mathbf{x}_k)$$

shows that $J_{T_1}^*(\mathbf{x}_k) \leq J_{T_2}^*(\mathbf{x}_k)$ for all $T_1 \leq T_2$. Negating equation (6.38), adding $J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k))$, and using $J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq J_\infty^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k))$, we get

$$\begin{aligned} & J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) - J_T^*(\mathbf{x}_k) \\ & \leq J_\infty^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) - J_{T-\Delta t}^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) - \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau. \end{aligned} \quad (6.39)$$

For further investigations, we define the function $\psi_T : X_\alpha \rightarrow \mathbb{R}$

$$\psi_T(\mathbf{x}) = \begin{cases} \frac{J_\infty^*(\mathbf{x}) - J_{T-\Delta t}^*(\mathbf{x})}{J_{\Delta t}^*(\mathbf{x})} & \text{if } \mathbf{x} \neq \mathbf{0} \\ \limsup_{x \rightarrow 0} \psi_T(\mathbf{x}) & \text{if } \mathbf{x} = \mathbf{0} \end{cases} \quad (6.40)$$

on the compact set

$$X_\alpha = \{\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k) \mid \tau \in [0, \Delta t], \mathbf{x}_k \in \Gamma_\alpha, T \in [\Delta t, \infty)\} \subset \mathbb{R}^n. \quad (6.41)$$

It is obvious for $\mathbf{x} \neq \mathbf{0}$ that $\psi_T(\mathbf{x})$ is continuous and monotonically decreasing for $T \in [\Delta t, \infty)$. It can be shown for the critical point $\mathbf{x} = \mathbf{0}$ that $\psi_T(\mathbf{0})$ is bounded and $\lim_{T \rightarrow \infty} \psi_T(\mathbf{0}) = 0$ holds [16]. Thus, we can summarize that $\psi_T(\mathbf{x})$ is continuous and bounded for all $\mathbf{x} \in X_\alpha$ and monotonically decreasing in T with the limit $\lim_{T \rightarrow \infty} \psi_T(\mathbf{x}) = 0$ for each fixed $\mathbf{x} \in X_\alpha$. At this point, we require *Dini's theorem* to proceed.

Lemma 6.3 (Dini's theorem) *Let $f_n : X \rightarrow \mathbb{R}$ be a sequence of continuous functions on a compact set X that monotonically converges towards a function $f : X \rightarrow \mathbb{R}$. If f is continuous, then the convergence is uniform on X .²*

Dini's theorem allows to draw the conclusion that every sequence $\psi_{T_n} : \Gamma \rightarrow \mathbb{R}$ with $\Delta t = T_1 < T_2 < \dots$ converges uniformly towards $\lim_{T \rightarrow \infty} \psi_T(\mathbf{x}) = 0$. The definition of uniform convergence implies that for every $\varepsilon \in (0, 1)$ there exists some $\hat{T} < \infty$ such that $\psi_T(\mathbf{x}) < \varepsilon$ for all $\mathbf{x} \in \Gamma_\alpha$. Thus, from (6.40) and for the point $\mathbf{x} = \bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k) \in X_\alpha$ (for $\mathbf{x}_k \in \Gamma_\alpha$) it follows that

$$J_\infty^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) - J_{T-\Delta t}^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq \varepsilon J_{\Delta t}^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)). \quad (6.42)$$

Some estimates are required to proceed from this point. With *Gronwall's lemma* and its inverse formulation [10], we get

$$\|\mathbf{x}_k\| e^{-\hat{L}\tau} \leq \|\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k)\| \leq \|\mathbf{x}_k\| e^{\hat{L}\tau}, \quad \tau \in [0, T], \quad \hat{L} = L_f(1 + L_\kappa), \quad (6.43)$$

where $L_f, L_\kappa > 0$ are Lipschitz constants of the system function $\mathbf{f}$ and of the optimal feedback law κ_T (let L_κ hold for all times $T \in [\Delta t, \infty)$). Using the left-side bound in (6.43) and a similar reasoning as in [4], we can show that for all $T \in [\Delta t, \infty)$ there exist constants $m_J, M_J > 0$ such that the following quadratic bounds hold for $J_T^*(\mathbf{x}_k)$

$$m_J \|\mathbf{x}_k\|^2 \leq J_T^*(\mathbf{x}_k) \leq M_J \|\mathbf{x}_k\|^2, \quad \mathbf{x}_k \in \Gamma_\alpha, \quad T \in [\Delta t, \infty). \quad (6.44)$$

With the estimates (6.43) and (6.44), the right side of (6.42) can be bounded as follows

$$\begin{aligned} \varepsilon J_{\Delta t}^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) &\stackrel{(6.44)}{\leq} \varepsilon M_J \|\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)\|^2 \\ &\stackrel{(6.43)}{\leq} \varepsilon M_J e^{2\hat{L}\Delta t} \|\mathbf{x}_k\|^2 \\ &\stackrel{(6.44)}{\leq} \varepsilon \frac{M_J}{m_J} e^{2\hat{L}\Delta t} J_{\Delta t}^*(\mathbf{x}_k) \\ &\leq \varepsilon \frac{M_J}{m_J} e^{2\hat{L}\Delta t} \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau. \end{aligned} \quad (6.45)$$

² The sequence $f_n : X \rightarrow \mathbb{R}$ with the limit $f : X \rightarrow \mathbb{R}$ is called *uniformly convergent*, if for every $\varepsilon > 0$ there exists an integer N such that for all $n \geq N$ and for all $\mathbf{x} \in X$ we have $|f_n(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$.

This shows that (6.39) can be written in the form (6.37) with the constant δ defined by

$$\delta = 1 - \varepsilon \frac{M_J}{m_J} e^{2\hat{L}\Delta t}. \quad (6.46)$$

Since $\varepsilon \in (0, 1)$ can be freely chosen, there exists a horizon length T_δ for every $\delta \in (0, 1)$ such that (6.37) holds for $T \geq T_\delta$.³

To conclude about asymptotic stability, the integral in (6.37) can be lower bounded by

$$\int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \geq J_{\Delta t}^*(\mathbf{x}_k) \stackrel{(6.44)}{\geq} m_J \|\mathbf{x}_k\|^2 \stackrel{(6.44)}{\geq} \sigma J_T^*(\mathbf{x}_k) \quad (6.47)$$

with $\sigma = m_J/M_J \in (0, 1)$. From (6.37), we therefore obtain the estimate $J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq (1 - \delta\sigma)J_T^*(\mathbf{x}_k)$ with $1 - \delta\sigma \in (0, 1)$ or, respectively, by means of the initial condition $J_T^*(\mathbf{x}_0) \leq \alpha$

$$J_T^*(\mathbf{x}_k) \leq \alpha(1 - \delta\sigma)^k \Rightarrow \|\mathbf{x}_k\| \stackrel{(6.44)}{\leq} \sqrt{\frac{J_T^*(\mathbf{x}_k)}{m_J}} \leq \sqrt{\frac{\alpha}{m_J}}(1 - \delta\sigma)^{k/2}. \quad (6.48)$$

This shows that $\|\mathbf{x}_k\|$ decreases asymptotically for $k \in \mathbb{N}_0^+$. To show stability in the continuous-time sense, it is sufficient to consider the upper bound from (6.43)

$$\|\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k)\| \stackrel{(6.43)}{\leq} \|\mathbf{x}_k\| e^{\hat{L}\tau} \stackrel{(6.48)}{\leq} \sqrt{\frac{\alpha}{m_J}} e^{\hat{L}\Delta t} (1 - \delta\sigma)^{k/2}, \quad \tau \in [0, \Delta t). \quad (6.49)$$

It can be shown that there exist constants $q_1, q_2 > 0$ such that the trajectories in closed-loop (6.9) are bounded by $\|\mathbf{x}(t)\| \leq q_1 e^{-q_2(t-t_0)}$, which implies exponential stability of the origin in closed-loop [17]. ■

Exponential stability as the result of Theorem 6.5 is a stronger statement than asymptotic stability in the sense of $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$. However, the proof of exponential stability requires the optimal feedback law in (6.8) to be locally Lipschitz.

The exact value of T_δ is problem-specific and must be determined numerically (i.e. in simulations). Nevertheless, Theorem 6.5 provides the interesting result that (under the stated assumptions) there always exists a minimum horizon length $T_\delta < \infty$ such that exponential stability holds for $T \geq T_\delta$.

Example 6.2 We consider again the MPC problem of Example 6.1 and drop the terminal cost (6.34). The MPC formulation therefore reads

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = \int_0^T 0.5\bar{x}_1^2 + 0.5\bar{x}_2^2 + \bar{u}^2 d\tau \quad (6.50a)$$

$$\text{s.t. } \dot{\bar{x}}_1 = \bar{x}_2 + (\mu + (1 - \mu)\bar{x}_1)\bar{u}, \quad \bar{x}_1(0) = \bar{x}_{1,k} = x_1(t_k) \quad (6.50b)$$

$$\dot{\bar{x}}_2 = \bar{x}_1 + (\mu - 4(1 - \mu)\bar{x}_2)\bar{u}, \quad \bar{x}_2(0) = \bar{x}_{2,k} = x_2(t_k) \quad (6.50c)$$

$$\bar{u}(\tau) \in [-2, 2], \quad \tau \in [0, T]. \quad (6.50d)$$

Figure 6.5 illustrates the influence of the horizon length T on the MPC formulation (6.50) for the values $T = 1.5$ and $T = 2.5$. The boundaries of the compact subsets $\Gamma_\alpha = \{\mathbf{x} \in \mathbb{R}^2 :$

³ Lipschitz estimates are usually very conservative. Therefore, the explicit computation of δ in (6.46) is a constructive approach to prove the existence of δ , which is rather unsuitable for the MPC design itself.

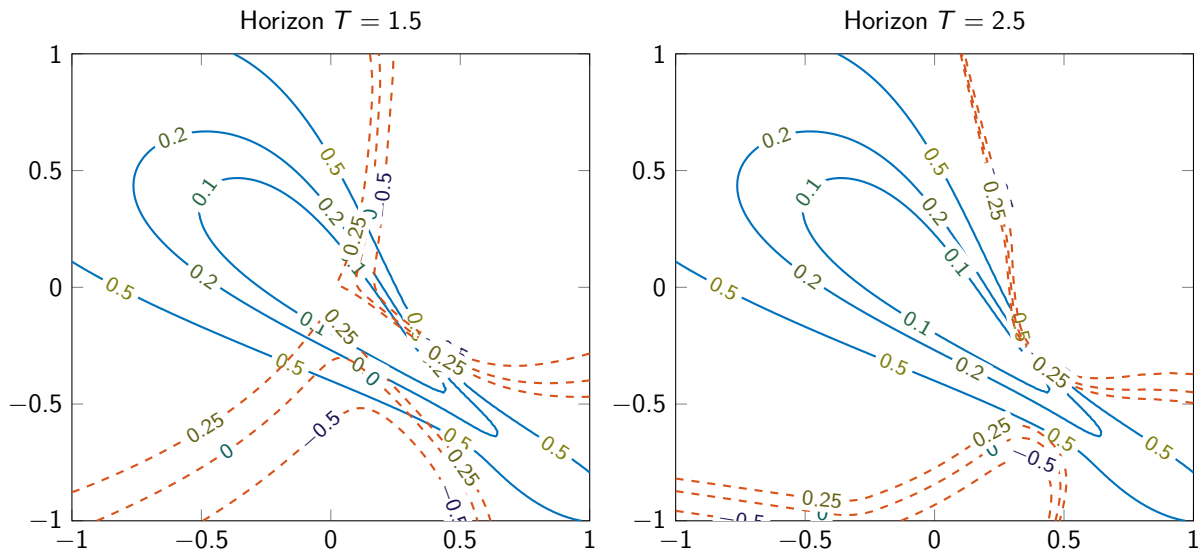


Figure 6.5: Contour lines of the domain of attraction Γ_α (—) and of the reduction factor δ (--) in the (x_1, x_2) -plane for the prediction horizon $T = 1.5$ and $T = 2.5$ (Example 6.2).

$J_\infty^*(\mathbf{x}) \leq \alpha$ were computed numerically. In addition, Figure 6.5 plots the contour lines for various values of the factor δ in (6.37).

The contour line for $\delta = 0$ represents the set of points $\mathbf{x}_k$ for which the next MPC step does not lead to a decrease of the cost $J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k))$. The cases $\delta = 0.25$ and $\delta = -0.5$ correspond to a decrease and increase of the cost functional (6.37). Thus, the set that is enclosed by the contour line $\delta = 0$ corresponds to the domain of attraction for the respective horizon length T . The plots for $T = 1.5$ and $T = 2.5$ in Figure 6.5 illustrate that the stability region increases for a longer prediction horizon.

Figure 6.5 also illustrates that T must be chosen sufficiently large if a given domain Γ_α is to be stabilized. For instance, the contour lines $\delta = 0$ show that stability over the region $\Gamma_{0.2}$ is only achieved for $T = 2.5$.

6.3 Real-time implementation

“All we have to decide is what to do with the time that is given to us.”

Gandalf the Grey

The last two MPC formulations with terminal cost or a sufficiently long horizon are particularly appropriate for an actual process implementation because of the reduced computational effort due to the absence of terminal constraints. Nevertheless, the real-time implementation of MPC is still a challenge, in particular for highly dynamical systems with sampling times in the (sub)millisecond range or if the MPC is implemented on hardware with limited resources, e.g. for embedded systems.

An intuitive approach for limiting the computational effort is the approximate solution of the optimal control problem, e.g. by using a fixed number of iterations per sampling step. In

the next step, the previous solution can be used to warm-start the optimization method with the intention to successively reduce the suboptimality of the solution over the runtime of the process. This incremental approach significantly differs from the optimal MPC case, where the optimal solution is assumed to be known in each sampling step.

There exist various efficient and suboptimal MPC schemes in the literature with different kinds of terminal constraints and conditions on the optimization algorithm that are not further elaborated at this point. Instead, we will focus on a general suboptimal solution strategy.

6.3.1 Suboptimal solution strategy

Since the main bottleneck for achieving real-time feasibility is the computational effort, we consider an MPC formulation without terminal conditions and state constraints

$$\min_{\bar{\mathbf{u}}(\cdot)} J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = V(\bar{\mathbf{x}}(T)) + \int_0^T l(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)) d\tau \quad (6.51a)$$

$$\text{s.t. } \dot{\bar{\mathbf{x}}}(\tau) = \mathbf{f}(\bar{\mathbf{x}}(\tau), \bar{\mathbf{u}}(\tau)), \quad \bar{\mathbf{x}}(0) = \mathbf{x}_k = \mathbf{x}(t_k) \quad (6.51b)$$

$$\bar{\mathbf{u}}(\tau) \in U, \quad \tau \in [0, T]. \quad (6.51c)$$

If no terminal cost is to be considered in (6.51a), we set $V(\mathbf{x}) = 0$ and $m_V = M_V = 0$ in the quadratic estimate (6.5).

In what follows, we assume the existence of an iterative optimization algorithm that computes a control and state trajectory

$$\bar{\mathbf{u}}_T^{(j)}(\tau; \mathbf{x}_k), \quad \bar{\mathbf{x}}_T^{(j)}(\tau; \mathbf{x}_k), \quad \tau \in [0, T] \quad (6.52)$$

in each iteration j . The corresponding (suboptimal) value of the cost functional is written as

$$J_T^{(j)}(\mathbf{x}_k) := J_T(\mathbf{x}_k, \bar{\mathbf{u}}_T^{(j)}) \geq J_T^*(\mathbf{x}_k).$$

Moreover, the optimization algorithm is supposed to be linearly convergent, i.e.

$$J_T^{(j+1)}(\mathbf{x}_k) - J_T^*(\mathbf{x}_k) \leq p \left(J_T^{(j)}(\mathbf{x}_k) - J_T^*(\mathbf{x}_k) \right) \quad (6.53)$$

with $\lim_{j \rightarrow \infty} J_T^{(j)}(\mathbf{x}_k) = J_T^*(\mathbf{x}_k)$. The factor $p \in (0, 1)$ is the linear convergence rate of the optimization algorithm.

The suboptimal solution strategy is supposed to stop the optimization algorithm after a fixed number of iterations

$$j = N$$

and to use the first part of the last suboptimal control trajectory $\bar{\mathbf{u}}_T^{(N)}(\tau; \mathbf{x}_k)$ as control input

$$\mathbf{u}(t_k + \tau) = \bar{\mathbf{u}}_T^{(N)}(\tau; \mathbf{x}_k), \quad \tau \in [0, \Delta t] \quad (6.54)$$

for the system (6.1). If no model uncertainties or disturbances are present, the next state at time instant t_{k+1} is $\mathbf{x}_{k+1} = \bar{\mathbf{x}}_T^{(N)}(\Delta t; \mathbf{x}_k)$. Overall, the closed-loop trajectories are given by

$$\begin{aligned} \mathbf{x}(t) &= \mathbf{x}(t_k + \tau) = \bar{\mathbf{x}}_T^{(N)}(\tau; \mathbf{x}_k), \\ \mathbf{u}(t) &= \mathbf{u}(t_k + \tau) = \bar{\mathbf{u}}_T^{(N)}(\tau; \mathbf{x}_k), \quad \tau \in [0, \Delta t], \quad k \in \mathbb{N}_0^+. \end{aligned} \quad (6.55)$$

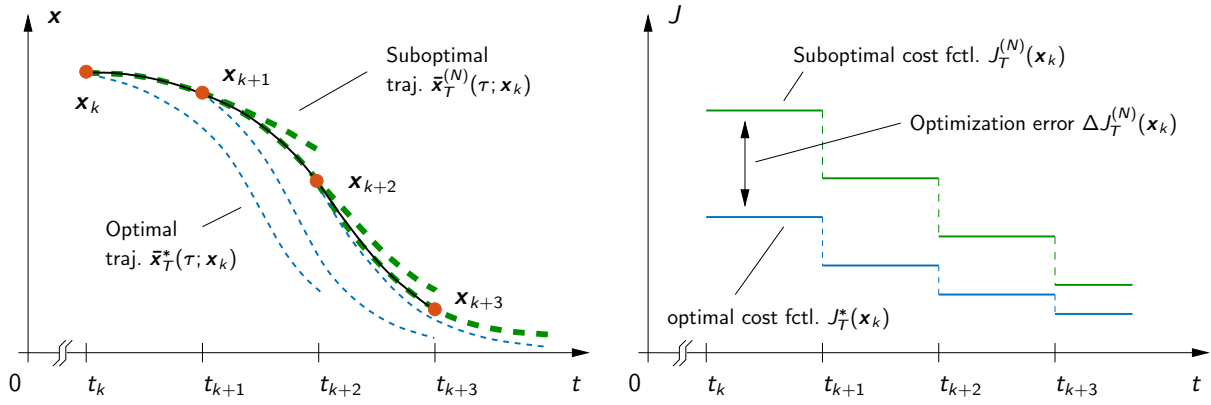


Figure 6.6: Principle of the suboptimal MPC strategy.

The trajectories are suboptimal and therefore lead to an *optimization error*

$$\Delta J_T^{(N)}(\mathbf{x}_k) := J_T^{(N)}(\mathbf{x}_k) - J_T^*(\mathbf{x}_k) \geq 0 \quad (6.56)$$

in each MPC step k . The goal of the suboptimal MPC strategy is to successively reduce the optimization error $\Delta J_T^{(N)}(\mathbf{x}_k)$ by using the last control trajectory $\bar{\mathbf{u}}_T^{(N)}(\tau; \mathbf{x}_k)$ to re-initialize the optimization method in the next step

$$\bar{\mathbf{u}}_T^{(0)}(\tau; \mathbf{x}_{k+1}) := \bar{\mathbf{u}}_T^{(N)}(\tau; \mathbf{x}_k), \quad \tau \in [0, T]. \quad (6.57)$$

Figure 6.6 illustrates the basic concept behind the suboptimal MPC strategy.

6.3.2 Stability and incremental improvement

For a more compact notation, we introduce the set

$$\mathcal{U}_{[0,T]} = \{\mathbf{u}(\cdot) \in L_m^\infty[0, T] : \mathbf{u}(t) \in U, t \in [0, T]\} \quad (6.58)$$

of all admissible control trajectories.⁴ The single iterations of an optimization algorithm $\bar{\mathbf{u}}_T^{(j)}(\cdot) \in \mathcal{U}_{[0,T]}$ in general differ from the optimal solution $\bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)$, $\tau \in [0, T]$ of (6.51), which requires to impose more restrictive assumptions than in the previous considerations. In particular, we impose the strict convexity condition

$$\|\mathbf{u} - \mathbf{u}_T^*(\cdot; \mathbf{x}_k)\|_{L_m^2[0,T]}^2 \leq C |J_T(\mathbf{x}_k, \mathbf{u}) - J_T^*(\mathbf{x}_k)| \quad (6.59)$$

for some constant $C > 0$. It can be shown that this assumption is always satisfied for linear systems with diagonal-quadratic cost functional.

⁴ The *infinite-dimensional* vector space $L_m^p[0, T]$, $1 \leq p < \infty$ is the space of all real-valued functions $u : [0, T] \rightarrow \mathbb{R}$ defined on the interval $[0, T]$ that is equipped with the norm

$$\|u\|_{L^p[0,T]} = \left(\int_0^T |u(t)|^p dt \right)^{1/p} < \infty \quad \text{or} \quad \|\mathbf{u}\|_{L_m^p[0,T]} = \left(\int_0^T \|\mathbf{u}(t)\|_2^p dt \right)^{1/p} < \infty$$

for vector-valued functions $\mathbf{u} : [0, T] \rightarrow \mathbb{R}^m$. In particular, the supremum norm is defined by $\|\mathbf{u}\|_{L_m^\infty[0,T]} = \sup(\|\mathbf{u}(t)\|_2, t \in [0, T])$.

Theorem 6.6 (Suboptimal MPC) Suppose the problem (6.51) is feasible for all $\mathbf{x}_k \in \mathbb{R}^n$ and that the following conditions are satisfied on a compact set $\Gamma_\alpha = \{\mathbf{x} \in \mathbb{R}^n : J_T^*(\mathbf{x}) \leq \alpha\}$ with $\alpha < \infty$:

1. The optimal control law in (6.8) is locally Lipschitz continuous.
2. $J_T(\mathbf{x}_k, \mathbf{u})$ is twice continuously differentiable in $\mathbf{x}_k$ for each $\mathbf{u} \in \mathcal{U}_{[0,T]}$.
3. The convexity condition (6.59) is satisfied for all $\mathbf{u} \in \mathcal{U}_{[0,T]}$ and $\mathbf{x}_k \in \Gamma_\alpha$.
4. The optimization method used for solving (6.51) satisfies (6.53) with a linear convergence rate $p \in (0, 1)$ for all $\mathbf{x}_k \in \Gamma_\alpha$.
5. There exists a constant $\delta \in (0, 1]$ such that

$$J_T^*(\bar{\mathbf{x}}_T^*(\Delta t; \mathbf{x}_k)) \leq J_T^*(\mathbf{x}_k) - \delta \int_0^{\Delta t} l(\bar{\mathbf{x}}_T^*(\tau; \mathbf{x}_k), \bar{\mathbf{u}}_T^*(\tau; \mathbf{x}_k)) d\tau \quad \forall \mathbf{x}_k \in \Gamma_\alpha. \quad (6.60)$$

Then, there exists a minimum number of iterations $\hat{N}$ and a constant $\Delta \hat{J} > 0$ such that for all $N \geq \hat{N}$ and all points $\mathbf{x}_0 \in \Gamma_\alpha$ with initial optimization error $\Delta J_T^{(0)}(\mathbf{x}_0) \leq \Delta \hat{J} p^{-N}$ the origin of the system (6.1) in closed-loop is exponentially stable and the optimization error (6.56) decays exponentially.

Proof: Similar to the procedure in [14, 11], it can be shown under the given assumptions that there exist constants $a \in (0, 1]$ and $b, c, d, e > 0$ such that $J_T^*(\mathbf{x}_k)$ and $\Delta J_T^{(N)}(\mathbf{x}_k)$ satisfy the estimates

$$\begin{aligned} J_T^*(\mathbf{x}_{k+1}) &\leq (1-a)J_T^*(\mathbf{x}_k) + b\sqrt{J_T^*(\mathbf{x}_k)}\sqrt{\Delta J_T^{(N)}(\mathbf{x}_k)} + c\Delta J_T^{(N)}(\mathbf{x}_k) \\ \Delta J_T^{(N)}(\mathbf{x}_{k+1}) &\leq p^N(1+d)\Delta J_T^{(N)}(\mathbf{x}_k) + p^N e J_T^*(\mathbf{x}_k) \end{aligned} \quad (6.61)$$

with $\mathbf{x}_{k+1} = \bar{\mathbf{x}}_T^{(N)}(\Delta t; \mathbf{x}_k)$ denoting the state in the next MPC step. By induction it can be shown that for the initial conditions

$$J_T^*(\mathbf{x}_0) \leq \alpha, \quad \Delta J_T^{(N)}(\mathbf{x}_0) \leq \Delta \hat{J} = \gamma^2 \alpha \quad \text{mit} \quad \gamma = \frac{-b + \sqrt{b^2 + 2ac}}{2c} \quad (6.62)$$

and a “sufficiently large” convergence rate

$$p^N \leq \frac{(1 - \frac{a}{2})\gamma^2}{(1+d)\gamma^2 + e} \quad (6.63)$$

the estimates

$$J_T^*(\mathbf{x}_k) \leq \left(1 - \frac{a}{2}\right)^k \alpha, \quad \Delta J_T^{(N)}(\mathbf{x}_k) \leq \gamma^2 \left(1 - \frac{a}{2}\right)^k \alpha \quad (6.64)$$

hold. The upper bound for p^N corresponds to a minimum number $\hat{N}$ of MPC iterations per MPC step. If this is satisfied, then $J_T^*(\mathbf{x}_k)$ as well as $\Delta J_T^{(N)}(\mathbf{x}_k)$ decay exponentially in discrete-time. The proof of exponential stability in closed-loop proceeds similar to the proof of Theorem 6.5, also see [14]. ■

Although the MPC formulation (6.51) could be solved with any of the methods presented in Chapter 5 (Section 5.6 and 5.7), the gradient method (see Section 5.6.4 and Table 5.1) is particularly favorable due to the formulation without terminal constraints. Moreover, it is shown in [6] that the gradient method satisfies the linear convergence property (6.53) under certain convexity and regularity assumptions.

As mentioned in Section 5.6.4, the input constraints (6.51c) can be considered via projection and the line search of the gradient method in Table 5.1 can be realized with an adaptive or explicit approach with a constant computation load in each MPC step [12]. The gradient method is also implemented in the MPC toolbox GRAMPC that uses an augmented Lagrangian formulation with a gradient-based minimization in the inner loop [7].

A further possibility is to solve the optimality conditions resulting from Pontryagin's maximum principle (Section 5.5.1) with a *fixed-point iteration scheme* that can be implemented with minimal computational footprint [11].

6.3.3 Application example

An illustrative MPC example is the laboratory-scale overhead crane in Figure 6.7 with five degrees-of-freedom (DOF) [13]. The crane consists of a fixed tower of height h , a rotating jib with a movable trolley and a cable with an attached load of mass m . The jib rotates about the vertical axis of the tower with the angle ϕ_1 . The radial motion of the trolley is measured from the tower axis to the trolley's center of mass in terms of the translational DOF s_1 . A cable winch fixed to the movable trolley together with a gimbal allow for the vertical actuation and an arbitrary pendulum motion of the cable with the attached load. The vertical motion is described in terms of the translational DOF s_2 measured from the gimbal to the load. The rotation of the gimbal is considered as a composition of two successive rotations with the rotational DOF ϕ_2 and ϕ_3 . In summary, the crane is an underactuated mechanical system with five DOFs and three controls.

The equations of motion can be derived via Lagrange's formalism. Under the assumption that the actuated DOF ϕ_1 , s_1 , and s_2 are controlled by sufficiently fast cascaded control loops, the dynamics of the system are described by the second-order differential equations

$$\ddot{s}_1 = u_1 \quad (6.65a)$$

$$\ddot{s}_2 = u_2 \quad (6.65b)$$

$$\ddot{\phi}_1 = u_3 \quad (6.65c)$$

$$\begin{aligned} \ddot{\phi}_2 = \frac{1}{s_2 \cos \phi_3} & \left(-2\dot{s}_2\dot{\phi}_1 \cos \phi_2 \sin \phi_3 - 2\dot{s}_2\dot{\phi}_2 \cos \phi_3 - 2\dot{\phi}_1\dot{\phi}_3 s_2 \cos \phi_2 \cos \phi_3 \right. \\ & + 2s_2\dot{\phi}_2\dot{\phi}_3 \sin \phi_3 - s_1\dot{\phi}_1^2 \cos \phi_2 + s_2\dot{\phi}_1^2 \sin \phi_2 \cos \phi_2 \cos \phi_3 \\ & \left. - g \sin \phi_2 + \cos \phi_2 u_1 - s_2 \cos \phi_2 \sin \phi_3 u_3 \right) \end{aligned} \quad (6.65d)$$

$$\begin{aligned} \ddot{\phi}_3 = \frac{1}{s_2} & \left(-2\dot{s}_1\dot{\phi}_1 \cos \phi_3 - 2\dot{s}_2\dot{\phi}_3 + 2\dot{s}_2\dot{\phi}_1 \sin \phi_2 + 2s_2\dot{\phi}_1\dot{\phi}_2 \cos \phi_2 \cos^2 \phi_3 \right. \\ & + s_1\dot{\phi}_1^2 \sin \phi_2 \sin \phi_3 - s_2\dot{\phi}_2^2 \sin \phi_3 \cos \phi_3 + s_2\dot{\phi}_1^2 \cos^2 \phi_2 \sin \phi_3 \cos \phi_3 \\ & \left. - g \cos \phi_2 \sin \phi_3 - \sin \phi_2 \sin \phi_3 u_1 + (s_2 \sin \phi_2 - s_1 \cos \phi_3) u_3 \right) \end{aligned} \quad (6.65e)$$

independent of the load mass m . The accelerations $\ddot{s}_1$, $\ddot{s}_2$, $\ddot{\phi}_1$ of the actuated DOF denote the control inputs of the crane system that are constrained by

$$\begin{aligned} u_1 & \in [u_1^-, u_1^+] = [-2, +2] \text{ m/s}^2 \\ u_2 & \in [u_2^-, u_2^+] = [-2, +2] \text{ m/s}^2 \\ u_3 & \in [u_3^-, u_3^+] = [-2, +2] \text{ rad/s}^2. \end{aligned} \quad (6.66)$$

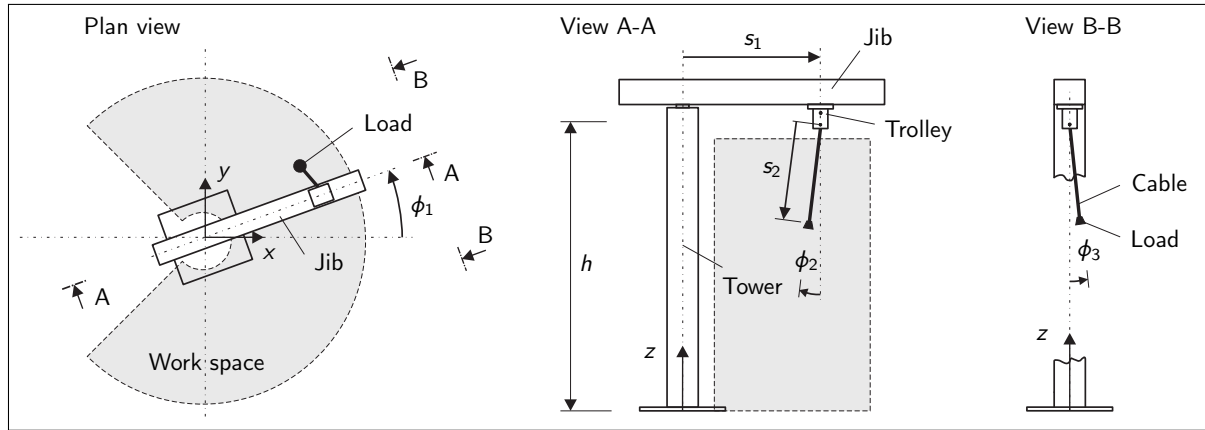


Figure 6.7: Schematic drawing of the overhead crane five degrees-of-freedom [13].

These values are obtained from the maximum motor currents with additional safety margins. The equations of motion (6.65) can be written in the general nonlinear form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}), \quad \mathbf{x}(0) = \mathbf{x}_0$$

with the state and input vectors

$$\mathbf{x} = [s_1, s_2, \phi_1, \phi_2, \phi_3, \dot{s}_1, \dot{s}_2, \dot{\phi}_1, \dot{\phi}_2, \dot{\phi}_3]^T, \quad \mathbf{u} = [u_1, u_2, u_3]^T. \quad (6.67)$$

The dynamical model of the crane therefore represents a nonlinear multiple-input system with $m = 3$ controls and $n = 10$ state variables.

The test scenario for the laboratory crane is a setpoint change from the initial position $\mathbf{x}_0$ to a desired one $\mathbf{x}_{des}$ with

$$\begin{aligned} \mathbf{x}_0 &= [0.7 \text{ m}, 0.7 \text{ m}, -60^\circ, 0, 0, 0, 0, 0, 0, 0]^T \\ \mathbf{x}_{des} &= [0.2 \text{ m}, 0.25 \text{ m}, 60^\circ, 0, 0, 0, 0, 0, 0, 0]^T. \end{aligned} \quad (6.68)$$

The setpoint change describes a turn of the jib about ϕ_1 with a simultaneous change of the trolley position s_1 and the cable length s_2 . The cost functional (6.51a) to accomplish the setpoint change is chosen as the quadratic form

$$J_T(\mathbf{x}_k, \bar{\mathbf{u}}) = \Delta \bar{\mathbf{x}}^T(T) \mathbf{P} \Delta \bar{\mathbf{x}}(T) + \int_0^T \Delta \bar{\mathbf{x}}^T(\tau) \mathbf{Q} \Delta \bar{\mathbf{x}}(\tau) + \bar{\mathbf{u}}^T(\tau) \mathbf{R} \bar{\mathbf{u}}(\tau) d\tau \quad (6.69)$$

with $\Delta \bar{\mathbf{x}} = \bar{\mathbf{x}} - \mathbf{x}_d$ and the diagonal matrices (neglecting SI units)

$$\begin{aligned} \mathbf{P} &= \text{diag}(10, 10, 10, 10, 10, 1, 1, 1, 1, 1) \\ \mathbf{Q} &= \text{diag}(1, 1, 1, 1, 1, 0.1, 0.1, 0.1, 1, 1) \\ \mathbf{R} &= \text{diag}(0.01, 0.01, 0.01). \end{aligned}$$

The prediction horizon and the sampling time are set to $T = 1.5 \text{ s}$ and $\Delta t = 2 \text{ ms}$.

The suboptimal MPC scheme is implemented with the toolbox GRAMPC that is based on an augmented Lagrangian formulation with a projected gradient method for the inner minimization problem. The canonical equations (see Table 5.1) within GRAMPC are solved with the second-order Heun method on an equidistant discretization of the time interval $[0, T]$ with 30 grid points.

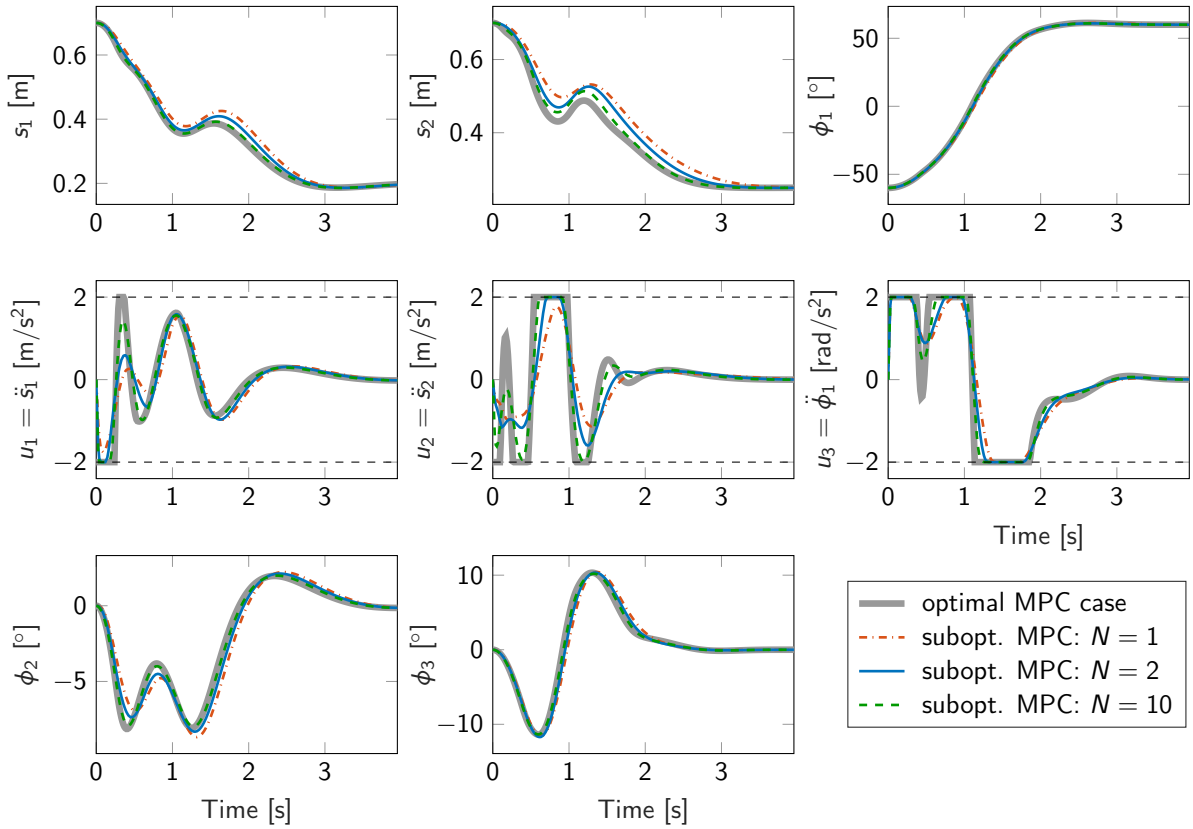


Figure 6.8: MPC simulation results for the setpoint change (6.68) of the overhead crane [13].

Figure 6.8 shows the simulation results for the setpoint change (6.68) with different numbers of iterations per MPC step N . The optimal MPC case is shown as reference. A good control performance can already be obtained for one iteration ($N = 1$) per sampling step. Increasing N mainly affects the control trajectories that show a more aggressive behavior and better exploitation of the constraints (6.66). It can also be seen that the trajectories approach the optimal solution for increasing values of N .

Figure 6.9 also shows the time profiles of the cost functional $J_T^{(N)}(\mathbf{x}_k)$ and optimization error $\Delta J_T^{(N)}(\mathbf{x}_k)$ that was introduced in (6.56). Both $J_T^{(N)}(\mathbf{x}_k)$ and $\Delta J_T^{(N)}(\mathbf{x}_k)$ decay exponentially, which shows the exponential stability in closed-loop and the incremental improvement of the suboptimal MPC scheme [14, 12].

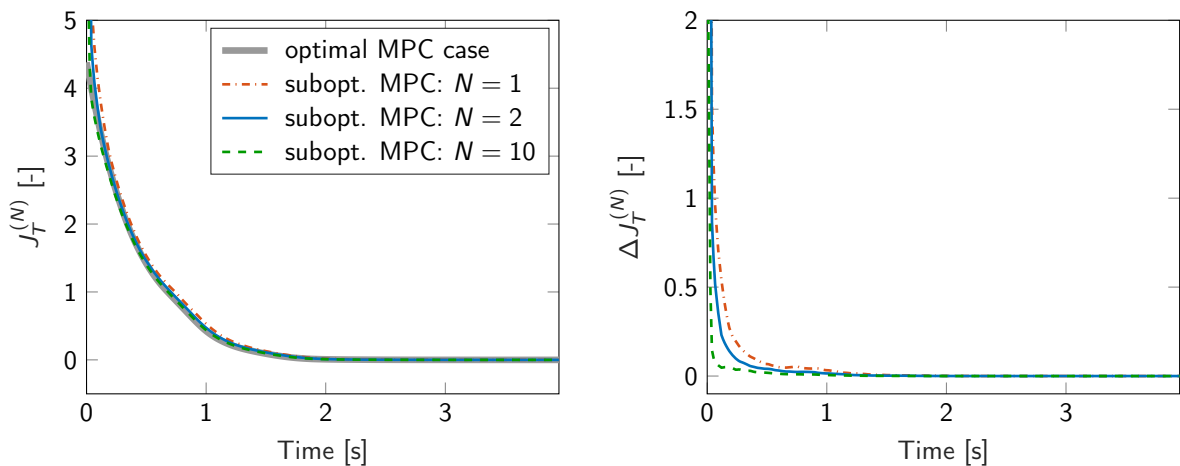


Figure 6.9: Cost $J_T^{(N)}(\mathbf{x}_k)$ and optimization error $\Delta J_T^{(N)}(\mathbf{x}_k)$ corresponding to Figure 6.8.

In addition, Table 6.1 lists the computation times of GRAMPC in terms of N as well as the standard deviation as a measure of the constant CPU load and real-time feasibility. The CPU results are obtained on a MacBook Pro with Apple M3 Max CPU.

Gradient steps N	CPU time [ms]	
	avg.	std. dev.
1	0.005	0.0004
2	0.009	0.0006
3	0.012	0.0006
5	0.020	0.0019
10	0.036	0.0018

Table 6.1: Computation times of GRAMPC for the overhead crane.

6.4 Software overview

Several MPC toolboxes are available today that are either open source or commercial. The following lines are a non-exhaustive selection of MPC toolboxes for linear and nonlinear systems. Further information can e.g. be found in the survey paper [9].

MPC toolboxes for linear systems

- CVXGEN (commercial, free for academic use)
<https://cvxgen.com/>
- FiOrdOs (open source licence)
<http://fiordos.ethz.ch/>
- Matlab MPC Toolbox (commercial)
<https://de.mathworks.com/products/mpc.html>
- μ AO-MPC (free)
<https://github.com/muaompc/muaompc>

MPC toolboxes for nonlinear systems

- acados (open source license)
<https://docs.acados.org>
- C/GMRES (free)
<http://www.ids.sys.i.kyoto-u.ac.jp/~ohtsuka/code/>
- GRAMPC (open source licence)
<https://github.com/grampc/grampc>
- HQP (open source licence)
<http://hqp.sourceforge.net>
- VIATOC (open source licence)
<https://sourceforge.net/projects/viatoc/>

6.5 References

- [1] L.D. Berkovitz. *Optimal Control Theory*. New York: Springer, 1974.
- [2] E. Camacho and C. Alba. *Model Predictive Control*. 2nd. London: Springer, 2004.
- [3] H. Chen. *Stability and Robstness Considerations in Nonlinear Model Predictive Control*. Vol. 674. Düsseldorf: Fortschritt-Berichte VDI Reihe 8, 1997.
- [4] H. Chen and F. Allgöwer. “A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability”. In: *Automatica* 34.10 (1998), pp. 1205–1217.
- [5] M. Diehl, H.J. Ferreau, and N. Haverbeke. “Efficient numerical methods for nonlinear MPC and moving horizon estimation”. In: *Nonlinear Model Predictive Control - Towards New Challenging Applications*. Ed. by L. Magni, D.M. Raimondo, and F. Allgöwer. 2009, pp. 391–417.
- [6] J.C. Dunn. “On L^2 conditions and the gradient projection method for optimal control problems”. In: *SIAM Journal on Control and Optimization* 34.4 (1996), pp. 1270–1290.
- [7] T. Englert, A. Völz, F. Mesmer, S. Rhein, and K. Graichen. “A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach (GRAMPC)”. In: *Optimization and Engineering* 20.3 (2019), pp. 769–809. DOI: 10.1007/s11081-018-9417-2.
- [8] R. Findeisen. *Nonlinear Model Predictive Control: A Sampled-Data Feedback Perspective*. Vol. 1087. <http://dx.doi.org/10.18419/opus-4059>. Fortschritt-Berichte VDI Reihe 8, 2006.
- [9] R. Findeisen, K. Graichen, and M. Mönnigmann. “Eingebettete Optimierung in der Regelungstechnik – Grundlagen und Herausforderungen”. In: *at – Automatisierungstechnik* 66.11 (2018), pp. 877–902.
- [10] H.E. Gollwitzer. “A note on a functional inequality”. In: *Proc. American Mathematical Society* 23.3 (1969), pp. 642–647.
- [11] K. Graichen. “A fixed-point iteration scheme for real-time model predictive control”. In: *Automatica* 48.7 (2012), pp. 1300–1305.
- [12] K. Graichen, M. Egretzberger, and A. Kugi. “Ein suboptimaler Ansatz zur schnellen modellprädiktiven Regelung nichtlinearer Systeme”. In: *Automatisierungstechnik* 58 (2010), pp. 447–456.
- [13] K. Graichen, M. Egretzberger, and A. Kugi. “Suboptimal model predictive control of a laboratory crane”. In: *Proc. 8th IFAC Symposium “Nonlinear Control Systems” (NOLCOS)*. Bologna (Italy), Sept. 2010, pp. 397–402.
- [14] K. Graichen and A. Kugi. “Stability and incremental improvement of suboptimal MPC without terminal constraints”. In: *IEEE Transactions on Automatic Control* 55.11 (2010), pp. 2576–2580.
- [15] L. Grüne and J. Pannek. *Nonlinear Model Predictive Control: Theory and Algorithms*. London: Springer, 2011.
- [16] A. Jadbabaie, J. Primbs, and J. Hauser. “Unconstrained receding horizon control with no terminal cost”. In: *Proc. American Control Conference (ACC)*. Arlington, VA, June 2001, pp. 3055–3060.
- [17] H.K. Khalil. *Nonlinear Systems*. 2nd. Prentice Hall, 1996.
- [18] E.B. Lee and L. Markus. *Foundations of Optimal Control*. New York: Wiley, 1967.

- [19] D. Limon, T. Alamo, F. Salas, and E.F. Camacho. “On the stability of constrained MPC without terminal constraint”. In: *IEEE Transaction on Automatic Control* 51.5 (2006), pp. 832–836.
- [20] D.Q. Mayne, J.B. Rawlings, C.V. Rao, and P.O.M. Scokaert. “Constrained model predictive control: stability and optimality”. In: *Automatica* 36.6 (2000), pp. 789–814.